CIS Center for Internet Security®

CIS Benchmarks™

CIS MCP Server Benchmark

v1.0.0

Terms of Use

Please see the below link for our current terms of use:

https://www.cisecurity.org/cis-securesuite/cis-securesuite-membership-terms-of-use/

For information on referencing and/or citing CIS Benchmarks in 3rd party documentation (including using portions of Benchmark Recommendations) please contact CIS Legal (legalnotices@cisecurity.org) and request guidance on copyright usage.

NOTE: It is **NEVER** acceptable to host a CIS Benchmark in **ANY** format (PDF, etc.) on a 3rd party (non-CIS owned) site.

Table of Contents

Terms of Use *1*
Overview *5*
Important Usage Information 5
Key Stakeholders 5
Apply the Correct Version of a Benchmark 6
Exceptions 6
Remediation 7
Summary 7
Target Technology Details 8
Intended Audience 8
Consensus Guidance *9*
Typographical Conventions *10*
Recommendation Definitions *11*
Title 11
Assessment Status 11
Automated 11
Manual 11
Profile 11
Description 11
Rationale Statement 11
Impact Statement 12
Audit Procedure 12
Remediation Procedure 12
Default Value 12
References 12
CIS Critical Security Controls (CIS Controls) 12
Additional Information 12
Profile Definitions *13*
Acknowledgements *14*
Recommendations *15*
1 Governance & Versioning 15
1.1 Ensure MCP protocol version is pinned and logged (Automated) 16
1.2 Ensure server capability baseline is established and documented (Manual) 29
1.3 Ensure newly advertised server capabilities are re-validated before invocation (Manual) 38
1.4 Ensure MCP client and server identity metadata is captured and validated against the asset inventory (Automated) 45
2 Transport & Connectivity 55
2.1 Ensure stdio is preferred for local, single-user servers (Manual) 56
2.2 Ensure TLS is required for HTTP and plaintext is disallowed (Automated) 60
2.3 Ensure authentication propagates through proxies to request-scoped SSE responses (Manual) .....
2.4 Ensure required MCP request metadata headers are present and consistent with the request body (Automated) 70
2.5 Ensure Origin header is validated on all Streamable HTTP requests to prevent DNS rebinding attacks (Automated) 80
3 Authentication & Authorization 84
3.1 Authentication and Credential Sourcing 84
3.1.1 Ensure stdio server credentials are sourced from the environment or OS credential store (Automated) 85
3.1.2 Ensure OIDC/OAuth 2.1 or short-lived API tokens are used for remote servers (Manual) 88

3.2 Authorization Enforcement **92**
3.2.1 Ensure per-tool authorization policies are enforced (Manual) 93
3.2.2 Ensure token passthrough to downstream APIs is forbidden (Manual) 98
3.2.3 Ensure server-provided tool annotations are not relied upon for authorization or HITL gating decisions (Manual) 102
3.3 OAuth Token and Metadata Hardening **107**
3.3.1 Ensure OAuth tokens are audience-bound to the MCP server using Resource Indicators (Automated) 108
3.3.2 Ensure OAuth discovery metadata is served over TLS and validated against the approved authorization server list (Automated) 111
3.3.3 Ensure shared downstream service account identities are prohibited across multiple MCP tools or servers (Manual) 120
3.3.4 Ensure OAuth scopes are minimized and elevated progressively (Manual) 123
3.3.5 Ensure confused-deputy safeguards are applied for static OAuth client IDs (Manual) 126
4 Client (Host) Configuration **130**
4.1 Consent & Human Oversight **130**
4.1.1 Ensure per-tool consent or pre-approved allowlists are required (Manual) 131
4.1.2 Ensure project-scoped MCP server definitions are not enabled or executed without explicit user approval (Manual) 136
4.1.3 Ensure Elicitation requests are consented, redacted, and identity-bound (Manual) 139
4.1.4 Ensure Sampling consent and redaction controls are implemented where Sampling is in use (Automated) 142
4.1.5 Ensure a human can deny tool invocations and sensitive or high-impact operations require confirmation (Manual) 145
4.2 Resource & Filesystem Restrictions **149**
4.2.1 Ensure filesystem scope is restricted to least privilege and enforced by OS-level controls (Manual) 150
4.2.2 Ensure MCP stdio server subprocess environment is restricted to the minimum required variables (Automated) 153
4.2.3 Ensure untrusted Resource URIs are not directly dereferenced outside MCP without scheme and destination policy (Manual) 156
4.3 MCP Apps **160**
4.3.1 Ensure MCP Apps content is rendered in sandboxed isolation with Content Security Policy enforced from declared metadata (Manual) 161
4.3.2 Ensure MCP Apps permission requests default-deny, all app-to-host messages are logged, and high-risk channels are consented (Manual) 164
5 Server Configuration **168**
5.1 Input & Schema Validation **168**
5.1.1 Ensure tool schemas and argument types are validated (Automated) 169
5.1.2 Ensure resource templates with explicit URI patterns and MIME types are used (Automated) 176
5.1.3 Ensure prompt templates declare and validate their arguments (Automated) 182
5.2 Session & Stream Hygiene **186**
5.2.1 Ensure listChanged notifications are rate-limited to prevent client flooding (Automated) 187
5.2.2 Ensure sessions are not used as authentication and, where present, are bound to user identity (Manual) 190
5.2.3 Ensure legacy Streamable HTTP session and stream-resumption mechanisms are disabled (Automated) .....
5.3 Protocol Stream Integrity **198**
5.3.1 Ensure logs are separated from the protocol stream in stdio mode (Automated) 199
5.4 Filesystem Path Safety **203**
5.4.1 Ensure path traversal and arbitrary filesystem access are prevented (Automated) 204
5.5 MCP Tasks **207**
5.5.1 Ensure authorization, scope, and expiry controls are enforced on MCP Tasks (Manual) 208
5.6 Idempotency & Side-Effect Control **213**
5.6.1 Ensure idempotency keys are required for side-effecting tool calls and duplicate side effects are prevented (Manual) 214
6 Data Protection & Privacy **219**
6.1 Ensure model context is minimized and MIME handling is honored (Manual) 220

6.2 Ensure tokens and keys are protected in clients and servers (Automated) 224
6.3 Ensure outbound proxy and DNS controls are applied for resource fetchers (Manual) 227
7 Observability & Audit 232
7.1 Audit Record Content & Integrity 232
7.1.1 Ensure lifecycle and invocation metadata is recorded (Automated) 233
7.1.2 Ensure non-null JSON-RPC request IDs are enforced and included in audit logs (Automated) 236
7.1.3 Ensure MCP audit records carry accurate, monotonic timestamps (Automated) 241
7.2 Detection & Alerting 244
7.2.1 Ensure alerts are generated on audience and issuer validation failures (Automated) 245
7.2.2 Ensure notifications/cancelled and notifications/progress messages are monitored for behavioral anomalies (Automated) 249
8 Supply Chain Security 254
8.1 Ensure third-party MCP servers and connectors are vetted and allowlists are enforced (Manual) 255
8.2 Ensure third-party MCP servers are pinned to a verified content hash and re-vetted before updates (Manual) 259
8.3 Ensure third-party MCP server artifacts are served exclusively from an internal private mirror (Manual) 266
9 Isolation & Execution Safety 272
9.1 Ensure remote MCP servers run in containers or VMs with least privilege and gateway ingress controls applied (Manual) 273
9.2 Ensure the MCP server process is confined to its working set by OS-level kernel-enforced controls (Automated) 280
9.3 Ensure local stdio MCP servers run under least-privilege identities with sandbox or container isolation (Manual) 283
10 Resource Limits & Caching 289
10.1 Ensure safe, static resources are cached with freshness limits and per-user data is never shared-cached (Manual) 290
10.2 Ensure request and response body size limits, token budgets, and per-principal quotas are enforced (Manual) 296
Appendix: Summary Table 300
Appendix: CIS Controls v8 IG 1 Mapped Recommendations 306
Appendix: CIS Controls v8 IG 2 Mapped Recommendations 308
Appendix: CIS Controls v8 IG 3 Mapped Recommendations 312
Appendix: Change History 316

Overview

All CIS Benchmarks focus on technical configuration settings used to maintain and/or increase the security of the addressed technology, and they should be used in **conjunction** with other essential cyber hygiene tasks like:

- Monitoring the base operating system and applications for vulnerabilities and quickly updating with the latest security patches.
- End-point protection (Antivirus software, Endpoint Detection and Response (EDR), etc.).
- Logging and monitoring user and system activity.

In the end, the Benchmarks are designed to be a key **component** of a comprehensive cybersecurity program.

Important Usage Information

All Benchmarks are available free for non-commercial use from the CIS Website. They can be used to manually assess and remediate systems and applications. In lieu of manual assessment and remediation, there are several tools available to assist with assessment:

- CIS Configuration Assessment Tool (CIS-CAT Pro Assessor)
- CIS Benchmarks Certified 3rd Party Tooling

These tools make the hardening process much more scalable for large numbers of systems and applications.

NOTE: Some tooling focuses only on the Benchmark Recommendations that can be fully automated (skipping ones marked **Manual**). It is important that **ALL** Recommendations (**Automated** and **Manual**) be addressed since all are important for properly securing systems and are typically in scope for audits.

Key Stakeholders

Cybersecurity is a collaborative effort, and cross functional cooperation is imperative within an organization to discuss, test, and deploy Benchmarks in an effective and efficient way. The Benchmarks are developed to be best practice configuration guidelines applicable to a wide range of use cases. In some organizations, exceptions to specific Recommendations will be needed, and this team should work to prioritize the problematic Recommendations based on several factors like risk, time, cost, and labor. These exceptions should be properly categorized and documented for auditing purposes.

Apply the Correct Version of a Benchmark

Benchmarks are developed and tested for a specific set of products and versions and applying an incorrect Benchmark to a system can cause the resulting pass/fail score to be incorrect. This is due to the assessment of settings that do not apply to the target systems. To assure the correct Benchmark is being assessed:

- **Deploy the Benchmark applicable to the way settings are managed in the environment:** An example of this is the Microsoft Windows family of Benchmarks, which have separate Benchmarks for Group Policy, Intune, and Stand-alone systems based upon how system management is deployed. Applying the wrong Benchmark in this case will give invalid results.
- **Use the most recent version of a Benchmark:** This is true for all Benchmarks, but especially true for cloud technologies. Cloud technologies change frequently and using an older version of a Benchmark may have invalid methods for auditing and remediation.

Exceptions

The guidance items in the Benchmarks are called recommendations and not requirements, and exceptions to some of them are expected and acceptable. The Benchmarks strive to be a secure baseline, or starting point, for a specific technology, with known issues identified during Benchmark development are documented in the Impact section of each Recommendation. In addition, organizational, system specific requirements, or local site policy may require changes as well, or an exception to a Recommendation or group of Recommendations (e.g. A Benchmark could Recommend that a Web server not be installed on the system, but if a system's primary purpose is to function as a Webserver, there should be a documented exception to this Recommendation for that specific server).

In the end, exceptions to some Benchmark Recommendations are common and acceptable, and should be handled as follows:

- The reasons for the exception should be reviewed cross-functionally and be well documented for audit purposes.
- A plan should be developed for mitigating, or eliminating, the exception in the future, if applicable.
- If the organization decides to accept the risk of this exception (not work toward mitigation or elimination), this should be documented for audit purposes.

It is the responsibility of the organization to determine their overall security policy, and which settings are applicable to their unique needs based on the overall risk profile for the organization.

Remediation

CIS has developed Build Kits for many technologies to assist in the automation of hardening systems. Build Kits are designed to correspond to Benchmark's "Remediation" section, which provides the manual remediation steps necessary to make that Recommendation compliant to the Benchmark.

When remediating systems (changing configuration settings on deployed systems as per the Benchmark's Recommendations), please approach this with caution and test thoroughly.

The following is a reasonable remediation approach to follow:

- CIS Build Kits, or internally developed remediation methods should never be applied to production systems without proper testing.
- Proper testing consists of the following:
    - Understand the configuration (including installed applications) of the targeted systems. Various parts of the organization may need different configurations (e.g., software developers vs standard office workers).
    - Read the Impact section of the given Recommendation to help determine if there might be an issue with the targeted systems.
    - Test the configuration changes with representative lab system(s). If issues arise during testing, they can be resolved prior to deploying to any production systems.
    - When testing is complete, initially deploy to a small sub-set of production systems and monitor closely for issues. If there are issues, they can be resolved prior to deploying more broadly.
    - When the initial deployment above is completed successfully, iteratively deploy to additional systems and monitor closely for issues. Repeat this process until the full deployment is complete.

Summary

Using the Benchmarks Certified tools, working as a team with key stakeholders, being selective with exceptions, and being careful with remediation deployment, it is possible to harden large numbers of deployed systems in a cost effective, efficient, and safe manner.

NOTE: As previously stated, the PDF versions of the CIS Benchmarks are available for free, non-commercial use on the CIS Website. All other formats of the CIS Benchmarks (MS Word, Excel, and Build Kits) are available for CIS SecureSuite members.

CIS-CAT Pro is also available to CIS SecureSuite members.

Target Technology Details

The CIS MCP Server Benchmark provides prescriptive, auditable hardening Recommendations for securely configuring MCP servers (local stdio and remote Streamable HTTP), the MCP host applications that spawn and mediate them (consent, subprocess environment, and content rendering), and their deployment environments, including containerized and VM-based hosting. Each Recommendation includes specific audit and remediation procedures, with CLI examples where a portable reference mechanism is appropriate, so compliance can be assessed and the intended configuration state understood.

The security boundary of an MCP server extends beyond the server process. Recommendations therefore also assess adjacent components where they directly enforce or materially affect the server's security posture: the host application that launches and mediates the server, reverse proxies and API gateways that terminate or route MCP traffic, authorization infrastructure that issues and validates credentials, and the operating system, container, or VM controls that constrain process, filesystem, and network access. The Benchmark does not attempt to comprehensively harden those supporting technologies outside their MCP-specific role.

This Benchmark targets the MCP specification revision `2026-07-28`. It does not cover program-level security practices such as vulnerability management, incident response planning, or penetration testing. For applying the CIS Controls to MCP environments at the program level, see the CIS Controls v8.1 Model Context Protocol (MCP) Companion Guide, the governance-level complement to this document.

Assessment Prerequisites

The Audit procedures in this Benchmark are written for bash on Ubuntu 24.04 and are not portable to `/bin/sh`. Most rely on `jq` and `curl`, which must be present on the auditing host, with `jq` required by 34 of the 55 Recommendations and not installed on minimal images. A small number of Recommendations require a specialized tool named in their Audit procedure (`ajv-cli`, `gitleaks`, `secret-tool`, `setfacl`, `docker`, `npm`, `pip`, `python3`). On-the-wire Audits require network reach from the auditing host to the MCP endpoint, and several Recommendations read a deployment-pinned MCP audit log path. Nine Recommendations include reverse-proxy Remediation shown in nginx syntax, to be adapted to the deployed gateway.

Intended Audience

The intended audience includes system administrators, platform and security engineers, DevSecOps practitioners, compliance auditors, developers of MCP servers and host applications, and solution vendors integrating MCP into enterprise environments. Familiarity with the MCP specification, JSON-RPC 2.0, OAuth 2.1, and basic Linux and container operations is assumed.

Consensus Guidance

This CIS Benchmark was created using a consensus review process comprised of a global community of subject matter experts. The process combines real world experience with data-based information to create technology specific guidance to assist users to secure their environments. Consensus participants provide perspective from a diverse set of backgrounds including consulting, software development, audit and compliance, security research, operations, government, and legal.

Each CIS Benchmark undergoes two phases of consensus review. The first phase occurs during initial Benchmark development. During this phase, subject matter experts convene to discuss, create, and test working drafts of the Benchmark. This discussion occurs until consensus has been reached on Benchmark recommendations. The second phase begins after the Benchmark has been published. During this phase, all feedback provided by the Internet community is reviewed by the consensus team for incorporation in the Benchmark. If you are interested in participating in the consensus process, please visit https://workbench.cisecurity.org/.

Typographical Conventions

The following typographical conventions are used throughout this guide:

Convention	Meaning
`Stylized Monospace font`	Used for blocks of code, command, and script examples. Text should be interpreted exactly as presented.
`Monospace font`	Used for inline code, commands, UI/Menu selections or examples. Text should be interpreted exactly as presented.
`<Monospace font in brackets>`	Text set in angle brackets denote a variable requiring substitution for a real value.
Italic font	Used to reference other relevant settings, CIS Benchmarks and/or Benchmark Communities. Also used to denote the title of a book, article, or other publication.
Bold font	Additional information or caveats such as **Notes**, **Warnings**, or **Cautions**.

Recommendation Definitions

The following defines the various components included in a CIS recommendation as applicable. If any of the components are not applicable it will be noted, or the component will not be included in the recommendation.

Title

Concise description for the recommendation's intended configuration.

Assessment Status

An assessment status is included for every recommendation. The assessment status indicates whether the given recommendation can be automated or requires manual steps to implement. Both statuses are equally important and are determined and supported as defined below:

Automated

Represents recommendations for which assessment of a technical control can be fully automated and validated to a pass/fail state. Recommendations will include the necessary information to implement automation.

Manual

Represents recommendations for which assessment of a technical control cannot be fully automated and requires all or some manual steps to validate that the configured state is set as expected. The expected state can vary depending on the environment.

Profile

A collection of recommendations for securing a technology or a supporting platform. Most benchmarks include at least a Level 1 and Level 2 Profile. Level 2 extends Level 1 recommendations and is not a standalone profile. The Profile Definitions section in the benchmark provides the definitions as they pertain to the recommendations included for the technology.

Description

Detailed information pertaining to the setting with which the recommendation is concerned. In some cases, the description will include the recommended value.

Rationale Statement

Detailed reasoning for the recommendation to provide the user a clear and concise understanding on the importance of the recommendation.

Impact Statement

Any security, functionality, or operational consequences that can result from following the recommendation.

Audit Procedure

Systematic instructions for determining if the target system complies with the recommendation.

Remediation Procedure

Systematic instructions for applying recommendations to the target system to bring it into compliance according to the recommendation.

Default Value

Default value for the given setting in this recommendation, if known. If not known, either not configured or not defined will be applied.

References

Additional documentation relative to the recommendation.

CIS Critical Security Controls (CIS Controls)

The mapping between a recommendation and the CIS Controls is organized by CIS Controls version, Safeguard, and Implementation Group (IG). The Benchmark in its entirety addresses the CIS Controls safeguards of (v7) "5.1 - Establish Secure Configurations" and (v8) "4.1 - Establish and Maintain a Secure Configuration Process" so individual recommendations will not be mapped to these safeguards.

Additional Information

Supplementary information that does not correspond to any other field but may be useful to the user.

Profile Definitions

The following configuration profiles are defined by this Benchmark:

- **Level 1**

  Items in this profile intend to:

  - be practical and prudent;
  - provide security-focused best practice hardening of a technology; and
  - limit the impact to the utility of the technology beyond acceptable means.

- **Level 2**

  This profile extends the "Level 1" profile. Items in this profile exhibit one or more of the following characteristics:

  - are intended for environments or use cases where security is more critical than manageability and usability
  - acts as a defense in depth measure
  - may impact the utility or performance of the technology
  - may include additional licensing, cost, or addition of third-party software

Acknowledgements

Contributor

Phil White
Tal Skverer
Tomer Schwartz
Shreyans Mehta

Author

Andrew Dannenberger

Recommendations

1 Governance & Versioning

An MCP deployment is only as controlled as its record of what each server exposes and which protocol revision it speaks. Every server advertises a protocol version and a set of capabilities and extensions. Left ungoverned, that surface changes silently as servers are updated, reconfigured, or replaced.

The Recommendations in this section treat the protocol version, the capability set, and the self-asserted identity of each peer as assets under change control. They require an approved allowlist of protocol revisions, a documented capability baseline, a gate that holds newly advertised capabilities until they are re-approved, and capture of client and server identity metadata for reconciliation against an enterprise registry. Together they establish the authoritative inventory the rest of the Benchmark depends on.

1.1 Ensure MCP protocol version is pinned and logged (Automated)

Profile Applicability:

- Level 1

Description:

MCP servers, hosts, and gateways serve only operator-approved protocol revisions, such as `2026-07-28`, and record the asserted protocol version for every request in structured logs. The allowlist is configured explicitly rather than inherited from SDK defaults, and it defines the revisions the endpoint serves, not only those it accepts per request. Serving older revisions alongside approved ones creates downgrade paths that bypass the allowlist. The allowlist is set no lower than `2025-06-18`, the revision that introduced the `MCP-Protocol-Version` header and header-body validation, because a server that serves earlier revisions can read a request carrying no header as an earlier revision and process it silently. In Streamable HTTP, the protocol version is asserted in both the `MCP-Protocol-Version` header and the `_meta.io.modelcontextprotocol/protocolVersion` field, the two values agree, and the deployment rejects any request where they differ. In stdio the version is carried only in `_meta`. A request that specifies a version outside the allowlist, omits a required version value, or carries conflicting values is rejected and logged rather than processed under a default protocol revision.

Rationale:

MCP protocol revisions can alter capability negotiation, authorization requirements, and transport behavior. Without version controls, a client, or an attacker interposing on the connection, may attempt a downgrade attack by asserting an older revision to bypass protections introduced in newer revisions.

Version pinning prevents this risk, while version logging provides the corresponding detective control. Recording the asserted protocol version for every request enables investigators to determine which security model governed a session and whether a downgrade occurred during an incident. Together, pinning and logging ensure protocol version selection is both intentional and auditable.

The specification permits two handlings of a request that omits the protocol version. A server that still supports clients predating `2025-06-18` may read a missing `MCP-Protocol-Version` header as revision `2025-03-26`, whereas a server that does not support those clients rejects the request. Because that fallback is a deployment choice rather than a protocol requirement, this Recommendation takes the stricter side: serve no revision earlier than `2025-06-18`, and reject any request that omits, misstates, or asserts an unapproved protocol version. Permitting the fallback creates a silent path onto a revision whose security model was never approved.

Impact:

Impact is low when an endpoint already serves only approved protocol revisions and moderate when older-version clients remain in use. Version pinning may modestly delay adoption of new revisions. The greater operational impact comes from restricting the revisions an endpoint serves, as clients using unapproved revisions may be disconnected. Organizations should inventory connected clients before enforcement. Rejecting requests that omit or misstate the protocol version may also expose non-conformant clients, which need to be updated to provide an approved version in both the `MCP-Protocol-Version` header and `_meta`.

Audit:

Compare the revisions the server advertises through `server/discover` against the approved allowlist, probe the server to confirm it rejects an unapproved version, an absent version, and a header and body that disagree, and inspect the logs to confirm the asserted version is recorded on every request. Run the probes against the origin server, because a fronting gateway that rejects a probe before it reaches the origin returns an empty or non-JSON response that the probes report as `ERROR`, so verify gateway enforcement separately. A response as a single JSON object or an SSE stream is accepted, and a non-JSON body is reported as `ERROR`.

Run the following command to compare the versions the server advertises against the approved allowlist, substituting your approved revision(s). Keep the allowlist as a newline-delimited file exported from the enterprise MCP registry, one revision per line, for example `<REGISTRY_DIR>/approved_protocol_versions.txt`. A revision the server advertises but the allowlist does not approve is a `FAIL`. A revision the allowlist approves but the server does not advertise is a `REVIEW` since the allowlist may be stale or the server misconfigured. An empty or unreadable allowlist, or a discovery result with no well-formed `supportedVersions` array, is an `ERROR`:

```
MCP_URL="https://<SERVER_URL>/mcp"
ALLOW="<REGISTRY_DIR>/approved_protocol_versions.txt"

resp=$(curl -sS --connect-timeout 5 --max-time 20 -X POST "$MCP_URL" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'MCP-Protocol-Version: 2026-07-28' \
 -H 'Mcp-Method: server/discover' \
 -d
'{"jsonrpc":"2.0","id":1,"method":"server/discover","params":{"_meta":{"io.modelconte
xtprotocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}')

json="$resp"
printf '%s' "$json" | jq -e 'has("id")' >/dev/null 2>&1 || json=$(printf '%s' "$resp"
| sed -n 's/^data: //p' | jq -c 'select(type == "object" and .id == 1)' 2>/dev/null |
head -n1)

WORK=$(mktemp -d)

if [ -z "$resp" ]; then
 echo "ERROR: no response from the MCP endpoint"
elif ! printf '%s' "$json" | jq -e . >/dev/null 2>&1; then
 echo "ERROR: response body is not JSON, verdict not attributable"
elif printf '%s' "$json" | jq -e 'has("error")' >/dev/null 2>&1; then
 echo "ERROR: server/discover rejected with code $(printf '%s' "$json" | jq -r
'.error.code'), probe not attributable"
elif ! printf '%s' "$json" | jq -e '.result.supportedVersions | type == "array" and
length > 0 and all(.[]; type == "string")' >/dev/null 2>&1; then
 echo "ERROR: the discovery result carries no well-formed supportedVersions array,
the served set cannot be enumerated and no verdict is attributable"
elif [ ! -r "$ALLOW" ]; then
 echo "ERROR: allowlist not readable at $ALLOW"
else
 tr -d '\r' < "$ALLOW" | grep -v '^[[:space:]]*$' | LC_ALL=C sort -u > "$WORK/allow"
 if [ ! -s "$WORK/allow" ]; then
  echo "ERROR: the allowlist at $ALLOW is empty, record the approved revisions before
auditing"
 else
  printf '%s' "$json" | jq -r '.result.supportedVersions[]' | LC_ALL=C sort -u >
"$WORK/adv"
  extra=$(LC_ALL=C comm -23 "$WORK/adv" "$WORK/allow")
  missing=$(LC_ALL=C comm -13 "$WORK/adv" "$WORK/allow")
  if [ -n "$missing" ]; then printf '%s\n' "$missing" | sed 's/^/approved but not
advertised: /'; fi
  if [ -n "$extra" ]; then
   printf '%s\n' "$extra" | sed 's/^/unapproved: /'
   echo "FAIL: the server advertises protocol revisions outside the approved
allowlist"
```

```
  elif [ -n "$missing" ]; then
   echo "REVIEW: approved revisions are not advertised, confirm whether the allowlist
is stale or the server is misconfigured"
  else
   echo "PASS: the advertised supported-version set matches the approved allowlist"
  fi
 fi
fi
rm -rf "$WORK"
```

Run the following command to confirm an unapproved version is rejected:

```
MCP_URL="https://<SERVER_URL>/mcp"

resp=$(curl -sS --connect-timeout 5 --max-time 20 -w '\n%{http_code}' -X POST
"$MCP_URL" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'Mcp-Method: tools/list' \
 -H 'MCP-Protocol-Version: 2024-01-01' \
 -d
'{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{"_meta":{"io.modelcontextpro
tocol/protocolVersion":"2024-01-
01","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}')
code=$(printf '%s' "$resp" | tail -n1)
body=$(printf '%s' "$resp" | sed '$d')

json="$body"
printf '%s' "$json" | jq -e 'has("id")' >/dev/null 2>&1 || json=$(printf '%s' "$body"
| sed -n 's/^data: //p' | jq -c 'select(type == "object" and .id == 1)' 2>/dev/null |
head -n1)
ec=$(printf '%s' "$json" | jq -r '.error.code // empty' 2>/dev/null)

if [ -z "$body" ] && [ "$code" = "000" ]; then
 echo "ERROR: no response from the MCP endpoint"
elif [ -z "$json" ]; then
 echo "ERROR: response body is not JSON, verdict not attributable, confirm the probe
reaches the origin MCP server"
 elif [ "$ec" = "-32022" ] && [ "$code" = "400" ]; then
 sup=$(printf '%s' "$json" | jq -c '.error.data.supported // empty' 2>/dev/null)
 echo "PASS: unapproved version rejected with HTTP 400 and
UnsupportedProtocolVersion, server supports $sup"
elif [ "$ec" = "-32022" ]; then
 echo "FAIL: UnsupportedProtocolVersion returned with HTTP status $code, the
specification requires 400"
elif [ -n "$ec" ]; then
 echo "REVIEW: rejected with unexpected error code $ec at HTTP $code, confirm the
rejection is version enforcement"
else
 echo "FAIL: request asserting an unapproved protocol version was accepted"
fi
```

Run the following command to confirm a request that omits the version header but carries a complete `_meta` envelope is rejected rather than silently downgraded. Because this Recommendation floors the served set at `2025-06-18`, a header-less request is rejected with HTTP `400` and HeaderMismatch, and a request that is instead accepted and served under an earlier revision is a `FAIL`. An `UnsupportedProtocolVersion` result here is a known reference-SDK divergence reported upstream, recorded as `REVIEW`:

```
MCP_URL="https://<SERVER_URL>/mcp"

resp=$(curl -sS --connect-timeout 5 --max-time 20 -w '\n%{http_code}' -X POST
"$MCP_URL" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'Mcp-Method: tools/list' \
 -d
'{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{"_meta":{"io.modelcontextpro
tocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}')
code=$(printf '%s' "$resp" | tail -n1)
body=$(printf '%s' "$resp" | sed '$d')

json="$body"
printf '%s' "$json" | jq -e 'has("id")' >/dev/null 2>&1 || json=$(printf '%s' "$body"
| sed -n 's/^data: //p' | jq -c 'select(type == "object" and .id == 1)' 2>/dev/null |
head -n1)
ec=$(printf '%s' "$json" | jq -r '.error.code // empty' 2>/dev/null)

if [ -z "$body" ] && [ "$code" = "000" ]; then
 echo "ERROR: no response from the MCP endpoint"
elif [ -z "$json" ]; then
 echo "ERROR: response body is not JSON, verdict not attributable, confirm the probe
reaches the origin MCP server"
 elif [ "$ec" = "-32020" ] && [ "$code" = "400" ]; then
 echo "PASS: request missing the protocol version header rejected with HTTP 400 and
HeaderMismatch"
elif [ "$ec" = "-32020" ]; then
 echo "FAIL: HeaderMismatch returned with HTTP status $code, the specification
requires 400"
elif [ "$ec" = "-32022" ]; then
 echo "REVIEW: server rejected the header-less request as an unsupported version
(HTTP $code) rather than HeaderMismatch. The transport specification requires
HeaderMismatch for a missing required header on a server that does not support pre-
2025-06-18 clients; this is a known reference SDK divergence, confirm the server's
client-support policy and record the finding"
elif [ -n "$ec" ]; then
 echo "REVIEW: rejected with unexpected error code $ec at HTTP $code, confirm the
rejection is version enforcement"
else
 echo "FAIL: request without the protocol version header was accepted"
fi
```

Run the following command to confirm a request that omits the version field from `_meta` while carrying the correct header is rejected as malformed with HTTP `400` and invalid params:

```
MCP_URL="https://<SERVER_URL>/mcp"

resp=$(curl -sS --connect-timeout 5 --max-time 20 -w '\n%{http_code}' -X POST
"$MCP_URL" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'Mcp-Method: tools/list' \
 -H 'MCP-Protocol-Version: 2026-07-28' \
 -d
'{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{"_meta":{"io.modelcontextpro
tocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}')
code=$(printf '%s' "$resp" | tail -n1)
body=$(printf '%s' "$resp" | sed '$d')

json="$body"
printf '%s' "$json" | jq -e 'has("id")' >/dev/null 2>&1 || json=$(printf '%s' "$body"
| sed -n 's/^data: //p' | jq -c 'select(type == "object" and .id == 1)' 2>/dev/null |
head -n1)
ec=$(printf '%s' "$json" | jq -r '.error.code // empty' 2>/dev/null)

if [ -z "$body" ] && [ "$code" = "000" ]; then
 echo "ERROR: no response from the MCP endpoint"
elif [ -z "$json" ]; then
 echo "ERROR: response body is not JSON, verdict not attributable, confirm the probe
reaches the origin MCP server"
 elif [ "$ec" = "-32602" ] && [ "$code" = "400" ]; then
 echo "PASS: request missing the required version field in _meta rejected with HTTP
400 and Invalid params"
elif [ "$ec" = "-32602" ]; then
 echo "FAIL: Invalid params returned with HTTP status $code, the specification
requires 400"
elif [ "$ec" = "-32020" ] && [ "$code" = "400" ]; then
 echo "REVIEW: server rejected the version-less body as a header mismatch, defensible
where header and body are compared before required-field validation, confirm the
request was not accepted anywhere"
elif [ "$ec" = "-32022" ]; then
 echo "REVIEW: server rejected the version-less body as an unsupported version (HTTP
$code), the specification defines this as a malformed request (-32602), record the
divergence"
elif [ -n "$ec" ]; then
 echo "REVIEW: rejected with unexpected error code $ec at HTTP $code, confirm the
rejection is version enforcement"
else
 echo "FAIL: request without a version field in the body was accepted"
fi
```

Run the following command to confirm the server rejects a header and `_meta` value that disagree. The probe carries the approved version in the body and a stale version in the header, so the only conforming outcome is HTTP `400` with `HeaderMismatch`. An `UnsupportedProtocolVersion` rejection here is a failure, because the body version is supported:

```
MCP_URL="https://<SERVER_URL>/mcp"

resp=$(curl -sS --connect-timeout 5 --max-time 20 -w '\n%{http_code}' -X POST
"$MCP_URL" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'Mcp-Method: tools/list' \
 -H 'MCP-Protocol-Version: 2025-03-26' \
 -d
'{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{"_meta":{"io.modelcontextpro
tocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}')
code=$(printf '%s' "$resp" | tail -n1)
body=$(printf '%s' "$resp" | sed '$d')

json="$body"
printf '%s' "$json" | jq -e 'has("id")' >/dev/null 2>&1 || json=$(printf '%s' "$body"
| sed -n 's/^data: //p' | jq -c 'select(type == "object" and .id == 1)' 2>/dev/null |
head -n1)
ec=$(printf '%s' "$json" | jq -r '.error.code // empty' 2>/dev/null)

if [ -z "$body" ] && [ "$code" = "000" ]; then
 echo "ERROR: no response from the MCP endpoint"
elif [ -z "$json" ]; then
 echo "ERROR: response body is not JSON, verdict not attributable, confirm the probe
reaches the origin MCP server"
 elif [ "$ec" = "-32020" ] && [ "$code" = "400" ]; then
 echo "PASS: disagreeing header and body versions rejected with HTTP 400 and
HeaderMismatch"
elif [ "$ec" = "-32020" ]; then
 echo "FAIL: HeaderMismatch returned with HTTP status $code, the specification
requires 400"
elif [ "$ec" = "-32022" ]; then
 echo "FAIL: server rejected the mismatch as an unsupported version. The body carries
a supported version, so no version-support rejection applies; the specification
requires HeaderMismatch when header and body disagree"
elif [ -n "$ec" ]; then
 echo "REVIEW: rejected with unexpected error code $ec at HTTP $code, confirm the
rejection is mismatch enforcement"
else
 echo "FAIL: disagreeing header and body versions were accepted"
fi
```

Run the following command to confirm the asserted version is recorded on every request. It reads the version from the `_meta` object the Description names, counts every request record, and excludes the audit client's own probes. Records with no direction field are counted separately rather than assumed to be requests:

```
LOG="<MCP_AUDIT_LOG>"
AUDIT_CLIENT="cis-benchmark-audit"
REQ='select(.direction? == "request") | select(.method? != null)'
SELF='((._meta["io.modelcontextprotocol/clientInfo"].name
// .params._meta["io.modelcontextprotocol/clientInfo"].name // "") == $ac)'
PV='(._meta["io.modelcontextprotocol/protocolVersion"]
// .params._meta["io.modelcontextprotocol/protocolVersion"] // .protocolVersion)'

if [ ! -r "$LOG" ]; then
 echo "ERROR: log not readable at $LOG"
else
 total=$(jq -s --arg ac "$AUDIT_CLIENT" "[.[] | $REQ | select($SELF | not)] | length"
"$LOG" 2>/dev/null)
 withver=$(jq -s --arg ac "$AUDIT_CLIENT" "[.[] | $REQ | select($SELF | not) |
select($PV != null)] | length" "$LOG" 2>/dev/null)
 nodir=$(jq -s '[.[] | select(has("direction") | not)] | length' "$LOG" 2>/dev/null)
 if ! [ "$total" -ge 0 ] 2>/dev/null || ! [ "$withver" -ge 0 ] 2>/dev/null; then
  echo "ERROR: log could not be parsed, expected one JSON object per line"
 elif [ "$total" -eq 0 ]; then
  echo "ERROR: no request records in log, confirm the log path, that records carry a
direction field, and that the capture window contains traffic from clients other than
the auditor"
 else
  if [ "${nodir:-0}" -gt 0 ]; then
   echo "records with no direction field: $nodir, excluded from evaluation, correct
the capture layer so every record carries a direction"
  fi
  echo "requests: $total, recording an asserted protocol version: $withver"
  if [ "$withver" -eq "$total" ]; then
   echo "PASS: every request records an asserted protocol version"
  else
   echo "FAIL: $((total - withver)) requests record no asserted protocol version"
  fi
 fi
fi
```

The system is compliant when the advertised supported-version set matches the allowlist, each of the four probes rejects its assertion (unapproved, header-less, version-less, and disagreeing) with the specified code at HTTP `400`, and the log records an asserted version on every request.

Remediation:

Configure the deployment to reject any request that asserts a version outside the allowlist, omits it, or carries disagreeing header and body values, and to record the asserted version on every request. The version is asserted twice per Streamable HTTP `POST`, in the `MCP-Protocol-Version` header and in `_meta` under `io.modelcontextprotocol/protocolVersion`, and the two are required to match. The MCP server, or an L7 gateway able to inspect the body, rejects these in request-handling middleware: an out-of-allowlist version with `-32022` (`UnsupportedProtocolVersion`), a header that disagrees with the body or a missing `MCP-Protocol-Version` header with `-32020` (`HeaderMismatch`), and a request whose `_meta` omits the protocol version value with `-32602` (`Invalid params`). All are rejected at HTTP `400`.

Record the asserted version, the request direction, and the client identity from `_meta` on every request, and ship the logs to the SIEM. The Audit reads the version from that `_meta` key, so a deployment that records it under a different name changes the Audit and this field together.

Record the approved revision(s) in the enterprise MCP registry and export them to the newline-delimited allowlist the Audit reads, one per line, with Unix line endings.

A gateway can reject unapproved or absent header values as a first filter, but it cannot compare the header against `_meta`, so it does not replace the server-side validation. The following nginx example defines the allowlist in the http context:

```
map $http_mcp_protocol_version $mcp_ver_ok {
 default 0;
 "2026-07-28" 1;
}
```

Then apply the check inside the MCP location block:

```
if ($mcp_ver_ok = 0) { return 400; }
```

The map alone only defines the variable. The `if` rejects the request. Both are required.

Default Value:

By default, an MCP server accepts any revision its SDK implements rather than an operator allowlist, and official SDKs commonly serve both `2026-07-28` and 2025-era revisions from one endpoint unless configured otherwise. A conformant server rejects a version it does not implement, but one that supports clients predating `2025-06-18` may treat a request with no `MCP-Protocol-Version` header as version `2025-03-26` rather than rejecting it. The asserted version is not necessarily recorded per request, and no alert is raised when an unexpected version appears.

References:

1. MCP specification (Base Protocol / Overview / Error Codes): https://modelcontextprotocol.io/specification/2026-07-28/basic/index#error-codes
2. MCP specification (Base Protocol / Transports / Streamable HTTP / Protocol Version Header): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http#protocol-version-header
3. MCP specification (Base Protocol / Versioning and Compatibility / Protocol Version Negotiation): https://modelcontextprotocol.io/specification/2026-07-28/basic/versioning#protocol-version-negotiation
4. MCP specification (Schema Reference): https://modelcontextprotocol.io/specification/2026-07-28/schema
5. SEP-2243 (HTTP Header Standardization for Streamable HTTP Transport): https://modelcontextprotocol.io/seps/2243-http-standardization

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	2.2 Ensure Authorized Software is Currently Supported Ensure that only currently supported software is designated as authorized in the software inventory for enterprise assets. If software is unsupported, yet necessary for the fulfillment of the enterprise's mission, document an exception detailing mitigating controls and residual risk acceptance. For any unsupported software without an exception documentation, designate as unauthorized. Review the software list to verify software support at least monthly, or more frequently.	●	●	●
v8	8.2 Collect Audit Logs Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.	●	●	●

1.2 Ensure server capability baseline is established and documented (Manual)

Profile Applicability:

- Level 1

Description:

A documented capability baseline is defined and maintained for each MCP server, recording which protocol components and settings it is approved to expose and which extensions it is approved to negotiate. Under the `2026-07-28` revision, a server advertises this configuration in the `capabilities` object returned by `server/discover`, where core components appear as top-level keys (Tools, Resources, Prompts, Completions), each with its own settings such as `tools.listChanged` or `resources.subscribe`, and extensions appear in the `extensions` map keyed by a reverse-DNS ID such as `io.modelcontextprotocol/tasks`. The baseline records the full object with its values, not just the names, because a configuration can change materially without any name being added, for example by enabling `tools.listChanged` or raising an extension limit. It is stored as the canonical JSON capability object, a versioned artifact in the enterprise MCP registry under change control, and re-approved on every server version change, configuration change, and scheduled re-discovery. The advertised configuration should not exceed or differ from the approved baseline, and any deviation is unauthorized drift the audit flags. Assessment is manual: the comparison is scripted, but approving the baseline and adjudicating withdrawals require human judgement.

Rationale:

An MCP server's advertised capabilities define the attack surface available to any client or model that reaches it. Without a documented baseline there is no authoritative record of what a server is expected to expose, so there is no way to detect when capabilities have been added, intentionally or otherwise. A compromised or misconfigured server that silently begins advertising support it was never approved to offer expands the attack surface without tripping any existing control. The drift that matters is not limited to new capability names: a server that enables `tools.listChanged`, turns on `resources.subscribe`, or changes an extension's settings gains new behavior while advertising exactly the same names, so a baseline that records names alone cannot see the change. The extensions framework widens this further, because an extension is negotiated by ID and can introduce whole method families that the core protocol never defined. A maintained baseline of the complete capability configuration is the primary detective mechanism for this drift, and the approval gate in front of it is the preventive one.

Impact:

Enabling this control adds a review gate. Each server needs an initial baseline approved, and every later capability or settings change needs re-review before it can go live, which delays legitimate changes by the review turnaround. Remediating a drifted server means disabling the unapproved capability, so any client already using it loses that function until the capability is reviewed and approved into the baseline.

Audit:

Compare the full capability configuration the server advertises through `server/discover` against the approved baseline, and flag any deviation as drift. The comparison covers every nested setting, not only the names, so a change to a value such as `tools.listChanged` or an extension setting is drift even though no name changed. Maintain the baseline per server as the canonical JSON capability object, for example `<BASELINE_DIR>/<SERVER>.capabilities.json`.

Run the following script to retrieve the advertised configuration and compare it against the baseline:

```
MCP_URL="https://<SERVER_URL>/mcp"
BASELINE="<BASELINE_DIR>/<SERVER>.capabilities.json"

LEAF='(paths(type | IN("string","number","boolean","null")), paths(type == "object"
and length == 0), paths(type == "array" and length == 0)) as $p | ($p | tojson) +
"\t" + (getpath($p) | tojson)'

resp=$(curl -sS --connect-timeout 5 --max-time 20 -X POST "$MCP_URL" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'MCP-Protocol-Version: 2026-07-28' \
 -H 'Mcp-Method: server/discover' \
 -d
'{"jsonrpc":"2.0","id":1,"method":"server/discover","params":{"_meta":{"io.modelconte
xtprotocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}')

json="$resp"
printf '%s' "$json" | jq -e . >/dev/null 2>&1 || json=$(printf '%s' "$resp" | sed -n
's/^data: //p' | head -n1)

WORK=$(mktemp -d)
TAB=$(printf '\t')

if [ -z "$resp" ]; then
 echo "ERROR: no response from the MCP endpoint"
elif ! printf '%s' "$json" | jq -e . >/dev/null 2>&1; then
 echo "ERROR: response body is not JSON, verdict not attributable"
elif printf '%s' "$json" | jq -e 'has("error")' >/dev/null 2>&1; then
 echo "ERROR: server/discover rejected with code $(printf '%s' "$json" | jq -r
'.error.code'), probe not attributable"
elif ! printf '%s' "$json" | jq -e '.result.capabilities | type ==
"object"' >/dev/null 2>&1; then
 echo "ERROR: result.capabilities is missing or not an object, no verdict is
attributable"
elif [ ! -r "$BASELINE" ]; then
 echo "ERROR: baseline not readable at $BASELINE"
elif ! jq -e 'type == "object"' "$BASELINE" >/dev/null 2>&1; then
 echo "ERROR: baseline is not a canonical JSON capability object, re-capture it with
the Remediation before auditing"
else
 printf '%s' "$json" | jq -S '.result.capabilities' > "$WORK/current.json"
 jq -r "$LEAF" "$WORK/current.json" | LC_ALL=C sort > "$WORK/cur"
 jq -r "$LEAF" "$BASELINE" | LC_ALL=C sort > "$WORK/base"
 comm -23 "$WORK/cur" "$WORK/base" > "$WORK/curonly"
 comm -13 "$WORK/cur" "$WORK/base" > "$WORK/baseonly"
 awk -F "$TAB" 'FILENAME == ARGV[1] { b[$1] = $2; next } ($1 in b) { print "changed:
" $1 " approved=" b[$1] " advertised=" $2 }' "$WORK/baseonly" "$WORK/curonly" >
"$WORK/changed"
```

```
 awk -F "$TAB" 'FILENAME == ARGV[1] { b[$1] = $2; next } !($1 in b) { print
"unapproved: " $1 "=" $2 }' "$WORK/baseonly" "$WORK/curonly" > "$WORK/added"
 awk -F "$TAB" 'FILENAME == ARGV[1] { c[$1] = $2; next } !($1 in c) { print
"withdrawn: " $1 "=" $2 }' "$WORK/curonly" "$WORK/baseonly" > "$WORK/gone"
 cat "$WORK/added" "$WORK/changed" "$WORK/gone"
 if [ -s "$WORK/added" ] || [ -s "$WORK/changed" ]; then
  echo "FAIL: the advertised capability configuration exceeds or differs from the
approved baseline"
 elif [ -s "$WORK/gone" ]; then
  echo "REVIEW: baselined capability settings are no longer advertised"
 else
  echo "PASS: the advertised capability configuration matches the approved baseline"
 fi
fi
rm -rf "$WORK"
```

Each advertised setting is compared to the baseline. A setting advertised but not approved prints `unapproved:` and is a `FAIL`. A setting present in both whose value differs prints `changed:` and is a `FAIL`, because the audit cannot judge the direction of an arbitrary value change. A setting approved but no longer advertised prints `withdrawn:` and is a `REVIEW`. Where a run produces both a `FAIL` and a `REVIEW`, `FAIL` takes precedence. An empty advertised object is a `PASS` against an empty baseline and a `REVIEW` against a populated one. A baseline that is not a canonical JSON capability object, including the earlier newline-delimited format, is an `ERROR`.

The system is compliant when the advertised configuration matches the baseline.

Remediation:

Establish the baseline from a server in a known-good state, have it reviewed, and store it in the enterprise registry under change control.

Run the following script to capture the baseline as the canonical JSON capability object. Capturing from a drifted server writes the drift into the baseline, so capture only from a server in its approved state. The script leaves any existing baseline untouched if the configuration cannot be retrieved or the server advertises an empty set:

```
MCP_URL="https://<SERVER_URL>/mcp"
BASELINE="<BASELINE_DIR>/<SERVER>.capabilities.json"
mkdir -p "$(dirname "$BASELINE")"

LEAF='(paths(type | IN("string","number","boolean","null")), paths(type == "object"
and length == 0), paths(type == "array" and length == 0)) as $p | $p | tojson'

resp=$(curl -sS --connect-timeout 5 --max-time 20 -X POST "$MCP_URL" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'MCP-Protocol-Version: 2026-07-28' \
 -H 'Mcp-Method: server/discover' \
 -d
'{"jsonrpc":"2.0","id":1,"method":"server/discover","params":{"_meta":{"io.modelconte
xtprotocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}')

json="$resp"
printf '%s' "$json" | jq -e . >/dev/null 2>&1 || json=$(printf '%s' "$resp" | sed -n
's/^data: //p' | head -n1)

if [ -z "$resp" ]; then
 echo "ERROR: no response from the MCP endpoint, baseline not written"
elif ! printf '%s' "$json" | jq -e . >/dev/null 2>&1; then
 echo "ERROR: response body is not JSON, baseline not written"
elif printf '%s' "$json" | jq -e 'has("error")' >/dev/null 2>&1; then
 echo "ERROR: server/discover rejected with code $(printf '%s' "$json" | jq -r
'.error.code'), baseline not written"
elif ! printf '%s' "$json" | jq -e '.result.capabilities | type ==
"object"' >/dev/null 2>&1; then
 echo "ERROR: no capability object retrieved, baseline not written"
else
 n=$(printf '%s' "$json" | jq -r ".result.capabilities | $LEAF" | grep -c .)
 if [ "$n" -eq 0 ]; then
  echo "ERROR: the server advertises an empty capability set, baseline not written.
If an empty set is the approved configuration, record it deliberately by writing {}
to $BASELINE under change control"
 else
  printf '%s' "$json" | jq -S '.result.capabilities' > "$BASELINE"
  echo "captured $n capability entries to $BASELINE"
 fi
fi
```

The script refuses to write an empty baseline since a zero-entry baseline recorded without review passes every later audit. If an empty set is genuinely approved, record it by writing `{}` to the baseline path under change control. Otherwise, review the captured file and commit it to the registry before treating it as approved. Re-capture any baseline still in the earlier newline-delimited format, which the Audit otherwise reports as `ERROR`.

Disable any capability not in the baseline through the server's own configuration since a proxy cannot enforce capability advertisement. Document intentional exceptions with approver, justification, and date, and re-establish the baseline whenever a version, configuration, or extension change alters the advertised configuration, down to a nested setting value.

Under the `2026-07-28` revision, the Logging capability is deprecated with a minimum twelve-month support window, so Logging continues to function during that window, but do not approve it into a new baseline. Roots and Sampling are also deprecated, but are client capabilities and do not appear in a server's advertised set.

Default Value:

An MCP server implementing `2026-07-28` answers `server/discover` and advertises the full set of capabilities and extensions it was built with. There is no default baseline, no approval workflow, and no mechanism that detects or alerts on a change to the advertised set. Discovery results may also carry `ttlMs` and `cacheScope`, so where a caching intermediary sits in the path an audit can read a capability set that is no longer current.

References:

1. MCP specification (Architecture, capability negotiation): https://modelcontextprotocol.io/specification/2026-07-28/architecture
2. MCP specification (Server Features / Discovery, server/discover capability advertisement): https://modelcontextprotocol.io/specification/2026-07-28/server/discover
3. SEP-2663 (Tasks Extension): https://modelcontextprotocol.io/seps/2663-tasks-extension
4. SEP-2577 (Deprecate Roots, Sampling, and Logging): https://modelcontextprotocol.io/seps/2577-deprecate-roots-sampling-and-logging

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	2.1 Establish and Maintain a Software Inventory Establish and maintain a detailed inventory of all licensed software installed on enterprise assets. The software inventory must document the title, publisher, initial install/use date, and business purpose for each entry; where appropriate, include the Uniform Resource Locator (URL), app store(s), version(s), deployment mechanism, and decommission date. Review and update the software inventory bi-annually, or more frequently.	●	●	●

1.3 Ensure newly advertised server capabilities are re-validated before invocation (Manual)

Profile Applicability:

- Level 2

Description:

When the set of capabilities a server advertises grows beyond its approved baseline, the client and host re-evaluate the additions before permitting invocation. Newly advertised capabilities do not inherit the approval granted to the baselined set. They are held in a staging state until an explicit re-approval is recorded, and the host denies any attempt to invoke a staged capability. Under the `2026-07-28` revision, a client learns of a capability change on a `subscriptions/listen` stream it has opened, filtered per notification type through `toolsListChanged`, `promptsListChanged`, and `resourcesListChanged`. A server sends no notification type the client has not requested, so the host opens and maintains one or more subscription streams that together cover the affected types. A host that does not subscribe receives no immediate change signal and can detect drift only through scheduled or freshness-driven list refreshes. List results also carry `ttlMs` and `cacheScope`, so a client may re-fetch a list when its freshness expires rather than in response to a notification. The staging gate therefore runs on a diff against the approved baseline at every refresh of the capability set, not only when a notification arrives.

Rationale:

Capability drift is a distinct attack vector. A compromised or malicious server can introduce new tools, resources, or prompts after the initial review and advertise them to connected clients, and a capability introduced this way silently inherits the trust established for the approved set even though it was never reviewed. This is a shadow-tool injection path, and it is not hypothetical. An SDK client configured with the `listChanged` option opens a subscription stream, re-fetches the affected list on every published change, and hands the refreshed set straight to the application, so a client that has opted in adopts whatever the server now advertises by default. Holding additions in a staging state is the preventive control that closes the path, and logging each change with a capability diff and its approval state is the detective control that proves the gate is operating.

Impact:

This control has two costs. A newly advertised capability is held until re-validation completes, so its adoption is delayed by the review turnaround. Separately, the host carries the standing operational cost of maintaining the subscription streams that detect capability changes and re-establishing them whenever they close.

Audit:

Confirm that a capability newly advertised beyond the approved baseline is denied until explicitly re-approved, and that each change is logged with a capability diff and an approval state.

Identify two capabilities to use as probes: one advertised beyond the baseline and not yet re-approved (the staged probe), and one already in the baseline and known to be invocable (the control probe). A denial of the staged probe is then attributable to the staging gate rather than to the probe itself, because the control probe confirms invocation otherwise works. Supply schema-valid arguments in `APPROVED_ARGS` and `PENDING_ARGS` since a tool invoked with invalid arguments returns an error indistinguishable from a gate denial. Extend `DENY_PATTERN` to match the wording the local gate emits.

Run the following script to attempt both invocations and compare the outcomes:

```
MCP_URL="https://<SERVER_URL>/mcp"
APPROVED_TOOL="<BASELINED_TOOL>"
PENDING_TOOL="<TOOL_ADVERTISED_SINCE_LAST_APPROVAL>"
APPROVED_ARGS='<JSON_ARGUMENTS_VALID_FOR_APPROVED_TOOL>'
PENDING_ARGS='<JSON_ARGUMENTS_VALID_FOR_PENDING_TOOL>'
DENY_PATTERN='pending|not approved|unapproved|unauthorized|awaiting
approval|forbidden|denied'

envelope='"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}'

call() {
 curl -sS --connect-timeout 5 --max-time 20 -X POST "$MCP_URL" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'MCP-Protocol-Version: 2026-07-28' \
 -H 'Mcp-Method: tools/call' \
 -H "Mcp-Name: $1" \
 -d
"{\"jsonrpc\":\"2.0\",\"id\":1,\"method\":\"tools/call\",\"params\":{\"name\":\"$1\",
\"arguments\":$2,$envelope}}"
}

norm() {
 printf '%s' "$1" | jq -e 'has("id")' >/dev/null 2>&1 && { printf '%s' "$1";
return; }
 printf '%s' "$1" | sed -n 's/^data: //p' | jq -c 'select(type == "object" and .id ==
1)' 2>/dev/null |  head -n1
}

text_of() { printf '%s' "$1" | jq -r
'[.result.content[]?|select(.type=="text")|.text]|join(" ")'; }

ctl_raw=$(call "$APPROVED_TOOL" "$APPROVED_ARGS")
ctl=$(norm "$ctl_raw")
if [ -z "$ctl_raw" ]; then
 echo "ERROR: no response to the control probe, verdict not attributable"
elif ! printf '%s' "$ctl" | jq -e . >/dev/null 2>&1; then
 echo "ERROR: control probe body is not JSON, verdict not attributable"
elif printf '%s' "$ctl" | jq -e 'has("error")' >/dev/null 2>&1; then
 echo "ERROR: control probe on $APPROVED_TOOL rejected with code $(printf '%s' "$ctl"
| jq -r '.error.code'), verdict not attributable"
elif ! printf '%s' "$ctl" | jq -e 'has("result")' >/dev/null 2>&1; then
 echo "ERROR: control probe returned neither result nor error, verdict not
attributable"
elif printf '%s' "$ctl" | jq -e '.result.isError == true' >/dev/null 2>&1; then
 printf 'control probe failed: %s\n' "$(text_of "$ctl")"
 echo "ERROR: the control probe on $APPROVED_TOOL did not execute cleanly, so a
failure on $PENDING_TOOL is not attributable to a staging gate. Correct APPROVED_ARGS
```

```
until this probe succeeds."
else
 tst_raw=$(call "$PENDING_TOOL" "$PENDING_ARGS")
 tst=$(norm "$tst_raw")
 if [ -z "$tst_raw" ]; then
 echo "ERROR: no response to the staged-capability probe"
 elif ! printf '%s' "$tst" | jq -e . >/dev/null 2>&1; then
 echo "ERROR: staged-capability response is not JSON, verdict not attributable"
 elif printf '%s' "$tst" | jq -e '.error.code == -32601 or .error.code == -
32602' >/dev/null 2>&1; then
 printf 'name did not resolve: code %s, message %s\n' \
 "$(printf '%s' "$tst" | jq -r '.error.code')" "$(printf '%s' "$tst" | jq -r
'.error.message')"
 echo "REVIEW: the staged capability name did not resolve, confirm it is the name
advertised by the list_changed diff"
 elif printf '%s' "$tst" | jq -e 'has("error")' >/dev/null 2>&1; then
 msg=$(printf '%s' "$tst" | jq -r '.error.message')
 printf 'denial carrier: protocol error, code %s, message %s\n' "$(printf '%s' "$tst"
| jq -r '.error.code')" "$msg"
 if printf '%s' "$msg" | grep -qiE "$DENY_PATTERN"; then
 echo "PASS: staged capability was denied and the denial names a pending or
unauthorized state"
 else
 echo "REVIEW: staged capability was rejected, but the message does not name a
pending or unauthorized state, so the rejection may not be the staging gate"
 fi
 elif printf '%s' "$tst" | jq -e '.result.isError == true' >/dev/null 2>&1; then
 msg=$(text_of "$tst")
 printf 'denial carrier: in-band tool error, message %s\n' "$msg"
 if printf '%s' "$msg" | grep -qiE "$DENY_PATTERN"; then
 echo "PASS: staged capability was denied and the denial names a pending or
unauthorized state"
 else
 echo "REVIEW: the staged capability returned an in-band error that does not name a
pending or unauthorized state. An input validation failure reads identically to a
gate denial on this carrier, so correct PENDING_ARGS and re-run before accepting this
as a denial"
 fi
 elif printf '%s' "$tst" | jq -e 'has("result")' >/dev/null 2>&1; then
 echo "FAIL: staged capability executed, no approval gate is enforced"
 else
 echo "ERROR: staged-capability response carries neither result nor error"
 fi
fi
```

A denial reaches the client on one of two carriers, and the script accepts either by inspecting the response body rather than the HTTP status. A gateway or middleware gate returns a JSON-RPC `error`. A gate inside the server's own tool handling returns a successful result carrying `isError: true` since a tool handler cannot emit a protocol error. On both carriers the script returns `PASS` only when the denial message matches `DENY_PATTERN`. A rejection that does not name a pending or unauthorized state is `REVIEW` since an input-validation failure reads identically on the `isError` carrier, as does a `-32601` or `-32602` for a name that does not resolve.

Run the following script to confirm capability changes are recorded with a diff and an approval state. It reads records as JSON documents so it tolerates pretty-printed logs:

```
LOG="<MCP_AUDIT_LOG>"
SEL='select((.method? // "") |
test("^notifications/(tools|resources|prompts)/list_changed$"))'
WF='((.capabilities_added | type == "array" and length > 0 and all(.[]; type ==
"string" and length > 0)) and (.approval_state | type == "string") and
(.approval_state | IN("pending","approved","denied")) and (if .approval_state ==
"pending" then true else ((.approver | type == "string" and length > 0) and
(.approved_at | type == "string" and length > 0)) end))'

if [ ! -r "$LOG" ]; then
  echo "ERROR: audit log not readable at $LOG"
else
  total=$(jq -s "[.[] | $SEL] | length" "$LOG" 2>/dev/null)
  good=$(jq -s "[.[] | $SEL | select($WF)] | length" "$LOG" 2>/dev/null)
  if ! [ "$total" -ge 0 ] 2>/dev/null || ! [ "$good" -ge 0 ] 2>/dev/null; then
    echo "ERROR: audit log could not be parsed as JSON records"
  elif [ "$total" -eq 0 ]; then
    echo "REVIEW: no list_changed events recorded, the control cannot be evaluated
from this log"
  else
    echo "list_changed events: $total, carrying well-formed diff and approval
evidence: $good"
    if [ "$good" -eq "$total" ]; then
      echo "PASS: every list_changed event is logged with a well-formed capability
diff and approval evidence"
    else
      echo "FAIL: $((total - good)) list_changed events lack a well-formed capability
diff or approval evidence"
    fi
  fi
fi
```

An event counts as well formed only when the capability diff is a non-empty array of non-empty strings and the approval state is `pending`, `approved`, or `denied`. An `approved` or `denied` event also carries a non-empty approver and timestamp, and a `pending` event may leave both `null`. The log field names are deployment-specific, so substitute them in the Audit and Remediation together since both read the same fields. The system is compliant when a capability advertised beyond the baseline is denied with a message naming a pending or unauthorized state, and every recorded change carries a well-formed diff and approval evidence.

Remediation:

Implement a capability staging gate at the application or host layer, or at a centralized gateway. A plain reverse proxy cannot host the gate since the decision depends on protocol-level capability state.

Open and maintain the `subscriptions/listen` streams covering every notification type in scope, and re-establish any stream that closes, because a server sends no type the client has not requested. A capability set can also refresh when its freshness expires, so run the diff on every refresh, not only on receipt of a notification.

On each refresh, diff the newly advertised set against the last approved set, place additions in a pending state, deny their invocation, and record the event. Where the gate is inside the server's own tool handling, the denial reaches the client as a successful result carrying `isError: true`, so make the denial text name the pending or unauthorized state.

Record each change in the audit log with the capabilities added as a non-empty list of names, the approval state (`pending`, `approved`, or `denied`), the approver, and the timestamp. The approver and timestamp may be `null` only while the state is pending. Use the same field names the Audit reads. Tie the approved set back to the capability baseline.

Default Value:

Under the `2026-07-28` revision, a client receives a capability change notification only for the types it has requested on a `subscriptions/listen` stream. Where a client has requested them, the common default is to refresh the affected list and treat it as immediately invocable. The TypeScript SDK's `listChanged` option opens the stream, re-fetches on every published change, and hands the result to the application, with no staging state and no re-approval step. Where a client has requested nothing, no change signal is delivered and drift goes unobserved. Neither default provides an approval gate, a pending state, or a capability diff in the audit log.

References:

1. MCP TypeScript SDK v2 (client subscriptions, the listChanged option): https://ts.sdk.modelcontextprotocol.io/v2/clients/subscriptions.html
2. MCP TypeScript SDK v2 (server errors): https://ts.sdk.modelcontextprotocol.io/v2/servers/errors.html
3. SEP-2243 (HTTP Header Standardization for Streamable HTTP Transport): https://modelcontextprotocol.io/seps/2243-http-standardization
4. SEP-2549 (TTL for List Results): https://modelcontextprotocol.io/seps/2549-TTL-for-list-results
5. SEP-2575 (Make MCP Stateless): https://modelcontextprotocol.io/seps/2575-stateless-mcp

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	2.5 Allowlist Authorized Software Use technical controls, such as application allowlisting, to ensure that only authorized software can execute or be accessed. Reassess bi-annually, or more frequently.		●	●

1.4 Ensure MCP client and server identity metadata is captured and validated against the asset inventory (Automated)

Profile Applicability:

- Level 1

Description:

The self-asserted identity metadata that MCP clients and servers exchange is captured and validated against the identities and capability profiles in the enterprise MCP registry. Under the `2026-07-28` revision, a client identifies itself in `_meta["io.modelcontextprotocol/clientInfo"]` on each request and a server in `_meta["io.modelcontextprotocol/serverInfo"]` on each response. Both fields carry a name and a version, and both are optional in the schema, so an implementation configured not to send one is still conforming. The specification does not define how a receiver treats a malformed identity value, and handling varies by implementation. This Recommendation therefore never accepts a malformed value as an approved inventory identity and records it as a metadata-validation finding. Server capabilities and supported protocol versions are obtained separately from the `server/discover` result. The enterprise standard is that every approved client and server emits these fields, the capturing layer records them, and each captured name and version is cross-referenced against the registry. An identity or version not in the registry, or an advertised capability set that differs from the registered profile for that identity, raises an alert and is treated as unauthorized asset expansion until reconciled. These fields are self-reported and are not authenticated by the protocol. They are treated as an inventory signal rather than proof of identity, and tool invocation is not gated on them. Where a deployment refuses an unrecognized server, the refusal is bound to an authenticated property of the connection, such as the mutual TLS identity, the OAuth client or resource identity, or the pinned artifact hash under the supply-chain Recommendations, with the captured `serverInfo` recorded alongside as the correlation key.

Rationale:

An MCP deployment accumulates servers faster than any manual process tracks them, and a server that no one approved is indistinguishable from one that everyone forgot about. Identity metadata is the cheapest signal for telling the two apart, because supported client and server SDK versions commonly stamp it on every request and response unless configured otherwise, and the capturing layer needs no protocol change to record it. Without that capture there is no inventory of what is actually running, so a new or modified server arriving through supply-chain compromise, misconfiguration, or insider action operates undetected until someone audits by hand.

The fields are self-reported and unverified, which sets the boundary of what this Recommendation claims. Cross-referencing them against the registry detects the unapproved, the stale, and the misconfigured, which is the large majority of real drift. It does not detect an adversary who forges the string, and treating it as though it did would substitute a detective control for a preventive one. The preventive control belongs on the authenticated identity of the channel. This Recommendation is the detective control that makes the inventory true, and the alert on an unregistered identity is what converts it from a log into a finding.

Impact:

Recording identity on every request and response produces log volume proportional to traffic, so size retention or sample accordingly. Capture requires the server, host, or an application-layer proxy able to read the JSON body.

Audit:

Confirm the capturing layer records client and server identity, and cross-reference each captured identity against the approved identities in the enterprise registry.

Set the log-schema variables at the head of each script to match the deployment: `RESP` and `REQ` select response and request records, `ERR` identifies an error response, and `SID` and `CID` locate the server and client identity objects. Error responses are excluded from the identity count rather than counted as failures since a JSON-RPC error has no `result` to carry `serverInfo`. Client identity is counted over requests and server identity over responses. The comparison excludes the audit client's own identity by name.

Run the following script to confirm identity is recorded on every captured server response. It reads records as JSON documents so it tolerates pretty-printed logs:

```
LOG="<MCP_AUDIT_LOG>"
RESP='select((.direction? // "") == "response")'
REQ='select((.direction? // "") == "request")'
ERR='((.error? != null) or ((.has_error? // false) == true))'
SID='(.result._meta["io.modelcontextprotocol/serverInfo"]
// ._meta["io.modelcontextprotocol/serverInfo"])'
CID='(._meta["io.modelcontextprotocol/clientInfo"]
// .params._meta["io.modelcontextprotocol/clientInfo"])'

if [ ! -r "$LOG" ]; then
 echo "ERROR: audit log not readable at $LOG"
else
 rtot=$(jq -s "[.[] | $RESP] | length" "$LOG" 2>/dev/null)
 rerr=$(jq -s "[.[] | $RESP | select($ERR)] | length" "$LOG" 2>/dev/null)
 rok=$(jq -s "[.[] | $RESP | select($ERR | not)] | length" "$LOG" 2>/dev/null)
 rid=$(jq -s "[.[] | $RESP | select($ERR | not) | select($SID.name != null and
$SID.version != null)] | length" "$LOG" 2>/dev/null)
 qtot=$(jq -s "[.[] | $REQ] | length" "$LOG" 2>/dev/null)
 qid=$(jq -s "[.[] | $REQ | select($CID.name != null and $CID.version != null)] |
length" "$LOG" 2>/dev/null)
 if ! [ "$rtot" -ge 0 ] 2>/dev/null || ! [ "$qtot" -ge 0 ] 2>/dev/null; then
 echo "ERROR: audit log could not be parsed as JSON records"
 elif [ "$qtot" -eq 0 ]; then
 echo "ERROR: no request records matched the selector, client identity cannot be
evaluated, confirm the log path and the selector"
 elif [ "$rtot" -eq 0 ]; then
 echo "ERROR: no response records matched the selector, server identity cannot be
evaluated, confirm the log path and the selector"
 elif [ "$rok" -eq 0 ]; then
 echo "ERROR: every recorded response carried a protocol error, so there is no
successful response to evaluate, widen the capture window"
 else
 echo "requests: $qtot, carrying clientInfo: $qid"
 echo "responses: $rtot, of which protocol errors: $rerr (excluded)"
 echo "successful responses: $rok, carrying serverInfo: $rid"
 mc=$((qtot - qid))
 ms=$((rok - rid))
 if [ "$mc" -eq 0 ] && [ "$ms" -eq 0 ]; then
 echo "PASS: client identity is recorded on every captured request and server
identity on every captured successful response"
 elif [ "$mc" -gt 0 ] && [ "$ms" -gt 0 ]; then
 echo "FAIL: $mc requests carry no client identity and $ms successful responses carry
no server identity"
 elif [ "$mc" -gt 0 ]; then
 echo "FAIL: $mc requests carry no client identity"
 else
 echo "FAIL: $ms successful responses carry no server identity"
 fi
 fi
fi
```

Maintain the approved identities as two registry exports of `name|version` pairs, one per line, for servers and for clients. Because each entry occupies one line delimited by the pipe, a registered name or version cannot contain the pipe delimiter, a carriage return, or a line break, and the script flags any observed identity containing the delimiter for review.

Run the following script to confirm every observed identity is registered on its own side:

```
LOG="<MCP_AUDIT_LOG>"
REG="<REGISTRY_DIR>/approved_identities.txt"
CREG="<REGISTRY_DIR>/approved_client_identities.txt"
AUDIT_CLIENT="cis-benchmark-audit"
RESP='select((.direction? // "") == "response")'
REQ='select((.direction? // "") == "request")'
SID='(.result._meta["io.modelcontextprotocol/serverInfo"]
// ._meta["io.modelcontextprotocol/serverInfo"])'
CID='(._meta["io.modelcontextprotocol/clientInfo"]
// .params._meta["io.modelcontextprotocol/clientInfo"])'
WORK=$(mktemp -d)

if [ ! -r "$LOG" ]; then
 echo "ERROR: audit log not readable at $LOG"
elif [ ! -r "$REG" ]; then
 echo "ERROR: server registry export not readable at $REG"
elif [ ! -r "$CREG" ]; then
 echo "ERROR: client registry export not readable at $CREG"
elif ! jq -r -s "[.[] | $RESP | $SID | select(.name != null) |
\"\(.name)|\(.version)\"] | unique[]" "$LOG" > "$WORK/sseen" 2>/dev/null; then
 echo "ERROR: audit log could not be parsed as JSON records"
elif ! jq -r -s --arg ac "$AUDIT_CLIENT" "[.[] | $REQ | $CID | select(.name != null)
| select(.name != \$ac) | \"\(.name)|\(.version)\"] | unique[]" "$LOG" >
"$WORK/cseen" 2>/dev/null; then
 echo "ERROR: audit log could not be parsed as JSON records"
elif [ ! -s "$WORK/sseen" ]; then
 echo "ERROR: no server identities present in the log, confirm the capture point and
the selector"
elif [ ! -s "$WORK/cseen" ]; then
 echo "ERROR: no client identities present in the log after excluding the auditor,
confirm the capture point and that the window contains traffic from clients other
than the auditor"
else
 PIPEQ='select(((.name | tostring) | contains("|")) or ((.version | tostring) |
contains("|")))'
 jq -r -s "[.[] | $RESP | $SID | select(.name != null) | $PIPEQ | \"server \(.name),
version \(.version)\"] | unique[]" "$LOG" > "$WORK/sbad" 2>/dev/null
 jq -r -s --arg ac "$AUDIT_CLIENT" "[.[] | $REQ | $CID | select(.name != null) |
select(.name != \$ac) | $PIPEQ | \"client \(.name), version \(.version)\"] |
unique[]" "$LOG" > "$WORK/cbad" 2>/dev/null
 LC_ALL=C sort -u "$WORK/sseen" -o "$WORK/sseen"
 LC_ALL=C sort -u "$WORK/cseen" -o "$WORK/cseen"
 tr -d '\r' < "$REG" | grep -v '^[[:space:]]*$' | LC_ALL=C sort -u > "$WORK/sregs"
 tr -d '\r' < "$CREG" | grep -v '^[[:space:]]*$' | LC_ALL=C sort -u > "$WORK/cregs"
 us=$(LC_ALL=C comm -23 "$WORK/sseen" "$WORK/sregs")
 uc=$(LC_ALL=C comm -23 "$WORK/cseen" "$WORK/cregs")
 as=$(LC_ALL=C comm -13 "$WORK/sseen" "$WORK/sregs")
 ac2=$(LC_ALL=C comm -13 "$WORK/cseen" "$WORK/cregs")
 if [ -n "$as" ]; then printf '%s\n' "$as" | sed 's/^/registered but not observed
(server): /'; fi
```

```
 if [ -n "$ac2" ]; then printf '%s\n' "$ac2" | sed 's/^/registered but not observed
(client): /'; fi
 if [ -n "$us" ]; then printf '%s\n' "$us" | sed 's/^/unregistered server: /'; fi
 if [ -n "$uc" ]; then printf '%s\n' "$uc" | sed 's/^/unregistered client: /'; fi
 if [ -n "$us" ] && [ -n "$uc" ]; then
  echo "FAIL: client and server identities were observed that are absent from the
registry exports"
 elif [ -n "$us" ]; then
  echo "FAIL: server identities were observed that are absent from the registry
export"
 elif [ -n "$uc" ]; then
  echo "FAIL: client identities were observed that are absent from the registry
export"
 elif [ -n "$as" ] || [ -n "$ac2" ]; then
  echo "REVIEW: every observed identity is registered, but registered identities were
not observed"
 elif [ -s "$WORK/sbad" ] || [ -s "$WORK/cbad" ]; then
  cat "$WORK/sbad" "$WORK/cbad" | sed 's/^/contains the registry delimiter: /'
  echo "REVIEW: observed identities contain the registry delimiter and cannot be
represented in the name|version export, apply the registry naming policy or
investigate the sender"
 else
  echo "PASS: every observed client and server identity is present in the registry
exports"
 fi
fi
rm -rf "$WORK"
```

An observed client or server identity absent from its export is unauthorized asset expansion and is a `FAIL`. A registered identity not seen in the log window is a `REVIEW` since it may be a decommissioned deployment or a quiet window. Where both apply, `FAIL` takes precedence. The system is compliant when client identity is recorded on every captured request, server identity on every captured successful response, and no unregistered identity appears.

Remediation:

Configure the MCP server, the host, or an application-layer gateway able to read the JSON body to record `clientInfo` from each request and `serverInfo` from each response into the audit log, with a correlation ID on each entry. The identity travels in the JSON body, not in an HTTP header, so a plain reverse proxy cannot capture it.

Configure the approved clients and servers to emit these fields. Both are optional, and supported Tier 1 SDKs commonly send them unless configured otherwise, so confirm nothing has disabled them.

Export the approved identities from the registry to two newline-delimited files at the paths the Audit reads, one `name|version` pair per line, servers in one file and clients in the other, using Unix line endings. Generate the export from the registry rather than by hand, and enforce a registry naming policy that excludes the pipe character, carriage returns, and line breaks so no identity can break the one-pair-per-line format.

Run the following script to create the export directory and confirm both exports are well formed:

```
REGDIR="<REGISTRY_DIR>"
REG="$REGDIR/approved_identities.txt"
CREG="$REGDIR/approved_client_identities.txt"

mkdir -p "$REGDIR"
missing=""
[ -s "$REG" ] || missing="$missing $REG"
[ -s "$CREG" ] || missing="$missing $CREG"
if [ -n "$missing" ]; then
 echo "ERROR: no registry export at$missing, export the approved server and client
identities from the enterprise MCP registry before running the audit"
else
 sbad=$(tr -d '\r' < "$REG" | grep -vcE '^[^|]+\|[^|]+$')
 cbad=$(tr -d '\r' < "$CREG" | grep -vcE '^[^|]+\|[^|]+$')
 if ! [ "$sbad" -ge 0 ] 2>/dev/null || ! [ "$cbad" -ge 0 ] 2>/dev/null; then
  echo "ERROR: registry exports could not be read under $REGDIR"
 elif [ "$sbad" -gt 0 ] || [ "$cbad" -gt 0 ]; then
  tr -d '\r' < "$REG" | grep -nvE '^[^|]+\|[^|]+$' | sed 's/^/malformed server line
/'
  tr -d '\r' < "$CREG" | grep -nvE '^[^|]+\|[^|]+$' | sed 's/^/malformed client line
/'
  echo "FAIL: $((sbad + cbad)) lines in the registry exports are not in name|version
form"
 else
  echo "PASS: $(tr -d '\r' < "$REG" | grep -c .) approved server and $(tr -d '\r' <
"$CREG" | grep -c .) approved client identities recorded in name|version form"
 fi
fi
```

Configure the capturing layer to alert when an observed identity or version is absent from the registry, or when an advertised capability set differs from the registered profile. Treat an unreconciled alert as an open finding.

These fields are self-reported, and an adversary bypasses them by changing a string, so do not block tool invocation on them. Where a deployment refuses an unrecognized server, bind the refusal to the authenticated channel identity under the authorization Recommendations or the pinned artifact hash under the supply-chain Recommendations, using the captured identity as the correlation key.

Default Value:

Under the `2026-07-28` revision, clients include `io.modelcontextprotocol/clientInfo` on each request and servers include `io.modelcontextprotocol/serverInfo` in each result's `_meta`, both optional in the schema. Supported Tier 1 SDK versions commonly stamp both by default, so the identity data is usually on the wire without configuration, but verify the specific SDK and entry point in use. No default behavior records these fields to an audit log, cross-references them against a registry, or alerts on an unrecognized value. Discovery results may also carry `ttlMs` and `cacheScope`, so where a caching intermediary sits in the path an audit can read a capability set that is no longer current.

References:

1. MCP specification (Key Changes, stateless core and per-request identity): https://modelcontextprotocol.io/specification/2026-07-28/changelog
2. MCP specification (Schema Reference, ResultMetaObject and io.modelcontextprotocol/serverInfo): https://modelcontextprotocol.io/specification/2026-07-28/schema
3. MCP Extensions (Overview, worked server/discover response): https://modelcontextprotocol.io/extensions/overview
4. SEP-2575 (Make MCP Stateless): https://modelcontextprotocol.io/seps/2575-stateless-mcp

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	1.1 Establish and Maintain Detailed Enterprise Asset Inventory Establish and maintain an accurate, detailed, and up-to-date inventory of all enterprise assets with the potential to store or process data, to include: end-user devices (including portable and mobile), network devices, non-computing/IoT devices, and servers. Ensure the inventory records the network address (if static), hardware address, machine name, enterprise asset owner, department for each asset, and whether the asset has been approved to connect to the network. For mobile end-user devices, MDM type tools can support this process, where appropriate. This inventory includes assets connected to the infrastructure physically, virtually, remotely, and those within cloud environments. Additionally, it includes assets that are regularly connected to the enterprise's network infrastructure, even if they are not under control of the enterprise. Review and update the inventory of all enterprise assets bi-annually, or more frequently.	●	●	●
v8	2.1 Establish and Maintain a Software Inventory Establish and maintain a detailed inventory of all licensed software installed on enterprise assets. The software inventory must document the title, publisher, initial install/use date, and business purpose for each entry; where appropriate, include the Uniform Resource Locator (URL), app store(s), version(s), deployment mechanism, and decommission date. Review and update the software inventory bi-annually, or more frequently.	●	●	●

2 Transport & Connectivity

MCP runs over more than one transport, and the security properties available to a deployment depend on which one it chooses and how that transport is configured. A local stdio server and a remote server reachable over HTTP present very different exposure. An HTTP endpoint that carries plaintext, trusts unvalidated request metadata, or accepts requests from any origin gives an attacker a foothold.

The Recommendations in this section prefer stdio where a server is local and single-user, require TLS and disallow plaintext for HTTP, ensure authentication survives intermediary proxies, require the request metadata the protocol's security model relies on, and validate the Origin header on Streamable HTTP requests.

2.1 Ensure stdio is preferred for local, single-user servers (Manual)

Profile Applicability:

- Level 1

Description:

The stdio transport is used for local development environments and single-user tool deployments, and Streamable HTTP is reserved for servers that must serve remote clients or multiple concurrent users. Each server's transport choice is inventoried, and a documented operational reason is kept on file for any server on a network-exposed transport where stdio would suffice. When a local stdio server is later promoted to serve multiple users, moving to Streamable HTTP means it must then meet the transport-security requirements, TLS and Origin validation, and the applicable client-configuration requirements.

Rationale:

The stdio transport confines all communication to local inter-process pipes, eliminating the network attack surface for servers that do not require remote access. A stdio server cannot be reached by external attackers over the network, cannot be subject to DNS rebinding, and requires no TLS, certificate management, or Origin validation. Using a network transport where stdio suffices exposes the server to all of these attack classes for no operational benefit. This recommendation makes stdio the enforced organizational preference for local, single-user servers.

Impact:

There is no impact for a server used only locally. For a server that was network-reachable without needing to be, moving to stdio removes that exposure at no functional cost.

Audit:

Inventory each MCP service unit and record whether the server process owns a TCP listener. A listener does not by itself prove Streamable HTTP, and its absence does not prove stdio, so determine the transport from the unit's `ExecStart`, the MCP client configuration, and the server configuration. Run the following command:

```
for unit in $(systemctl list-units --type=service --no-legend 'mcp*' | awk '{print
$1}'); do
 pid=$(systemctl show -p MainPID --value "$unit" 2>/dev/null)
 if [ -n "$pid" ] && [ "$pid" -gt 0 ] 2>/dev/null \
  && ss -ltnp 2>/dev/null | grep -q "pid=$pid,"; then
  echo "NETWORK-LISTENER $unit (TCP listener detected, determine whether it carries
MCP)"
 else
  echo "NO-PROCESS-LISTENER $unit (no TCP listener owned by this process, inspect the
configured MCP transport)"
 fi
done
```

A local, single-user server carrying MCP over a network listener is a finding. The system is compliant when every local, single-user server is confirmed to use stdio, and any server exposing a network listener either does not carry MCP over it or genuinely serves multiple users or remote clients.

Remediation:

Reconfigure the server to use stdio and restart it. Then run the following command to confirm the server process owns no TCP listener, substituting the service unit name. The check reports `ERROR` and stops if the service did not restart, is not active, or has no active `MainPID`:

```
MCP_UNIT="${MCP_UNIT:-<MCP_UNIT>}"
sudo systemctl restart "$MCP_UNIT" || {
 echo "ERROR: service restart failed for $MCP_UNIT"
 exit 1
}
if ! systemctl is-active --quiet "$MCP_UNIT"; then
 echo "ERROR: $MCP_UNIT is not active after restart"
 exit 1
fi
pid=$(systemctl show -p MainPID --value "$MCP_UNIT" 2>/dev/null)
if [ -z "$pid" ] || [ "$pid" -le 0 ] 2>/dev/null; then
 echo "ERROR: $MCP_UNIT has no active MainPID, cannot attribute the listener check"
 exit 1
fi
if ss -ltnp 2>/dev/null | grep -q "pid=$pid,"; then
 echo "FAIL: a TCP listener is owned by the MCP server process"
else
 echo "PASS: no TCP listener is owned by the MCP server process"
fi
```

A `PASS` confirms no TCP listener is owned by the MCP server process. It does not by itself prove the transport is stdio, which the manual assessment confirms from the server configuration.

Default Value:

By default, MCP server implementations may be configured for either transport. There is no automatic enforcement that local or single-user servers use stdio, and no alert when a stdio-eligible server is network-exposed.

References:

1. MCP specification (Base Protocol / Transports / Overview): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports
2. MCP specification (Base Protocol / Transports / stdio): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/stdio

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.8 Uninstall or Disable Unnecessary Services on Enterprise Assets and Software Uninstall or disable unnecessary services on enterprise assets and software, such as an unused file sharing service, web application module, or service function.		●	●

2.2 Ensure TLS is required for HTTP and plaintext is disallowed (Automated)

Profile Applicability:

- Level 1

Description:

MCP Streamable HTTP servers that carry sensitive data are expected to be deployed over a secure transport. The MCP specification requires Origin validation and recommends authentication for HTTP deployments, but it does not prescribe a minimum TLS protocol revision. TLS 1.2 or higher is the baseline established here, consistent with current enterprise TLS configuration guidance. A server that serves plaintext HTTP, negotiates a legacy TLS version, or presents an expired certificate does not meet this baseline.

Rationale:

Transmitting MCP messages, OAuth tokens, and tool-call payloads over unencrypted HTTP exposes all of it to interception. An attacker positioned on the network can read or modify every message. Authorization tokens captured from plaintext can then be replayed to invoke tools as the victim. TLS is the baseline defense against both. Permitting TLS 1.0 or 1.1 reopens known transport weaknesses that TLS 1.2 closes.

Impact:

Certificate expiry is the main operational risk, mitigated by automated renewal. Clients pinned to TLS 1.0 or 1.1 fail to connect once weak versions are refused and must be updated.

Audit:

Verify that the plaintext HTTP port is not served or redirects to HTTPS, that legacy TLS versions are refused while TLS 1.2 or 1.3 is negotiated, and that the served certificate is within its validity period. Substitute the plaintext URL, the TLS endpoint URL, and the TLS host, port, and SNI name. Because a refused plaintext port on a dead deployment proves nothing, the probe first confirms the TLS endpoint answers. Run the following command to confirm plaintext behavior:

```
HTTP_URL="${HTTP_URL:-<PLAINTEXT_HTTP_URL>}"
TLS_URL="${TLS_URL:-<TLS_ENDPOINT_URL>}"
curl -sk -o /dev/null --connect-timeout 5 --max-time 15 "$TLS_URL" 2>/dev/null
TRC=$?
if [ "$TRC" -eq 6 ] || [ "$TRC" -eq 7 ] || [ "$TRC" -eq 28 ]; then
  echo "ERROR: the TLS endpoint is unreachable (curl exit $TRC), so plaintext
behavior is not attributable to a served deployment"
else
R=$(curl -sS -o /dev/null --connect-timeout 5 --max-time 15 -w '%{http_code}'
"$HTTP_URL" 2>/dev/null)
RC=$?
T=$(curl -sS -o /dev/null --connect-timeout 5 --max-time 15 -w '%{redirect_url}'
"$HTTP_URL" 2>/dev/null)
if [ "$RC" -eq 7 ] || [ -z "$R" ] || [ "$R" = "000" ]; then
  echo "plaintext probe: connection refused or no response (curl rc=$RC)"
  echo "PASS: the plaintext HTTP port is not served while the TLS endpoint answers"
else
  echo "plaintext probe: HTTP $R redirect_target=${T:-none}"
  case "$R" in
    301|302|307|308)
      case "$T" in
        https://*) echo "PASS: plaintext HTTP redirects to an HTTPS target ($R)" ;;
        *) echo "FAIL: plaintext HTTP redirects to a non-HTTPS target ($R to ${T:-an
empty target})" ;;
      esac ;;
    200) echo "FAIL: plaintext HTTP is served with a 200 response" ;;
    426) echo "PASS: plaintext HTTP requires upgrade to TLS (426)" ;;
    *) echo "REVIEW: plaintext HTTP returned $R, inspect whether the endpoint serves
cleartext content before concluding" ;;
  esac
fi
fi
```

Run the following command to confirm legacy TLS versions are refused. Because a default modern client cannot offer TLS 1.0 or 1.1 at all, the legacy offers use the OpenSSL security-level opt-in:

```
HOST="${HOST:-<TLS_HOST>}"
PORT="${PORT:-<TLS_PORT>}"
SNI="${SNI:-<TLS_SNI>}"
if ! echo | timeout 10 openssl s_client -connect "$HOST:$PORT" -servername
"$SNI" >/dev/null 2>&1; then
  echo "ERROR: no TLS handshake possible with $HOST:$PORT, endpoint unreachable or
not serving TLS"
else
FAILN=0; ERRN=0
for ver in tls1 tls1_1; do
  if ! echo | timeout 10 openssl s_client -connect "$HOST:$PORT" -servername "$SNI" -
"$ver" -cipher DEFAULT@SECLEVEL=0 >/dev/null 2>/tmp/cis22_cap.$$; then
    if grep -qi 'no protocols available\|unknown option\|unsupported protocol'
/tmp/cis22_cap.$$; then
      echo "$ver: the auditing openssl cannot offer this version even at security
level 0, so a refusal here is not attributable to the server"
      ERRN=$((ERRN+1))
    else
      echo "$ver: refused by the server"
    fi
  else
    echo "$ver: NEGOTIATED (the server accepted a weak TLS version)"
    FAILN=$((FAILN+1))
  fi
done
rm -f /tmp/cis22_cap.$$
echo | timeout 10 openssl s_client -connect "$HOST:$PORT" -servername "$SNI" -
tls1_2 >/dev/null 2>&1 && T12=ok || T12=no
echo | timeout 10 openssl s_client -connect "$HOST:$PORT" -servername "$SNI" -
tls1_3 >/dev/null 2>&1 && T13=ok || T13=no
echo "tls1_2=$T12 tls1_3=$T13"
if [ "$FAILN" -gt 0 ]; then
  echo "FAIL: a legacy TLS version was negotiated by the server"
elif [ "$ERRN" -gt 0 ]; then
  echo "ERROR: the auditing host cannot offer one or more legacy versions, run from a
host whose openssl can offer TLS 1.0 and 1.1 or the negative test proves nothing"
elif [ "$T12" = "ok" ] || [ "$T13" = "ok" ]; then
  echo "PASS: legacy TLS versions are refused and TLS 1.2 or 1.3 is negotiated"
else
  echo "REVIEW: legacy TLS refused but neither TLS 1.2 nor 1.3 negotiated, inspect
the endpoint's protocol configuration before concluding"
fi
fi
```

Run the following command to confirm the served certificate is currently valid:

```
HOST="${HOST:-<TLS_HOST>}"
PORT="${PORT:-<TLS_PORT>}"
SNI="${SNI:-<TLS_SNI>}"
PEM=$(echo | timeout 10 openssl s_client -connect "$HOST:$PORT" -servername "$SNI"
2>/dev/null | openssl x509 2>/dev/null)
if [ -z "$PEM" ]; then
 echo "ERROR: could not retrieve a certificate from $HOST:$PORT, confirm the endpoint
is reachable and terminates TLS"
else
 DATES=$(printf '%s' "$PEM" | openssl x509 -noout -subject -issuer -dates
2>/dev/null)
 echo "$DATES"
 if printf '%s' "$PEM" | openssl x509 -noout -checkend 0 >/dev/null 2>&1; then
  echo "PASS: the certificate is currently valid"
 else
  echo "FAIL: the certificate is expired or not yet valid"
 fi
fi
```

The system is compliant when the plaintext, protocol, and certificate probes all print `PASS`. An `ERROR` means the evidence is not attributable, usually an unreachable endpoint or an auditor host whose openssl cannot offer legacy versions.

Remediation:

Terminate TLS in front of the MCP server and redirect or refuse plaintext. The following nginx example defines the upstream, redirects port 80 to HTTPS, restricts protocols to TLS 1.2 and 1.3, and proxies the MCP path. Substitute the upstream address, the server name, and the certificate paths:

```
upstream mcp_upstream {
    server <MCP_UPSTREAM_ADDR>;
}
server {
    listen 80;
    server_name <MCP_SERVER_NAME>;
    return 308 https://$host$request_uri;
}
server {
    listen 443 ssl;
    server_name <MCP_SERVER_NAME>;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_certificate <TLS_CERTIFICATE_PATH>;
    ssl_certificate_key <TLS_CERTIFICATE_KEY_PATH>;
    location /mcp { proxy_pass http://mcp_upstream; }
}
```

After applying the configuration, run the Audit to confirm all three probes print `PASS`.

Default Value:

By default, most MCP server frameworks accept both HTTP and HTTPS, with TLS enforcement and plaintext rejection or redirection left to explicit configuration at the server or reverse-proxy layer.

References:

1. MCP specification (Base Protocol / Transports / Streamable HTTP, security and endpoint): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http
2. NIST SP 800-52 Revision 2, Guidelines for the Selection, Configuration, and Use of Transport Layer Security (TLS) Implementations: https://csrc.nist.gov/pubs/sp/800/52/r2/final

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 Encrypt Sensitive Data in Transit Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).		●	●

2.3 Ensure authentication propagates through proxies to request-scoped SSE responses (Manual)

Profile Applicability:

- Level 2

Description:

Under the `2026-07-28` revision, each JSON-RPC request on the Streamable HTTP transport is sent as its own HTTP `POST`, and the server answers with either a single JSON response or a request-scoped Server-Sent Events stream that carries any request-related notifications and then the final response. There is no standalone `GET` stream and no `Last-Event-ID` resumption. A request-scoped SSE stream is not resumable, so closing or dropping it cancels that request. If the client retries, the retry is a new HTTP `POST` authenticated independently. When the MCP server is reached through one or more proxies, every hop preserves the authentication requirement, so an unauthenticated request cannot reach a streamed response.

Rationale:

Misconfigured gateways can strip or cache credentials, causing either an authentication failure or credential leakage. A stripped credential is an availability failure, the request is rejected. A cached, logged, or forwarded credential is the security failure, because a token exposed at an intermediary can be captured and replayed to act as the user. The risk is greatest in gateway-mediated deployments with multiple proxy hops, especially where a proxy terminates TLS and re-encrypts to the backend, where header-stripping and header-logging bugs commonly appear on the inner hop.

Impact:

Meeting this requirement may need gateway configuration changes, or pinning the proxy and client to versions that handle streaming authentication correctly. The cost is minimal where authentication already terminates at a conformant gateway.

Audit:

Send a `tools/call` request to the endpoint that returns a request-scoped SSE stream, through the same proxy path production clients use, once without a credential and once with a valid one. Confirm the unauthenticated request is rejected, the authenticated request is accepted, and the authenticated response is an SSE stream. Substitute the streaming endpoint URL, the approved protocol version, a safe streaming tool and its arguments, and a valid bearer token:

```
URL="<STREAMING_ENDPOINT_URL>"
VER="<APPROVED_PROTOCOL_VERSION>"
TOOL="<SAFE_STREAMING_TOOL>"
ARGS='<SAFE_STREAMING_TOOL_ARGS_JSON>'
TOKEN="<VALID_BEARER_TOKEN>"
BODY=$(jq -nc --arg v "$VER" --arg t "$TOOL" --argjson a "$ARGS"
'{jsonrpc:"2.0",id:31,method:"tools/call",params:{name:$t,arguments:$a,_meta:{"io.mod
elcontextprotocol/protocolVersion":$v,"io.modelcontextprotocol/clientInfo":{name:"cis
-benchmark-audit",version:"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}'
2>/dev/null)
NAME_HDR=$TOOL
case $TOOL in *[!-A-Za-z0-9._~]*|"") NAME_HDR="=?base64?$(printf '%s' "$TOOL" |
base64 | tr -d '\r\n')?=" ;; esac
if [ -z "$BODY" ]; then
 echo "ERROR: probe build failed, check that ARGS is valid JSON"
else
 NO=$(curl -sS -o /dev/null --connect-timeout 5 --max-time 20 -w '%{http_code}' -X
POST "$URL" -H 'Accept: application/json, text/event-stream' -H 'Content-Type:
application/json' -H "MCP-Protocol-Version: $VER" -H 'Mcp-Method: tools/call' -H
"Mcp-Name: $NAME_HDR" --data-binary "$BODY" 2>/dev/null)
 YES_HDRS=$(curl -sS -D - -o /dev/null --connect-timeout 5 --max-time 20 -X POST
"$URL" -H 'Accept: application/json, text/event-stream' -H 'Content-Type:
application/json' -H "MCP-Protocol-Version: $VER" -H 'Mcp-Method: tools/call' -H
"Mcp-Name: $NAME_HDR" -H "Authorization: Bearer $TOKEN" --data-binary "$BODY"
2>/dev/null)
 YES=$(printf '%s' "$YES_HDRS" | awk 'NR==1{print $2}')
 YES_CT=$(printf '%s\n' "$YES_HDRS" | tr -d '\r' | awk -F': ' 'tolower($1)=="content-
type"{print tolower($2)}')
 echo "no_auth=$NO with_auth=$YES with_auth_content_type=${YES_CT:-none}"
 if [ "$NO" = "000" ] || [ -z "$YES" ]; then
  echo "ERROR: endpoint unreachable or timed out, the credential path could not be
exercised"
 else
  case "$NO" in
   401|403)
    case "$YES" in
     2*)
      case "$YES_CT" in
       *text/event-stream*) echo "PASS: authentication is enforced before a request-
scoped SSE stream is established, and enforcement holds across every proxy hop on the
path" ;;
       *) echo "REVIEW: authentication is enforced and the authenticated request
succeeded ($YES), but the response was $YES_CT rather than an SSE stream, so the
server chose a non-streamed response and the SSE-specific assertion could not be
exercised, re-run against a tool or condition that yields a streamed response" ;;
      esac ;;
     401|403) echo "REVIEW: the valid credential was also rejected ($YES), confirm
the token and that it survives every proxy hop to the enforcement point" ;;
     *) echo "REVIEW: the unauthenticated request was rejected but the authenticated
request returned $YES, inspect the proxy path before concluding" ;;
```

```
    esac ;;
   2*) echo "FAIL: an unauthenticated request was accepted, authentication is not
enforced before the response is established" ;;
   *) echo "REVIEW: the unauthenticated request returned $NO, expected 401 or 403,
inspect before concluding" ;;
  esac
 fi
fi
```

A `PASS` requires the unauthenticated request rejected with `401` or `403`, the authenticated request accepted with `2xx`, and the authenticated response carrying `Content-Type: text/event-stream`. A `2xx` that is not an SSE stream is a `REVIEW`, so re-run against a tool that yields a streamed response. An accepted unauthenticated request is a `FAIL`. An unreachable endpoint is `ERROR`.

Remediation:

Configure each proxy hop to forward the `Authorization` header to the upstream for both the initial stream and any reconnect, and to pass the SSE response through without buffering so the stream is not held or broken. Because a reconnect is a new `POST`, the same rule covers it, and there is no separate resumption path. The following nginx example is illustrative. Other proxies use their own header-forwarding directives:

```
proxy_set_header Authorization $http_authorization;
proxy_pass_request_headers on;
proxy_buffering off;
proxy_read_timeout 3600s;
```

Pin proxy and client or server versions known to handle streaming authentication correctly, and re-run the Audit after any proxy upgrade.

Default Value:

By default, reverse proxies do not guarantee preservation of `Authorization` headers across streaming responses. Behavior varies by proxy vendor and configuration.

References:

1. MCP specification (Base Protocol / Transports / Streamable HTTP, receiving messages and request metadata): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http
2. MCP specification (Base Protocol / Message Patterns / Multi Round-Trip Requests): https://modelcontextprotocol.io/specification/2026-07-28/basic/patterns/mrtr

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.7 Centralize Access Control Centralize access control for all enterprise assets through a directory service or SSO provider, where supported.		●	●

2.4 Ensure required MCP request metadata headers are present and consistent with the request body (Automated)

Profile Applicability:

- Level 1

Description:

Under the `2026-07-28` revision, every Streamable HTTP request mirrors selected request-body fields into HTTP headers so that gateways, load balancers, and observability tooling can route and inspect requests without parsing the body. The `MCP-Protocol-Version` and `Mcp-Method` headers are required on every `POST`, with values matching the protocol version in the body's `_meta` and the JSON-RPC method respectively. The `Mcp-Name` header is required on `tools/call`, `resources/read`, and `prompts/get` requests, with a value that matches the tool name, resource URI, or prompt name in the body. Values that cannot be represented as plain ASCII are carried in the specification's Base64 sentinel format, which the server decodes before comparing. A server that processes the request body rejects any request with a missing required header or a header that disagrees with the body, using HTTP `400` and the `HeaderMismatch` JSON-RPC error (`-32020`). This Recommendation ensures those rejections are enforced, so a request cannot be routed or rate-limited as one operation while executing as another.

Rationale:

In the stateless model, routing, rate-limiting, and observability middleware act on the mirrored headers rather than parsing the body. A missing header leaves a gateway unable to route or apply policy correctly. Headers allowed to disagree with the body let different components in the request path rely on different sources of truth, so a request can be routed or rate-limited as one operation while executing as another, which is a request-smuggling and confusion vector. The specification therefore requires servers that process the body to reject missing or mismatched headers with HTTP `400` and the `HeaderMismatch` error, and to decode Base64 sentinel-encoded values before comparison. A validator that compares encoded values raw either admits encoded disagreement or rejects conformant clients. This Recommendation verifies the rejection is enforced on the wire rather than assumed from the SDK.

Impact:

There is no impact for conformant clients. Non-conformant clients that omit or contradict the required headers are rejected with HTTP `400` and need to be updated. One deployment risk to watch is an SDK that validates headers without decoding Base64 sentinel values, which wrongly rejects conformant clients whose names are not header-safe and needs a server or SDK update.

Audit:

Verify that the server rejects Streamable HTTP requests whose required metadata headers are missing or disagree with the body, and that Base64 sentinel-encoded values are decoded before comparison. Substitute the endpoint URL, the approved protocol version, and a baselined tool with schema-valid arguments. The probes only read, apart from the baselined tool invocations, which is why a baselined tool is used. Run the following command to confirm a request missing a required routing header is rejected with HTTP `400` and `HeaderMismatch` (`-32020`):

```
URL="<MCP_ENDPOINT_URL>"
VER="<APPROVED_PROTOCOL_VERSION>"
TOOL="<BASELINED_TOOL>"
ARGS='<BASELINED_TOOL_ARGS_JSON>'
B1=$(jq -nc --arg v "$VER"
'{jsonrpc:"2.0",id:41,method:"tools/list",params:{_meta:{"io.modelcontextprotocol/pro
tocolVersion":$v,"io.modelcontextprotocol/clientInfo":{name:"cis-benchmark-
audit",version:"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}'
2>/dev/null)
B2=$(jq -nc --arg v "$VER" --arg t "$TOOL" --argjson a "$ARGS"
'{jsonrpc:"2.0",id:42,method:"tools/call",params:{name:$t,arguments:$a,_meta:{"io.mod
elcontextprotocol/protocolVersion":$v,"io.modelcontextprotocol/clientInfo":{name:"cis
-benchmark-audit",version:"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}'
2>/dev/null)
FAILN=0; ERRN=0; REVN=0
if [ -z "$B1" ] || [ -z "$B2" ]; then
 ERRN=1
 echo "probe build failed: check that ARGS is valid JSON"
else
 for probe in 1 2; do
 if [ "$probe" = "1" ]; then
 R=$(curl -sS --connect-timeout 5 --max-time 20 -w '\n%{http_code}' -X POST "$URL" -H
'Accept: application/json, text/event-stream' -H 'Content-Type: application/json' -H
"MCP-Protocol-Version: $VER" --data-binary "$B1" 2>/dev/null)
 RID=41; LBL="missing Mcp-Method (tools/list)"
 else
 R=$(curl -sS --connect-timeout 5 --max-time 20 -w '\n%{http_code}' -X POST "$URL" -H
'Accept: application/json, text/event-stream' -H 'Content-Type: application/json' -H
"MCP-Protocol-Version: $VER" -H 'Mcp-Method: tools/call' --data-binary "$B2"
2>/dev/null)
 RID=42; LBL="missing Mcp-Name (tools/call)"
 fi
 CODE=$(printf '%s' "$R" | tail -n 1)
 BODY=$(printf '%s' "$R" | sed '$d')
 EC=$(printf '%s' "$BODY" | jq -r --argjson i "$RID" 'select(.id==$i) | .error.code
// empty' 2>/dev/null)
 [ -z "$EC" ] && EC=$(printf '%s' "$BODY" | sed -n 's/^data:[[:space:]]*//p' | jq -r
--argjson i "$RID" 'select(.id==$i) | .error.code // empty' 2>/dev/null | head -n 1)
 echo "$LBL: HTTP ${CODE:-none} jsonrpc_error ${EC:-none}"
 case "$CODE" in
 2*) FAILN=$((FAILN+1)) ;;
 400) case "$EC" in
 -32020) : ;;
 -32022) ERRN=$((ERRN+1)); echo " endpoint rejected the probe protocol version; align
VER with the approved allowlist and re-run" ;;
 *) ERRN=$((ERRN+1)); echo " 400 without a HeaderMismatch error body is not
attributable to the MCP implementation; if a gateway enforces presence, verify with
gateway evidence and probe the origin directly" ;;
 esac ;;
 000|"") ERRN=$((ERRN+1)); echo " endpoint unreachable or timed out" ;;
```

```
 *) REVN=$((REVN+1)) ;;
 esac
 done
fi
if [ "$FAILN" -gt 0 ]; then echo "FAIL: a request missing a required routing header
was accepted"
elif [ "$ERRN" -gt 0 ]; then echo "ERROR: no attributable evidence for one or more
probes; see detail lines"
elif [ "$REVN" -gt 0 ]; then echo "REVIEW: unexpected status for one or more probes;
see detail lines"
else echo "PASS: requests missing Mcp-Method or Mcp-Name are rejected with HTTP 400
and HeaderMismatch (-32020)"
fi
```

Run the following command to confirm a request whose `Mcp-Name` header contradicts the body is rejected:

```
URL="<MCP_ENDPOINT_URL>"
VER="<APPROVED_PROTOCOL_VERSION>"
TOOL="<BASELINED_TOOL>"
ARGS='<BASELINED_TOOL_ARGS_JSON>'
B=$(jq -nc --arg v "$VER" --arg t "$TOOL" --argjson a "$ARGS"
'{jsonrpc:"2.0",id:43,method:"tools/call",params:{name:$t,arguments:$a,_meta:{"io.mod
elcontextprotocol/protocolVersion":$v,"io.modelcontextprotocol/clientInfo":{name:"cis
-benchmark-audit",version:"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}'
2>/dev/null)
if [ -z "$B" ]; then
 echo "ERROR: probe build failed, check that ARGS is valid JSON"
else
 R=$(curl -sS --connect-timeout 5 --max-time 20 -w '\n%{http_code}' -X POST "$URL" -H
'Accept: application/json, text/event-stream' -H 'Content-Type: application/json' -H
"MCP-Protocol-Version: $VER" -H 'Mcp-Method: tools/call' -H 'Mcp-Name: cis-mismatch-
probe' --data-binary "$B" 2>/dev/null)
 CODE=$(printf '%s' "$R" | tail -n 1)
 BODY=$(printf '%s' "$R" | sed '$d')
 EC=$(printf '%s' "$BODY" | jq -r 'select(.id==43) | .error.code // empty'
2>/dev/null)
 [ -z "$EC" ] && EC=$(printf '%s' "$BODY" | sed -n 's/^data:[[:space:]]*//p' | jq -r
'select(.id==43) | .error.code // empty' 2>/dev/null | head -n 1)
 echo "header/body mismatch probe: HTTP ${CODE:-none} jsonrpc_error ${EC:-none}"
 case "$CODE" in
 2*) echo "FAIL: a request whose Mcp-Name header contradicts the body was
accepted" ;;
 400) case "$EC" in
 -32020) echo "PASS: header/body mismatch rejected with HTTP 400 and HeaderMismatch
(-32020)" ;;
 -32022) echo "ERROR: endpoint rejected the probe protocol version; align VER with
the approved allowlist and re-run" ;;
 *) echo "ERROR: 400 without a HeaderMismatch error body is not attributable to
header validation; if a gateway enforces headers, verify with gateway evidence and
probe the origin directly" ;;
 esac ;;
 000|"") echo "ERROR: endpoint unreachable or timed out" ;;
 *) echo "REVIEW: unexpected HTTP status ${CODE}; inspect the response before
concluding" ;;
 esac
fi
```

Run the following command to confirm the server decodes Base64 sentinel-encoded header values before comparing, accepting an encoded value that decodes to the body value and rejecting one that does not:

```
URL="<MCP_ENDPOINT_URL>"
VER="<APPROVED_PROTOCOL_VERSION>"
TOOL="<BASELINED_TOOL>"
ARGS='<BASELINED_TOOL_ARGS_JSON>'
B=$(jq -nc --arg v "$VER" --arg t "$TOOL" --argjson a "$ARGS"
'{jsonrpc:"2.0",id:44,method:"tools/call",params:{name:$t,arguments:$a,_meta:{"io.mod
elcontextprotocol/protocolVersion":$v,"io.modelcontextprotocol/clientInfo":{name:"cis
-benchmark-audit",version:"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}'
2>/dev/null)
ENC_MATCH="=?base64?$(printf '%s' "$TOOL" | base64 | tr -d '\r\n')?="
ENC_MISM="=?base64?$(printf '%s' 'cis-mismatch-probe' | base64 | tr -d '\r\n')?="
if [ -z "$B" ]; then
 echo "ERROR: probe build failed, check that ARGS is valid JSON"
else
 RM=$(curl -sS --connect-timeout 5 --max-time 20 -w '\n%{http_code}' -X POST "$URL" -
H 'Accept: application/json, text/event-stream' -H 'Content-Type: application/json' -
H "MCP-Protocol-Version: $VER" -H 'Mcp-Method: tools/call' -H "Mcp-Name: $ENC_MATCH"
--data-binary "$B" 2>/dev/null)
 CM=$(printf '%s' "$RM" | tail -n 1)
 RX=$(curl -sS --connect-timeout 5 --max-time 20 -w '\n%{http_code}' -X POST "$URL" -
H 'Accept: application/json, text/event-stream' -H 'Content-Type: application/json' -
H "MCP-Protocol-Version: $VER" -H 'Mcp-Method: tools/call' -H "Mcp-Name: $ENC_MISM" -
-data-binary "$B" 2>/dev/null)
 CX=$(printf '%s' "$RX" | tail -n 1)
 BX=$(printf '%s' "$RX" | sed '$d')
 EX=$(printf '%s' "$BX" | jq -r 'select(.id==44) | .error.code // empty' 2>/dev/null)
 [ -z "$EX" ] && EX=$(printf '%s' "$BX" | sed -n 's/^data:[[:space:]]*//p' | jq -r
'select(.id==44) | .error.code // empty' 2>/dev/null | head -n 1)
 echo "encoded matching Mcp-Name: HTTP ${CM:-none}"
 echo "encoded mismatching Mcp-Name: HTTP ${CX:-none} jsonrpc_error ${EX:-none}"
 case "$CX" in
 2*) echo "FAIL: an encoded Mcp-Name that decodes to a different value than the body
was accepted" ;;
 400) if [ "$EX" = "-32020" ]; then
 case "$CM" in
 2*) echo "PASS: encoded values are decoded before comparison and encoded
disagreement is rejected with HeaderMismatch (-32020)" ;;
 400) echo "REVIEW: encoded disagreement is rejected but a conformant encoded value
that decodes to the body value is also rejected; the server appears to compare
without decoding, which rejects conformant clients" ;;
 000|"") echo "ERROR: endpoint unreachable or timed out on the matching probe" ;;
 *) echo "REVIEW: unexpected HTTP status ${CM} on the matching probe; inspect before
concluding" ;;
 esac
 elif [ "$EX" = "-32022" ]; then echo "ERROR: endpoint rejected the probe protocol
version; align VER with the approved allowlist and re-run"
 else echo "ERROR: 400 without a HeaderMismatch error body is not attributable to
header validation; if a gateway enforces headers, verify with gateway evidence and
probe the origin directly"
 fi ;;
```

```
 000|"") echo "ERROR: endpoint unreachable or timed out" ;;
 *) echo "REVIEW: unexpected HTTP status ${CX}; inspect the response before
concluding" ;;
 esac
fi
```

A `400` without a `HeaderMismatch` error body is `ERROR`, not a pass, because a gateway may reject with a plain `400` and the specification permits intermediaries to omit the JSON-RPC error body. Because gateway rejection never substitutes for origin validation, verify gateway enforcement with gateway evidence and probe the origin directly. The system is compliant when all three commands print `PASS`.

Remediation:

Configure the MCP server to reject with HTTP `400` and the `HeaderMismatch` error (`-32020`) any request that is missing the `MCP-Protocol-Version` or `Mcp-Method` header, is missing the `Mcp-Name` header on `tools/call`, `resources/read`, or `prompts/get`, or whose header values disagree with the body after Base64 sentinel-encoded values are decoded. Because behavior varies by SDK version and adapter, deploy a server build or SDK version that implements `2026-07-28` request metadata validation and confirm the entry point enforces it with the Audit.

As a layered defense, the gateway can enforce header presence, which needs no body inspection. This layer assumes recommendation 1.1 already enforces an approved protocol version. Without that check, an intermediary acting on the mirrored `Mcp-Method` or `Mcp-Name` headers can be fed unvalidated headers by a downgraded request. The following nginx example is illustrative. The three map directives go in the http context, the maps flag a missing `Mcp-Method` on any request and a missing `Mcp-Name` on the three methods that require it, and the location line places the checks inside the MCP location block ahead of `proxy_pass`:

```
map $http_mcp_method $cis_mcp_method_missing {
    default 0;
    "" 1;
}
map $http_mcp_method $cis_mcp_name_required {
    default 0;
    "tools/call" 1;
    "resources/read" 1;
    "prompts/get" 1;
}
map "$cis_mcp_name_required:$http_mcp_name" $cis_mcp_name_missing {
    default 0;
    "1:" 1;
}
server {
    location /mcp { if ($cis_mcp_method_missing) { return 400; } if
($cis_mcp_name_missing) { return 400; } proxy_pass http://mcp_upstream; }
}
```

Apply with a configuration test and reload, for example `nginx -t && systemctl reload nginx`. The gateway enforces presence only. It cannot compare header values to the body, so it does not replace server-side consistency validation. When the gateway layer is deployed, run the Audit against the origin endpoint directly, because the gateway intercepts the missing-header probes with a plain `400` that the Audit records as `ERROR`. Confirm all three commands print `PASS` against the origin.

Default Value:

The `2026-07-28` revision requires the `MCP-Protocol-Version`, `Mcp-Method`, and `Mcp-Name` headers on the requests that carry them, and requires servers that process the body to reject a missing or disagreeing header. Clients built on current Tier 1 SDKs send these headers, though browser clients, hand-rolled clients, and cross-era bridges may not. Server-side enforcement is more uneven: it is present in some current SDKs, was added only recently in others, and the header-absent path in particular has been observed to skip validation, including a reference implementation that serves a header-absent request with HTTP `200`. Verify the specific server build in use with the Audit rather than assuming enforcement.

References:

1. MCP specification (Base Protocol / Transports / Streamable HTTP, request metadata and server validation): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http
2. MCP specification (Key Changes, SEP-2243 header requirement and -32020 allocation): https://modelcontextprotocol.io/specification/2026-07-28/changelog
3. SEP-2243 (HTTP Header Standardization for Streamable HTTP Transport): https://modelcontextprotocol.io/seps/2243-http-standardization

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

2.5 Ensure Origin header is validated on all Streamable HTTP requests to prevent DNS rebinding attacks (Automated)

Profile Applicability:

- Level 1

Description:

Streamable HTTP servers validate the Origin header on every incoming request. When the Origin header is present and is not on the operator-configured allowlist, the server returns HTTP `403`. This is enforced on the request itself, not only on a CORS preflight. Because non-browser MCP clients routinely send no Origin, the specification does not require an absent Origin to be rejected, and a server may accept it or, under a stricter local policy, reject it. The control targets a present, hostile Origin, which is the DNS-rebinding shape. For locally bound servers, bind to localhost and reject non-localhost Origins.

Rationale:

DNS rebinding lets a remote web page re-resolve its own domain to a loopback address and then issue requests to a locally bound MCP server, which is why developer-facing localhost deployments are the prime target. Without Origin validation the server cannot distinguish a legitimate same-origin request from a rebinding attack, which enables cross-origin tool invocation. The specification's Streamable HTTP security section requires Origin validation on all incoming connections and a `403` on a present-and-invalid Origin.

Impact:

There is no impact for legitimate clients that send an allowlisted Origin. A legitimate non-browser client that sends no Origin is unaffected where the server accepts an absent Origin, and is rejected only under a stricter local policy that requires one. The control adds strong protection against DNS rebinding, including for localhost-bound developer servers.

Audit:

Send a well-formed Streamable HTTP request carrying a hostile Origin and confirm the server returns HTTP `403` on the request itself, not only on a CORS preflight. Then repeat with an allowlisted Origin and with no Origin. Because acceptance or rejection of an absent Origin does not determine compliance unless policy requires one, record the no-Origin result for compatibility awareness rather than as a pass or fail. Substitute the endpoint URL, the approved protocol version, and an allowlisted Origin. Run the following command:

```
URL="<MCP_ENDPOINT_URL>"
VER="<APPROVED_PROTOCOL_VERSION>"
ALLOWED_ORIGIN="<ALLOWED_ORIGIN>"
HOSTILE_ORIGIN="https://cis-rebinding-probe.example"
BODY=$(jq -nc --arg v "$VER"
'{jsonrpc:"2.0",id:51,method:"tools/list",params:{_meta:{"io.modelcontextprotocol/pro
tocolVersion":$v,"io.modelcontextprotocol/clientInfo":{name:"cis-benchmark-
audit",version:"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}'
2>/dev/null)
if [ -z "$BODY" ]; then
 echo "ERROR: probe build failed"
else
 EV=$(curl -sS -o /dev/null --connect-timeout 5 --max-time 15 -w '%{http_code}' -X
POST "$URL" -H 'Accept: application/json, text/event-stream' -H 'Content-Type:
application/json' -H "MCP-Protocol-Version: $VER" -H 'Mcp-Method: tools/list' -H
"Origin: $HOSTILE_ORIGIN" --data-binary "$BODY" 2>/dev/null)
 GD=$(curl -sS -o /dev/null --connect-timeout 5 --max-time 15 -w '%{http_code}' -X
POST "$URL" -H 'Accept: application/json, text/event-stream' -H 'Content-Type:
application/json' -H "MCP-Protocol-Version: $VER" -H 'Mcp-Method: tools/list' -H
"Origin: $ALLOWED_ORIGIN" --data-binary "$BODY" 2>/dev/null)
 NO=$(curl -sS -o /dev/null --connect-timeout 5 --max-time 15 -w '%{http_code}' -X
POST "$URL" -H 'Accept: application/json, text/event-stream' -H 'Content-Type:
application/json' -H "MCP-Protocol-Version: $VER" -H 'Mcp-Method: tools/list' --data-
binary "$BODY" 2>/dev/null)
 echo "hostile_origin=$EV allowed_origin=$GD no_origin=$NO"
 if [ "$EV" = "000" ] || [ "$GD" = "000" ] || [ "$NO" = "000" ]; then
 echo "ERROR: endpoint unreachable or timed out on one or more probes"
 elif [ "$EV" = "400" ] || [ "$GD" = "400" ] || [ "$NO" = "400" ]; then
 echo "ERROR: a probe was rejected 400 before Origin was evaluated (likely a routing-
header mismatch); align the probe headers and re-run"
 elif [ "$EV" = "403" ]; then
 case "$GD" in
 2*|3*) echo "PASS: the hostile Origin is rejected with 403 on the request itself and
the allowlisted Origin is accepted" ;;
 403) echo "REVIEW: the allowlisted Origin is also rejected with 403; confirm
ALLOWED_ORIGIN matches the server's allowlist exactly" ;;
 *) echo "REVIEW: hostile Origin rejected but the allowlisted Origin returned $GD;
inspect before concluding" ;;
 esac
 else
 echo "FAIL: the hostile Origin was not rejected with 403 (got $EV) on the request
itself"
 fi
fi
```

The system is compliant when the hostile Origin is rejected with `403` on the actual request and the allowlisted Origin is accepted. Whether the server accepts or rejects a request with no Origin is informational, not part of the verdict. A probe rejected with `400` indicates a routing-header mismatch rather than an Origin decision, so correct the probe headers and re-run.

Remediation:

Configure the MCP server to validate the Origin header on every Streamable HTTP request and return HTTP `403` for any present Origin not on an explicit allowlist, logging the rejection as a security event. For locally bound servers, bind to localhost and reject non-localhost Origins. As layered defense, the gateway can enforce the allowlist. The following nginx example is illustrative:

```
map $http_origin $cis_mcp_origin_ok {
    default 0;
    "https://app.example.com" 1;
}
server {
    location /mcp { if ($cis_mcp_origin_ok) { set $cis_block 0; } if ($http_origin)
{ set $cis_block "${cis_block}1"; } if ($cis_block = 1) { return 403; } proxy_pass
http://mcp_upstream; }
}
```

The gateway enforces the allowlist for present Origins but does not replace server-side validation, which the specification requires. Where a web UI is served alongside the MCP endpoint, set an explicit `Access-Control-Allow-Origin` allowlist, do not combine `Access-Control-Allow-Credentials: true` with a wildcard origin, and set a restrictive `Content-Security-Policy: frame-ancestors`.

Default Value:

By default, most MCP server frameworks do not validate the `Origin` header, and all incoming Streamable HTTP requests are accepted regardless of `Origin`.

References:

1. MCP specification (Base Protocol / Transports / Streamable HTTP, security and endpoint): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http
2. MCP specification (security best practices, SSRF and DNS rebinding): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	13.4 Perform Traffic Filtering Between Network Segments Perform traffic filtering between network segments, where appropriate.		●	●

3 Authentication & Authorization

MCP servers act on behalf of users and models, often reaching downstream APIs and data with real privilege, so establishing who is calling and constraining what they may do is critical. Weak or absent authentication lets an unapproved caller reach tools directly, and coarse authorization lets an authenticated caller do far more than the task requires. OAuth adds failure modes including tokens replayed against the wrong resource, confused-deputy abuse of a shared client, over-broad scopes, and forged discovery metadata.

The Recommendations in this section cover both halves of the problem. For authentication, they require modern authentication for HTTP transports and safe credential sourcing for local servers. For authorization, they enforce per-tool policies and forbid passing a caller's token directly to downstream services. They then harden OAuth specifically, binding tokens to the MCP server as their audience, applying confused-deputy safeguards, minimizing and progressively elevating scopes, prohibiting shared downstream service identities, and validating discovery metadata over TLS.

3.1 Authentication and Credential Sourcing

3.1.1 Ensure stdio server credentials are sourced from the environment or OS credential store (Automated)

Profile Applicability:

- Level 1

Description:

Source the stdio server's own credentials from environment variables, an OS keychain or secret store, or injected process credentials, and do not use MCP's HTTP browser-based authorization flow to authenticate the client-to-server stdio connection. This does not prohibit a tool or server from independently using OAuth to access a downstream service. Under the `2026-07-28` specification, implementations using an stdio transport do not follow the HTTP authorization specification and instead retrieve credentials from the environment.

Rationale:

Using MCP's interactive HTTP authorization flow for a stdio connection cannot work and creates a credential-exposure risk, because a stdio server has no browser and no redirect URI. Sourcing the server's own secrets from the environment or an OS secret store keeps them out of command lines, process listings, and logs. A stdio server that acts as a client of a downstream service may still use OAuth for that downstream call, which is out of scope for the client-to-server stdio connection this Recommendation covers.

Impact:

There is no impact on functionality. Sourcing credentials from the environment keeps them out of process listings and shell history.

Audit:

Inspect the stdio server's launch configuration and process environment, and confirm that credentials come from the environment or a secret store and that no interactive OAuth flow authenticates the stdio connection. The commands assume systemd, so adapt them to the process manager. Run the following command to confirm no interactive or browser OAuth is configured. The command reads the unit first and treats a missing unit as an error:

```
UNIT="<MCP_STDIO_UNIT>"
unit_text=$(systemctl cat "$UNIT" 2>/dev/null) || { echo "ERROR: cannot read unit
$UNIT (does it exist? is this a systemd deployment?)"; exit 0; }
[ -n "$unit_text" ] || { echo "ERROR: unit $UNIT returned no configuration"; exit
0; }
if printf '%s' "$unit_text" | grep -iqE 'oauth|authorization_code|redirect_uri|--
browser|open-browser'; then
  printf '%s\n' "$unit_text" | grep -inE 'oauth|authorization_code|redirect_uri|--
browser|open-browser'
  echo "FAIL: interactive-auth indicators in launch config (inspect above)"
else
  echo "PASS: no interactive OAuth indicators in launch config"
fi
```

Run the following command to confirm the credential is injected from the environment or secret store, not the command line. The command also treats a missing unit as an error:

```
UNIT="<MCP_STDIO_UNIT>"
CRED_VAR_NAME="<CRED_VAR_NAME>"
unit_text=$(systemctl cat "$UNIT" 2>/dev/null) || { echo "ERROR: cannot read unit
$UNIT"; exit 0; }
env_line=$(systemctl show "$UNIT" -p Environment 2>/dev/null)
if printf '%s' "$env_line" | grep -q "$CRED_VAR_NAME"; then echo "PASS: credential
supplied via environment"
else echo "REVIEW: credential not found in unit Environment (check secret-store
injection)"; fi
if printf '%s' "$unit_text" | grep -E '^ExecStart=' | grep -qiE "$CRED_VAR_NAME|--
token|--api-key"; then
  echo "FAIL: credential appears on the command line (visible in process list)"
else echo "PASS: no credential on the command line"; fi
```

The system is compliant when no interactive OAuth indicators appear, no credential appears on the command line, and the credential is supplied through the environment or a secret store. A missing or unreadable unit is `ERROR`. A `REVIEW` on the environment-presence check means the credential was not visible in the unit `Environment`, and the reviewer confirms whether a secret-store wrapper injects it at launch.

Remediation:

Configure stdio servers to receive credentials from the environment or an OS secret store, not through an interactive OAuth flow or a command-line argument. The following example uses `secret-tool` from libsecret, so adapt it to your store. Place the export in the unit's environment setup (an `EnvironmentFile` or wrapper script), not on the command line, then exec the server. Run the following command to retrieve the credential and export it before launch:

```
export MCP_API_TOKEN="$(secret-tool lookup app mcp-server)"
```

Remove any interactive OAuth configuration and any credential passed as a command-line argument. This does not affect a downstream OAuth client the server uses to reach a third-party service. Restart the server and re-run the Audit.

Default Value:

By default, stdio MCP servers may be launched with browser-style OAuth redirect flows or with credentials passed via command-line arguments, neither of which suits an automated subprocess.

References:

1. MCP specification (Base Protocol / Authorization, environment-based credentials for stdio): https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization
2. MCP specification (security best practices, local server credential handling): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●

3.1.2 Ensure OIDC/OAuth 2.1 or short-lived API tokens are used for remote servers (Manual)

Profile Applicability:

- Level 1

Description:

Remote (HTTP) MCP servers use transport-level authentication, OAuth 2.1/OIDC for user access or the OAuth `client_credentials` flow for service-to-service access. Tokens are scoped and audience-bound narrowly (see the Resource Indicators Recommendation), token lifetimes are kept short, and rotation is automated. Where a deployment uses short-lived API tokens instead of interactive OAuth, those tokens are audience-bound to this server, validated server-side, issued by an enterprise-controlled issuer, and never long-lived static keys. Under the `2026-07-28` revision, clients validate the `iss` parameter on authorization responses. Client registration uses Client ID Metadata Documents as the standard mechanism, where the client identifies itself with a portable HTTPS URL rather than a per-server registration, with pre-registration and Dynamic Client Registration retained for backwards compatibility. A client that registers through Dynamic Client Registration or pre-registration declares its OpenID Connect `application_type`, so the authorization server does not default a desktop or CLI client to web and reject its localhost redirect URI. It also binds registered credentials to the issuing authorization server's issuer, re-registering when a resource migrates between authorization servers rather than sending credentials to the wrong party.

Rationale:

Strong authentication limits the blast radius of token theft and cross-service misuse. Short token lifetimes bound the window in which a stolen token is useful, and audience binding prevents a token issued for one resource from being replayed at another. `iss` validation mitigates a class of mix-up attack that is more prevalent in MCP's single-client, many-server pattern.

Impact:

Adopting OAuth 2.1/OIDC or the `client_credentials` flow is a moderate one-time integration effort, standing up or connecting to an authorization server, registering the client, and wiring token validation into the request path. The security value is high: it establishes who is calling and bounds the damage from a stolen token through short lifetimes and audience binding.

Audit:

Review the server authentication middleware, token lifetimes, and audiences. Confirm unauthenticated requests are rejected and issued tokens are short-lived. Audience binding is assessed under Recommendation 3.3.1. Run the following command to confirm an unauthenticated request returns `401`, ideally with a `WWW-Authenticate` header pointing at resource metadata:

```
URL="https://<SERVER_URL>/mcp"
PROTO="2026-07-28"
code=$(curl -sS -o /dev/null -w '%{http_code}' -X POST "$URL" -H 'Content-Type:
application/json' -H 'Accept: application/json, text/event-stream' -H "MCP-Protocol-
Version: $PROTO" -H 'Mcp-Method: tools/list' -d
"{\"jsonrpc\":\"2.0\",\"id\":1,\"method\":\"tools/list\",\"params\":{\"_meta\":{\"io.
modelcontextprotocol/protocolVersion\":\"$PROTO\",\"io.modelcontextprotocol/clientInf
o\":{\"name\":\"cis-benchmark-audit\",\"version\":\"1.0\"}}}}")
case "$code" in
 401) echo "PASS: unauthenticated request rejected with 401" ;;
 403) echo "REVIEW: unauthenticated request returned 403, expected 401 for missing
credentials, confirm this is a deliberate gateway policy" ;;
 000) echo "ERROR: server unreachable (not a PASS)" ;;
 2*) echo "FAIL: unauthenticated request accepted ($code)" ;;
 *) echo "REVIEW: returned $code" ;;
esac
```

The `aud` claim is shown for context, and its validation is assessed under Recommendation 3.3.1. Run the following command to decode a live JWT access token and confirm it is short-lived:

```
TOKEN='<JWT_ACCESS_TOKEN>'
seg=$(printf '%s' "$TOKEN" | cut -d. -f2 | tr '_-' '/+')
pad=$(( (4 - ${#seg} % 4) % 4 )); seg="$seg$(printf '=%.0s' $(seq 1 $pad))"
claims=$(printf '%s' "$seg" | base64 -d 2>/dev/null)
[ -z "$claims" ] && { echo "ERROR: could not decode token (opaque token? use
introspection)"; exit 0; }
printf '%s' "$claims" | jq empty 2>/dev/null || { echo "ERROR: decoded payload is not
JSON (opaque token? use introspection)"; exit 0; }
echo "$claims" | jq '{iss, aud, exp, iat, scope, scp}'
now=$(date +%s)
exp=$(echo "$claims" | jq -r '.exp // empty')
iat=$(echo "$claims" | jq -r '.iat // empty')
[ -z "$exp" ] && { echo "REVIEW: token has no exp claim, lifetime cannot be
assessed"; exit 0; }
remaining=$(( exp - now )); echo "remaining_seconds=$remaining"
[ "$remaining" -le 0 ] && echo "note: sampled token is already expired, remaining
life is not the issued lifetime"
if [ -n "$iat" ]; then
  lifetime=$(( exp - iat )); echo "issued_lifetime_seconds=$lifetime"
  if [ "$lifetime" -gt 0 ] && [ "$lifetime" -le 3600 ]; then echo "PASS: issued token
lifetime is within the 1h guideline"
  else echo "REVIEW: issued token lifetime is $lifetime (longer than the 1h
guideline, or non-positive)"; fi
else
  echo "REVIEW: token has no iat claim, issued lifetime cannot be computed from the
token; confirm expires_in from the token endpoint response is short-lived"
fi
```

Confirm `iss` validation on authorization responses by inspecting the authorization-server metadata and the client since the resource-server boundary cannot exercise it. In the metadata, confirm `authorization_response_iss_parameter_supported` is `true` and that the server emits `iss`. In the client, confirm it records the expected issuer before redirecting and rejects a response whose `iss` does not match, or is absent when the server advertises support. The system is compliant when unauthenticated requests are rejected with `401`, issued tokens are short-lived, and the client validates `iss`.

Remediation:

Configure remote MCP servers to require OAuth 2.1/OIDC, or `client_credentials` for machine-to-machine access. They issue short-lived audience-bound tokens and validate iss on authorization responses. This spans the client, server, and authorization-server layers with no single runnable command. Set a short access-token TTL (a common baseline is `<= 1 hour`) with refresh-token rotation, require PKCE for public clients, and enforce audience and `iss` validation on the MCP server. Register clients through Client ID Metadata Documents where supported. A client using Dynamic Client Registration or pre-registration declares its OpenID Connect `application_type` and binds registered credentials to the issuer, re-registering on resource migration. Record these as authorization-server and server configuration in the enterprise registry. Verify with the Audit.

Default Value:

By default, many remote MCP deployments use long-lived static tokens or no authentication, and OAuth 2.1/OIDC is not pre-configured in most SDK templates.

References:

1. MCP specification (Base Protocol / Authorization): https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization
2. MCP specification (Key Changes, authorization hardening): https://modelcontextprotocol.io/specification/2026-07-28/changelog
3. SEP-837 (application_type requirement in Dynamic Client Registration): https://github.com/modelcontextprotocol/modelcontextprotocol/pull/837
4. SEP-2352 (Clarify authorization server binding and migration): https://github.com/modelcontextprotocol/modelcontextprotocol/pull/2352
5. SEP-2468 (Recommend Issuer iss Parameter in MCP Auth Responses): https://modelcontextprotocol.io/seps/2468-recommend-issuer-claim-for-auth
6. RFC 9207 (OAuth 2.0 Authorization Server Issuer Identification): https://datatracker.ietf.org/doc/html/rfc9207

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.7 Centralize Access Control Centralize access control for all enterprise assets through a directory service or SSO provider, where supported.		●	●

3.2 Authorization Enforcement

3.2.1 Ensure per-tool authorization policies are enforced (Manual)

Profile Applicability:

- Level 2

Description:

Authorization for tool invocation is deny-by-default: tools are allowlisted per role or identity, and every authorization decision (allow and deny) is logged with the tool name, calling identity, the policy rule that matched, a timestamp, and a request or correlation identifier that ties the decision to the invocation. Authorization is enforced server-side or at the gateway, never in the model or client.

Rationale:

Tools can have powerful side effects, and without per-tool authorization any caller that reaches the server can invoke every tool it exposes, including destructive, irreversible, or data-exfiltrating ones. Role-based, deny-by-default authorization limits which identities can invoke which tools, so a compromised client, or a confused or injected model, is confined to the tools its identity is granted rather than the full set.

Impact:

This adds ongoing policy-management overhead to define and maintain each tool's access policy, justified by preventing an over-permissioned caller from invoking every tool the server exposes.

Audit:

Compare the tool registry to the role-based authorization policy, then confirm a least-privileged identity is denied a privileged tool that an authorized identity is allowed. Both decisions are logged and correlated to the controlled requests. Set `TOOL`, `ARGS`, `AUTH_TOKEN` (an identity the policy permits, the positive control), and `LP_TOKEN` (a least-privileged identity that should be denied). Set `AUTHZ_ERR_JQ` to the boolean matching the deployment's structured authorization-denial error, for example `.error.code==<AUTHZ_DENY_CODE>`. Run the following command:

```
URL="https://<SERVER_URL>/mcp"
TOOL="<PRIVILEGED_TOOL>"
ARGS='<VALID_ARGUMENTS_JSON>'
AUTH_TOKEN="<AUTHORIZED_TOKEN>"
LP_TOKEN="<LEAST_PRIVILEGED_TOKEN>"
PROTO="2026-07-28"
AUTHZ_ERR_JQ="${AUTHZ_ERR_JQ:-.error.code==-32003}"
CORR=$(head -c16 /dev/urandom 2>/dev/null | od -An -tx1 | tr -d ' \n'); [ -n
"$CORR" ] || CORR=$(date +%s%N | sha1sum | cut -c1-32)
echo "$CORR" > /tmp/cis_3_3_corr
TP="00-$CORR-$(printf '%016x' 1)-01"
mkbody() { printf
'{"jsonrpc":"2.0","id":1,"method":"tools/call","params":{"name":"%s","arguments":%s,"
_meta":{"io.modelcontextprotocol/protocolVersion":"%s","io.modelcontextprotocol/trace
parent":"%s","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"}}}}' "$1" "$2" "$PROTO" "$TP"; }
norm() { case "$(printf '%s' "$1" | sed -e 's/^[[:space:]]*//' | cut -c1)" in '{')
printf '%s' "$1";; *) printf '%s' "$1" | sed -n 's/^data: //p' | while IFS= read -r
l; do printf '%s' "$l" | jq -e 'select(.id==1)' 2>/dev/null && break; done;; esac; }
bf=$(mktemp)
pcs=$(curl -sS --connect-timeout 5 -m 10 -o "$bf" -w '%{http_code}' -X POST "$URL" -H
'Content-Type: application/json' -H 'Accept: application/json, text/event-stream' -H
"Authorization: Bearer $AUTH_TOKEN" -H "MCP-Protocol-Version: $PROTO" -H 'Mcp-Method:
tools/call' -H "Mcp-Name: $TOOL" -d "$(mkbody "$TOOL" "$ARGS")" 2>/dev/null)
pc=$(norm "$(cat "$bf")")
if [ "$pcs" != "200" ] || ! printf '%s' "$pc" | jq -e '.result and
(.error|not)' >/dev/null 2>&1; then
  echo "ERROR: positive control failed (authorized identity status=$pcs), transport
or auth broken, cannot attribute the denial"; rm -f "$bf"; exit 0
fi
ds=$(curl -sS --connect-timeout 5 -m 10 -o "$bf" -w '%{http_code}' -X POST "$URL" -H
'Content-Type: application/json' -H 'Accept: application/json, text/event-stream' -H
"Authorization: Bearer $LP_TOKEN" -H "MCP-Protocol-Version: $PROTO" -H 'Mcp-Method:
tools/call' -H "Mcp-Name: $TOOL" -d "$(mkbody "$TOOL" "$ARGS")" 2>/dev/null)
d=$(norm "$(cat "$bf")"); rm -f "$bf"
if [ "$ds" = "403" ]; then echo "PASS: least-privileged identity denied by
attributable authorization response (403)"
elif printf '%s' "$d" | jq -e "$AUTHZ_ERR_JQ" >/dev/null 2>&1; then echo "PASS:
least-privileged identity denied by documented authorization error"
elif [ "$ds" = "200" ] && printf '%s' "$d" | jq -e '.result and
(.error|not)' >/dev/null 2>&1; then echo "FAIL: least-privileged identity invoked the
privileged tool"
else de=$(printf '%s' "$d" | jq -r '.error.code // "none"'); echo "ERROR: denial not
attributable to authorization (status=$ds, error=$de)"; fi
```

Run the following command to confirm both decisions were logged and correlated to the controlled requests. Set `LOG` to the authorization decision log and adjust the `jq` selectors to the deployment's log schema. The command scopes to the correlation identifier from the previous command:

```
LOG="<MCP_AUDIT_LOG>"
CORR=$(cat /tmp/cis_3_3_corr 2>/dev/null)
[ -n "$CORR" ] || { echo "ERROR: no correlation id from the wire audit, run the wire
block first"; exit 0; }
scoped=$(grep -F "$CORR" "$LOG" 2>/dev/null)
n=$(printf '%s\n' "$scoped" | sed '/^$/d' | wc -l | tr -d ' ')
echo "count: $n log records match this audit's correlation id"
allow=$(printf '%s\n' "$scoped" | jq -c 'select(.event=="authz_allow" and .tool!=null
and .identity!=null and .rule!=null and .ts!=null and .corr!=null)' 2>/dev/null |
head -1)
deny=$(printf '%s\n' "$scoped" | jq -c 'select(.event=="authz_deny" and .tool!=null
and .identity!=null and .rule!=null and .ts!=null and .corr!=null)' 2>/dev/null |
head -1)
if [ -n "$allow" ] && [ -n "$deny" ]; then echo "PASS: both allow and deny decisions
logged for the controlled requests with tool, identity, rule, timestamp, and
correlation id"
elif [ -n "$allow" ] || [ -n "$deny" ]; then echo "REVIEW: only one of the allow or
deny decisions was logged with the required fields for this audit"
else echo "FAIL: no correlated authorization decision records with the required
fields"; fi
```

The system is compliant when a least-privileged identity is denied a privileged tool by an attributable authorization response (a `403` or the documented authorization error) while an authorized identity is allowed the same tool under the same request, and both decisions are logged with tool, identity, matched rule, timestamp, and correlation identifier. A denial not attributable to authorization is `ERROR`.

Remediation:

Configure the server or gateway to enforce deny-by-default per-tool authorization keyed to role and identity, and to log every decision. This is application, server, or gateway logic with no portable config or command. Define a deny-by-default policy that allowlists tools per role, map identities to roles from the authenticated token, and reference the enterprise tool risk classification rather than server-provided annotations (see Recommendation 3.2.3). For every invocation, emit an authorization decision log entry (allow and deny) with the tool name, calling identity, matched rule, timestamp, and a correlation identifier. Verify with the Audit using least-privileged and authorized test identities.

Default Value:

By default, MCP servers expose every registered tool to any authenticated caller, with no per-tool role or identity checks. Even where authentication is in place, the specification provides no per-tool authorization, so deny-by-default access is not pre-configured.

References:

1. MCP specification (Base Protocol / Authorization): https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization
2. MCP specification (Base Protocol / Transports / Streamable HTTP, MCP-Protocol-Version, Mcp-Method, and Mcp-Name headers): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http
3. MCP specification (security best practices, least privilege and per-operation authorization): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.8 Define and Maintain Role-Based Access Control Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●

3.2.2 Ensure token passthrough to downstream APIs is forbidden (Manual)

Profile Applicability:

- Level 1

Description:

An MCP server never forwards a client-presented inbound token to a downstream API. Under the MCP authorization specification, the server accepts only tokens whose audience is the MCP server itself. To call a downstream API, the server presents a separate token whose audience is that downstream API, never the inbound token. It obtains that token through OAuth token exchange when acting on behalf of a user, or through `client_credentials` when acting as itself. Because the inbound token's audience is the MCP server and not the downstream API, it is never valid there.

Rationale:

The MCP authorization specification forbids token passthrough because it circumvents security controls such as rate limiting, request validation, and audience checks, breaks the audit trail by making downstream logs record the wrong origin identity, and widens the trust boundary so a token accepted at one service is honored at others.

Impact:

Enforcing this may require a one-time gateway or middleware change so the server acquires a separate downstream token rather than forwarding the inbound token.

Audit:

Inspect the proxy or gateway configuration and the token-handling middleware, and confirm the token presented downstream is a separate token audience-bound to the downstream API, not the inbound token. Run the following command to decode the inbound token and confirm its `aud` includes the MCP server and not the downstream API:

```
TOKEN='<CLIENT_INBOUND_JWT>'
MCP_SERVER_AUDIENCE="<MCP_SERVER_AUDIENCE>"
DOWNSTREAM_API_AUDIENCE="<DOWNSTREAM_API_AUDIENCE>"
seg=$(printf '%s' "$TOKEN" | cut -d. -f2 | tr '_-' '/+')
pad=$(( (4 - ${#seg} % 4) % 4 )); seg="$seg$(printf '=%.0s' $(seq 1 $pad))"
claims=$(printf '%s' "$seg" | base64 -d 2>/dev/null)
[ -z "$claims" ] && { echo "ERROR: could not decode token (opaque token? use AS
introspection)"; exit 0; }
printf '%s' "$claims" | jq empty 2>/dev/null || { echo "ERROR: decoded payload is not
JSON (opaque token? use AS introspection)"; exit 0; }
echo "$claims" | jq '{aud, iss}'
echo "$claims" | jq -e --arg mcp "$MCP_SERVER_AUDIENCE" --arg down
"$DOWNSTREAM_API_AUDIENCE" '
 (.aud | if type=="array" then . else [.] end) as $a
 | ($a | index($mcp)) != null
 and ($a | index($down)) == null' >/dev/null \
 && echo "PASS: client token is bound to the MCP server, not the downstream API" \
 || echo "REVIEW: client token audience does not match the expected MCP-only binding
(missing the MCP audience, or including the downstream API)"
```

Run the following command to confirm the inbound token does not appear in egress to the downstream host. The command scopes to that host, confirms that host captures Authorization headers, and matches the inbound token by fingerprint, not by raw bearer string:

```
LOG="<MCP_EGRESS_LOG>"
TOKEN='<CLIENT_INBOUND_JWT>'
DOWNSTREAM_HOST="<DOWNSTREAM_HOST>"
[ -s "$LOG" ] || { echo "ERROR: egress log not found or empty (capture egress for the
controlled request)"; exit 0; }
FP=$(printf '%s' "$TOKEN" | sha256sum | cut -c1-32)
[ -n "$FP" ] || { echo "ERROR: cannot compute token fingerprint (sha256sum
unavailable)"; exit 0; }
scope=$(grep -F "$DOWNSTREAM_HOST" "$LOG")
[ -n "$scope" ] || { echo "REVIEW: no egress lines to $DOWNSTREAM_HOST in the log
window, cannot assess passthrough to this destination"; exit 0; }
printf '%s' "$scope" | grep -qE 'Bearer |Authorization:' || { echo "REVIEW: egress to
$DOWNSTREAM_HOST does not capture Authorization headers, so absence of the token
cannot prove no passthrough"; exit 0; }
hit=""
while IFS= read -r line; do
 bt=$(printf '%s' "$line" | sed -n 's/.*[Bb]earer \([A-Za-z0-9._-]*\).*/\1/p')
 [ -z "$bt" ] && continue
 h=$(printf '%s' "$bt" | sha256sum | cut -c1-32)
 [ "$h" = "$FP" ] && hit=1
done <<EOF
$scope
EOF
if [ -n "$hit" ]; then echo "FAIL: inbound client token found by fingerprint in
egress to $DOWNSTREAM_HOST (passthrough)"
else echo "PASS: inbound client token not present in egress to $DOWNSTREAM_HOST and
that egress captures Authorization headers"; fi
```

The system is compliant when the inbound token is bound to the MCP server and not the downstream API, and does not appear by fingerprint in egress to a downstream destination that is demonstrably capturing Authorization headers. A reviewer adjudicates any `REVIEW`.

Remediation:

This is server or gateway logic with no portable command. On an inbound request, validate that the token's audience is this MCP server and reject it otherwise. To call a downstream API, obtain a separate token whose audience is that downstream API, using token exchange when acting on behalf of a user or `client_credentials` when the server acts as itself, and never forward the inbound token. Pair with audience and issuer alerting (see Recommendation 7.2.1) so passthrough or wrong-audience tokens raise an alert in production.

Default Value:

By default, deployments do not restrict token passthrough. A client-presented token may be forwarded downstream unless blocked by application code or gateway rules.

References:

1. MCP specification (Base Protocol / Authorization, token audience restriction): https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization
2. MCP specification (security best practices, token passthrough): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

3.2.3 Ensure server-provided tool annotations are not relied upon for authorization or HITL gating decisions (Manual)

Profile Applicability:

- Level 1

Description:

Authorization policies, RBAC rules, and human-in-the-loop (HITL) gating treat MCP tool annotations (`readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`) and any other server-provided behavior annotation as untrusted and exclude them from access-control decisions. The specification requires clients to consider tool annotations untrusted unless they come from a trusted server. All authorization and HITL decisions instead rest on an enterprise-controlled tool risk classification that maps tool names and identifiers to approved risk tiers. Annotations from verified internal servers may inform UI decisions (for example, showing a confirmation dialog) but never substitute for enforcement.

Rationale:

Tool annotations are unsigned, server-provided metadata that the client or host consumes without independent validation, so a compromised, misconfigured, or malicious server can set `readOnlyHint: true` on a tool that performs destructive writes or `openWorldHint: false` on a tool that reaches external systems. Any authorization or HITL logic that trusts the annotation then misclassifies the tool as safe and skips approval gates. Enterprise authorization therefore rests on operator-controlled classification, not server-asserted hints. The Tools specification is explicit that clients treat tool annotations as untrusted unless they come from a trusted server, so this Recommendation audits at the enterprise policy layer what the specification already requires the client to enforce.

Impact:

Maintaining the enterprise tool risk classification is ongoing effort since the registry must be updated as tools are added or their behavior changes.

Audit:

Run the following command to confirm that a destructive tool falsely advertising `readOnlyHint: true` is still gated, attributable to the classification rather than to a malformed request. Set `SAFE_TOOL` to a permitted tool with valid arguments (the positive control) and `DEST_TOOL` to a tool classified destructive in the registry that advertises `readOnlyHint: true`. Set `GATING_JQ` to the deployment's structured HITL or authorization signal, for example `.result.hitl_required==true`:

```
URL="https://<SERVER_URL>/mcp"
SAFE_TOOL="<PERMITTED_TOOL>"
SAFE_ARGS='<VALID_SAFE_ARGUMENTS_JSON>'
DEST_TOOL="<DESTRUCTIVE_TOOL_FALSELY_ANNOTATED_READONLY>"
DEST_ARGS='<VALID_DEST_ARGUMENTS_JSON>'
AUTH_TOKEN="<AUTH_TOKEN>"
PROTO="2026-07-28"
GATING_JQ="${GATING_JQ:-.result.hitl_required==true}"
normalize() {
  raw="$1"
  case "$(printf '%s' "$raw" | sed -e 's/^[[:space:]]*//' | cut -c1)" in
    '{') printf '%s' "$raw" ;;
    *) printf '%s' "$raw" | sed -n 's/^data: //p' | while IFS= read -r line; do
printf '%s' "$line" | jq -e 'select(.id==1)' 2>/dev/null && break; done ;;
  esac
}
mkbody() { printf
'{"jsonrpc":"2.0","id":1,"method":"tools/call","params":{"name":"%s","arguments":%s,"
_meta":{"io.modelcontextprotocol/protocolVersion":"%s","io.modelcontextprotocol/clien
tInfo":{"name":"cis-benchmark-audit","version":"1.0"}}}}' "$1" "$2" "$PROTO"; }
bfile=$(mktemp)
pcs=$(curl -sS --connect-timeout 5 -m 10 -o "$bfile" -w '%{http_code}' -X POST "$URL"
-H 'Content-Type: application/json' -H 'Accept: application/json, text/event-stream'
-H "Authorization: Bearer $AUTH_TOKEN" -H "MCP-Protocol-Version: $PROTO" -H 'Mcp-
Method: tools/call' -H "Mcp-Name: $SAFE_TOOL" -d "$(mkbody "$SAFE_TOOL"
"$SAFE_ARGS")" 2>/dev/null)
pc=$(normalize "$(cat "$bfile")")
if [ "$pcs" != "200" ] || ! printf '%s' "$pc" | jq -e '.result and
(.error|not)' >/dev/null 2>&1; then
  echo "ERROR: positive control failed (permitted tool status=$pcs), transport or
auth broken, cannot attribute gating"
  rm -f "$bfile"; exit 0
fi
ds=$(curl -sS --connect-timeout 5 -m 10 -o "$bfile" -w '%{http_code}' -X POST "$URL"
-H 'Content-Type: application/json' -H 'Accept: application/json, text/event-stream'
-H "Authorization: Bearer $AUTH_TOKEN" -H "MCP-Protocol-Version: $PROTO" -H 'Mcp-
Method: tools/call' -H "Mcp-Name: $DEST_TOOL" -d "$(mkbody "$DEST_TOOL"
"$DEST_ARGS")" 2>/dev/null)
d=$(normalize "$(cat "$bfile")"); rm -f "$bfile"
if [ "$ds" = "403" ]; then echo "PASS: destructive tool gated by attributable
authorization denial (403)"
elif printf '%s' "$d" | jq -e "$GATING_JQ" >/dev/null 2>&1; then echo "PASS:
destructive tool gated by documented HITL or authorization signal"
elif [ "$ds" = "200" ] && printf '%s' "$d" | jq -e '.result and
(.error|not)' >/dev/null 2>&1; then echo "FAIL: destructive tool executed without
gating, annotation was trusted"
else de=$(printf '%s' "$d" | jq -r '.error.code // "none"' 2>/dev/null); echo "ERROR:
destructive-tool response not attributable to gating (status=$ds, error=$de)"; fi
```

Review the policy and HITL gating configuration, whose location is environment-specific. Confirm it references tool names and identifiers from the enterprise registry, and that no rule branches on `readOnlyHint`, `destructiveHint`, `idempotentHint`, or `openWorldHint`. The system is compliant when a destructive tool falsely advertising `readOnlyHint: true` is still gated by an attributable signal while a permitted tool succeeds under the same request, and no rule branches on annotation fields. A response that is neither an attributable gating signal nor a clean execution is `ERROR`.

Remediation:

Configure authorization and human-in-the-loop logic to key on the enterprise tool risk classification, never on server-provided annotations. Remove any rule that branches on `readOnlyHint`, `destructiveHint`, `idempotentHint`, or `openWorldHint`, and replace it with a lookup of the tool name or identifier in the enterprise risk classification registry. Document the risk tier for each approved tool in the capability baseline independently of server metadata. Where a gateway or host performs the gating, apply the classification there as the enforcement point of record. Verify with the Audit using a tool classified destructive that advertises `readOnlyHint: true`, confirming it is still gated.

Default Value:

By default, annotations are set by the MCP server and no validation or signing of annotation values is performed before they are consumed by authorization or HITL logic.

References:

1. MCP specification (Base Protocol / Transports / Streamable HTTP): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http
2. MCP specification (security best practices, annotations are untrusted hints): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices
3. MCP specification (Server Features / Tools): https://modelcontextprotocol.io/specification/2026-07-28/server/tools
4. MCP GitHub discussion 1138 (tool annotation hints): https://github.com/modelcontextprotocol/modelcontextprotocol/discussions/1138
5. MCP specification (security best practices, annotations are untrusted hints): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

3.3 OAuth Token and Metadata Hardening

3.3.1 Ensure OAuth tokens are audience-bound to the MCP server using Resource Indicators (Automated)

Profile Applicability:

- Level 1

Description:

OAuth tokens are bound to the MCP server they are used with, so a token minted for one server cannot be replayed at another. Clients send the Resource Indicators `resource` parameter in both the authorization request and the token request, identifying this MCP server by its canonical URI, so the authorization server mints a token whose audience is this server. The MCP server validates that an inbound token's audience is itself and rejects a token bound to a different resource. Validating a present `iss` parameter on authorization responses is a related client responsibility required under the `2026-07-28` revision, not a resource-server check, and is covered by Recommendation 3.1.2.

Rationale:

Resource Indicators bind a token's audience to a specific resource, so the authorization server issues a token scoped to one MCP server rather than an over-broad credential accepted anywhere. Without this binding, a token obtained for or stolen from one server can be replayed at another that accepts any valid-looking bearer token, letting an attacker pivot from one service to the next on a single credential. This cross-resource replay, or token audience confusion, is more damaging in MCP's single-client, many-server pattern, where one client holds tokens for many servers at once. Audience binding closes it by construction: a token minted for MCP server A cannot be replayed at server B when B validates that the audience is itself.

Impact:

The effort is low, requiring a client request parameter and a server-side audience check. Once the audience check is enforced, tokens not bound to this server, including those from clients that do not send the `resource` parameter, are rejected, so existing integrations may need updating first.

Audit:

Verify that issued tokens carry an `aud` equal to this MCP server and that a token whose audience is a different resource is rejected. Where an OAuth trace is available, also confirm that the `resource` parameter carrying this server's canonical URI appears in both the authorization and token requests, as a supplementary review. Run the following command to decode a token issued for this server and check its `aud`:

```
TOKEN='<JWT_ISSUED_FOR_THIS_SERVER>'
MCP_SERVER_AUDIENCE="<MCP_SERVER_AUDIENCE>"
seg=$(printf '%s' "$TOKEN" | cut -d. -f2 | tr '_-' '/+')
pad=$(( (4 - ${#seg} % 4) % 4 )); seg="$seg$(printf '=%.0s' $(seq 1 $pad))"
claims=$(printf '%s' "$seg" | base64 -d 2>/dev/null)
[ -z "$claims" ] && { echo "ERROR: could not decode token (opaque token? use AS
introspection)"; exit 0; }
printf '%s' "$claims" | jq empty 2>/dev/null || { echo "ERROR: decoded payload is not
JSON (opaque token? use AS introspection)"; exit 0; }
echo "$claims" | jq -e --arg aud "$MCP_SERVER_AUDIENCE" '
 (.aud | if type=="array" then . else [.] end) as $a
 | ($a | index($aud)) != null' >/dev/null \
 && echo "PASS: token audience-bound to this MCP server" \
 || echo "FAIL: token audience does not include this MCP server"
```

Run the following command with a token issued for a different resource and confirm the server rejects it with `401`:

```
URL="https://<SERVER_URL>/mcp"
PROTO="2026-07-28"
WRONG='<JWT_FOR_DIFFERENT_RESOURCE>'
code=$(curl -sS -o /dev/null -w '%{http_code}' -X POST "$URL" -H 'Content-Type:
application/json' -H 'Accept: application/json, text/event-stream' -H "MCP-Protocol-
Version: $PROTO" -H "Authorization: Bearer $WRONG" -H 'Mcp-Method: tools/list' -d
"{\"jsonrpc\":\"2.0\",\"id\":1,\"method\":\"tools/list\",\"params\":{\"_meta\":{\"io.
modelcontextprotocol/protocolVersion\":\"$PROTO\",\"io.modelcontextprotocol/clientInf
o\":{\"name\":\"cis-benchmark-audit\",\"version\":\"1.0\"}}}}")
case "$code" in
  401) echo "PASS: wrong-audience token rejected with 401" ;;
  403) echo "REVIEW: wrong-audience token returned 403, expected 401 for a token not
valid for this resource, confirm this is a deliberate policy" ;;
  000) echo "ERROR: server unreachable (not a PASS)" ;;
  2*)  echo "FAIL: wrong-audience token accepted ($code)" ;;
  *)   echo "REVIEW: returned $code" ;;
esac
```

The system is compliant when a token issued for this server carries `aud` equal to this server and a token bound to a different resource is rejected with `401`. Where a trace is available, the `resource` parameter is present in both OAuth exchanges.

Remediation:

Bind token audience across the client, the authorization server, and the MCP server since no single command configures all three. Configure the client to include the `resource` parameter identifying this server in both the authorization and token requests, and the authorization server to set the token `aud` accordingly. Configure the MCP server to validate that `aud` equals this server on every request and to reject a token bound to a different resource. Client-side `iss` validation is configured on the client under Recommendation 3.1.2. Verify with the Audit.

Default Value:

By default, an authorization server issues tokens without a resource-scoped audience unless the client sends the `resource` parameter, and an MCP server does not validate a token's audience against itself unless configured to.

References:

1. MCP specification (Base Protocol / Authorization, resource parameter and audience validation): https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization
2. RFC 8707 (Resource Indicators for OAuth 2.0): https://datatracker.ietf.org/doc/html/rfc8707
3. RFC 9207 (OAuth 2.0 Authorization Server Issuer Identification): https://datatracker.ietf.org/doc/html/rfc9207
4. SEP-2468 (Recommend Issuer iss Parameter in MCP Auth Responses): https://modelcontextprotocol.io/seps/2468-recommend-issuer-claim-for-auth

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.7 Centralize Access Control Centralize access control for all enterprise assets through a directory service or SSO provider, where supported.		●	●

3.3.2 Ensure OAuth discovery metadata is served over TLS and validated against the approved authorization server list (Automated)

Profile Applicability:

- Level 2

Description:

The OAuth discovery chain for every remote MCP server resolves only to approved authorization servers, over TLS, from documents protected against unauthorized modification. Under the `2026-07-28` revision, a client discovers where to obtain tokens by reading the server's Protected Resource Metadata, reached either through the `resource_metadata` parameter of a `WWW-Authenticate` challenge or, where the challenge is absent, through the well-known protected-resource URIs (the sub-path form first, then the root form). The challenge is optional under this revision, so a server that serves only the well-known document is conformant and the audit treats a missing challenge as normal. The Protected Resource Metadata document names at least one authorization server in `authorization_servers`, and an empty or absent list is protocol-invalid and fails. The client then discovers each authorization server through its own metadata, retrieved by trying the OAuth Authorization Server Metadata forms and the OpenID Connect Discovery forms in turn, including path-insertion and path-append variants where the issuer has a path component. The Protected Resource Metadata document is served only over TLS, its `resource` value matches the server's canonical URI (the RFC 8707 resource identifier the audience-binding Recommendation relies on), and every entry in its `authorization_servers` list appears in a change-controlled export of the enterprise's approved authorization servers. Each listed authorization server's metadata publishes an `issuer` identical to the approved entry by simple string comparison, which is the identity check the revision requires of clients. The files or configuration that generate these documents are writable only by the deployment pipeline, enforced through Remediation and confirmed out of band. An advertised authorization server absent from the approved export is a failure, and an approved authorization server not currently advertised is flagged for review.

Rationale:

The discovery chain is the root of trust for every token-based control in this section. Audience binding, scope minimization, and issuer validation all assume the client obtained its token from the intended authorization server. A single unauthorized entry in `authorization_servers`, introduced through misconfiguration or tampering, redirects the entire authentication flow to a party the enterprise never approved, and every downstream token check then validates tokens from the wrong issuer perfectly. The revision makes the issuer-identity check normative: a client rejects authorization server metadata whose `issuer` differs from the identifier used to construct the discovery URL, so this Recommendation audits at the server what the specification already requires the client to enforce. This is a preventive control at the point of maximum leverage since one metadata document governs the authorization path for every client of that server.

Impact:

The impact is low for a single enterprise identity provider, where the approved export is one line. Deployments with multiple authorization servers maintain the export under change control, and a stale export produces review findings until updated.

Audit:

Verify each remote MCP server's discovery chain against the approved export. The command below checks that a discovery path (the challenge, or the well-known sub-path or root form) serves one consistent Protected Resource Metadata document over TLS whose `resource` matches the canonical URI. Set `ALLOW_PRIVATE` to `1` only for internal or lab deployments whose discovery hosts are legitimately non-public, and leave it unset for public deployments so a URL whose host is a private, loopback, link-local, or other non-public address is refused as a possible SSRF target. This refusal is a lexical check on the URL host and does not resolve DNS, so a hostname resolving to an internal address is not caught. Run the following command, substituting the angle-bracket placeholders:

```
MCP_URL="<MCP_URL>"
CANON="<CANONICAL_URI>"
ALLOW_PRIVATE="${ALLOW_PRIVATE:-0}"
PROTO="2026-07-28"
host_of() {
 printf '%s' "$1" | sed -E 's#^[a-zA-Z]+://\[([^]]+)\].*#\1#; t; s#^[a-zA-
Z]+://([^/@:]+).*#\1#' | tr 'A-F' 'a-f'
}
is_private_host() {
 case "$1" in
 localhost|localhost.*|*.internal|*.local) return 0 ;;
 127.*|0.*|10.*|169.254.*|192.168.*) return 0 ;;
 172.1[6-9].*|172.2[0-9].*|172.3[0-1].*) return 0 ;;
 100.6[4-9].*|100.[789][0-9].*|100.1[01][0-9].*|100.12[0-7].*) return 0 ;;
 ::1|::|0:*) return 0 ;;
 ::ffff:*) return 0 ;;
 fc*:*|fd*:*) return 0 ;;
 fe8*:*|fe9*:*|fea*:*|feb*:*) return 0 ;;
 esac
 return 1
}
guard() {
 gh=$(host_of "$1")
 if [ "$ALLOW_PRIVATE" != "1" ] && is_private_host "$gh"; then
 echo "FAIL: server-provided discovery URL resolves to a private, loopback, link-
local, or mapped host ($gh), set ALLOW_PRIVATE=1 only for internal deployments"
 exit 0
 fi
}
case "$MCP_URL" in https://*) : ;; *) echo "FAIL: MCP endpoint is not served over TLS
($MCP_URL)"; exit 0 ;; esac
ORIGIN=$(printf '%s' "$MCP_URL" | sed -E 's#^(https://[^/]+).*#\1#')
RPATH=$(printf '%s' "$MCP_URL" | sed -E 's#^https://[^/]+##')
WK_SUB="$ORIGIN/.well-known/oauth-protected-resource$RPATH"
WK_ROOT="$ORIGIN/.well-known/oauth-protected-resource"
body="{\"jsonrpc\":\"2.0\",\"id\":1,\"method\":\"tools/list\",\"params\":{\"_meta\":{
\"io.modelcontextprotocol/protocolVersion\":\"$PROTO\"}}}"
chal=$(curl -sS --connect-timeout 5 -m 10 -o /dev/null -D - -X POST "$MCP_URL" -H
"Content-Type: application/json" -H "Accept: application/json, text/event-stream" -H
"MCP-Protocol-Version: $PROTO" -H "Mcp-Method: tools/list" -d "$body" 2>/dev/null |
tr -d '\r' | sed -n 's/.*resource_metadata="\([^"]*\)".*/\1/p')
sub=$(curl -sS --connect-timeout 5 -m 10 "$WK_SUB" 2>/dev/null)
wkdoc=""
if printf '%s' "$sub" | jq -e . >/dev/null 2>&1; then
 wkdoc="$sub"
else
 root=$(curl -sS --connect-timeout 5 -m 10 "$WK_ROOT" 2>/dev/null)
 printf '%s' "$root" | jq -e . >/dev/null 2>&1 && wkdoc="$root"
fi
if [ -z "$wkdoc" ] && [ -z "$chal" ]; then
```

```
 echo "ERROR: discovery unavailable, no WWW-Authenticate challenge and neither well-
known protected-resource path returned JSON"
 exit 0
fi
if [ -n "$chal" ]; then
 guard "$chal"
 case "$chal" in https://*) : ;; *) echo "FAIL: challenge-advertised metadata URL is
not TLS ($chal)"; exit 0 ;; esac
 chdoc=$(curl -sS --connect-timeout 5 -m 10 "$chal" 2>/dev/null)
 printf '%s' "$chdoc" | jq -e . >/dev/null 2>&1 || { echo "ERROR: challenge-
advertised metadata at $chal unreachable or not JSON"; exit 0; }
 if [ -n "$wkdoc" ]; then
 a=$(printf '%s' "$wkdoc" | jq -S .); b=$(printf '%s' "$chdoc" | jq -S .)
 [ "$a" = "$b" ] || { echo "FAIL: the challenge and well-known discovery paths serve
different metadata documents"; exit 0; }
 else
 wkdoc="$chdoc"
 fi
fi
res=$(printf '%s' "$wkdoc" | jq -r '.resource // empty')
[ "$res" = "$CANON" ] && echo "PASS: discovery served over TLS, paths consistent,
resource matches canonical URI" || echo "FAIL: metadata resource is '$res', expected
canonical URI '$CANON'"
```

Run the following command to compare the advertised authorization servers against the approved export and fail an empty list. The comparison is line-wise so a value is never split on internal whitespace:

```
PRM_URL="<PRM_URL>"
EXPORT="<APPROVED_AS_EXPORT>"
[ -f "$EXPORT" ] || { echo "ERROR: approved authorization server export not found at
$EXPORT"; exit 0; }
approved=$(tr -d '\r' < "$EXPORT" | sed '/^[[:space:]]*$/d' | sort -u)
[ -n "$approved" ] || { echo "ERROR: approved authorization server export is empty,
record the approval before auditing"; exit 0; }
doc=$(curl -sS --connect-timeout 5 -m 10 "$PRM_URL" 2>/dev/null)
printf '%s' "$doc" | jq -e '.authorization_servers' >/dev/null 2>&1 || { echo "ERROR:
metadata at $PRM_URL unreachable or missing authorization_servers"; exit 0; }
n=$(printf '%s' "$doc" | jq '.authorization_servers | length')
[ "$n" -ge 1 ] 2>/dev/null || { echo "FAIL: Protected Resource Metadata
authorization_servers is empty, which is protocol-invalid (the specification requires
at least one authorization server)"; exit 0; }
observed=$(printf '%s' "$doc" | jq -r '.authorization_servers[]' | sort -u)
exp_file=$(mktemp); wd_file=$(mktemp)
printf '%s\n' "$observed" | while IFS= read -r o; do [ -z "$o" ] && continue; printf
'%s\n' "$approved" | grep -Fxq "$o" || printf '%s\n' "$o" >> "$exp_file"; done
printf '%s\n' "$approved" | while IFS= read -r a; do [ -z "$a" ] && continue; printf
'%s\n' "$observed" | grep -Fxq "$a" || printf '%s\n' "$a" >> "$wd_file"; done
[ -s "$exp_file" ] && while IFS= read -r e; do echo "detail: advertised but not
approved: $e"; done < "$exp_file"
[ -s "$wd_file" ] && while IFS= read -r w; do echo "detail: approved but not
advertised: $w"; done < "$wd_file"
if [ -s "$exp_file" ]; then echo "FAIL: metadata advertises authorization servers
outside the approved export"
elif [ -s "$wd_file" ]; then echo "REVIEW: approved authorization servers absent from
metadata, confirm decommissioning"
else echo "PASS: advertised authorization servers exactly match the approved export";
fi
rm -f "$exp_file" "$wd_file"
```

Run the following command to confirm each approved authorization server publishes metadata whose issuer matches the approved entry. It tries the OAuth Authorization Server Metadata forms and the OpenID Connect Discovery forms in turn:

```
EXPORT="<APPROVED_AS_EXPORT>"
[ -f "$EXPORT" ] || { echo "ERROR: approved authorization server export not found at
$EXPORT"; exit 0; }
tmp=$(mktemp)
tr -d '\r' < "$EXPORT" | sed '/^[[:space:]]*$/d' > "$tmp"
[ -s "$tmp" ] || { rm -f "$tmp"; echo "ERROR: approved authorization server export is
empty"; exit 0; }
try_urls() {
 iss="$1"
 origin=$(printf '%s' "$iss" | sed -E 's#^(https?://[^/]+).*#\1#')
 ipath=$(printf '%s' "$iss" | sed -E 's#^https?://[^/]+##')
 if [ -n "$ipath" ] && [ "$ipath" != "/" ]; then
 printf '%s\n' "$origin/.well-known/oauth-authorization-server$ipath"
 printf '%s\n' "$iss/.well-known/oauth-authorization-server"
 printf '%s\n' "$origin/.well-known/openid-configuration$ipath"
 printf '%s\n' "$iss/.well-known/openid-configuration"
 else
 printf '%s\n' "$origin/.well-known/oauth-authorization-server"
 printf '%s\n' "$origin/.well-known/openid-configuration"
 fi
}
fails=$(mktemp); errs=$(mktemp)
while IFS= read -r iss; do
 [ -z "$iss" ] && continue
 got=""
 for u in $(try_urls "$iss"); do
 d=$(curl -sS --connect-timeout 5 -m 10 "$u" 2>/dev/null)
 v=$(printf '%s' "$d" | jq -r '.issuer // empty' 2>/dev/null)
 [ -n "$v" ] && { got="$v"; break; }
 done
 if [ -z "$got" ]; then printf '%s\n' "$iss" >> "$errs"; echo "detail: no issuer
retrievable for $iss"
 elif [ "$got" != "$iss" ]; then printf '%s\n' "$iss" >> "$fails"; echo "detail:
issuer mismatch for $iss (metadata says $got)"; fi
done < "$tmp"
rm -f "$tmp"
if [ -s "$fails" ]; then echo "FAIL: authorization server metadata issuer does not
match the approved entry"
elif [ -s "$errs" ]; then echo "ERROR: one or more approved authorization servers had
no retrievable metadata"
else echo "PASS: every approved authorization server publishes metadata with an
exactly matching issuer"; fi
rm -f "$fails" "$errs"
```

The system is compliant when the discovery chain serves one consistent Protected Resource Metadata document over TLS whose `resource` matches the canonical URI, `authorization_servers` is non-empty, every advertised authorization server appears in the approved export, and each publishes metadata with an exactly matching `issuer`. An advertised authorization server absent from the export is a `FAIL`, an approved server not advertised is a `REVIEW`, and an empty `authorization_servers` list is a `FAIL` as protocol-invalid. A discovery chain that resolves to nothing is an `ERROR` for discovery-unavailable, distinct from a server that accepts unauthenticated requests.

Remediation:

Configure the MCP server, or the gateway terminating its OAuth surface, to serve Protected Resource Metadata only over TLS, with `authorization_servers` containing exactly the approved entries and `resource` set to the server's canonical URI. Serve the same Protected Resource Metadata document on every discovery path the server supports, whether the `WWW-Authenticate` challenge or the well-known URIs. Generate the approved-authorization-server export from the identity governance system under change control, one issuer per line with carriage returns stripped, following the Section 1 export conventions. Restrict write access on the metadata documents or their generating configuration to the deployment pipeline identity. Where a gateway serves the metadata, apply the same restrictions there and treat gateway serving as the enforcement point of record for this surface.

Default Value:

By default, SDK-generated Protected Resource Metadata reflects whatever authorization server the operator configured, with no comparison against an approved list, though the exact behavior varies by SDK and entry point. Whether the document is served TLS-only and write-protected depends on the deployment.

References:

1. MCP specification (Base Protocol / Authorization / Authorization Server Discovery): https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization/authorization-server-discovery
2. MCP specification (Base Protocol / Authorization, authorization server metadata validation): https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization
3. RFC 8707 (Resource Indicators for OAuth 2.0): https://datatracker.ietf.org/doc/html/rfc8707
4. RFC 8414 (OAuth 2.0 Authorization Server Metadata): https://datatracker.ietf.org/doc/html/rfc8414
5. RFC 9728 (OAuth 2.0 Protected Resource Metadata): https://datatracker.ietf.org/doc/html/rfc9728
6. OpenID Connect Discovery 1.0: https://openid.net/specs/openid-connect-discovery-1_0.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.6 Establish and Maintain an Inventory of Authentication and Authorization Systems Establish and maintain an inventory of the enterprise's authentication and authorization systems, including those hosted on-site or at a remote service provider. Review and update the inventory, at a minimum, annually, or more frequently.		●	●

3.3.3 Ensure shared downstream service account identities are prohibited across multiple MCP tools or servers (Manual)

Profile Applicability:

- Level 2

Description:

Each MCP server, and each tool that accesses backend systems or downstream APIs, uses a dedicated downstream service identity with the minimum permissions required for that function. Sharing a service account, workload identity, or API credential across multiple tools or servers is prohibited. An exception is allowed only when explicitly approved with a documented justification, an identified owner, a defined expiry date, and compensating controls on file, and approved exceptions are reviewed at least annually. The MCP specification does not mandate per-tool service identities, so this Recommendation is a hardening measure the specification leaves open.

Rationale:

Shared service accounts undermine both attribution and blast-radius containment. When multiple tools or servers share one downstream identity, downstream audit logs cannot attribute an action to a specific tool or server, and compromising any one component that holds the identity grants access to everything that identity can reach. The risk is acute when tools of different risk tiers share a credential, because compromising a read-only tool that shares its credential with a write tool yields write access. The MCP specification supports least privilege but does not itself require per-tool identities.

Impact:

Giving each tool or server its own downstream identity is a moderate one-time IAM effort, plus ongoing provisioning as new tools or servers are added.

Audit:

Review the identity mapping inventory as the authoritative evidence, with the credential-reuse scan below as a supplementary check. In the inventory, confirm each MCP server and tool maps to a distinct downstream service principal or workload identity, and that any shared identity has a documented, time-bound exception with an owner and compensating controls. The scan extracts credential-bearing values, hashes them so the value is never printed, and flags a hash appearing in more than one file. An empty extraction can mean workload identity federation is in use with no literal credentials to scan, or that the configuration directory or search pattern is wrong, so it is inconclusive on its own. Run the following command to detect literal credential values reused across distinct configuration files:

```
CONF_DIR="<MCP_SERVERS_CONFIG_DIR>"
command -v sha256sum >/dev/null 2>&1 || { echo "ERROR: sha256sum unavailable, cannot
scan for duplicate literal credentials"; exit 0; }
tmp=$(mktemp)
grep -rEn 'service_account|client_id|api_key|client_secret' "$CONF_DIR" 2>/dev/null \
 | sed -E 's/^([^:]+:[0-
9]+):.*(service_account|client_id|api_key|client_secret)[=: ]+["'"'"']?([^"'"'"' ]+).
*/\1|\2|\3/' \
 | while IFS='|' read -r loc key val; do
 [ -n "$val" ] || continue
 file=$(printf '%s' "$loc" | sed -E 's/:[0-9]+$//')
 printf '%s %s %s\n' "$(printf '%s' "$val" | sha256sum | cut -c1-12)" "$key" "$file"
 done > "$tmp"
count=$(wc -l < "$tmp" | tr -d ' ')
dups=$(awk '{print $1, $3}' "$tmp" | sort -u | awk '{print $1}' | sort | uniq -d)
if [ "$count" -eq 0 ]; then
 echo "REVIEW: no literal credentials extracted. Either this deployment uses workload
identity federation (assess via the identity mapping inventory), or CONF_DIR or the
pattern is wrong"
elif [ -z "$dups" ]; then
 echo "PASS: no literal credential value shared across distinct configs ($count
references scanned)"
else
 echo "FAIL: literal credential value(s) shared across distinct configs (hash ->
files):"
 for h in $dups; do printf ' %s -> ' "$h"; awk -v h="$h" '$1==h{print $3}' "$tmp" |
sort -u | tr '\n' ' '; echo; done
fi
rm -f "$tmp"
```

The system is compliant when the inventory shows each server and tool using a distinct downstream identity with any sharing covered by a documented exception, and the scan finds no identical literal credential across distinct files. An empty scan is a `REVIEW` directing the reviewer to the inventory, not a pass.

Remediation:

This is IAM and documentation work with no portable command. Maintain an identity mapping inventory that records, for each server and tool, the downstream system it reaches, the service principal or workload identity it uses, and the permissions granted. Configure each MCP tool and server to use a dedicated downstream service identity, and for each shared-identity case create a least-privilege dedicated identity and update each configuration to use its own. Use workload identity federation and per-workload IAM roles for cloud deployments, and separate OS user accounts or container identities per stdio server. Prohibit sharing absent a documented, time-bound exception, and record any case where separation is not yet feasible as an approved exception with owner, justification, expiry, and compensating controls, reviewed at least annually. Verify with the Audit.

Default Value:

By default, MCP server configurations do not enforce per-tool or per-server service account separation. Teams commonly reuse shared API keys or service accounts across multiple servers.

References:

1. MCP specification (Base Protocol / Authorization): https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization
2. MCP specification (security best practices, dedicated service identities and least privilege): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.5 Establish and Maintain an Inventory of Service Accounts Establish and maintain an inventory of service accounts. The inventory, at a minimum, must contain department owner, review date, and purpose. Perform service account reviews to validate that all active accounts are authorized, on a recurring schedule at a minimum quarterly, or more frequently.		●	●

3.3.4 Ensure OAuth scopes are minimized and elevated progressively (Manual)

Profile Applicability:

- Level 2

Description:

MCP servers advertise the minimal set of scopes needed for basic functionality in Protected Resource Metadata `scopes_supported` and do not expose an over-broad or wildcard scope surface. During initial authorization a client requests only the scopes indicated by the `WWW-Authenticate` challenge or, absent a challenge scope, the advertised `scopes_supported`. When a privileged operation returns an `insufficient_scope` challenge, the client elevates through a step-up flow, requesting the union of its previously requested scopes and the scopes named in the challenge, so previously granted permissions are preserved. A granted token may therefore carry more than the operation in progress requires, but every granted scope stays within an approved cumulative scope set that the deployment records in the capability baseline, together with the per-tool scopes that compose it. Broad wildcard scopes such as `admin:*` are not granted unless a documented exception approves them.

Rationale:

Over-broad scopes amplify token theft. A token scoped to `files:*`, `db:*`, and `admin:*` simultaneously enables lateral access, privilege chaining, and hard revocation. The damage from a stolen token is bounded by its scopes, so constraining the granted set to an approved, least-privilege cumulative model, and refusing broad wildcard grants, reduces that bound. Progressive elevation keeps the granted set as small as the workflow allows at each point while still preserving permissions the client legitimately accumulated, rather than issuing a broad token up front.

Impact:

Constraining scopes may require a one-time re-consent when existing broad tokens are revoked. Progressive elevation also adds an authorization round-trip the first time a session needs a higher scope, and a client without step-up support sees that round-trip as a failed privileged operation.

Audit:

Inspect the Protected Resource Metadata for the scopes advertised in `scopes_supported`, then compare a token's granted scopes against the approved cumulative scope set in the capability baseline. Run the following command to read the scopes on a token in use. The decode is padding-safe, and scopes may be in the `scope` or `scp` claim:

```
TOKEN='<JWT_ACCESS_TOKEN>'
seg=$(printf '%s' "$TOKEN" | cut -d. -f2 | tr '_-' '/+')
pad=$(( (4 - ${#seg} % 4) % 4 )); seg="$seg$(printf '=%.0s' $(seq 1 $pad))"
claims=$(printf '%s' "$seg" | base64 -d 2>/dev/null)
[ -z "$claims" ] && { echo "ERROR: could not decode (opaque token? introspect
instead)"; exit 0; }
printf '%s' "$claims" | jq empty 2>/dev/null || { echo "ERROR: decoded payload is not
JSON (opaque token? introspect instead)"; exit 0; }
granted=$(echo "$claims" | jq -r '(.scope // (.scp // [] | join(" ")))')
echo "granted_scopes: $granted"
```

Set `APPROVED` to the deployment's approved cumulative scope set from the capability baseline, space-delimited (for example `mcp:read mcp:write`). Run the following command to flag any granted scope outside the approved set and any broad wildcard grant:

```
APPROVED="<APPROVED_CUMULATIVE_SCOPES>"
[ -z "$granted" ] && { echo "ERROR: token carries no scopes (opaque token, or scopes
conveyed out of band); assess via introspection"; exit 0; }
extra=""; wild=""
for s in $granted; do
  case "$s" in *:\*|\*) wild="$wild $s" ;; esac
  case " $APPROVED " in *" $s "*) ;; *) extra="$extra $s" ;; esac
done
if [ -n "$extra" ]; then
  echo "FAIL: token carries scopes outside the approved cumulative set:$extra"
elif [ -n "$wild" ]; then
  echo "REVIEW: all granted scopes are approved but the token carries broad wildcard
scopes:$wild"
else
  echo "PASS: every granted scope is within the approved cumulative scope set and
none are broad wildcards"
fi
```

The system is compliant when every scope on a token in use is within the approved cumulative set and the token carries no broad wildcard scope. A token with no decodable scopes is `ERROR` and is assessed by introspection. An approved wildcard scope is `REVIEW` for a human to confirm the documented exception.

Remediation:

Trim the scopes advertised in Protected Resource Metadata `scopes_supported` to the minimal set the server's tools need, and do not advertise or grant broad wildcard scopes such as `admin:*` unless a documented exception approves them. Configure the authorization server so that a privileged operation returns an `insufficient_scope` challenge naming all required scopes in a single challenge rather than one at a time. Clients elevate through the step-up flow, requesting the union of previously requested scopes and the challenge scopes so accumulated permissions are preserved. Document the approved cumulative set, and the per-tool scopes that compose it, in the capability baseline using the same scope names the Audit reads. For servers with read and write tools, separate scope tiers (for example `mcp:read`, `mcp:write`, `mcp:admin`). Verify with the Audit.

Default Value:

By default, many MCP authorization configurations expose the full supported scope set in Protected Resource Metadata, and clients may request all available scopes rather than scoping to current needs.

References:

1. MCP specification (Base Protocol / Authorization, scope selection and step-up authorization flow): https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization
2. MCP specification (security best practices, least privilege and scope minimization): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices
3. SEP-2350 (Clarify client-side scope accumulation in step-up authorization): https://github.com/modelcontextprotocol/modelcontextprotocol/pull/2350

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●

3.3.5 Ensure confused-deputy safeguards are applied for static OAuth client IDs (Manual)

Profile Applicability:

- Level 2

Description:

An MCP server acting as an OAuth proxy with a shared static client ID toward a downstream third-party authorization server, that lets MCP clients dynamically register (each obtaining its own client ID), and whose third-party authorization server sets a consent cookie for the static client ID, implements the specification's confused-deputy mitigations. The third-party authorization server often lacks dynamic client registration, which is why the proxy uses one static client ID for all requests. Three mitigations apply. (1) Per-client consent keyed to each dynamically-registered MCP client's `client_id`, checked before any redirect to the third-party authorization server, and never reused across registrations. (2) Exact-string `redirect_uri` validation with no wildcards or pattern matching, rejecting any request whose `redirect_uri` changed without re-registration. (3) State parameter validation, with a cryptographically random single-use state stored server-side only after consent is approved and validated at the callback. Consent records are logged with the registered MCP client's identity and a timestamp.

Rationale:

The confused-deputy attack arises when a proxy uses one static client ID on behalf of many MCP clients. An attacker registers a malicious client with an attacker-controlled `redirect_uri`, then sends the victim a crafted authorization link. Because the third-party authorization server already set a consent cookie for the static client ID during a prior legitimate flow, it skips consent, and the authorization code is redirected to the attacker. Per-client consent closes this skip-consent path, while exact `redirect_uri` matching and single-use state validation harden the surrounding flow against code redirection to an unregistered URI and against code interception and CSRF. The specification mandates all three.

Impact:

There is minor UX friction: an extra consent prompt the first time each MCP client registers.

Audit:

Review the proxy's OAuth flow and confirm a newly registered MCP client triggers fresh consent, a changed `redirect_uri` is rejected by exact match, and a missing or mismatched state is rejected.

```
AUTHZ="https://<SERVER_URL>/authorize"
CID="<REGISTERED_CLIENT_ID>"
GOOD_REDIRECT="<REGISTERED_REDIRECT_URI>"
BAD_REDIRECT="https://attacker.example.com/cb"
CH="<CODE_CHALLENGE>"
base="client_id=$CID&response_type=code&state=test123&code_challenge=$CH&code_challen
ge_method=S256"
gc=$(curl -sS -o /dev/null -w '%{http_code}'
"$AUTHZ?$base&redirect_uri=$GOOD_REDIRECT")
bc=$(curl -sS -o /dev/null -w '%{http_code}'
"$AUTHZ?$base&redirect_uri=$BAD_REDIRECT")
echo "control_valid_redirect=$gc varied_bad_redirect=$bc"
case "$gc" in
  000) echo "ERROR: authorization endpoint unreachable"; exit 0 ;;
  4*)  echo "ERROR: positive control was itself rejected ($gc), the request is
missing required parameters, cannot attribute the redirect_uri rejection"; exit 0 ;;
esac
case "$bc" in
  400|401|403) echo "PASS: with only redirect_uri changed the request is rejected
while the valid one is accepted, rejection attributable to exact redirect_uri
validation" ;;
  302) echo "REVIEW: the bad redirect_uri produced a 302, confirm it is an error
redirect and not an authorization code sent to the attacker URI" ;;
  000) echo "ERROR: authorization endpoint unreachable" ;;
  *)   echo "REVIEW: bad redirect_uri returned $bc" ;;
esac
```

Run the following command to confirm per-client consent records exist keyed to the registered MCP client's client_id with a timestamp. This block is reconstructed from the consent-record shape the Description and Remediation specify and needs lab confirmation of the actual consent-log field names before sign-off:

```
CB="https://<SERVER_URL>/callback"
nostate=$(curl -sS -o /dev/null -w '%{http_code}' "$CB?code=fakecode")
badstate=$(curl -sS -o /dev/null -w '%{http_code}' "$CB?code=fakecode&state=wrong-
value")
echo "no_state=$nostate bad_state=$badstate"
echo "REVIEW: both should be 4xx, but a fabricated code is rejected regardless of
state, confirm state validation with a controlled transaction varying only state, or
with callback-validation logs"
```

Run the following command to confirm per-client consent records exist keyed to a downstream `client_id` with a timestamp. This block is reconstructed from the consent-record shape the Description and Remediation specify and needs lab confirmation of the actual consent-log field names before sign-off:

```
LOG="<MCP_CONSENT_LOG>"
CLIENT="<DOWNSTREAM_CLIENT_ID>"
if [ ! -s "$LOG" ]; then
 echo "ERROR: consent log not found or empty (cannot verify, capture consent
records)"
else
 grep -E '"event":"consent_granted"' "$LOG" \
 | jq -e --arg c "$CLIENT" 'select(.client_id==$c and .timestamp!=null)' >/dev/null \
 && echo "PASS: per-client consent record found for the downstream client with a
timestamp" \
 || echo "REVIEW: no per-client consent record for this client_id (confirm log field
names and that consent runs before the redirect)"
fi
```

The system is compliant when a mismatched `redirect_uri` is rejected, a missing or mismatched state is rejected, and per-client consent is recorded per registered MCP client and enforced before the third-party redirect. A reviewer adjudicates any `REVIEW`, in particular confirming a `302` is an error redirect and not a leaked authorization code.

Remediation:

This is application or server logic with no portable command. Maintain a per-user registry of the MCP client `client_id` values that have been consented, check it before any redirect to the third-party authorization server, and show an MCP-owned consent page that names the client and the third-party scopes, with CSRF protection and anti-framing. Validate `redirect_uri` by exact string match and reject any change without re-registration. Generate a cryptographically random state per request, store it server-side only after consent is approved, validate it at the callback, and make it single-use and short-lived. Log consent per MCP client. Verify with the Audit.

Default Value:

By default, MCP proxy servers using static OAuth client IDs do not implement per-downstream-client consent, and a consent cookie at the third-party AS may be reused across client registrations.

References:

1. MCP specification (security best practices, confused deputy and per-client consent): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.1 Establish an Access Granting Process Establish and follow a process, preferably automated, for granting access to enterprise assets upon new hire, rights grant, or role change of a user.	●	●	●

4 Client (Host) Configuration

The host is the trust boundary between the user and the servers it connects to. It receives tool descriptions, elicitation and sampling requests, resource URIs, and rendered content from servers that may be third-party or compromised. A host that invokes tools without consent, exposes an over-broad filesystem, dereferences whatever URI a server supplies, or renders server content without isolation hands control of the user's environment to whoever controls the server.

The Recommendations in this section keep the host defaulting to least privilege and explicit human control. They require consent or pre-approved allowlists before invocation, human denial of sensitive or high-impact actions, consent and redaction on elicitation and sampling, a restricted filesystem scope, care with untrusted resource URIs and stdio subprocess environments, sandboxed rendering and default-deny permissions for MCP Apps, and review before project-scoped server definitions are enabled.

4.1 Consent & Human Oversight

4.1.1 Ensure per-tool consent or pre-approved allowlists are required (Manual)

Profile Applicability:

- Level 1

Description:

MCP hosts display the canonical tool definition to the operator before a tool is executed for the first time, and require either an explicit user decision or a pre-approved allowlist entry before execution proceeds. The canonical tool definition includes at least the tool name, title, description, and input schema, the output schema where present, and the annotations, though server-provided annotations are untrusted and do not determine enterprise risk. The pre-approved allowlist path and the per-invocation consent path are each configured explicitly and are not interchangeable. A tool newly advertised or modified through a `notifications/tools/list_changed` notification is not added to the allowlist automatically. The tool is quarantined until re-validated under the capability baseline, after which it becomes eligible for allowlist addition or per-invocation consent.

Rationale:

Tool invocation consent is the primary way human operators keep awareness and control of what an MCP-connected model does on their behalf. Without consent or a reviewed allowlist, a model can invoke any tool the server exposes, including tools with destructive, irreversible, or data-exfiltrating effects. This matters most for third-party servers, where tool descriptions and schemas are untrusted metadata that may not reflect actual behavior. Presenting the full canonical tool definition before first execution ensures the operator sees what the tool claims to do before authorizing it.

Impact:

Consent prompts add slight friction on first use of a new tool, and the automated path avoids that friction at the cost of configuring and maintaining the allowlist. Both paths require explicit configuration.

Audit:

Review the host configuration and audit log to confirm that an authorizing decision is recorded and precedes each tool's first execution, and that the full canonical tool definition is shown before execution is permitted. Set the field names to match the deployment's log schema if they differ. Run the following command:

```
LOG="<MCP_AUDIT_LOG>"
command -v jq >/dev/null 2>&1 || { echo "ERROR: jq is required and not installed
(cannot verify)"; exit 0; }
[ -s "$LOG" ] || { echo "ERROR: audit log <MCP_AUDIT_LOG> is missing or empty (cannot
verify)"; exit 0; }
recs=$(grep -E '"event":"tool_(consent|allowlist)"' "$LOG")
total=$(printf '%s\n' "$recs" | grep -c .)
[ "$total" -gt 0 ] || { echo "REVIEW: no tool consent or allowlist records found
(control not evidenced)"; exit 0; }
printf '%s\n' "$recs" | jq -e . >/dev/null 2>&1 || { echo "ERROR: audit log contains
unparseable records (cannot verify)"; exit 0; }
bad=$(printf '%s\n' "$recs" | jq -c '
  select(
    (.tool==null or .tool=="") or
    (.manifest_shown!=true) or
    (((.decision//"")|test("^(user_approved|user_denied|allowlist_match)$"))|not) or
    (((.decision//"")|test("^user_")) and (.decision_source!="user")) or
    ((.decision//"")=="allowlist_match" and ((.allowlist_ref//"")==""))
  )')
if [ -n "$bad" ]; then echo "FAIL: tool records without the manifest shown, or with a
default or automatic grant not backed by an explicit user decision or an approved
allowlist match"; exit 0; fi
calls=$(grep -E '"event":"tool_call"' "$LOG")
ncalls=$(printf '%s\n' "$calls" | grep -c .)
if [ "$ncalls" -eq 0 ]; then echo "PASS (records): decisions are complete and
explicit. REVIEW (ordering): no tool_call events in the log, decision-before-first-
execution not evidenced, confirm by review"; exit 0; fi
printf '%s\n' "$calls" | jq -e . >/dev/null 2>&1 || { echo "ERROR: tool_call records
are not parseable (cannot verify ordering)"; exit 0; }
auth=$(printf '%s\n' "$recs" | jq -c 'select((.decision=="user_approved"
or .decision=="allowlist_match") and (.ts!=null)) | {tool, ts}')
viol=""
while IFS= read -r c; do
  [ -n "$c" ] || continue
  ct=$(printf '%s' "$c" | jq -r '.tool'); cts=$(printf '%s' "$c" | jq -r '.ts')
  earliest=$(printf '%s\n' "$auth" | jq -r --arg t "$ct" 'select(.tool==$t) | .ts' |
sort | head -1)
  if [ -z "$earliest" ] || [ "$earliest" \> "$cts" ]; then viol="${viol}
${ct}@${cts}"; fi
done <<EOF
$(printf '%s\n' "$calls" | jq -c 'select(.ts!=null) | {tool, ts}')
EOF
if [ -n "$viol" ]; then echo "FAIL (ordering): a tool_call executed before an
authorizing decision for:$viol"
else echo "PASS: decisions are complete and explicit, and every tool_call was
preceded by an authorizing decision for that tool"; fi
```

Confirm separately that a one-time server-level approval is not used in place of per-tool gating, because a single server-level approval does not ensure each individual tool was reviewed, and confirm the full canonical tool definition is shown at the point of decision. This Recommendation is compliant when the audit reports `PASS` on both the decision records and the call ordering, the canonical tool definition is shown, and no server-level blanket approval substitutes for per-tool gating. Where the log contains no `tool_call` events, the ordering check returns `REVIEW` and a reviewer confirms decision-before-first-execution by inspection.

Remediation:

Configure the host to require per-tool consent or a pre-approved allowlist before first execution, and to present the full, unsummarized canonical tool definition (name, description, input schema, output schema where present, title, and annotations) at the point of decision. This behavior is set in host or client configuration, with no portable command. Populate the pre-approved allowlist from the operator's assessment of which tools are sensitive or high-impact and from the capability baseline, and configure the host so that a newly advertised or modified tool from a `notifications/tools/list_changed` notification is quarantined for re-validation rather than auto-added. Record each consent and allowlist decision to the audit log with the tool name, the decision (an explicit user approval, an explicit user denial, or an approved allowlist match with a reference), the decision source, and confirmation that the canonical tool definition was shown. Record tool executions with the tool name and a timestamp, so the decision can be confirmed to precede first execution.

Default Value:

By default, MCP hosts may execute tools without displaying the manifest or requiring per-tool consent, and some hosts implement a one-time server-level approval that does not gate individual tools.

References:

1. MCP specification (security best practices): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices
2. MCP specification (Server Features / Tools): https://modelcontextprotocol.io/specification/2026-07-28/server/tools

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	2.5 Allowlist Authorized Software Use technical controls, such as application allowlisting, to ensure that only authorized software can execute or be accessed. Reassess bi-annually, or more frequently.		●	●
v8	6.1 Establish an Access Granting Process Establish and follow a process, preferably automated, for granting access to enterprise assets upon new hire, rights grant, or role change of a user.	●	●	●

4.1.2 Ensure project-scoped MCP server definitions are not enabled or executed without explicit user approval (Manual)

Profile Applicability:

- Level 1

Description:

A project-scoped server definition is one declared in configuration carried inside a project or repository, rather than in the user's own global configuration. The MCP client or host does not enable or execute such a definition without an explicit user review and approval of it. A server definition specifies a command the client will launch, so trusting it is equivalent to agreeing to run that command. Auto-trust and auto-approval of project-scoped servers are disabled, so opening a shared or cloned workspace does not enable or launch its declared servers before the user has seen and approved them. Approval is per-project and precedes any launch of, or command execution from, a project-scoped definition.

Rationale:

A server definition is not passive data. It names an executable and its arguments, and enabling it runs that command. When a client honors project-scoped definitions automatically, a definition placed in a repository or shared workspace by someone other than the user runs on their machine as soon as they open the project, before any review. This has been a live vulnerability class in deployed agentic clients. A project-scoped setting that auto-approved servers, or a project configuration that launched a server on open, produced remote code execution from nothing more than opening an untrusted repository. The defense is a trust boundary at the point of enabling the definition: the user reviews which servers a project declares and approves them before any run. This is the client-side counterpart to the server-side consent controls, the difference between reading a project's server list and executing it unseen.

Impact:

Opening a newly cloned or shared workspace that declares its own servers requires an explicit approval before those servers are enabled, which adds a confirmation step at onboarding. For the common case of a workspace with no project-scoped servers there is no added friction.

Audit:

Confirm by configuration review that the client's project auto-trust or auto-approval behavior is disabled, so project-scoped server definitions are not enabled automatically. Then perform a behavioral test: in a throwaway workspace, add a benign project-scoped server definition, open the workspace with the client, and confirm the client does not enable or launch that server without presenting an explicit approval, and that declining the approval prevents the launch. For a client that exposes a named auto-trust setting, the following optional command confirms it is not enabled in the located configuration, as a convenience rather than the verdict:

```
FILES="<CLIENT_CONFIG_FILE_LIST>"
found=0; enabled=""
for f in $FILES; do
  [ -e "$f" ] || continue
  found=$((found+1))
  grep -Eq
'"(enableAllProjectMcpServers|enabledMcpjsonServers|autoApprove|autoMode)"[[:space:]]
*:[[:space:]]*true' "$f" 2>/dev/null && enabled="${enabled} ${f}"
done
if [ "$found" -eq 0 ]; then
  echo "REVIEW: none of the given config paths exist (confirm the client's config
location)"
elif [ -n "$enabled" ]; then
  echo "FAIL: project auto-trust or auto-approval is enabled in:$enabled"
else
  echo "PASS: no known project auto-trust or auto-approval flag is enabled in the
located config"
fi
```

This Recommendation is compliant when the behavioral test shows a project-scoped server definition cannot be enabled or executed without an explicit, per-project user approval, and no known client auto-trust setting is left enabled.

Remediation:

These are client configuration settings, with no portable command. Disable the client's project auto-trust or auto-approval of servers, so project-scoped definitions are presented for approval rather than enabled automatically. Where the client supports it, block auto-trust settings from being honored when they originate in project scope. Require an explicit per-project approval before a project's server definitions are enabled or any command from them runs. Keep the client updated to a version that gates project-scoped definitions behind that approval.

Default Value:

Behavior varies by client and version. Several clients have historically honored project-scoped server definitions, including auto-approval settings, before an explicit trust decision, and later versions gate that behind an approval prompt.

References:

1. MCP specification (security best practices): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices
2. MCP specification (Server Features / Tools, human oversight of tool execution): https://modelcontextprotocol.io/specification/2026-07-28/server/tools

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.1 Establish an Access Granting Process Establish and follow a process, preferably automated, for granting access to enterprise assets upon new hire, rights grant, or role change of a user.	●	●	●

4.1.3 Ensure Elicitation requests are consented, redacted, and identity-bound (Manual)

Profile Applicability:

- Level 1

Description:

The client presents each server-initiated elicitation with clear attribution of the requesting server, provides decline and cancel options, lets the user review form responses before they are sent, and requires an explicit user decision before any form data is submitted or any URL is navigated to. Redaction filters remove personally identifiable information, secrets, and other sensitive data from context surfaced during an elicitation exchange, and the client records that the redaction check ran. Server-initiated elicitation is rate-limited. Each elicitation event is logged with the requesting server identity, the mode, a request identifier, the user's decision and its source, the redaction result, and the mode-specific fields.

The `2026-07-28` specification sets two hard requirements. First, servers do not use form-mode elicitation to request secrets such as passwords, API keys, access tokens, or payment credentials, and use URL mode for those interactions. Second, for URL-mode flows the server securely associates any stored state with the individual user and verifies that the completing user is the one for whom the elicitation was generated, deriving identity from MCP authorization credentials where possible, to prevent phishing-based account takeover.

Rationale:

Elicitation is the path by which a server solicits information from the user. Without consent gates and clear server identity, a compromised or malicious server can prompt the user for data under a misleading pretext. Form-mode elicitation exposes its data to the client and model, so using it for secrets would leak them. URL mode keeps the data between the user and the server, which is why the specification requires it for sensitive interactions. The URL-mode same-user check closes a phishing path where an attacker triggers an elicitation and tricks a different user into completing it, binding the resulting tokens to the wrong identity. Rate limits prevent a server from flooding the user with prompts, which would otherwise cause denial-of-service or consent fatigue.

Impact:

Consent gates add small UX overhead to server-initiated elicitation, and redaction adds minimal computational overhead.

Audit:

Inspect the elicitation logs for an explicit user decision, a completed redaction check, and the mode-specific fields, and confirm by configuration review that form-mode elicitation cannot request secrets, that such requests use URL mode, that URL-mode flows verify the completing user, and that rate limiting is applied. Set the field names to match the deployment's log schema if they differ. Run the following command:

```
LOG="<MCP_AUDIT_LOG>"
command -v jq >/dev/null 2>&1 || { echo "ERROR: jq is required and not installed
(cannot verify)"; exit 0; }
[ -s "$LOG" ] || { echo "ERROR: audit log <MCP_AUDIT_LOG> is missing or empty (cannot
verify)"; exit 0; }
events=$(grep -E '"event":"elicitation"' "$LOG")
n=$(printf '%s\n' "$events" | grep -c .)
[ "$n" -gt 0 ] || { echo "REVIEW: no elicitation events found (control not
evidenced)"; exit 0; }
printf '%s\n' "$events" | jq -e . >/dev/null 2>&1 || { echo "ERROR: elicitation
records are not parseable (cannot verify)"; exit 0; }
bad=$(printf '%s\n' "$events" | jq -c '
  select(
    (.server_identity==null or .server_identity=="") or
    (((.mode//"")=="form" or (.mode//"")=="url")|not) or
    (((.decision//"")|test("^(accepted|declined|cancelled)$"))|not) or
    (.decision_source!="user") or
    (.redaction_checked!=true) or
    (.mode=="form" and (.requestedSchema==null)) or
    (.mode=="url" and ((.url//"")=="" and (.target_origin//"")==""))
  )')
if [ -n "$bad" ]; then
  echo "FAIL: elicitation events without an explicit user decision, a completed
redaction check, a valid mode, or the mode-specific fields"
else
  echo "PASS: every elicitation event carries server identity, mode, an explicit user
decision, a completed redaction check, and the mode-specific fields"
fi
```

This Recommendation is compliant when the audit reports `PASS` and the configuration review confirms the form-mode secret prohibition, URL mode for those requests, URL-mode same-user verification, and rate limiting.

Remediation:

These are host and server configuration settings, with no portable command. Configure the host to present server-initiated elicitation with clear server attribution and to require an explicit user decision before form data is submitted or a URL is navigated to. Apply a redaction filter that removes API keys, passwords, and common PII formats from any surfaced context, and record that the redaction check ran. Rate-limit server-initiated elicitation. In the server's elicitation handling, route any request for secrets through URL mode and never form mode, and for URL-mode flows securely associate any stored state with the individual user and verify that the same user completes the flow, deriving identity from MCP authorization credentials where possible. Log every elicitation event with the server identity, the mode, the user's decision and its source, the redaction result, and the mode-specific fields (`requestedSchema` for form mode, the target origin or URL for URL mode).

Default Value:

By default, many MCP elicitation implementations do not enforce the form-mode secret prohibition or the URL-mode same-user verification, and redaction filters and rate limits are often absent from SDK defaults.

References:

1. MCP specification (Client Features / Elicitation): https://modelcontextprotocol.io/specification/2026-07-28/client/elicitation
2. MCP specification (security best practices): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices
3. SEP-2322 (Multi Round-Trip Requests): https://modelcontextprotocol.io/seps/2322-MRTR

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.13 Deploy a Data Loss Prevention Solution Implement an automated tool, such as a host-based Data Loss Prevention (DLP) tool to identify all sensitive data stored, processed, or transmitted through enterprise assets, including those located onsite or at a remote service provider, and update the enterprise's sensitive data inventory.			●
v8	6.1 Establish an Access Granting Process Establish and follow a process, preferably automated, for granting access to enterprise assets upon new hire, rights grant, or role change of a user.	●	●	●

4.1.4 Ensure Sampling consent and redaction controls are implemented where Sampling is in use (Automated)

Profile Applicability:

- Level 2

Description:

The host requires an explicit user decision before any server-initiated sampling request is forwarded to the model. Redaction filters remove personally identifiable information, secrets, and other sensitive data from context before it is included in a sampling payload, and the host records that the redaction check ran. Server-initiated sampling is rate-limited. Each sampling event is logged with the requesting server identity, a request identifier, the request type, the user's decision and its source, and the redaction result, without logging raw sensitive prompts.

Under the `2026-07-28` revision, the Sampling feature is deprecated. It is a client capability, advertised per request in `_meta.io.modelcontextprotocol/clientCapabilities`, and delivered through a Multi Round-Trip Request rather than the standalone `sampling/createMessage` method used before this revision. These controls apply to any deployment still using sampling, and the documented migration path is to integrate the LLM provider directly rather than relying on sampling.

Rationale:

Sampling lets a server ask the host to send context to the model on the server's behalf. Without consent gates, a compromised server can drive that context, including credentials, internal data, or personal information, to the model without the user's knowledge. Without redaction, secrets and PII present in host context may be forwarded to an external model API. Without rate limits, a server can flood the host with sampling requests, creating denial-of-service or consent fatigue. Sampling is the most security-sensitive of the three features deprecated in the `2026-07-28` revision, which is why these controls remain important wherever it is still in use.

Impact:

Consent gates add small UX overhead to server-initiated sampling, and redaction adds minimal computational overhead. Because sampling is deprecated, weigh migrating off it entirely rather than investing further in these controls.

Audit:

Inspect the sampling logs for an explicit user decision and a completed redaction check, and confirm by configuration review that consent is enforced before a payload is forwarded, that the redaction filter is applied, and that rate limiting is in force. To determine whether this Recommendation is not applicable, confirm that the assessed host or client does not advertise `sampling` in its per-request client capabilities and that the server or application does not generate a `sampling/createMessage` request or a Multi Round-Trip Request sampling input request. Do not look for a server-advertised sampling capability, because sampling is a client capability.

Run the following command to search the audit log for sampling events and confirm each carries server identity, an explicit user decision, and a completed redaction check. Set the field names to match the deployment's log schema if they differ:

```
LOG="<MCP_AUDIT_LOG>"
command -v jq >/dev/null 2>&1 || { echo "ERROR: jq is required and not installed
(cannot verify)"; exit 0; }
[ -s "$LOG" ] || { echo "ERROR: audit log <MCP_AUDIT_LOG> is missing or empty (cannot
verify)"; exit 0; }
events=$(grep -E '"event":"sampling"' "$LOG")
n=$(printf '%s\n' "$events" | grep -c .)
[ "$n" -gt 0 ] || { echo "REVIEW: no sampling events found (sampling may not be in
use, confirm the client capability and any sampling input requests separately)"; exit
0; }
printf '%s\n' "$events" | jq -e . >/dev/null 2>&1 || { echo "ERROR: sampling records
are not parseable (cannot verify)"; exit 0; }
bad=$(printf '%s\n' "$events" | jq -c '
  select(
    (.server_identity==null or .server_identity=="") or
    (((.decision//"")|test("^(accepted|declined|cancelled)$"))|not) or
    (.decision_source!="user") or
    (.redaction_checked!=true)
  )')
if [ -n "$bad" ]; then
  echo "FAIL: sampling events without a server identity, an explicit user decision,
or a completed redaction check"
else
  echo "PASS: every sampling event carries a server identity, an explicit user
decision, and a completed redaction check"
fi
```

This Recommendation is compliant, where sampling is in use, when the audit reports `PASS` and the configuration review confirms consent before forwarding, redaction of the payload, and rate limiting. Where sampling is not in use, it is not applicable, confirmed by the client-capability and input-request checks above.

Remediation:

These are host and server configuration settings, with no portable command. Configure the host to require an explicit user decision before forwarding a server-initiated sampling request to the model. Apply a redaction filter that removes API keys, passwords, and common PII formats from context before it is sent, and record that the redaction check ran. Rate-limit server-initiated sampling. Log every sampling event with the server identity, a request or correlation identifier, the request type, the user's decision and its source, and the redaction result, without logging raw sensitive prompts. Because sampling is deprecated under the `2026-07-28` revision, plan migration to direct LLM-provider integration, and until that migration completes keep these controls in force wherever sampling remains enabled.

Default Value:

By default, many MCP sampling implementations do not require explicit consent before forwarding context to the model, and redaction filters and rate limits are often absent from SDK defaults.

References:

1. MCP specification (Client Features / Sampling): https://modelcontextprotocol.io/specification/2026-07-28/client/sampling
2. MCP specification (Deprecated Features): https://modelcontextprotocol.io/specification/2026-07-28/deprecated
3. SEP-2577 (Deprecate Roots, Sampling, and Logging): https://modelcontextprotocol.io/seps/2577-deprecate-roots-sampling-and-logging

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.13 Deploy a Data Loss Prevention Solution Implement an automated tool, such as a host-based Data Loss Prevention (DLP) tool to identify all sensitive data stored, processed, or transmitted through enterprise assets, including those located onsite or at a remote service provider, and update the enterprise's sensitive data inventory.			●
v8	6.1 Establish an Access Granting Process Establish and follow a process, preferably automated, for granting access to enterprise assets upon new hire, rights grant, or role change of a user.	●	●	●

4.1.5 Ensure a human can deny tool invocations and sensitive or high-impact operations require confirmation (Manual)

Profile Applicability:

- Level 2

Description:

A human can always deny a tool invocation, and sensitive or high-impact operations require explicit confirmation before they execute. A compliant host makes clear which tools are exposed to the model, shows a clear indicator when a tool is invoked, and shows the tool inputs to the user before the call. Routine, non-sensitive read operations proceed without a prompt but are logged. Sensitive operations, and high-impact or irreversible operations such as external communications, financial or legal commitments, security-posture changes, and destructive writes, require explicit human confirmation or approval before execution. In agentic multi-step workflows, the same confirmation applies to a sequence of operations that is collectively high-impact, so the human's ability to deny covers the aggregate effect, not only a single call. The operator maintains the judgment of what is sensitive or high-impact independently of server-provided tool annotations, which are untrusted unless they come from a trusted server.

Rationale:

The MCP Tools specification recommends that there always be a human in the loop able to deny a tool invocation, and that applications present confirmation prompts for operations and make clear which tools are invoked. Applying identical oversight to every invocation produces one of two bad outcomes: uniform high-friction oversight that causes consent fatigue, or uniform low-friction oversight that leaves irreversible actions unreviewed. Requiring confirmation for sensitive and high-impact operations while letting routine reads proceed matches oversight to actual risk and keeps the human able to deny the actions that matter. The confirmation step is also a prompt-injection mitigation: an injected instruction cannot silently trigger a high-impact action that needs human approval, so the approval breaks the attack chain at the execution layer even when input-layer detection fails. Because server-provided annotations are untrusted unless from a trusted server, the operator classifies what is sensitive or high-impact rather than relying on a tool's self-description.

Impact:

Sensitive and high-impact operations require explicit human confirmation before execution, which adds overhead in workflows that frequently invoke them. In most deployments the majority of invocations are routine reads and proceed without added friction.

Audit:

Confirm the host or gateway keeps a human able to deny any tool invocation, and that sensitive or high-impact operations require confirmation. Verify behavior at the enforcement point, not by calling the MCP server directly unless the server is explicitly the enforcement point: attempt a high-impact operation through the host or gateway and confirm a human approval is required before execution, and separately confirm a routine read executes without a prompt but is logged.

Set `ENFORCEMENT` to the host control API, gateway, or policy engine that enforces the controls (only the MCP server if the server itself is the enforcement point), set `HIGH_IMPACT_TOOL` to a tool the operator classifies as sensitive or high-impact, set `TOKEN` to a bearer token the enforcement point accepts, and set `APPROVAL_JQ` to the boolean the deployment documents as its attributable human-approval signal, for example a host approval state. Run the following command:

```
ENFORCEMENT="<HITL_ENFORCEMENT_URL>"
HIGH_IMPACT_TOOL="<HIGH_IMPACT_TOOL>"
PROTO="2026-07-28"
APPROVAL_JQ="${APPROVAL_JQ:-.result.approval.state==\"pending_human\"}"
TOKEN="<BEARER_TOKEN>"
command -v jq >/dev/null 2>&1 || { echo "ERROR: jq is required and not installed
(cannot verify)"; exit 0; }
body="{\"jsonrpc\":\"2.0\",\"id\":1,\"method\":\"tools/call\",\"params\":{\"name\":\"
$HIGH_IMPACT_TOOL\",\"arguments\":{},\"_meta\":{\"io.modelcontextprotocol/protocolVer
sion\":\"$PROTO\",\"io.modelcontextprotocol/clientInfo\":{\"name\":\"cis-benchmark-
audit\",\"version\":\"1.0\"}}}}"
raw=$(curl -sS --connect-timeout 3 -m 10 -X POST "$ENFORCEMENT" -H 'Content-Type:
application/json' -H 'Accept: application/json, text/event-stream' -H "MCP-Protocol-
Version: $PROTO" -H 'Mcp-Method: tools/call' -H "Mcp-Name: $HIGH_IMPACT_TOOL" -H
"Authorization: Bearer $TOKEN" -d "$body" 2>/dev/null)
case "$(printf '%s' "$raw" | sed -e 's/^[[:space:]]*//' | cut -c1)" in
 '{') resp="$raw" ;;
 *) resp=$(printf '%s\n' "$raw" | sed -n 's/^data: //p' | while IFS= read -r l; do
printf '%s' "$l" | jq -e 'select(.id==1)' 2>/dev/null && break; done) ;;
esac
[ -z "$resp" ] && { echo "ERROR: no response from the enforcement point
(unreachable), cannot verify"; exit 0; }
printf '%s' "$resp" | jq -e . >/dev/null 2>&1 || { echo "ERROR: response is not valid
JSON, cannot verify"; exit 0; }
if printf '%s' "$resp" | jq -e '.error != null' >/dev/null 2>&1; then
 echo "ERROR: request errored (for example an auth or method error), which does not
demonstrate a human approval gate, cannot verify"
elif printf '%s' "$resp" | jq -e "$APPROVAL_JQ" >/dev/null 2>&1; then
 echo "PASS: the high-impact operation produced the deployment's attributable human-
approval signal"
elif printf '%s' "$resp" | jq -e '.result.resultType=="input_required"' >/dev/null
2>&1; then
 echo "REVIEW: the response is input_required, which is not by itself proof of human
approval (it can be elicitation, sampling, or roots), confirm the input request
represents human approval in this deployment"
elif printf '%s' "$resp" | jq -e '.result != null' >/dev/null 2>&1; then
 echo "FAIL: the high-impact operation returned a completed result with no human
approval step"
else
 echo "REVIEW: response did not clearly indicate approval or execution, inspect it"
fi
```

This Recommendation is compliant when a human can deny any invocation, sensitive and high-impact operations are gated with human confirmation at the enforcement point, and the operator's classification is maintained independently of server annotations.

Remediation:

These are host and gateway configuration settings, with no portable command. Configure human-in-the-loop controls at the host or the gateway, the enforcement point, so a human can deny any tool invocation and sensitive or high-impact operations require explicit confirmation or approval before execution, while routine reads proceed without a prompt but are logged. Maintain the operator's classification of which operations are sensitive or high-impact independently of server-provided annotations (see Recommendation 3.2.3), and review it whenever a tool is added or its behavior changes. Apply the confirmation to a multi-step sequence that is collectively high-impact, not only to an individual call. Show the tool inputs to the user before the call so the confirmation is informed.

Default Value:

By default, hosts apply the same confirmation setting to every tool call, without regard to how risky the call is. Either all calls run without asking, or all calls prompt, so a destructive or high-impact operation is treated no differently from a routine read.

References:

1. MCP specification (security best practices): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices
2. MCP specification (Server Features / Tools, humans retain ability to deny invocations): https://modelcontextprotocol.io/specification/2026-07-28/server/tools
3. SEP-2322 (Multi Round-Trip Requests): https://modelcontextprotocol.io/seps/2322-MRTR

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

4.2 Resource & Filesystem Restrictions

4.2.1 Ensure filesystem scope is restricted to least privilege and enforced by OS-level controls (Manual)

Profile Applicability:

- Level 1

Description:

Each MCP server process is restricted to the minimum filesystem paths its declared functionality requires, and that restriction is enforced with OS-level access controls such as filesystem ACLs, mandatory access control policies (SELinux or AppArmor), or container bind mounts, which the kernel imposes regardless of server behavior. The intended scope may be expressed through MCP Roots or, under the `2026-07-28` revision where Roots is deprecated, through workspace paths passed as tool parameters or resource URIs, and how scope is expressed does not change the requirement. Whatever the expression mechanism, treat it as an advisory hint and do not rely on it for containment. The OS-level controls are the actual security boundary, and they enforce the same least-privilege scope at the kernel level.

Rationale:

Any scope hint conveyed through the protocol, whether MCP Roots or a path passed as a tool parameter, is advisory: a server that ignores or mishandles it can access any file its OS user account can reach. The MCP specification is explicit that Roots are coordination hints, not a security boundary, and the same is true of paths passed as tool arguments. Only OS-level controls such as ACLs, SELinux or AppArmor, and container bind mounts are enforced by the kernel and cannot be bypassed by application code, which makes them the effective boundary. Scope hints help well-behaved servers stay in bounds. OS controls contain servers that do not.

Impact:

Narrowing scope and adding OS sandboxing may require refactoring file-access paths in server logic to stay within the approved set.

Audit:

Review how the server's intended filesystem scope is expressed (MCP Roots, or under the `2026-07-28` revision where Roots is deprecated, workspace paths passed as tool parameters or resource URIs) and confirm the approved scope is the minimum the server's declared functionality requires. Then verify that OS-level controls deny the service account access outside that scope. Run the following command to confirm the service account cannot read a chosen sensitive path outside its approved scope, pointing the probe at a path the remediation denies, for example a file under the sensitive directory or a system secret such as `/etc/shadow`:

```
SVC_ACCT="<MCP_SERVICE_ACCOUNT>"
OUT_OF_SCOPE="<SENSITIVE_PATH_OUTSIDE_SCOPE>"
command -v sudo >/dev/null 2>&1 || { echo "ERROR: sudo is required to switch to the
service account (cannot verify)"; exit 0; }
id "$SVC_ACCT" >/dev/null 2>&1 || { echo "ERROR: service account
<MCP_SERVICE_ACCOUNT> not found (cannot verify)"; exit 0; }
[ -e "$OUT_OF_SCOPE" ] || { echo "ERROR: probe path <SENSITIVE_PATH_OUTSIDE_SCOPE>
not found (choose a sensitive path outside the approved scope)"; exit 0; }
sudo -n -u "$SVC_ACCT" true 2>/dev/null || { echo "ERROR: cannot switch to
<MCP_SERVICE_ACCOUNT> via sudo (run as root or a sudoer), not a PASS"; exit 0; }
if sudo -n -u "$SVC_ACCT" test -r "$OUT_OF_SCOPE"; then
  echo "FAIL: service account can read $OUT_OF_SCOPE, a path outside its approved
scope"
else
  echo "PASS: service account cannot read $OUT_OF_SCOPE (this probed path only, not
proof of the whole deny-by-default boundary)"
fi
```

A `PASS` covers only the probed path, so repeat against several representative sensitive paths outside the approved scope. This Recommendation is compliant when the audit reports `PASS` on those paths and a reviewer confirms that the approved scope is itself minimal and that the OS boundary denies access by default rather than only on the probed paths.

Remediation:

Enforce containment with a control that denies by default. The strongest options are a mandatory access control profile (AppArmor or SELinux) confining the process to its working set, or running the server in a mount namespace or container where only the work directory is bind-mounted in (read-only where writes are not needed) so paths outside it are not visible at all. POSIX ACLs are a useful supplement but are a grant mechanism, not a default-deny boundary. They constrain only the paths you name and do not remove the account's standard-permission access to everything else. Where ACLs are used as a supplement, restrict the work directory, set a default ACL so newly created files inherit it, and apply targeted denies to known-sensitive directories. Run the following commands to apply the supplementary ACLs:

```
setfacl -R -m u:<MCP_SERVICE_ACCOUNT>:rX <WORK_DIR>
setfacl -R -d -m u:<MCP_SERVICE_ACCOUNT>:rX <WORK_DIR>
setfacl -R -m u:<MCP_SERVICE_ACCOUNT>:--- <SENSITIVE_DIR>
```

The MAC profile or bind-mount is the boundary, and the ACLs harden within it.

Default Value:

By default, the scope hint, whether Roots or a path argument, is advisory, and most MCP servers do not validate that file access stays within it. The service account can reach any file permitted by its OS user context.

References:

1. MCP specification (Client Features / Roots): https://modelcontextprotocol.io/specification/2026-07-28/client/roots
2. MCP specification (Deprecated Features): https://modelcontextprotocol.io/specification/2026-07-28/deprecated
3. SEP-2577 (Deprecate Roots, Sampling, and Logging): https://modelcontextprotocol.io/seps/2577-deprecate-roots-sampling-and-logging

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●

4.2.2 Ensure MCP stdio server subprocess environment is restricted to the minimum required variables (Automated)

Profile Applicability:

- Level 1

Description:

When spawning stdio MCP server subprocesses, the host constructs the subprocess environment from a documented allowlist of required variables rather than inheriting the full parent environment. Only variables required for the server's declared functionality, as documented in the capability baseline, are forwarded, and all others are excluded by default. The allowlist is a per-server configuration artifact, reviewed whenever the server's capability baseline changes.

Rationale:

stdio servers run as subprocesses, and when a process spawns a child the operating system passes down the full parent environment by default unless the caller supplies a restricted one, so a server inherits the spawning process's complete environment. This is runtime behavior of the host and the operating system, not something MCP defines. On developer workstations and CI hosts that environment commonly holds sensitive values: cloud credentials, personal API tokens, internal service credentials, and SSH key material. A malicious or compromised stdio server can read any inherited variable with no filesystem access or elevated privileges since the values are simply present in its environment. Constructing the subprocess environment from an allowlist limits what any one server can access to only what it was explicitly granted.

Impact:

Building and maintaining per-server allowlists is a moderate one-time effort per server. The protection against credential exfiltration from a compromised stdio server is substantial, especially in developer environments carrying many sensitive variables.

Audit:

Inspect the actual environment of a running stdio server subprocess and confirm it contains only the variables on that server's documented allowlist, with no inherited sensitive variables. The allowlist file lists one approved variable name per line. Run the following command to validate the allowlist format and compare the subprocess environment against it:

```
PID="<STDIO_SERVER_PID>"
ALLOW="<ALLOWLIST_FILE>"
if [ ! -r "/proc/$PID/environ" ]; then
  echo "ERROR: cannot read /proc/$PID/environ (wrong PID or insufficient privilege)"
elif [ ! -s "$ALLOW" ]; then
  echo "ERROR: allowlist file <ALLOWLIST_FILE> missing or empty (cannot verify)"
else
  malformed=$(grep -vE '^[A-Za-z_][A-Za-z0-9_]*$' "$ALLOW" | grep -v '^[[:space:]]*$'
|| true)
  if [ -n "$malformed" ]; then
    echo "REVIEW: allowlist contains malformed entries (expect one valid environment-
variable name per line): $(printf '%s' "$malformed" | tr '\n' ' ')"
  else
    vars=$(tr '\0' '\n' < "/proc/$PID/environ" | sed 's/=.*//' | sort -u)
    if [ -z "$vars" ]; then
      echo "ERROR: no environment read (empty result, not a PASS)"
    else
      leaked=$(printf '%s\n' "$vars" | grep -vxF -f "$ALLOW" || true)
      if [ -z "$leaked" ]; then
        echo "PASS: subprocess environment is within the allowlist"
      else
        echo "FAIL: non-allowlisted variables present: $(printf '%s' "$leaked" | tr
'\n' ' ')"
      fi
    fi
  fi
fi
```

A `FAIL` listing a sensitive variable confirms the parent environment is leaking into the subprocess. This Recommendation is compliant when the audit reports `PASS`, meaning the subprocess environment contains only allowlisted variables.

Remediation:

Configure the host spawn logic to pass only the variables on each server's documented allowlist to that stdio subprocess, rather than inheriting the parent environment. This is host spawn-configuration, and the allowlist is a per-server artifact. Maintain the allowlist with the minimum required variables for each server, one valid environment-variable name per line and nothing else. The following creates the allowlist file the audit reads:

```
cat > <ALLOWLIST_FILE> <<'EOF'
MCP_API_TOKEN
MCP_LOG_LEVEL
PATH
EOF
```

This command creates the allowlist artifact only and does not change the running subprocess environment, so the Audit verdict flips from `FAIL` to `PASS` only after the host spawn logic is configured to pass exactly the allowlisted variables, a step with no single portable command. Review the allowlist whenever the server's capability baseline changes, and document which secrets each server requires.

Default Value:

Most operating-system process-spawn APIs give a child process the full parent environment unless the caller supplies a restricted one, so a stdio server subprocess inherits the complete environment of the spawning host process by default, including all secrets and credentials in scope for that user context. MCP does not define subprocess environment inheritance, so the default depends on the host implementation.

References:

1. MCP specification (Base Protocol / Transports / stdio): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/stdio

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

4.2.3 Ensure untrusted Resource URIs are not directly dereferenced outside MCP without scheme and destination policy (Manual)

Profile Applicability:

- Level 1

Description:

The host does not directly dereference an untrusted Resource URI or resource link through the browser, an OS protocol handler, or an arbitrary network fetcher unless the scheme and destination are explicitly permitted. A Resource URI is an identifier, and its scheme does not authorize local action. A `file://` Resource URI identifies a resource that behaves like a filesystem but need not map to a physical local file, so the host reads it through the server with `resources/read` rather than opening a local path. The host distinguishes three handling paths. For MCP-mediated resource access, the host sends the URI to the MCP server in a `resources/read` request and does not dereference it locally. For direct external fetch or navigation, which the specification allows for `https://` resources a client can load on its own, the host applies a scheme and destination allowlist, requires user confirmation for destinations that are not pre-approved, and applies SSRF protections. For OS protocol handler invocation, the host defaults to deny unless the scheme is explicitly approved. The allowlist covers both Resource URIs returned directly by the server and URIs the model or client constructs from tool outputs.

Rationale:

A URI that the host directly dereferences without policy can trigger unintended behavior. A `file:` URI opened locally can expose filesystem contents to an attacker-controlled destination. A `data:` URI can carry content that triggers application handlers. A custom protocol handler can invoke a native application with attacker-supplied arguments. An `https:` URI fetched directly can be pointed at an internal address for server-side request forgery. These are risks of direct dereferencing, not of the Resource URI itself, which is why this Recommendation restricts local navigation and external fetch rather than the identifier a server returns for `resources/read`. The risk is compounded in agentic workflows where the model follows links without per-link human review.

Impact:

Friction is minimal for workflows that use only standard HTTPS resource links, and rises where a workflow relies on other schemes or on destinations that are not pre-approved.

Audit:

Where the host exposes a direct-open surface for URIs, confirm it rejects a disallowed scheme while handling an approved one, so the rejection is attributable to scheme filtering. This tests the host's direct-navigation policy, it does not test `resources/read`, and it does not imply that `file://` Resource URIs are invalid. Run the following command against a host direct-open surface, which probes both an approved `https` URI and a disallowed `file:` URI and sends each URI as a percent-encoded query parameter so a query-based scheme filter such as the nginx example in the remediation can inspect it:

```
CTRL="<HOST_RESOURCE_OPEN_URL>"
APPROVED_URI="<APPROVED_HTTPS_URI>"
DISALLOWED_URI="file:///etc/passwd"
allow=$(curl -sS -G --connect-timeout 3 -m 6 -o /dev/null -w '%{http_code}' --data-
urlencode "uri=$APPROVED_URI" "$CTRL")
deny=$(curl -sS -G --connect-timeout 3 -m 6 -o /dev/null -w '%{http_code}' --data-
urlencode "uri=$DISALLOWED_URI" "$CTRL")
if [ "$allow" = "000" ] || [ "$deny" = "000" ]; then
 echo "ERROR: control surface unreachable (cannot verify)"
elif printf '%s' "$deny" | grep -qE '^2'; then
 echo "FAIL: disallowed scheme accepted (file: returned $deny)"
elif printf '%s' "$allow" | grep -qE '^(4|5)'; then
 echo "REVIEW: approved https scheme also rejected ($allow), so the file: rejection
($deny) is not attributable to scheme filtering"
elif printf '%s' "$deny" | grep -qE '^(400|403)'; then
 echo "PASS: disallowed scheme rejected ($deny) while the approved scheme was handled
($allow)"
else
 echo "REVIEW: disallowed scheme returned $deny, not a clear accept or reject,
inspect the control surface"
fi
```

Where the host has no such surface, confirm by configuration review that the host applies the scheme and destination allowlist to direct dereferencing and prompts for confirmation on external navigation. This Recommendation is compliant when direct dereferencing of a disallowed scheme is rejected attributably and the allowlist covers both server-returned and model-constructed URIs, while MCP-mediated `resources/read` of any legitimate Resource URI scheme is unaffected.

Remediation:

Configure the host so that direct dereferencing of a Resource URI is governed by a scheme and destination allowlist covering only what legitimate workflows require (typically `https` only), and so that the host prompts for user confirmation on any external navigation that is not pre-approved. This is client configuration, and there is no portable command. Do not restrict the Resource URI identifier itself, which the host reads through the server with `resources/read`. The allowlist must cover both resource URIs returned directly by the server and URIs the model or client may construct from tool outputs. Where direct navigation is proxied through a gateway, a scheme filter can be added there as a coarse backstop that does not replace the client-side allowlist. The following nginx example rejects non-HTTP(S) direct-navigation requests, with the `location` path set to the site's actual direct-navigation route:

```
map $arg_uri $uri_scheme_ok {
 default 0;
 "~*^https?(://|%3a%2f%2f)" 1;
}

location /resource-navigation/ {
 if ($uri_scheme_ok = 0) { return 400; }
}
```

For a gateway that percent-decodes the argument before matching, for example through njs or a decode module, the literal `"~*^https?://"` test is sufficient, and the encoded alternative is not needed. Choose whichever matches how the gateway reads the argument.

Default Value:

The MCP protocol does not define a universal client-side policy for direct navigation or a universal allowlist of permitted schemes. Each host decides whether to directly dereference a Resource URI, and under what scheme and destination policy, so this behavior must be verified for the deployed client.

References:

1. MCP specification (Server Features / Resources): https://modelcontextprotocol.io/specification/2026-07-28/server/resources
2. MCP specification (security best practices, SSRF): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	9.3 Maintain and Enforce Network-Based URL Filters Enforce and update network-based URL filters to limit an enterprise asset from connecting to potentially malicious or unapproved websites. Example implementations include category-based filtering, reputation-based filtering, or through the use of block lists. Enforce filters for all enterprise assets.		●	●

4.3 MCP Apps

4.3.1 Ensure MCP Apps content is rendered in sandboxed isolation with Content Security Policy enforced from declared metadata (Manual)

Profile Applicability:

- Level 1

Description:

Where the MCP Apps extension (`io.modelcontextprotocol/ui`) is in use, the host renders all UI resource content inside a sandboxed execution context with a Content Security Policy constructed from, and never looser than, the domains declared in the resource's metadata. The extension delivers HTML documents (MIME type `text/html;profile=mcp-app`, URI scheme `ui://`) that the host renders in a sandboxed iframe. The extension specification requires sandboxed rendering for all view content and requires the host to build the CSP from the declared `csp` metadata (`connectDomains`, `resourceDomains`, `frameDomains`, `baseUriDomains`). When that metadata is absent, a restrictive default applies (`default-src 'none'`, with `frame-src 'none'` and `base-uri 'self'` where those are not declared). The host may restrict further but must not allow any undeclared external domain. Web-based hosts additionally interpose the sandbox proxy on a separate origin, as the extension specifies. The host logs the effective CSP applied to each rendered resource.

Rationale:

MCP Apps places a browser execution context, running HTML from the MCP server, inside the host application's trust boundary. The extension's own threat model names malicious HTML delivery, sandbox escape, unauthorized tool execution, and data exfiltration. The sandbox and the CSP are the containment boundary. With them, a malicious or compromised UI resource is confined to its frame and its declared destinations. Without them, it executes with the reach of the host page and can move data to arbitrary origins. The no-loosening rule matters as much as the sandbox itself, because a host that helpfully adds domains beyond the declaration turns the server's auditable, reviewable declaration into a dead letter. Enforcing from declared metadata also makes exfiltration destinations enumerable in advance: a reviewer of a server's UI resource can read exactly which origins its frames may contact.

Impact:

UI resources that legitimately need external origins must declare them, and an undeclared destination breaks at render time. That breakage is intended and surfaces during server onboarding, rather than as silent exposure later. Web-based hosts take on the operational cost of the separate-origin sandbox proxy. Hosts that do not support the extension are unaffected, because the extension is optional and negotiated, and their tools degrade to text-only results.

Audit:

Confirm by host configuration review and rendering inspection that UI resources render inside a sandboxed iframe, and for web-based hosts that the sandbox proxy runs on an origin distinct from the host's. Render a test UI resource with declared `csp` metadata and inspect the effective `Content-Security-Policy` on the frame, confirming it permits no undeclared external domain. The host may be stricter, and the mandatory restrictive defaults may include local, `'self'`, or `data:` sources defined by the extension, which are not server-declared domains and are expected. Render a resource with no csp metadata and confirm the restrictive default applies (`default-src 'none'`, with no external connect, resource, or frame origins). Attempt a fetch from the rendered view to an undeclared origin and confirm it is blocked. As defense-in-depth, confirm the servers permitted to deliver `ui://` resources are limited to the approved allowlist and that UI resource content participates in content-hash pinning.

The command below checks logging completeness only: it confirms each rendered view is logged as sandboxed with an effective CSP and an attributed server, but a non-empty yet permissive CSP still passes, so CSP correctness is adjudicated by the render inspection above. Run it after setting the log path:

```
LOG="<HOST_APP_LOG>"
command -v jq >/dev/null 2>&1 || { echo "ERROR: jq is required and not installed
(cannot verify)"; exit 0; }
[ -s "$LOG" ] || { echo "ERROR: host application log <HOST_APP_LOG> not found or
empty (cannot verify)"; exit 0; }
renders=$(grep -E '"event":"app_render"' "$LOG")
n=$(printf '%s\n' "$renders" | grep -c .)
[ "$n" -gt 0 ] || { echo "REVIEW: no MCP Apps render events found (extension may not
be in use)"; exit 0; }
printf '%s\n' "$renders" | jq -e . >/dev/null 2>&1 || { echo "ERROR: render records
are not parseable (cannot verify)"; exit 0; }
echo "count: $n app_render events in window"
bad=$(printf '%s\n' "$renders" | jq -c 'select((.csp==null or .csp=="") or
(.sandboxed!=true) or (.server_identity==null or .server_identity==""))')
if [ -n "$bad" ]; then
  echo "FAIL: render events missing an effective CSP, the sandbox flag, or server
identity (logging completeness)"
else
  echo "PASS: every rendered app view is logged sandboxed with a non-empty effective
CSP and server identity (logging completeness, not CSP correctness)"
fi
```

This Recommendation is compliant when sandboxing is unconditional, the effective policy permits no undeclared external domain (the host may be stricter), the restrictive default applies in the absence of a declaration, undeclared-origin requests are blocked, and effective policies are logged.

Remediation:

These are host configuration and host implementation settings, with no portable command. Configure the host to render all MCP Apps content in sandboxed iframes, building the Content Security Policy from the resource's declared csp metadata and applying the extension's restrictive default when the declaration is absent. Do not add external domains beyond the declaration, though the host may apply a stricter policy than it. For web-based hosts, deploy the extension's sandbox proxy on a separate origin and route all view communication through it. Enable logging of the effective CSP per rendered resource. Restrict UI-delivering servers to the approved allowlist and include UI resource content in content-hash verification.

Default Value:

The extension specification defines a restrictive default CSP when metadata is absent and requires sandboxing unconditionally. Whether a given host implements these requirements, and whether CSP logging is enabled, varies by host implementation. Verify the specific host in use.

References:

1. MCP Extensions (Overview): https://modelcontextprotocol.io/extensions/overview
2. MCP Apps extension repository: https://github.com/modelcontextprotocol/ext-apps
3. MCP Apps extension specification: https://github.com/modelcontextprotocol/ext-apps/blob/main/specification/2026-01-26/apps.mdx

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

4.3.2 Ensure MCP Apps permission requests default-deny, all app-to-host messages are logged, and high-risk channels are consented (Manual)

Profile Applicability:

- Level 1

Description:

Where the MCP Apps extension (`io.modelcontextprotocol/ui`) is in use, the host treats an app's requested capabilities as default-deny, logs every message the app sends to the host, and requires explicit user consent for the high-risk channels. The extension lets a UI resource request the camera, microphone, geolocation, and `clipboardWrite` permissions and tells apps not to assume any are granted. The host does not honor a requested permission automatically. It grants one only on a recorded per-permission user decision and denies the rest by default. All View-to-host JSON-RPC is auditable, so the host logs every app-to-host message, including `ui/message`, `ui/update-model-context`, `ui/open-link`, `tools/call`, and `resources/read`, with the requesting server identity. The extension leaves per-message consent to the host, so as a requirement beyond the extension, the host additionally gates the two high-risk channels behind explicit user consent: `ui/message`, which sends content to the host chat, and `ui/update-model-context`, which injects content into the model context. The host also enforces the `_meta.ui.visibility` declaration, keeping tools whose visibility omits `"model"` out of the agent's tool list and rejecting an app `tools/call` for any tool whose visibility omits `"app"`, so an app cannot invoke a tool it was not granted.

Rationale:

An MCP App runs in the host's trust boundary and can ask for device access, inject content into the model's context, send messages into the chat, and call tools. The ui/update-model-context channel is a direct prompt-injection vector because an untrusted UI resource can place text in front of the model. Default-deny permissions keep a compromised or over-eager app from silently reaching the camera, microphone, location, or clipboard. Because the extension makes all View-to-host communication auditable but leaves per-message consent to the host, the host logs every message for attribution and gates the two high-risk channels behind explicit consent, making context injection and chat insertion visible, attributable, and mediated rather than silent. Visibility enforcement stops an app from calling tools never meant to be reachable from a UI, which matters because app-invoked tool calls bypass the model and its own guardrails.

Impact:

Per-permission prompts and message consent add interaction cost when an app first requests a capability or injects context, though most read-only apps request nothing and proceed without friction. Enforcing visibility means an app that relies on calling a tool not visible to apps breaks, which is the intended containment and surfaces during onboarding. Hosts that do not support the extension are unaffected.

Audit:

Confirm by host configuration review that requested permissions are default-deny and granted only on a recorded user decision, that all app-to-host messages are logged with attribution, and that the two high-risk channels are consented. Attempt an app `tools/call` for a tool whose declared visibility does not include `"app"` and confirm the host rejects it, and confirm a tool whose visibility excludes `"model"` does not appear in the agent's tool list. Run the following command:

```
LOG="<HOST_APP_LOG>"
command -v jq >/dev/null 2>&1 || { echo "ERROR: jq is required and not installed
(cannot verify)"; exit 0; }
[ -s "$LOG" ] || { echo "ERROR: host application log <HOST_APP_LOG> not found or
empty (cannot verify)"; exit 0; }
events=$(grep -E '"event":"app_permission"|"event":"app_message"' "$LOG")
n=$(printf '%s\n' "$events" | grep -c .)
[ "$n" -gt 0 ] || { echo "REVIEW: no MCP Apps permission or message events found
(extension may not be in use)"; exit 0; }
printf '%s\n' "$events" | jq -e . >/dev/null 2>&1 || { echo "ERROR: records are not
parseable (cannot verify)"; exit 0; }
echo "count: $n app permission/message events in window"
autogrant=$(printf '%s\n' "$events" | jq -c 'select(.event=="app_permission"
and .granted==true and ((.decision_source!="user") or
(((.decision//"")|test("^(user_granted|user_denied)$"))|not)))')
missattr=$(printf '%s\n' "$events" | jq -c 'select(.server_identity==null
or .server_identity=="")')
hirisk=$(printf '%s\n' "$events" | jq -c 'select(.event=="app_message" and
(((.channel//"")=="ui/message") or ((.channel//"")=="ui/update-model-context")) and
((.decision_source!="user") or
(((.consent//"")|test("^(accepted|declined)$"))|not)))')
if [ -n "$autogrant" ]; then
  echo "FAIL: a permission was granted without an explicit user decision (default-
deny not enforced)"
elif [ -n "$missattr" ]; then
  echo "FAIL: an app permission or message event is missing server identity (not
attributable or logged)"
elif [ -n "$hirisk" ]; then
  echo "FAIL: a high-risk channel message (ui/message or ui/update-model-context)
lacks an explicit user consent decision"
else
  echo "PASS: permissions are granted only on an explicit user decision, all app-to-
host messages are attributed, and the high-risk channels carry explicit user consent"
fi
```

This Recommendation is compliant when permissions are default-deny, all app-to-host messages are logged with attribution, the high-risk channels are consented, and the host enforces tool visibility.

Remediation:

These are host configuration and host implementation settings, with no portable command. Configure the host to deny all four MCP Apps permissions by default and to grant one only on an explicit per-permission user decision recorded with a user decision source, never automatically from the resource's request. Log every app-to-host message with the requesting server identity, and gate the two high-risk channels, `ui/message` and `ui/update-model-context`, behind an explicit user consent decision recorded with a user decision source. Enforce the `_meta.ui.visibility` declaration so that an app cannot call a tool whose visibility omits `"app"`, and the agent's tool list omits any tool whose visibility omits `"model"`.

Default Value:

The extension leaves it to the host whether to honor a requested permission, and instructs apps not to assume any are granted, so a host that simply sets the requested `allow` attributes would grant them without a decision. Tool visibility defaults to `["model", "app"]` when the declaration is omitted. Whether a given host default-denies permissions, gates the message channels, logs decisions, and enforces visibility varies by host.

References:

1. MCP Apps extension repository: https://github.com/modelcontextprotocol/ext-apps
2. MCP Apps extension specification: https://github.com/modelcontextprotocol/ext-apps/blob/main/specification/2026-01-26/apps.mdx
3. MCP Extensions (Overview): https://modelcontextprotocol.io/extensions/overview

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

5 Server Configuration

A server that trusts its inputs, conflates transport features with security, or exposes more of the filesystem than a request needs becomes the weak point behind every transport and authorization control in front of it. MCP servers accept structured arguments, resolve resource and prompt templates, manage sessions, and perform side-effecting operations, and each of these is an opportunity to accept something malformed, replay something twice, or reach a path that was never meant to be reachable.

The Recommendations in this section harden the server's own handling of what it accepts and how it maintains state. They validate tool schemas, resource templates, and prompt template arguments before use, separate logs from the protocol stream, rate-limit change notifications, keep sessions out of the authentication path, disable legacy resumption behavior, prevent path traversal and arbitrary filesystem access, enforce authorization and expiry on the server side, and require idempotency keys for side-effecting tools. The theme is that the server validates everything it accepts and never lets a convenience of the transport stand in for a security decision.

5.1 Input & Schema Validation

5.1.1 Ensure tool schemas and argument types are validated (Automated)

Profile Applicability:

- Level 1

Description:

The server validates every tool input and output against the tool's advertised schema at the server middleware layer before the tool executes. Validation uses the JSON Schema dialect each schema declares in `$schema`, defaulting to `2020-12` when none is declared. A `tools/call` request that fails the `CallToolRequest` structure is a protocol error. A well-formed call whose arguments fail the tool's input schema is surfaced as a tool execution error in a normal result with `isError` set to `true`, not as a `-32602` protocol error. Where a tool declares an `outputSchema`, the server returns structured results that conform to it. The server sanitizes the whole tool schema, including tool names, parameter names, descriptions, and outputs, not only `inputSchema`. The validator accepts the full schema vocabulary the `2026-07-28` revision permits, including composition keywords, conditionals, and internal `$ref` and `$defs`. `inputSchema` keeps a `type: "object"` root, and `outputSchema` may be any valid schema, including arrays and primitives. Implementations do not automatically dereference external `$ref` URIs, and schema depth and validation time are bounded, as the revision recommends.

Rationale:

Unvalidated tool arguments let a malformed or malicious call reach tool logic, and unsanitized schema fields let a malicious server inject instructions the model reads when choosing how to call a tool. Validating at the middleware layer before dispatch closes the first path, and sanitizing all schema fields closes the second. Returning an input-validation failure as a tool execution error rather than a protocol error keeps it actionable to the caller, which is the behavior the specification defines, while a structurally malformed request stays a protocol error. Auto-dereferencing an external `$ref` URI turns schema validation into a server-side request forgery primitive, and unbounded composition or nesting depth turns it into a denial-of-service primitive, so the revision's prohibition on external `$ref` and its recommended depth and time bounds close both.

Impact:

Schema validation adds minor per-call overhead. Bounding schema depth and validation time is a one-time configuration cost. Returning input-validation failures as tool execution errors rather than protocol errors is a small change to the error path for servers that currently answer such calls with `-32602`. Tools whose schemas were generated against an older draft continue to validate as long as they declare that draft.

Audit:

Verify that every advertised input and output schema compiles under its declared dialect and that a schema-violating call is not executed. Run the following command to compile each advertised schema under its declared dialect, substituting the endpoint. It requires `ajv`:

```
MCP_ENDPOINT="<MCP_ENDPOINT>"
raw=$(curl -sS --connect-timeout 5 -m 10 -X POST "$MCP_ENDPOINT" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'MCP-Protocol-Version: 2026-07-28' \
 -H 'Mcp-Method: tools/list' \
 -d
'{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{"_meta":{"io.modelcontextpro
tocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}'
2>/dev/null)
[ -n "$raw" ] || { echo "ERROR: no response from $MCP_ENDPOINT"; exit 0; }
obj=$(printf '%s' "$raw" | jq -c 'select(.jsonrpc) | select(.id==1)' 2>/dev/null |
head -n1)
if [ -z "$obj" ]; then
 obj=$(printf '%s\n' "$raw" | sed -n 's/^data:[[:space:]]*//p' | while IFS= read -r
d; do printf '%s' "$d" | jq -e 'select(.id==1)' >/dev/null 2>&1 && { printf '%s'
"$d"; break; }; done)
fi
[ -n "$obj" ] || { echo "ERROR: no JSON-RPC response with id 1 in tools/list reply";
exit 0; }
tmpd=$(mktemp -d)
printf '%s' "$obj" | jq -c '.result.tools[]? | {t:.name, k:"inputSchema",
s:.inputSchema}, (select(.outputSchema!=null) | {t:.name, k:"outputSchema",
s:.outputSchema})' > "$tmpd/schemas.jsonl" 2>/dev/null
[ -s "$tmpd/schemas.jsonl" ] || { echo "ERROR: no inputSchema present on any tool";
rm -rf "$tmpd"; exit 0; }
bad=0; review=0; total=0
while IFS= read -r rec; do
 total=$((total+1))
 tname=$(printf '%s' "$rec" | jq -r '.t')
 kind=$(printf '%s' "$rec" | jq -r '.k')
 printf '%s' "$rec" | jq -c '.s' > "$tmpd/s.json"
 ext=$(jq '[.. | objects | select(has("$ref")) | .["$ref"]] | map(select(test("^[a-
z][a-z0-9+.-]*://";"i"))) | length' "$tmpd/s.json" 2>/dev/null)
 [ -n "$ext" ] || ext=0
 if [ "$ext" -gt 0 ]; then
  review=$((review+1)); echo "detail: $tname $kind declares an external \$ref, a
hardened validator refuses to dereference it"; continue
 fi
 dialect=$(jq -r '.["$schema"] // ""' "$tmpd/s.json" 2>/dev/null)
 case "$dialect" in
  ""|*2020-12*) spec=draft2020 ;;
  *2019-09*)    spec=draft2019 ;;
  *draft-07*)   spec=draft7 ;;
  *) review=$((review+1)); echo "detail: $tname $kind declares an unsupported dialect
$dialect"; continue ;;
 esac
 ajv compile -s "$tmpd/s.json" --spec=$spec >/dev/null 2>&1 || { bad=$((bad+1)); echo
```

```
"detail: $tname $kind does not compile under $spec"; }
done < "$tmpd/schemas.jsonl"
rm -rf "$tmpd"
if [ "$bad" -gt 0 ]; then
 echo "FAIL: $bad of $total advertised schema(s) do not compile under their declared
dialect"
elif [ "$review" -gt 0 ]; then
 echo "REVIEW: $review of $total advertised schema(s) use an unsupported dialect or
an external \$ref, attribute before concluding"
else
 echo "PASS: all $total advertised schema(s) compile under their declared JSON Schema
dialect"
fi
```

Then confirm the server does not execute a schema-violating call. Run the following command, substituting a tool name and an argument object that violates that tool's input schema, for example an omitted required field or a wrong-typed field:

```
MCP_ENDPOINT="<MCP_ENDPOINT>"
TOOL="<TOOL_NAME>"
ARGS='<VIOLATING_ARGS_JSON>'
resp=$(curl -sS --connect-timeout 5 -m 10 -X POST "$MCP_ENDPOINT" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'MCP-Protocol-Version: 2026-07-28' \
 -H 'Mcp-Method: tools/call' \
 -H "Mcp-Name: $TOOL" \
 -d
"{\"jsonrpc\":\"2.0\",\"id\":2,\"method\":\"tools/call\",\"params\":{\"name\":\"$TOOL
\",\"arguments\":$ARGS,\"_meta\":{\"io.modelcontextprotocol/protocolVersion\":\"2026-
07-28\",\"io.modelcontextprotocol/clientInfo\":{\"name\":\"cis-benchmark-
audit\",\"version\":\"1.0\"},\"io.modelcontextprotocol/clientCapabilities\":{}}}}"
2>/dev/null)
[ -n "$resp" ] || { echo "ERROR: no response to tools/call probe"; exit 0; }
obj=$(printf '%s' "$resp" | jq -c 'select(.jsonrpc) | select(.id==2)' 2>/dev/null |
head -n1)
if [ -z "$obj" ]; then
 obj=$(printf '%s\n' "$resp" | sed -n 's/^data:[[:space:]]*//p' | while IFS= read -r
d; do printf '%s' "$d" | jq -e 'select(.id==2)' >/dev/null 2>&1 && { printf '%s'
"$d"; break; }; done)
fi
[ -n "$obj" ] || { echo "ERROR: no JSON-RPC response with id 2 in tools/call reply";
exit 0; }
haserr=$(printf '%s' "$obj" | jq -r 'has("error")' 2>/dev/null)
iserr=$(printf '%s' "$obj" | jq -r '.result.isError // empty' 2>/dev/null)
hasresult=$(printf '%s' "$obj" | jq -r 'has("result")' 2>/dev/null)
if [ "$iserr" = "true" ]; then
 echo "PASS: the server surfaced the invalid input as a tool execution error
(isError) rather than executing it"
elif [ "$haserr" = "true" ]; then
 code=$(printf '%s' "$obj" | jq -r '.error.code // empty' 2>/dev/null)
 echo "REVIEW: the server rejected the invalid input with a protocol error (code
$code) rather than a tool execution error, attribute before concluding"
elif [ "$hasresult" = "true" ]; then
 echo "FAIL: the server returned a normal tool result for schema-violating input, the
input was not validated before execution"
else
 echo "ERROR: no result or error field in the tools/call response"
fi
```

Then confirm by configuration review that the validator does not auto-dereference external `$ref` URIs, that schema depth and validation time are bounded, and that all schema fields are sanitized before presentation to the model. This Recommendation is compliant when every advertised input and output schema compiles under its declared dialect, a schema-violating call is surfaced as a tool execution error rather than executed, and the validator configuration refuses external `$ref` and bounds schema depth and time.

Remediation:

Add a validation middleware layer that runs before tool dispatch and validates every tool call's arguments against the tool's input schema under the dialect the schema declares. Return a schema-violating call as a tool execution error in a normal result with `isError` set `true`, and reject a structurally malformed `tools/call` request as a protocol error. Where a tool declares an `outputSchema`, validate the structured result against it before returning. Sanitize all schema fields, meaning tool names, parameter names, descriptions, and outputs, not only `inputSchema`, before they reach the model or downstream use. Tighten schemas to strict types, disallow `additionalProperties` where appropriate, and mark required fields. Configure the validator so it does not auto-dereference external `$ref` URIs, and bound schema depth and validation time. This is server middleware and configuration, and the exact validator is implementation-specific, for example `ajv` for TypeScript servers, so there is no portable command.

Default Value:

By default, an MCP server may advertise tool schemas without enforcing them at runtime, may answer an invalid-argument call with a protocol error rather than an actionable tool execution error, and typically passes schema fields other than descriptions to the model unsanitized unless validation is added.

References:

1. MCP specification (Base Protocol / Transports / Streamable HTTP): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http
2. MCP specification (Key Changes): https://modelcontextprotocol.io/specification/2026-07-28/changelog
3. MCP specification (Server Features / Tools, inputSchema and outputSchema with JSON Schema 2020-12): https://modelcontextprotocol.io/specification/2026-07-28/server/tools
4. SEP-1303 (Input Validation Errors as Tool Execution Errors): https://modelcontextprotocol.io/seps/1303-input-validation-errors-as-tool-execution-errors
5. SEP-2106 (Tools inputSchema & outputSchema Conform to JSON Schema 2020-12): https://modelcontextprotocol.io/seps/2106-json-schema-2020-12

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

5.1.2 Ensure resource templates with explicit URI patterns and MIME types are used (Automated)

Profile Applicability:

- Level 1

Description:

The server exposes resources through predictable, well-defined URI template patterns with an explicit MIME type for each resource type. The specification makes `mimeType` optional on resource definitions and templates, and this Recommendation requires it be set explicitly so the client interprets each resource type unambiguously, which is stricter than the base protocol. The server validates every substituted URI-template value against the expected pattern before constructing the final URI. A read of a non-existent resource is rejected with a JSON-RPC `-32602` error, the code the specification defines for that case. Resource templates advertise parameterized resources and are not the exhaustive set a server serves, because a server may also expose individual resources that no template describes. Where a deployment intends the approved templates to be an access-control namespace so that a read outside it is denied, that is a stricter CIS posture, not a base-protocol requirement. Where a deployment also serves resource bytes over HTTP directly to a user agent, for example a separate download endpoint or an `https://` resource the host renders, that HTTP surface sets `X-Content-Type-Options: nosniff` to prevent content-type sniffing. That surface is outside the MCP `resources/read` mechanism, where content is delivered inside the JSON-RPC result with a declared `mimeType`.

Rationale:

Explicit URI templates and MIME types let the client interpret resources predictably and prevent content-type confusion. The specification leaves `mimeType` optional, so an operator can advertise resources without one, and this Recommendation says which side to choose because an absent MIME type forces the client to guess the type. Validating template variables before substitution prevents injection through resource URIs, which the specification also requires. A read of a non-existent resource is rejected with `-32602` rather than answered with an ambiguous empty result, which the specification requires. Treating the approved template set as an access-control namespace is a stronger posture an operator may choose, but it is not implied by the templates alone since the protocol allows a server to serve resources outside any template. Where resource bytes are served over HTTP directly to a user agent, `X-Content-Type-Options: nosniff` stops that agent from reinterpreting a response as a more dangerous type than declared, though that surface is separate from `resources/read`.

Impact:

Impact is low. Defining URI templates and explicit MIME types is straightforward and improves both security and interoperability. Requiring an explicit `mimeType` where the specification makes it optional costs only the one-time work of annotating each resource type. For deployments that serve resource bytes over HTTP directly, the `X-Content-Type-Options: nosniff` header is a standard web security header with no operational downside.

Audit:

Verify that every advertised resource template declares a non-empty URI pattern and MIME type and that a read of a non-existent resource is rejected. Run the following command to list the templates, substituting the endpoint:

```
MCP_ENDPOINT="<MCP_ENDPOINT>"
raw=$(curl -sS --connect-timeout 5 -m 10 -X POST "$MCP_ENDPOINT" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'MCP-Protocol-Version: 2026-07-28' \
 -H 'Mcp-Method: resources/templates/list' \
 -d
'{"jsonrpc":"2.0","id":1,"method":"resources/templates/list","params":{"_meta":{"io.m
odelcontextprotocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}'
2>/dev/null)
[ -n "$raw" ] || { echo "ERROR: no response from $MCP_ENDPOINT"; exit 0; }
obj=$(printf '%s' "$raw" | jq -c 'select(.jsonrpc) | select(.id==1)' 2>/dev/null |
head -n1)
if [ -z "$obj" ]; then
 obj=$(printf '%s\n' "$raw" | sed -n 's/^data:[[:space:]]*//p' | while IFS= read -r
d; do printf '%s' "$d" | jq -e 'select(.id==1)' >/dev/null 2>&1 && { printf '%s'
"$d"; break; }; done)
fi
[ -n "$obj" ] || { echo "ERROR: no JSON-RPC response with id 1 in
resources/templates/list reply"; exit 0; }
count=$(printf '%s' "$obj" | jq '.result.resourceTemplates | length' 2>/dev/null)
[ -n "$count" ] && [ "$count" != "null" ] || { echo "ERROR: resources/templates/list
returned no resourceTemplates array"; exit 0; }
[ "$count" -gt 0 ] || { echo "ERROR: no resource templates advertised, confirm
whether the server exposes templated resources"; exit 0; }
missing=$(printf '%s' "$obj" | jq '[.result.resourceTemplates[] |
select((.uriTemplate|type!="string") or (.uriTemplate=="") or
(.mimeType|type!="string") or (.mimeType==""))] | length' 2>/dev/null)
[ -n "$missing" ] || missing=0
if [ "$missing" -eq 0 ]; then
 echo "PASS: all $count resource template(s) declare a non-empty uriTemplate and an
explicit non-empty mimeType"
else
 echo "FAIL: $missing of $count resource template(s) lack a non-empty uriTemplate or
an explicit non-empty mimeType"
fi
```

Then confirm the server rejects a read of a non-existent resource. Run the following command, substituting the endpoint and a URI within a scheme the server serves but that points to no existing resource:

```
MCP_ENDPOINT="<MCP_ENDPOINT>"
MISSING_URI="<KNOWN_NONEXISTENT_RESOURCE_URI>"
resp=$(curl -sS --connect-timeout 5 -m 10 -X POST "$MCP_ENDPOINT" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'MCP-Protocol-Version: 2026-07-28' \
 -H 'Mcp-Method: resources/read' \
 -H "Mcp-Name: $MISSING_URI" \
 -d
"{\"jsonrpc\":\"2.0\",\"id\":2,\"method\":\"resources/read\",\"params\":{\"uri\":\"$M
ISSING_URI\",\"_meta\":{\"io.modelcontextprotocol/protocolVersion\":\"2026-07-
28\",\"io.modelcontextprotocol/clientInfo\":{\"name\":\"cis-benchmark-
audit\",\"version\":\"1.0\"},\"io.modelcontextprotocol/clientCapabilities\":{}}}}"
2>/dev/null)
[ -n "$resp" ] || { echo "ERROR: no response to resources/read probe"; exit 0; }
obj=$(printf '%s' "$resp" | jq -c 'select(.jsonrpc) | select(.id==2)' 2>/dev/null |
head -n1)
if [ -z "$obj" ]; then
 obj=$(printf '%s\n' "$resp" | sed -n 's/^data:[[:space:]]*//p' | while IFS= read -r
d; do printf '%s' "$d" | jq -e 'select(.id==2)' >/dev/null 2>&1 && { printf '%s'
"$d"; break; }; done)
fi
[ -n "$obj" ] || { echo "ERROR: no JSON-RPC response with id 2 in resources/read
reply"; exit 0; }
errcode=$(printf '%s' "$obj" | jq -r '.error.code // empty' 2>/dev/null)
hasresult=$(printf '%s' "$obj" | jq -r 'has("result")' 2>/dev/null)
case "$errcode" in
 -32602) echo "PASS: a read of a non-existent resource was rejected with -32602
(Invalid Params)" ;;
 -32002) echo "REVIEW: the server used legacy -32002 for resource-not-found, accepted
for backward compatibility but confirm the deployment's target revision" ;;
 "") if [ "$hasresult" = "true" ]; then
       echo "FAIL: the server returned a result for a non-existent resource instead
of a -32602 error"
     else
       echo "ERROR: no error code and no result in the resources/read response"
     fi ;;
 *) echo "REVIEW: the server returned error code $errcode rather than -32602,
attribute before concluding" ;;
esac
```

Where a deployment treats the approved template set as an access-control namespace, confirm a read outside that namespace is denied. Where a deployment serves resource bytes over HTTP directly to a user agent, confirm that HTTP surface sets `X-Content-Type-Options: nosniff`. This Recommendation is compliant when every advertised resource template declares a non-empty URI pattern and MIME type, a read of a non-existent resource is rejected with `-32602`, and any applicable namespace-denial and `nosniff` conditions above hold.

Remediation:

Define a URI template and an explicit MIME type for every resource type in the server configuration, and validate substituted template values against their expected pattern before building the final URI. Configure the server to reject a read of a non-existent resource with `-32602` rather than returning an ambiguous empty result. Where the deployment treats the approved template set as an access-control namespace, deny a read outside that namespace and state the policy explicitly because the base protocol does not require it. Where the deployment also serves resource bytes over HTTP directly to a user agent, configure that HTTP surface, or the reverse proxy fronting it, to set `X-Content-Type-Options: nosniff` on those responses. The following nginx directive sets the header on a proxied resource surface:

```
add_header X-Content-Type-Options "nosniff" always;
```

Default Value:

By default, resources may be advertised without an explicit MIME type because the specification makes mimeType optional. Where a deployment serves resource bytes over HTTP directly, MIME sniffing on that surface is not disabled unless the serving layer sets the `X-Content-Type-Options: nosniff` header.

References:

1. MCP specification (Key Changes): https://modelcontextprotocol.io/specification/2026-07-28/changelog
2. MCP specification (Server Features / Resources, resource templates and resources/read validation): https://modelcontextprotocol.io/specification/2026-07-28/server/resources

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

5.1.3 Ensure prompt templates declare and validate their arguments (Automated)

Profile Applicability:

- Level 1

Description:

The server publishes MCP prompts with an explicit `arguments` declaration for every parameter. Each argument names the parameter with a non-empty `name`, and where it marks whether the argument is required it uses a boolean `required` value, so the client knows what to supply. The base protocol makes the `arguments` list optional, so requiring an explicit declaration for parameterized prompts is a stricter posture. The server validates provided argument values before processing. This Recommendation requires it to reject an invalid prompt name or a missing required argument with a JSON-RPC `-32602` error, which the specification recommends and this Recommendation elevates to a deterministic requirement for auditing. By design, prompts are user-controlled and explicitly selected, so a deployment runs prompt logic only in response to an explicit, identifiable `prompts/get` call and not as a side effect of discovery or another operation. The server records the prompt name for every invocation to support supply-chain traceability. Where the deployment versions prompts out of band, it records that version because the prompt model carries no version field.

Rationale:

Declaring each prompt argument with a `name` and a `required` flag lets client and server agree on what a prompt needs. Validating provided values before processing prevents malformed or injected input from reaching prompt logic, which the specification requires when it states that implementations validate all prompt inputs and outputs to prevent injection. The specification recommends a `-32602` error for an invalid prompt name or a missing required argument, and this Recommendation elevates that to a deterministic requirement so the check audits to a clean `PASS` or `FAIL`. The specification's design that prompts are user-controlled and explicitly selected makes explicit invocation the safe posture because it prevents a malicious server from running prompt logic as a side effect of another operation. Recording the prompt name supports traceability if a prompt is later found to be malicious.

Impact:

Impact on legitimate workflows is low, because a well-formed client already supplies the declared arguments and selects prompts explicitly. The remaining costs are one-time and small: declaring `arguments` for each parameterized prompt, recording the prompt name and any out-of-band version in the audit log, and removing any path that runs a prompt as a side effect of another operation.

Audit:

Verify that every advertised prompt declares well-formed arguments and that the server rejects a `prompts/get` missing a required argument. Run the following command to list the prompts and validate their argument declarations, substituting the endpoint:

```
MCP_ENDPOINT="<MCP_ENDPOINT>"
raw=$(curl -sS --connect-timeout 5 -m 10 -X POST "$MCP_ENDPOINT" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'MCP-Protocol-Version: 2026-07-28' \
 -H 'Mcp-Method: prompts/list' \
 -d
'{"jsonrpc":"2.0","id":1,"method":"prompts/list","params":{"_meta":{"io.modelcontextp
rotocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}'
2>/dev/null)
[ -n "$raw" ] || { echo "ERROR: no response from $MCP_ENDPOINT"; exit 0; }
obj=$(printf '%s' "$raw" | jq -c 'select(.jsonrpc) | select(.id==1)' 2>/dev/null |
head -n1)
if [ -z "$obj" ]; then
 obj=$(printf '%s\n' "$raw" | sed -n 's/^data:[[:space:]]*//p' | while IFS= read -r
d; do printf '%s' "$d" | jq -e 'select(.id==1)' >/dev/null 2>&1 && { printf '%s'
"$d"; break; }; done)
fi
[ -n "$obj" ] || { echo "ERROR: no JSON-RPC response with id 1 in prompts/list
reply"; exit 0; }
count=$(printf '%s' "$obj" | jq '.result.prompts | length' 2>/dev/null)
[ -n "$count" ] && [ "$count" != "null" ] || { echo "ERROR: prompts/list returned no
prompts array"; exit 0; }
[ "$count" -gt 0 ] || { echo "ERROR: no prompts advertised, confirm whether the
server exposes prompts"; exit 0; }
badargs=$(printf '%s' "$obj" | jq '[.result.prompts[]? | .arguments[]? |
select((.name|type!="string") or (.name=="") or ((has("required")) and
(.required|type!="boolean")))] | length' 2>/dev/null)
[ -n "$badargs" ] || badargs=0
if [ "$badargs" -eq 0 ]; then
 echo "PASS: $count prompt(s) advertised, every declared argument has a non-empty
name and a boolean required flag where present"
else
 echo "FAIL: $badargs declared argument(s) have an empty or non-string name or a non-
boolean required flag"
fi
```

Then confirm the server validates a missing required argument. Run the following command, substituting the endpoint, a prompt that has a required argument, and a valid arguments object for that prompt:

```
MCP_ENDPOINT="<MCP_ENDPOINT>"
PROMPT="<PROMPT_WITH_REQUIRED_ARG>"
VALID_ARGS='<VALID_ARGS_JSON>'
call(){ curl -sS --connect-timeout 5 -m 10 -X POST "$MCP_ENDPOINT" \
 -H 'Content-Type: application/json' -H 'Accept: application/json, text/event-stream'
\
 -H 'MCP-Protocol-Version: 2026-07-28' -H 'Mcp-Method: prompts/get' -H "Mcp-Name:
$PROMPT" \
 -d
"{\"jsonrpc\":\"2.0\",\"id\":$1,\"method\":\"prompts/get\",\"params\":{\"name\":\"$PR
OMPT\",\"arguments\":$2,\"_meta\":{\"io.modelcontextprotocol/protocolVersion\":\"2026
-07-28\",\"io.modelcontextprotocol/clientInfo\":{\"name\":\"cis-benchmark-
audit\",\"version\":\"1.0\"},\"io.modelcontextprotocol/clientCapabilities\":{}}}}"
2>/dev/null; }
pick(){ o=$(printf '%s' "$1" | jq -c "select(.jsonrpc) | select(.id==$2)" 2>/dev/null
| head -n1); [ -n "$o" ] || o=$(printf '%s\n' "$1" | sed -n 's/^data:[[:space:]]*//p'
| while IFS= read -r d; do printf '%s' "$d" | jq -e "select(.id==$2)" >/dev/null 2>&1
&& { printf '%s' "$d"; break; }; done); printf '%s' "$o"; }
r1=$(call 2 "$VALID_ARGS"); r2=$(call 3 '{}')
[ -n "$r1" ] && [ -n "$r2" ] || { echo "ERROR: no response to one or both prompts/get
probes"; exit 0; }
o1=$(pick "$r1" 2); o2=$(pick "$r2" 3)
valid1=$(printf '%s' "$o1" | jq -r 'if has("result") and (.result|has("messages"))
then "ok" else "no" end' 2>/dev/null)
if [ "$valid1" != "ok" ]; then
 echo "ERROR: the control prompts/get with valid arguments did not return a prompt
result, cannot attribute a missing-argument rejection (check the prompt name and
VALID_ARGS)"; exit 0
fi
code2=$(printf '%s' "$o2" | jq -r '.error.code // empty' 2>/dev/null)
has2=$(printf '%s' "$o2" | jq -r 'has("result")' 2>/dev/null)
case "$code2" in
 -32602) echo "PASS: with a valid control call succeeding, omitting the required
argument was rejected with -32602 (Invalid Params)" ;;
 -32002) echo "REVIEW: the server used legacy -32002, confirm the deployment's target
revision" ;;
 "") if [ "$has2" = "true" ]; then echo "FAIL: the server returned a prompt result
for a prompts/get missing the required argument"; else echo "ERROR: no error code and
no result in the missing-argument response"; fi ;;
 *) echo "REVIEW: the server returned error code $code2 rather than -32602, attribute
before concluding" ;;
esac
```

Then, where the working group keeps explicit invocation in scope, confirm by behavioral and configuration review that the server runs prompt logic only via an explicit `prompts/get` call, not as a side effect of discovery or another operation, and that each invocation records the prompt name in the audit log. This Recommendation is compliant when every advertised prompt declares well-formed arguments and a `prompts/get` missing a required argument is rejected with `-32602`.

Remediation:

Declare an `arguments` entry for every parameter of each prompt in the server configuration, naming the parameter with a non-empty `name` and, where the argument is required, marking it with a boolean `required` value. Configure the server to validate provided argument values and, as the CIS-chosen deterministic behavior, reject an invalid prompt name or a missing required argument with `-32602`. Run prompt logic only in response to an explicit `prompts/get` call, and disable any path that runs a prompt as a side effect of discovery or another operation. Configure the audit log to record the prompt name on every invocation, and its out-of-band version where the deployment versions prompts. This is server configuration, and there is no portable command.

Default Value:

By default, prompts may be advertised without declared arguments, and a server may not validate provided argument values or restrict prompt execution to explicit invocation unless configured to do so.

References:

1. MCP specification (Key Changes, promptsListChanged via subscriptions/listen): https://modelcontextprotocol.io/specification/2026-07-28/changelog
2. MCP specification (Server Features / Prompts, prompts/list and prompts/get argument validation): https://modelcontextprotocol.io/specification/2026-07-28/server/prompts

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

5.2 Session & Stream Hygiene

5.2.1 Ensure listChanged notifications are rate-limited to prevent client flooding (Automated)

Profile Applicability:

- Level 2

Description:

The server applies rate limiting or debouncing per notification type to `list_changed` notifications and other server-to-client event emissions, to prevent client floods and denial-of-service conditions. Under the `2026-07-28` revision, these notifications flow on the `subscriptions/listen` stream to clients that opted into the corresponding type through `toolsListChanged`, `promptsListChanged`, or `resourcesListChanged`. Each type is an independent subscription, so the throttle applies per type on that stream. The server enforces a minimum interval between two successive `list_changed` notifications of the same type, and batches or applies cool-down logic to high-frequency change events. The server logs the rate of its notification emissions so anomalous spikes can be detected.

Rationale:

A server that emits `list_changed` notifications without throttling can flood a client that has opened a `subscriptions/listen` stream and opted into that type, causing denial-of-service or forcing repeated re-fetches of capability lists. More important for security, rapid `toolsListChanged` emission accelerates the shadow-tool injection attack that Recommendation 1.3 addresses. A server that can fire many `toolsListChanged` events quickly can introduce newly advertised tools faster than re-validation and quarantine can review them. Throttling per type bounds the emission rate, which both prevents the flood and gives detection and re-validation more time to catch unauthorized capability expansion. Per-server emission logging lets a monitor flag a server whose emission rate is abnormal, which can indicate compromise or misconfiguration.

Impact:

Throttling `list_changed` notifications may slightly delay a client's awareness of new capabilities when changes occur in rapid succession. This tradeoff is acceptable because the goal is stability rather than maximum notification throughput. Under the `2026-07-28` revision, the downside is smaller still, because list results carry `ttlMs` and `cacheScope`, so a client re-fetches the list once its cached copy expires, regardless of whether a `list_changed` notification arrived.

Audit:

Confirm the server does not emit two successive `list_changed` notifications of the same type within a configured minimum interval. Run the following command, substituting the deployment's audit log and the minimum interval in seconds:

```
LOG="<MCP_AUDIT_LOG>"
MIN_INTERVAL="<MIN_INTERVAL_SECONDS>"
[ -f "$LOG" ] || { echo "ERROR: audit log not found at $LOG"; exit 0; }
pairs=$(grep -E '"method":"notifications/(tools|resources|prompts)/list_changed"'
"$LOG" | while IFS= read -r ln; do
 m=$(printf '%s' "$ln" | grep -oE
'notifications/(tools|resources|prompts)/list_changed' | head -n1)
 t=$(printf '%s' "$ln" | grep -oE '"timestamp":"[^"]+"' | head -n1 | sed
's/"timestamp":"//; s/"$//')
 [ -n "$m" ] && [ -n "$t" ] && printf '%s\t%s\n' "$m" "$t"
done)
[ -n "$pairs" ] || { echo "ERROR: no listChanged notifications with timestamps in
$LOG"; exit 0; }
assessed=0; fail=0; worst=""; worst_type=""
for type in notifications/tools/list_changed notifications/resources/list_changed
notifications/prompts/list_changed; do
 epochs=$(printf '%s\n' "$pairs" | awk -F'\t' -v ty="$type" '$1==ty{print $2}' |
while IFS= read -r t; do date -d "$t" +%s 2>/dev/null; done | sort -n | sed '/^$/d')
 cnt=$(printf '%s\n' "$epochs" | sed '/^$/d' | wc -l | tr -d ' ')
 if [ "$cnt" -lt 2 ]; then
  echo "detail: $type had $cnt emission(s), insufficient to assess an interval for
this type"
  continue
 fi
 assessed=$((assessed+1))
 mn=$(printf '%s\n' "$epochs" | awk 'NR>1{d=$1-p; if(m==""||d<m)m=d} {p=$1} END{print
m}')
 echo "detail: $type minimum interval ${mn}s across $cnt emissions"
 if [ "$mn" -lt "$MIN_INTERVAL" ]; then
  fail=$((fail+1))
  if [ -z "$worst" ] || [ "$mn" -lt "$worst" ]; then worst="$mn"; worst_type="$type";
fi
 fi
done
if [ "$assessed" -eq 0 ]; then
 echo "ERROR: no notification type emitted twice in the log window, cannot assess a
same-type emission rate"
elif [ "$fail" -gt 0 ]; then
 echo "FAIL: $worst_type emitted ${worst}s apart, below the ${MIN_INTERVAL}s minimum
interval for a single notification type"
else
 echo "PASS: no notification type emitted two successive notifications closer than
the ${MIN_INTERVAL}s minimum interval"
fi
```

This Recommendation is compliant when, for each notification type, no two successive notifications of that type are emitted closer together than the configured minimum interval.

Remediation:

Configure a minimum interval between two successive `list_changed` notifications of the same type on the `subscriptions/listen` stream, and batch or debounce rapid change events of a type into consolidated notifications rather than one per change. Configure the audit log to record notification emissions with a timestamp and the notification method so anomalous per-type rates can be detected. This is server configuration, and there is no portable command.

Default Value:

By default, MCP servers may emit a `list_changed` notification per change with no minimum interval or batching.

References:

1. MCP specification (Base Protocol / Message Patterns / Subscriptions): https://modelcontextprotocol.io/specification/2026-07-28/basic/patterns/subscriptions
2. MCP specification (Key Changes, subscriptions/listen replaces the GET stream): https://modelcontextprotocol.io/specification/2026-07-28/changelog
3. MCP specification (Server Features / Resources, listChanged and subscriptions): https://modelcontextprotocol.io/specification/2026-07-28/server/resources

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

5.2.2 Ensure sessions are not used as authentication and, where present, are bound to user identity (Manual)

Profile Applicability:

- Level 1

Description:

Under the `2026-07-28` revision, the protocol-level session and the `Mcp-Session-Id` header are removed. Every request carries its protocol version, self-reported client identity metadata, and client capabilities in `_meta`, and none of these is authentication. For a remote HTTP server, authentication and authorization are established independently for each request through the transport authorization framework, and an invalid or absent credential is rejected, so a session identifier or a continuity handle is never proof of identity. Where an application needs continuity across calls, it uses an explicit, server-issued handle passed as an ordinary tool argument, which the specification treats as an application value rather than a credential. Building on the specification's non-normative guidance for such handles, this Recommendation requires that the handle be generated from a cryptographically secure random source, be non-guessable, be bound to the authenticated identity, be re-validated against that identity on every use, and expire after a server-defined duration. For deployments still on `2025-11-25`, the same principle applies to the protocol session.

Rationale:

Treating a session identifier or a continuity handle as authentication lets anyone who captures or guesses it impersonate the user. Because the `2026-07-28` revision makes the protocol stateless, application state now lives in explicit handles that flow through the model and the client and can be logged or leaked. This Recommendation therefore requires that they be unguessable, bound to identity, and re-checked on every use. Authentication is established independently per request through the transport authorization framework, not through the self-reported identity in `_meta`. The specification treats such a handle as an application value and gives non-normative guidance that it carry sufficient entropy and a bounded lifetime, and this Recommendation makes that a requirement. Expiry bounds the window in which a leaked handle is useful.

Impact:

Re-validating identity per request adds minor overhead. Deployments that span multiple nodes must store the handle-to-identity binding in a shared store, a modest infrastructure requirement.

Audit:

First, confirm the server rejects a request that carries a captured handle but no authorization credential. Run the following command, substituting the endpoint, a tool that accepts a continuity-handle argument, and a captured handle or session value. The request is otherwise a valid tool call, so a rejection can only be due to the missing authorization and not to some other defect in the request:

```
URL="<MCP_ENDPOINT>"
CAPTURED_HANDLE='<CAPTURED_HANDLE_OR_SESSION_VALUE>'
TOOL="<TOOL_ACCEPTING_A_HANDLE>"
code=$(curl -sS -o /dev/null -w '%{http_code}' --connect-timeout 5 -m 10 -X POST
"$URL" \
 -H 'Content-Type: application/json' \
 -H 'Accept: application/json, text/event-stream' \
 -H 'MCP-Protocol-Version: 2026-07-28' \
 -H 'Mcp-Method: tools/call' \
 -H "Mcp-Name: $TOOL" \
 -d
"{\"jsonrpc\":\"2.0\",\"id\":1,\"method\":\"tools/call\",\"params\":{\"name\":\"$TOOL
\",\"arguments\":{\"handle\":\"$CAPTURED_HANDLE\"},\"_meta\":{\"io.modelcontextprotoc
ol/protocolVersion\":\"2026-07-
28\",\"io.modelcontextprotocol/clientInfo\":{\"name\":\"cis-benchmark-
audit\",\"version\":\"1.0\"},\"io.modelcontextprotocol/clientCapabilities\":{}}}}"
2>/dev/null)
case "$code" in
 401) echo "PASS: a request presenting only a captured handle with no valid
authorization was rejected with 401 (authorization required or token invalid)" ;;
 403) echo "REVIEW: rejected with 403, which signals insufficient scope for an
authenticated caller rather than a missing credential, confirm the server rejects the
unauthenticated request as 401" ;;
 000) echo "ERROR: server unreachable" ;;
 400) echo "REVIEW: rejected with 400, confirm it is an authorization rejection and
not a header-mismatch or malformed-request rejection before concluding" ;;
 2*) echo "FAIL: a captured handle with no valid authorization was accepted ($code),
the handle is being treated as authentication" ;;
 *) echo "REVIEW: server returned $code, attribute before concluding" ;;
esac
```

Then inspect the handle-generation mechanism to confirm a cryptographically secure random source, and confirm by configuration review that handles are bound to the authenticated identity, stored server-side, re-validated per request, and expire. This Recommendation is compliant when a request presenting only a captured handle with no valid authorization is rejected with `401`, and inspection confirms the handle is non-guessable, identity-bound, re-validated on each use, and expiring.

Remediation:

Establish authentication per request through the transport authorization framework, never through a session identifier or a continuity handle, and do not rely on a protocol session under the `2026-07-28` revision. For cross-call continuity, issue an explicit handle from a tool. Generate the handle from a cryptographically secure random source, bind it to the authenticated identity, store the binding server-side in a shared store for multi-node deployments, re-validate it against the identity on every use, and expire it after a server-defined duration. Reject any request that presents only a handle without independent authentication. This is server middleware and configuration, and there is no portable command.

Default Value:

By default, application code may treat a session identifier or handle as sufficient identity, and continuity handles may be generated without a cryptographic source or identity binding.

References:

1. MCP specification (Base Protocol / Authorization): https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization
2. MCP specification (Base Protocol / Transports / Streamable HTTP): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http
3. MCP specification (Key Changes): https://modelcontextprotocol.io/specification/2026-07-28/changelog
4. MCP specification (Server Features / Tools): https://modelcontextprotocol.io/specification/2026-07-28/server/tools

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

5.2.3 Ensure legacy Streamable HTTP session and stream-resumption mechanisms are disabled (Automated)

Profile Applicability:

- Level 1

Description:

Under the `2026-07-28` revision, a server does not implement the legacy Streamable HTTP session and stream-resumption mechanisms that earlier revisions defined. A server that supports only this revision responds to a standalone `GET` or a `DELETE` on the MCP endpoint with `405`, ignores a client-supplied `Mcp-Session-Id` header and neither mints nor echoes one, and ignores a `Last-Event-ID` header while still serving the request normally, because streams are not resumable. These mechanisms were removed in `2026-07-28`, so a server that still honors them exposes a legacy session or replay surface an attacker can use to receive another identity's server-initiated messages or to resume a stream that should not be resumable. Deployments that stream must also scope each streamed response to its originating authenticated request so it cannot be retrieved across identities. This request-scoped SSE property is confirmed by reviewing the transport and authentication configuration.

Rationale:

Earlier revisions let a server assign a session through the `Mcp-Session-Id` header, let a client open a standalone SSE stream with GET to receive server-initiated messages, and let streams be resumed through `Last-Event-ID`. The `2026-07-28` revision removes all three, and each request now carries its own identity in `_meta`. A server that still exposes any of them keeps a legacy attack surface: a standalone `GET` stream or a resumable stream can leak or replay events across contexts, and a minted session identifier reintroduces the connection-level state the stateless model eliminated. Rejecting `GET` and `DELETE`, ignoring `Last-Event-ID`, and never minting a session identifier close that surface. Because the revision removes stream resumability and message redelivery, there is no cross-stream replay to abuse and each SSE response carries only the originating request's notifications.

Impact:

There is a minor impact. Rejecting legacy `GET` and `DELETE` and ignoring the legacy headers is the default posture of a server built for `2026-07-28`, so the cost is confirming rather than adding behavior. A deployment that carried forward legacy session or resumption handling from an earlier revision removes it, which also removes the connection-level state a stateless deployment does not need.

Audit:

Verify that the MCP endpoint rejects legacy `GET` and `DELETE`, ignores `Last-Event-ID`, and neither honors nor mints a protocol-level session identifier. Run the following command to confirm a standalone `GET` and a `DELETE` are rejected, substituting the endpoint:

```
URL="<MCP_ENDPOINT>"
get_code=$(curl -sS -o /dev/null -w '%{http_code}' --connect-timeout 5 -m 8 -X GET
"$URL" -H 'Accept: text/event-stream' 2>/dev/null)
del_code=$(curl -sS -o /dev/null -w '%{http_code}' --connect-timeout 5 -m 8 -X DELETE
"$URL" 2>/dev/null)
echo "detail: GET=$get_code DELETE=$del_code (2026-07-28-only servers respond 405 to
both)"
if [ "$get_code" = "000" ] || [ "$del_code" = "000" ]; then
 echo "ERROR: server unreachable on the GET or DELETE probe"
elif [ "$get_code" = "405" ] && [ "$del_code" = "405" ]; then
 echo "PASS: standalone GET and DELETE to the MCP endpoint return 405, no legacy
stream or session surface is exposed"
else
 case "$get_code $del_code" in
  2[0-9][0-9]\ *|*\ 2[0-9][0-9]) echo "FAIL: a standalone GET or DELETE is served on
the MCP endpoint (GET=$get_code DELETE=$del_code)" ;;
  *) echo "REVIEW: expected 405 for both, got GET=$get_code DELETE=$del_code, both
are rejections but not the recommended status, attribute before concluding" ;;
 esac
fi
```

Then confirm the server ignores a `Last-Event-ID` header by comparing a request that carries it against a control request that does not. Run the following command:

```
URL="<MCP_ENDPOINT>"
body='{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{"_meta":{"io.modelconte
xtprotocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}'
common="-H Content-Type:application/json -H Accept:application/json,text/event-stream
-H MCP-Protocol-Version:2026-07-28 -H Mcp-Method:tools/list"
ctrl=$(curl -sS --connect-timeout 5 -m 8 -X POST "$URL" $common -d "$body"
2>/dev/null)
withid=$(curl -sS --connect-timeout 5 -m 8 -X POST "$URL" $common -H 'Last-Event-ID:
1' -d "$body" 2>/dev/null)
[ -n "$ctrl" ] && [ -n "$withid" ] || { echo "ERROR: server unreachable on one or
both probes"; exit 0; }
norm(){ obj=$(printf '%s' "$1" | jq -c 'select(.jsonrpc) | select(.id==1)'
2>/dev/null | head -n1); [ -n "$obj" ] || obj=$(printf '%s\n' "$1" | sed -n
's/^data:[[:space:]]*//p' | while IFS= read -r d; do printf '%s' "$d" | jq -e
'select(.id==1)' >/dev/null 2>&1 && { printf '%s' "$d"; break; }; done); printf '%s'
"$obj" | jq -r 'if has("result") and (.result|has("tools")) then "toolsresult" elif
has("error") then "error" else "other" end' 2>/dev/null; }
c=$(norm "$ctrl"); w=$(norm "$withid")
echo "detail: control_post=$c last_event_id_post=$w (both must be a normal tools/list
result)"
if [ "$c" = "toolsresult" ] && [ "$w" = "toolsresult" ]; then
 echo "PASS: the server returned the same normal tools/list result with and without
Last-Event-ID, the header is ignored"
elif [ "$c" = "toolsresult" ] && [ "$w" != "toolsresult" ]; then
 echo "FAIL: the Last-Event-ID request was handled differently from the control, the
header is not ignored"
else
 echo "ERROR: the control request did not return a tools/list result, cannot
attribute the Last-Event-ID behavior"
fi
```

Then confirm the server ignores a client-supplied `Mcp-Session-Id` and never mints or echoes one. Run the following command:

```
URL="<MCP_ENDPOINT>"
body='{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{"_meta":{"io.modelconte
xtprotocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}'
common="-H Content-Type:application/json -H Accept:application/json,text/event-stream
-H MCP-Protocol-Version:2026-07-28 -H Mcp-Method:tools/list"
hdr_ctrl=$(mktemp); hdr_sess=$(mktemp)
b_ctrl=$(curl -sS --connect-timeout 5 -m 8 -D "$hdr_ctrl" -X POST "$URL" $common -d
"$body" 2>/dev/null)
b_sess=$(curl -sS --connect-timeout 5 -m 8 -D "$hdr_sess" -X POST "$URL" $common -H
'Mcp-Session-Id: cis-audit-supplied-000' -d "$body" 2>/dev/null)
[ -n "$b_ctrl" ] && [ -n "$b_sess" ] || { echo "ERROR: server unreachable on one or
both probes"; rm -f "$hdr_ctrl" "$hdr_sess"; exit 0; }
minted=$(cat "$hdr_ctrl" "$hdr_sess" | grep -ic '^Mcp-Session-Id:')
rm -f "$hdr_ctrl" "$hdr_sess"
sess_ok=$(printf '%s' "$b_sess" | jq -r 'if has("result") and (.result|has("tools"))
then "ok" else "notok" end' 2>/dev/null)
echo "detail: mcp_session_id_response_headers=$minted
post_with_client_session_id=$sess_ok"
if [ "$minted" -gt 0 ]; then
 echo "FAIL: the server minted or echoed an Mcp-Session-Id response header, a
protocol-level session surface is present"
elif [ "$sess_ok" = "ok" ]; then
 echo "PASS: the server ignored a client-supplied Mcp-Session-Id and did not mint or
echo one"
else
 echo "REVIEW: no Mcp-Session-Id was minted, but the request carrying a client
session id did not return a normal result, attribute before concluding"
fi
```

Then confirm by configuration review that any streamed response is scoped to its originating authenticated request and cannot be retrieved across identities. This Recommendation is compliant when standalone `GET` and `DELETE` return `405`, a `Last-Event-ID` request behaves identically to a control request, and no `Mcp-Session-Id` is honored, minted, or echoed, and, for deployments that stream, each streamed response is request-scoped and not retrievable across identities.

Remediation:

For the `2026-07-28` revision, do not implement resumable streams or protocol-level sessions. Reject a standalone `GET` and a `DELETE` on the MCP endpoint with `405`, ignore a `Last-Event-ID` header, and neither mint nor echo an `Mcp-Session-Id` header. Where a deployment streams responses, scope each streamed response to its originating authenticated request and maintain per-identity separation so events cannot be retrieved across identities. Where a tool uses the multi round-trip pattern, bind the `requestState` blob to the originating identity and authorize it on the follow-up call. Where the SDK in use exposes no server-side switch for the legacy surface, enforce the rejection at a front proxy or gateway, returning `405` to a standalone `GET` or `DELETE` on the MCP endpoint and stripping `Mcp-Session-Id` and `Last-Event-ID` before the origin. Gateway enforcement covers only paths that traverse the gateway, so restrict direct network reach to the origin where this layer is the enforcement point. This is server middleware and configuration, and there is no portable command.

Default Value:

By default, a server carried forward from an earlier revision may still honor a standalone `GET` stream, a `DELETE` session termination, a `Last-Event-ID` resumption cursor, or a minted `Mcp-Session-Id`, any of which reintroduces the legacy session or replay surface on a deployment that has not disabled it.

References:

1. MCP specification (Base Protocol / Overview): https://modelcontextprotocol.io/specification/2026-07-28/basic
2. MCP specification (Base Protocol / Transports / Streamable HTTP): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http
3. MCP specification (Key Changes): https://modelcontextprotocol.io/specification/2026-07-28/changelog

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

5.3 Protocol Stream Integrity

5.3.1 Ensure logs are separated from the protocol stream in stdio mode (Automated)

Profile Applicability:

- Level 1

Description:

In stdio transport mode, `stdout` is reserved exclusively for valid JSON-RPC protocol messages. A valid message is a response correlated by `id`, or a notification that relates to an in-flight request or an active subscription. A server does not write JSON-RPC requests to `stdout` under the `2026-07-28` revision because server-initiated interaction uses the multi round-trip pattern inside a response rather than a server-originated request. No log output, diagnostic message, progress banner, or debug line may be written to `stdout`, and all logging goes to `stderr` or a file sink. The requirement is unconditional: a single invalid line on `stdout` is a failure. Logger configuration is verified at startup to confirm no appender or handler writes to `stdout` before the server accepts connections.

Rationale:

stdio framing depends on `stdout` carrying only well-formed JSON-RPC responses and notifications. Any log line, banner, or stray write corrupts the stream and breaks the client, and a line that happens to contain a `jsonrpc` field but is not a valid message is contamination just the same. Validating that each line is a valid response or notification, rather than merely that a field is present, is what distinguishes a clean protocol channel from a contaminated one. Directing all logging to `stderr` or a file keeps the channel clean. Under the `2026-07-28` revision, the Logging capability is deprecated, and its advised replacement is `stderr` or OpenTelemetry, which aligns with this requirement.

Impact:

Operational impact is minimal. Directing logs to `stderr` or a file provides equivalent observability without interfering with the protocol stream. Redirecting logging away from `stdout` and adding a startup guard is one-time configuration for a server that does not already do so.

Audit:

Confirm that the stdio server's `stdout` carries only valid JSON-RPC messages and stays clean during startup. First capture the server's `stdout` into a file while sending it tool calls and any requests that trigger notifications, then run the following command to validate each captured line:

```
OUT="<CAPTURED_STDOUT_FILE>"
[ -f "$OUT" ] || { echo "ERROR: captured stdout file not found at $OUT"; exit 0; }
[ -s "$OUT" ] || { echo "ERROR: captured stdout file is empty, no representative
exchange to check"; exit 0; }
bad=0; review=0; total=0
while IFS= read -r line; do
 [ -z "$line" ] && continue
 total=$((total+1))
 cls=$(printf '%s' "$line" | jq -r '
   if type != "object" then "bad"
   elif (.jsonrpc != "2.0") then "bad"
   elif (has("method") and has("id")) then "request"
   elif has("method") then "notification"
   elif (has("id") and has("result") and (has("error")|not)) then "response"
   elif (has("id") and has("error") and (has("result")|not)) then "response"
   else "bad" end' 2>/dev/null)
 [ -n "$cls" ] || cls="bad"
 case "$cls" in
  request) review=$((review+1)); echo "detail: server-originated request on stdout, a
2026-07-28 server does not emit requests here: $line" ;;
  notification|response) : ;;
  *) bad=$((bad+1)); echo "detail: non-JSON-RPC or malformed line on stdout:
$line" ;;
 esac
done < "$OUT"
if [ "$bad" -gt 0 ]; then
 echo "FAIL: $bad of $total stdout line(s) are not valid JSON-RPC messages, the
protocol stream is contaminated"
elif [ "$review" -gt 0 ]; then
 echo "REVIEW: $review of $total stdout line(s) are server-originated requests, which
a 2026-07-28 server should not emit on stdout, attribute before concluding"
else
 echo "PASS: all $total stdout line(s) are valid JSON-RPC responses or notifications"
fi
```

Then confirm `stdout` stays clean during startup, before any protocol input arrives. Run the following command, substituting the server's startup command:

```
SERVER_CMD="<SERVER_STARTUP_COMMAND>"
out=$(mktemp); err=$(mktemp)
timeout 5 sh -c "$SERVER_CMD" </dev/null 1>"$out" 2>"$err"
status=$?
case "$status" in
 127) echo "ERROR: startup command not found (127), cannot assess stdout hygiene"; rm
-f "$out" "$err"; exit 0 ;;
 126) echo "ERROR: startup command not executable (126), cannot assess stdout
hygiene"; rm -f "$out" "$err"; exit 0 ;;
 124) state="still running when the timeout fired" ;;
 0)   state="exited cleanly before the timeout" ;;
 *)   echo "ERROR: startup command exited $status before the timeout, the server did
not stay up, cannot assess stdout hygiene (stderr head: $(head -c 200 "$err" | tr
'\n' ' '))"; rm -f "$out" "$err"; exit 0 ;;
esac
if [ -s "$out" ]; then
 echo "FAIL: content on stdout during startup before any protocol input ($state):
$(head -c 200 "$out" | tr '\n' ' ')"
else
 echo "PASS: stdout clean during startup with no protocol input ($state)"
fi
rm -f "$out" "$err"
```

Then review the logging configuration to confirm no log output targets `stdout`, with all destinations being `stderr` or a file. This Recommendation is compliant when every captured `stdout` line is a valid JSON-RPC response or notification, `stdout` stays clean during an attributable startup, and the logging configuration directs all diagnostic output to `stderr` or a file.

Remediation:

Configure the server's logging so all output goes to `stderr` or a file and never to `stdout`. The exact setting is framework-specific. In Python, initialize the logging module against a `stderr` stream and avoid `print`, which writes to `stdout`. In Node.js, use `console.error` rather than `console.log` for diagnostics, and confirm any logging library defaults to `stderr`. Under the `2026-07-28` revision, prefer `stderr` or OpenTelemetry over the deprecated Logging capability. Add a startup guard to the launch wrapper that fails the launch if anything reaches `stdout` before protocol input arrives, and that distinguishes a failed launch from a clean start rather than treating an empty `stdout` as success. This is server configuration, and there is no single portable command.

Default Value:

Defaults vary by language and library. In Node.js, `console.log` writes to `stdout` and only `console.error` writes to `stderr`, so a stray `console.log` diagnostic corrupts the protocol stream. Some Node logging libraries also default to `stdout`. Python's standard `logging` module defaults to `stderr`, which is safe, but `print` writes to `stdout`.

References:

1. MCP specification (Base Protocol / Overview): https://modelcontextprotocol.io/specification/2026-07-28/basic
2. MCP specification (Base Protocol / Transports / stdio): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/stdio
3. MCP specification (Key Changes): https://modelcontextprotocol.io/specification/2026-07-28/changelog

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.2 Collect Audit Logs Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.	●	●	●

5.4 Filesystem Path Safety

5.4.1 Ensure path traversal and arbitrary filesystem access are prevented (Automated)

Profile Applicability:

- Level 1

Description:

The server normalizes and canonicalizes every filesystem path supplied as a tool or resource parameter before any access-control check or filesystem operation. Canonicalization resolves symbolic links, for example `os.path.realpath()` in Python or `fs.realpathSync()` in Node.js, before comparing the resolved path against the configured allowlist of approved directories. Symlink bypass and `../` directory traversal are two independent attack paths, and each is guarded. The read is rejected unless the resolved real path is the approved root itself or a descendant of it, compared by path component and not by string prefix, so an approved root such as `/srv/approved` does not admit a sibling such as `/srv/approved_evil`. Every rejected attempt is logged as a security event.

Rationale:

Without canonicalization, a `../` sequence or a symlink can resolve to a file outside the approved scope while passing a naive check performed before resolution. Resolving symlinks and `../` first, then checking the resolved real path against the allowlist by path component, closes both paths. Comparing by component rather than by string prefix prevents a sibling directory whose name begins with the root's name from aliasing into the root. Logging rejected attempts surfaces probing.

Impact:

Per-call overhead is negligible. Configuring the path-handling middleware and the approved-directory allowlist is one-time setup.

Audit:

Confirm that a resource read whose URI escapes the approved root is denied, while an allowed read under the same conditions succeeds. Run the following command, substituting the endpoint, an allowed resource URI under the approved root, and a traversal URI that points outside it, for example `file:///approved/../../etc/passwd`:

```
MCP_ENDPOINT="<MCP_ENDPOINT>"
ALLOWED_URI="<ALLOWED_RESOURCE_URI>"
TRAVERSAL_URI="<TRAVERSAL_RESOURCE_URI>"
read_uri(){ curl -sS --connect-timeout 5 -m 10 -X POST "$MCP_ENDPOINT" \
 -H 'Content-Type: application/json' -H 'Accept: application/json, text/event-stream'
\
 -H 'MCP-Protocol-Version: 2026-07-28' -H 'Mcp-Method: resources/read' -H "Mcp-Name:
$2" \
 -d
"{\"jsonrpc\":\"2.0\",\"id\":$1,\"method\":\"resources/read\",\"params\":{\"uri\":\"$
2\",\"_meta\":{\"io.modelcontextprotocol/protocolVersion\":\"2026-07-
28\",\"io.modelcontextprotocol/clientInfo\":{\"name\":\"cis-benchmark-
audit\",\"version\":\"1.0\"},\"io.modelcontextprotocol/clientCapabilities\":{}}}}"
2>/dev/null; }
pick(){ o=$(printf '%s' "$1" | jq -c "select(.jsonrpc) | select(.id==$2)" 2>/dev/null
| head -n1); [ -n "$o" ] || o=$(printf '%s\n' "$1" | sed -n 's/^data:[[:space:]]*//p'
| while IFS= read -r d; do printf '%s' "$d" | jq -e "select(.id==$2)" >/dev/null 2>&1
&& { printf '%s' "$d"; break; }; done); printf '%s' "$o"; }
c=$(read_uri 1 "$ALLOWED_URI"); t=$(read_uri 2 "$TRAVERSAL_URI")
[ -n "$c" ] && [ -n "$t" ] || { echo "ERROR: no response to one or both
resources/read probes"; exit 0; }
oc=$(pick "$c" 1); ot=$(pick "$t" 2)
ctrl_ok=$(printf '%s' "$oc" | jq -e '.result.contents | length > 0' >/dev/null 2>&1
&& echo yes || echo no)
if [ "$ctrl_ok" != "yes" ]; then
 echo "ERROR: the positive-control read of an allowed file did not return contents,
cannot attribute the traversal result to path confinement (check ALLOWED_URI and
authorization)"; exit 0
fi
t_contents=$(printf '%s' "$ot" | jq -e '.result.contents | length > 0' >/dev/null
2>&1 && echo yes || echo no)
t_err=$(printf '%s' "$ot" | jq -r '.error.code // empty' 2>/dev/null)
if [ "$t_contents" = "yes" ]; then
 echo "FAIL: the allowed read succeeded and the traversal read returned contents, the
path boundary was escaped"
elif [ -n "$t_err" ]; then
 echo "PASS: the allowed read succeeded and the traversal read was denied with error
$t_err, path confinement is enforced"
else
 echo "REVIEW: the allowed read succeeded but the traversal read returned neither
contents nor an error, attribute before concluding"
fi
```

Then, using a filesystem-backed environment, stage a symlink inside the approved directory that points outside it and repeat the read, to confirm the symlink path is guarded independently of the `../` path. Apply the same probe, a positive control paired with a traversal attempt, to every tool that accepts a path argument. This Recommendation is compliant when an allowed read under the approved root succeeds, a traversal read that escapes the root is denied with no contents returned, and the symlink path is guarded independently across every path-taking surface.

Remediation:

Configure the server's path-handling middleware to canonicalize every supplied path, resolving symlinks and `../` via `realpath` or `realpathSync`, before any access check. Reject the read unless the resolved real path is the approved root or a descendant of it, compared by path component rather than by string prefix, and log rejected attempts as security events. Set the approved directories in server configuration. For defense in depth, add kernel-enforced confinement that resolves beneath the root without following symlinks, and apply the OS-level isolation Recommendations in Section 9. This is server middleware, so there is no portable command for the canonicalization logic itself.

Default Value:

By default, path handling may compare unresolved paths against an allowlist, allowing `../` or symlink bypass.

References:

1. MCP specification (Key Changes): https://modelcontextprotocol.io/specification/2026-07-28/changelog
2. MCP specification (Server Features / Resources, security considerations): https://modelcontextprotocol.io/specification/2026-07-28/server/resources

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

5.5 MCP Tasks

5.5.1 Ensure authorization, scope, and expiry controls are enforced on MCP Tasks (Manual)

Profile Applicability:

- Level 2

Description:

MCP Tasks are long-running operations that return a task handle for asynchronous status polling and result retrieval. Because the Tasks extension defines task lifecycle management but leaves access control to implementations, authorization is enforced independently at task creation, status polling, and result retrieval. The server ensures callers can only access tasks they created and re-validates authorization when results are retrieved since credentials may be revoked or expire during execution. Task handles are generated from a cryptographically secure random source, remain non-guessable, and never serve as proof of identity on their own. Although the extension makes `ttlMs` nullable and expiration optional, Level 2 deployments enforce a finite, non-null retention period through `ttlMs` as a CIS hardening measure. In the `2026-07-28` revision, Tasks is an extension (`io.modelcontextprotocol/tasks`) negotiated through extension capabilities and implemented through `tasks/get`, `tasks/update`, and `tasks/cancel`. The extension defines no `tasks/list` method, so callers cannot enumerate tasks.

Rationale:

A guessable or unauthorized task handle could let an attacker access another user's task status or results. Because the Tasks extension enforces authorization only when the server implements it, task security depends on per-identity ownership controls, non-guessable handles, and authorization re-validation during result retrieval. The removal of `tasks/list` prevents task enumeration across users but does not prevent access to a specific task handle that has been exposed, which makes ownership validation and reauthorization checks essential. Since the extension treats expiration as optional, requiring a finite, non-null retention period limits the usefulness of a leaked handle and is a Level 2 requirement under this Recommendation.

Impact:

Re-validating authorization at each stage adds minor overhead. Enforcing per-identity ownership and finite `ttlMs` cleanup is one-time implementation work.

Audit:

Confirm that a task retrievable by its owning identity cannot be retrieved by a second identity. Run the following command, substituting the endpoint, a task handle owned by a first identity, that identity's bearer token, and a second identity's bearer token. The command sets `Mcp-Name` to the task id as the extension requires:

```
URL="<MCP_ENDPOINT>"
OWNER_TASK='<TASK_HANDLE_OWNED_BY_FIRST_IDENTITY>'
OWNER_TOKEN='<FIRST_IDENTITY_BEARER_TOKEN>'
OTHER_TOKEN='<SECOND_IDENTITY_BEARER_TOKEN>'
gettask(){ curl -sS --connect-timeout 5 -m 10 -X POST "$URL" \
 -H 'Content-Type: application/json' -H 'Accept: application/json, text/event-stream'
\
 -H 'MCP-Protocol-Version: 2026-07-28' -H 'Mcp-Method: tasks/get' -H "Mcp-Name:
$OWNER_TASK" \
 -H "Authorization: Bearer $1" \
 -d
"{\"jsonrpc\":\"2.0\",\"id\":$2,\"method\":\"tasks/get\",\"params\":{\"taskId\":\"$OW
NER_TASK\",\"_meta\":{\"io.modelcontextprotocol/protocolVersion\":\"2026-07-
28\",\"io.modelcontextprotocol/clientInfo\":{\"name\":\"cis-benchmark-
audit\",\"version\":\"1.0\"},\"io.modelcontextprotocol/clientCapabilities\":{\"extens
ions\":{\"io.modelcontextprotocol/tasks\":{}}}}}}" 2>/dev/null; }
pick(){ o=$(printf '%s' "$1" | jq -c "select(.jsonrpc) | select(.id==$2)" 2>/dev/null
| head -n1); [ -n "$o" ] || o=$(printf '%s\n' "$1" | sed -n 's/^data:[[:space:]]*//p'
| while IFS= read -r d; do printf '%s' "$d" | jq -e "select(.id==$2)" >/dev/null 2>&1
&& { printf '%s' "$d"; break; }; done); printf '%s' "$o"; }
own=$(gettask "$OWNER_TOKEN" 1); oth=$(gettask "$OTHER_TOKEN" 2)
[ -n "$own" ] && [ -n "$oth" ] || { echo "ERROR: no response to one or both tasks/get
probes"; exit 0; }
oo=$(pick "$own" 1); ot=$(pick "$oth" 2)
own_code=$(printf '%s' "$oo" | jq -r '.error.code // empty' 2>/dev/null)
[ "$own_code" = "-32601" ] && { echo "ERROR: tasks extension not implemented (-
32601), nothing to assess"; exit 0; }
owner_ok=$(printf '%s' "$oo" | jq -e '.result.taskId != null' >/dev/null 2>&1 && echo
yes || echo no)
if [ "$owner_ok" != "yes" ]; then
 echo "ERROR: the owning identity could not retrieve its own task, cannot attribute a
cross-identity denial to ownership (check OWNER_TASK and OWNER_TOKEN)"; exit 0
fi
other_result=$(printf '%s' "$ot" | jq -e '.result.taskId != null' >/dev/null 2>&1 &&
echo yes || echo no)
other_err=$(printf '%s' "$ot" | jq -r '.error.code // empty' 2>/dev/null)
if [ "$other_result" = "yes" ]; then
 echo "FAIL: the owner retrieved its task and a second identity also retrieved the
same task, cross-identity task access is not prevented"
elif [ -n "$other_err" ]; then
 echo "PASS: the owner retrieved its task and a second identity was denied (error
$other_err), cross-identity access is prevented at the wire, confirm the denial
reason from the backend record"
else
 echo "REVIEW: the owner retrieved its task but the second identity returned neither
a task nor an error, attribute before concluding"
fi
```

Then confirm from the backend authorization record that the second identity's denial is an ownership decision rather than an unrelated error or a task that never existed, inspect the handle-generation mechanism for a cryptographically secure source, and confirm tasks carry a finite, non-null `ttlMs` and are cleaned up. This Recommendation is compliant when a second identity cannot retrieve a task its owner can, the backend record establishes the denial as an ownership decision, handles are non-guessable, and tasks carry a finite retention period.

Remediation:

Configure the server to generate task handles from a cryptographically secure random source, for example a version-4 UUID. Enforce per-identity ownership at both `tasks/get` and result retrieval, and re-validate authorization at retrieval time. Set a finite, non-null `ttlMs` with scheduled cleanup. The extension makes expiry optional, so a finite retention period is a Level 2 requirement under this Recommendation. Negotiate the Tasks extension through the extension capability, with the client declaring `io.modelcontextprotocol/tasks` in its per-request capabilities. The extension defines no `tasks/list` method, so do not rely on task enumeration. This is server middleware, and there is no portable command.

Default Value:

By default, task handle generation and per-user ownership enforcement are not in place unless explicitly implemented. Without ownership enforcement, a task created by one caller may be pollable by another.

References:

1. MCP specification (Key Changes): https://modelcontextprotocol.io/specification/2026-07-28/changelog
2. MCP Extensions (Tasks): https://modelcontextprotocol.io/extensions/tasks/overview
3. SEP-2663 (Tasks Extension): https://modelcontextprotocol.io/seps/2663-tasks-extension

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

5.6 Idempotency & Side-Effect Control

5.6.1 Ensure idempotency keys are required for side-effecting tool calls and duplicate side effects are prevented (Manual)

Profile Applicability:

- Level 2

Description:

The server requires a client-supplied idempotency key for every side-effecting tool call and uses that key to de-duplicate retries, so a request is not executed twice. In MCP's stateless model, interrupted operations are recovered by re-issuing the request with a new request ID, which makes the idempotency key, rather than request tracking, the mechanism that guarantees safe replay. Duplicate detection is scoped to the authenticated principal, target server or resource, tool name, idempotency key, and a canonical fingerprint of the request arguments. A request with the same key and fingerprint returns the previously stored result instead of re-executing. A request that reuses a key with a different fingerprint is rejected as an idempotency conflict, to prevent payload substitution or cross-principal result exposure. Multi-node deployments require a shared idempotency store because any node may process a retried request. The `Idempotency-Key` header and cached-result indicator are conventions defined by this Recommendation and are not MCP specification fields.

Rationale:

Without idempotency keys, a retried or replayed side-effecting call executes twice, for example sending an email or charging a payment twice. Under the stateless model a broken stream is recovered by re-issuing the request with a new request ID, so detecting a repeated request ID would not catch a legitimate re-issue and is not a mechanism the protocol provides since a request ID only has to be unique among the sender's still-outstanding requests. The client-supplied idempotency key is what makes the re-issue safe, by returning a stored result instead of re-executing. Scoping the key to the principal, server, tool, and request fingerprint prevents three failures: two principals colliding on a key, one principal's stored result reaching another, and the same key with different arguments returning an unrelated result. A shared store across load-balanced instances is required so a duplicate routed to a different node is still caught.

Impact:

Idempotency tracking adds a small per-call lookup, a canonical fingerprint computation, and a shared-store dependency for multi-node deployments. Removing request-ID replay tracking simplifies the server since it no longer maintains historical request-ID state the protocol does not require.

Audit:

Identify the tools in the capability baseline that perform writes or trigger external actions, then confirm three properties against a safely reversible or dedicated test tool so the probes cause no real side effect. First, confirm a duplicate key returns a cached result. Run the following command:

```
URL="<MCP_ENDPOINT>"
KEY="cis-audit-key-001"
TOOL="<SIDE_EFFECTING_OR_TEST_TOOL>"
mkbody(){ printf
'{"jsonrpc":"2.0","id":%s,"method":"tools/call","params":{"name":"%s","arguments":{},
"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}' "$1"
"$TOOL"; }
extract(){ obj=$(printf '%s' "$1" | jq -c 'select(.jsonrpc)' 2>/dev/null | head -n1);
[ -n "$obj" ] || obj=$(printf '%s\n' "$1" | sed -n 's/^data:[[:space:]]*//p' | head -
n1); printf '%s' "$obj" | jq -r 'if (.result|type=="object") and
(.result|has("cached")) then (.result.cached|tostring) else "absent" end'
2>/dev/null; }
hdrs=(-H 'Content-Type: application/json' -H 'Accept: application/json, text/event-
stream' -H 'MCP-Protocol-Version: 2026-07-28' -H "Idempotency-Key: $KEY" -H 'Mcp-
Method: tools/call' -H "Mcp-Name: $TOOL")
r1=$(curl -sS --connect-timeout 5 -m 10 -X POST "$URL" "${hdrs[@]}" -d "$(mkbody 1)"
2>/dev/null)
r2=$(curl -sS --connect-timeout 5 -m 10 -X POST "$URL" "${hdrs[@]}" -d "$(mkbody 2)"
2>/dev/null)
[ -n "$r1" ] && [ -n "$r2" ] || { echo "ERROR: server unreachable on one or both
submissions"; exit 0; }
first=$(extract "$r1"); [ -n "$first" ] || first=absent
second=$(extract "$r2"); [ -n "$second" ] || second=absent
echo "detail: first_cached=$first second_cached=$second (same Idempotency-Key,
request ids 1 then 2)"
case "$second" in
 true)  echo "PASS: a duplicate idempotency key on a re-issued request returned a
cached result" ;;
 false) echo "FAIL: the server re-executed a duplicate idempotency key instead of
returning a cached result" ;;
 *)     echo "REVIEW: no cached indicator in the second response, confirm the
deployment's cached-result field" ;;
esac
```

Then, to confirm the key is bound to the request payload, run the following command:

```
URL="<MCP_ENDPOINT>"
TOOL="<SIDE_EFFECTING_OR_TEST_TOOL>"
body='{"jsonrpc":"2.0","id":1,"method":"tools/call","params":{"name":"'"$TOOL"'","arg
uments":{},"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}'
resp=$(curl -sS --connect-timeout 5 -m 10 -X POST "$URL" -H 'Content-Type:
application/json' -H 'Accept: application/json, text/event-stream' -H 'MCP-Protocol-
Version: 2026-07-28' -H 'Mcp-Method: tools/call' -H "Mcp-Name: $TOOL" -d "$body"
2>/dev/null)
[ -n "$resp" ] || { echo "ERROR: no response to no-key tools/call probe"; exit 0; }
obj=$(printf '%s' "$resp" | jq -c 'select(.jsonrpc) | select(.id==1)' 2>/dev/null |
head -n1)
[ -n "$obj" ] || obj=$(printf '%s\n' "$resp" | sed -n 's/^data:[[:space:]]*//p' |
head -n1)
[ -n "$obj" ] || { echo "ERROR: no JSON-RPC response with id 1 to no-key probe"; exit
0; }
haserr=$(printf '%s' "$obj" | jq -r 'has("error")' 2>/dev/null)
iserr=$(printf '%s' "$obj" | jq -r '.result.isError // empty' 2>/dev/null)
if [ "$haserr" = "true" ] || [ "$iserr" = "true" ]; then
 echo "PASS: the server rejected a side-effecting call submitted without an
idempotency key"
else
 echo "FAIL: the server accepted and executed a side-effecting call with no
idempotency key"
fi
```

Then, to confirm the key is bound to the request payload, run the following command:

```
URL="<MCP_ENDPOINT>"
KEY="cis-audit-key-002"
TOOL="<SIDE_EFFECTING_OR_TEST_TOOL>"
mkbody(){ printf
'{"jsonrpc":"2.0","id":%s,"method":"tools/call","params":{"name":"%s","arguments":%s,
"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}' "$1"
"$TOOL" "$2"; }
hdrs=(-H 'Content-Type: application/json' -H 'Accept: application/json, text/event-
stream' -H 'MCP-Protocol-Version: 2026-07-28' -H "Idempotency-Key: $KEY" -H 'Mcp-
Method: tools/call' -H "Mcp-Name: $TOOL")
r1=$(curl -sS --connect-timeout 5 -m 10 -X POST "$URL" "${hdrs[@]}" -d "$(mkbody 1
'{"amount":1}')" 2>/dev/null)
r2=$(curl -sS --connect-timeout 5 -m 10 -X POST "$URL" "${hdrs[@]}" -d "$(mkbody 2
'{"amount":2}')" 2>/dev/null)
[ -n "$r1" ] && [ -n "$r2" ] || { echo "ERROR: server unreachable on one or both
submissions"; exit 0; }
obj=$(printf '%s' "$r2" | jq -c 'select(.jsonrpc)' 2>/dev/null | head -n1)
[ -n "$obj" ] || obj=$(printf '%s\n' "$r2" | sed -n 's/^data:[[:space:]]*//p' | head
-n1)
[ -n "$obj" ] || { echo "ERROR: no parseable response to the second submission"; exit
0; }
haserr=$(printf '%s' "$obj" | jq -r 'has("error")' 2>/dev/null)
if [ "$haserr" = "true" ]; then
 echo "PASS: the server rejected a reused idempotency key carrying a different
request payload as a conflict"
else
 echo "FAIL: the server returned a result for a reused key with a different payload,
the key is not bound to the request fingerprint"
fi
```

Then confirm from backend execution records that the side effect occurred exactly once across the duplicate submissions, and that a reuse under a different principal is denied. This Recommendation is compliant when a side-effecting tool requires an idempotency key, a duplicate key with the same payload returns a cached result, and a reused key with a different payload is rejected as a conflict. Backend records must additionally confirm the side effect ran once and that a key reused under a different principal is denied.

Remediation:

Add server middleware that requires an `Idempotency-Key` header for side-effecting tools, classified per the tool risk tiers of Recommendation 4.1.5. Key the deduplication store on the authenticated principal, the target server or resource, the tool name, the idempotency key, and a canonical fingerprint of the request arguments, with a time-to-live of at least 24 hours. Return the stored result when the same key arrives with the same fingerprint. Reject a reuse of the same key with a different fingerprint as an idempotency conflict, rather than returning the stored result or executing. For multi-node deployments store keys in a shared distributed cache such as Redis so duplicates are caught regardless of which instance serves the retry. This is server middleware and configuration, and there is no portable command.

Default Value:

By default, MCP servers do not require idempotency keys and may re-execute retried or replayed side-effecting calls.

References:

1. MCP specification (Base Protocol / Overview): https://modelcontextprotocol.io/specification/2026-07-28/basic
2. MCP specification (Key Changes): https://modelcontextprotocol.io/specification/2026-07-28/changelog
3. MCP specification (Server Features / Tools): https://modelcontextprotocol.io/specification/2026-07-28/server/tools

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

6 Data Protection & Privacy

MCP moves potentially sensitive context between models, tools, and external services, and each hop is a place where more data than necessary can be exposed, a credential can leak, or a request can be steered to an unintended destination. Minimizing context and controlling where it flows directly reduces what an incident can expose.

The Recommendations in this section limit and protect data in motion. They minimize the context handed to the model and honor declared MIME handling, protect tokens and keys in both clients and servers, and apply outbound proxy and DNS controls so that resource fetching cannot be pointed at arbitrary destinations.

6.1 Ensure model context is minimized and MIME handling is honored (Manual)

Profile Applicability:

- Level 1

Description:

The server sends only the minimum context necessary for the model to complete the current operation. Where the server requests a model generation through sampling, that request travels inside an `InputRequiredResult` under the multi round-trip request pattern. The `includeContext` parameter defaults to `none`, with the `thisServer` and `allServers` values deprecated, so a compliant request does not pull in server or cross-server context unless the operator opts in. Content filters and size constraints are applied to resource content before it is placed in a sampling payload or forwarded directly to a model provider. Arbitrary or unvalidated bytes are not injected into text context, and where image or audio context is intentionally supported, it is validated and passed as the provider's typed image or audio representation (base64 data with a `mimeType`) rather than as text. Secrets, credentials, and other sensitive data are redacted or excluded before context is forwarded to the model. A maximum context payload size is defined and enforced at the server middleware layer, coordinated with recommendation 10.2. Because sampling is deprecated as of the `2026-07-28` revision, the same minimization and scrubbing apply to any context sent to a model provider through a direct integration.

Rationale:

Oversized or unnecessary context increases the chance that secrets or sensitive data are forwarded to an external model API and raises cost and latency. Distinguishing binary from text prevents corrupt or wasteful payloads. Redaction before forwarding keeps credentials and PII out of the model context. Enforcing a maximum payload size bounds both the data-exposure and denial-of-service surface.

Impact:

Context filtering and redaction add minor processing overhead. Defining and enforcing a maximum context payload size is one-time middleware configuration.

Audit:

Inspect model-context payloads, whether sent through sampling or forwarded directly to a model provider, and confirm each carries only the content needed for the operation, contains no raw or unvalidated bytes in text fields, and contains no credentials or secrets. Set `MAX_CONTEXT_BYTES` to the configured maximum payload size, then run the following command, which reads each log line as JSON, selects the sampling-payload records, and fails any that exceed the maximum:

```
LOG="<MCP_AUDIT_LOG>"
MAX_CONTEXT_BYTES="<MAX_CONTEXT_BYTES>"
if [ ! -r "$LOG" ]; then
  echo "ERROR: audit log \"$LOG\" not found or not readable (set MCP_AUDIT_LOG)"
else
  sizes=$(jq -rR '(fromjson? // null) as $o | select($o != null and $o.event ==
"sampling_payload") | ($o.payload_bytes // empty)' "$LOG" 2>/dev/null | sort -nr)
  if [ -z "$sizes" ]; then
    echo "REVIEW: no sampling_payload events in log (nothing to assess, or the
deployment integrates a model provider directly and the direct-provider path must be
inspected instead)"
  elif printf '%s' "$MAX_CONTEXT_BYTES" | grep -q '<MAX_CONTEXT_BYTES>'; then
    echo "sampling_payload sizes (bytes), largest first:"
    printf '%s\n' "$sizes" | head
    echo "REVIEW: set MAX_CONTEXT_BYTES to the configured maximum, then confirm no
record exceeds it and that content carries no secrets or raw binary"
  else
    over=$(printf '%s\n' "$sizes" | awk -v m="$MAX_CONTEXT_BYTES" '$1+0 > m+0')
    if [ -n "$over" ]; then
      echo "FAIL: sampling_payload record(s) exceed
MAX_CONTEXT_BYTES=$MAX_CONTEXT_BYTES (bytes):"
      printf '%s\n' "$over" | head
    else
      echo "PASS: all sampling_payload records within
MAX_CONTEXT_BYTES=$MAX_CONTEXT_BYTES (confirm by content review that none carry
secrets or raw binary)"
    fi
  fi
fi
```

Then confirm by content review that the sampled payloads include no secrets and no raw binary in text fields. Where the deployment has migrated off deprecated sampling to a direct model-provider integration, inspect that provider telemetry rather than treating the absence of sampling records as evidence of compliance. This Recommendation is compliant when model-context payloads carry only the content the operation needs, no payload exceeds the configured maximum size, and content review confirms no secrets or raw binary in text fields, whether context is sent through sampling or forwarded directly to a model provider.

Remediation:

Configure the server middleware to filter context to the minimum needed for the operation, and to scrub secrets and personally identifiable information before forwarding. Keep unvalidated bytes out of text context, and pass intentional image or audio content as the provider's typed representation. Enforce a maximum context payload size, coordinated with the deployment's request and response body-size limits. Because sampling is deprecated as of the `2026-07-28` revision, apply the same minimization and scrubbing to any context sent to a model provider through a direct integration. This is server middleware and configuration, so there is no portable command.

Default Value:

The `includeContext` parameter defaults to `none`, so by default a sampling request does not pull in server or cross-server context. The protocol does not otherwise constrain the size or content of the context a server places in a sampling request or forwards to a model provider. No redaction or maximum payload size applies unless the operator configures it, so secrets present in host context may be forwarded to the model.

References:

1. MCP specification (Client Features / Sampling): https://modelcontextprotocol.io/specification/2026-07-28/client/sampling
2. MCP specification (Deprecated Features): https://modelcontextprotocol.io/specification/2026-07-28/deprecated
3. MCP specification (Server Features / Resources): https://modelcontextprotocol.io/specification/2026-07-28/server/resources
4. SEP-2577 (Deprecate Roots, Sampling, and Logging): https://modelcontextprotocol.io/seps/2577-deprecate-roots-sampling-and-logging

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.1 Establish and Maintain a Data Management Process Establish and maintain a data management process. In the process, address data sensitivity, data owner, handling of data, data retention limits, and disposal requirements, based on sensitivity and retention standards for the enterprise. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●
v8	3.13 Deploy a Data Loss Prevention Solution Implement an automated tool, such as a host-based Data Loss Prevention (DLP) tool to identify all sensitive data stored, processed, or transmitted through enterprise assets, including those located onsite or at a remote service provider, and update the enterprise's sensitive data inventory.			●

6.2 Ensure tokens and keys are protected in clients and servers (Automated)

Profile Applicability:

- Level 1

Description:

All credentials (API tokens, OAuth client secrets, service-account keys) are stored in system keychains, managed secret stores, or KMS-backed stores. As a CIS hardening position, secret values are not embedded in deployed configuration or artifacts in any form, including encrypted, and are referenced from the managed store at runtime instead. Automated credential rotation is in place, with alerts for credentials approaching expiry.

Rationale:

Credentials embedded in deployed configuration or artifacts are a leading cause of compromise: they propagate through backups and images and are a primary target of automated scanning by attackers. A managed secret store keeps secret material out of configuration, supports rotation, and centralizes access control.

Impact:

Migrating to a secret store is a moderate one-time effort. Ongoing, rotation adds little overhead and substantially reduces credential-compromise risk.

Audit:

Scan the deployed configuration and installed artifacts for inline credentials, and confirm that deployment manifests reference secrets from a managed store rather than defining them inline. Set `DEPLOY` to the directory holding the deployed configuration and installed artifacts. Run the following command, which invokes the `gitleaks` secret scanner, to detect embedded credentials in that directory:

```
DEPLOY="${DEPLOY:-<MCP_DEPLOY_DIR>}"
if ! command -v gitleaks >/dev/null 2>&1; then echo "ERROR: gitleaks not installed
(cannot scan)"; exit 0; fi
if [ ! -d "$DEPLOY" ]; then echo "ERROR: deployment directory \"$DEPLOY\" not found
(set MCP_DEPLOY_DIR)"; exit 0; fi
GLOUT=$(mktemp)
gitleaks detect --no-banner --no-git --source "$DEPLOY" --redact >"$GLOUT" 2>&1
rc=$?
if [ "$rc" -eq 0 ]; then
 echo "PASS: no inline credentials detected in the deployed configuration and
artifacts"
elif [ "$rc" -eq 1 ]; then
 echo "FAIL: gitleaks reported potential inline credentials in the deployed files:"
 tail -n 20 "$GLOUT"
else
 echo "ERROR: gitleaks scan failed (exit $rc), cannot assess, inspect the scanner
installation and flags"
fi
rm -f "$GLOUT"
```

Then inspect deployment manifests to confirm secrets are referenced from a managed store, not defined inline. This Recommendation is compliant when the scan reports no findings, the command prints `PASS`, and manifests contain only secret references.

Remediation:

Migrate all inline or embedded secrets to a managed secret store or system keychain, and update deployment configuration to reference them from the store at runtime rather than inline. This is a deployment and configuration change, and the exact store is environment-specific. Implement automated rotation with alerts for credentials approaching expiry. Rotate any secret that was previously embedded in deployed configuration or an artifact since it must be treated as compromised.

Default Value:

By default, SDK templates and example configurations often place credentials in environment files or inline configuration.

References:

1. MCP specification (Base Protocol / Authorization / Client Registration): https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization/client-registration
2. MCP specification (security best practices, credential handling): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices
3. SEP-2352 (Clarify authorization server binding and migration): https://github.com/modelcontextprotocol/modelcontextprotocol/pull/2352

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.11 Leverage Vetted Modules or Services for Application Security Components Leverage vetted modules or services for application security components, such as identity management, encryption, and auditing and logging. Using platform features in critical security functions will reduce developers' workload and minimize the likelihood of design or implementation errors. Modern operating systems provide effective mechanisms for identification, authentication, and authorization and make those mechanisms available to applications. Use only standardized, currently accepted, and extensively reviewed encryption algorithms. Operating systems also provide mechanisms to create and maintain secure audit logs.		●	●

6.3 Ensure outbound proxy and DNS controls are applied for resource fetchers (Manual)

Profile Applicability:

- Level 1

Description:

All outbound network access from MCP server processes is forced through a configured egress proxy. DNS resolution is restricted to approved internal resolvers, and FQDN allowlists are enforced for all resource-retrieval operations. Direct outbound connections to arbitrary IP addresses, or to external hostnames not on the approved FQDN list, are blocked. All outbound connection attempts, including blocked ones, are logged as security events.

Rationale:

An MCP server that fetches resources can be induced to connect to internal services or attacker-controlled hosts through SSRF, a malicious resource URI, or DNS rebinding. Forcing egress through a proxy with an FQDN allowlist and restricting DNS to approved resolvers bounds where the server can connect and provides a logging chokepoint. This is only effective if direct, non-proxied egress is actually blocked, not merely if a proxy is available. A test that only observes a failed direct connection cannot prove egress is bounded, because the destination may simply be unreachable. The audit therefore establishes reachability through the approved proxy first, then treats a failed direct attempt to the same destination as evidence that egress is enforced.

Impact:

Routing egress through a proxy and restricting DNS adds configuration and a dependency on the proxy and resolvers.

Audit:

Confirm outbound connections are forced through the egress proxy and that a direct connection bypassing the proxy is blocked, then confirm DNS resolution uses approved resolvers. Set `PROXY` to the approved egress proxy and `TEST_DEST` to a destination that is reachable through that proxy and is not on any host-local allowlist, then run the following command. The command reaches the destination through the proxy as a positive control, then attempts the same destination directly with no proxy. It reports egress enforced only when the proxy path works and the direct path fails:

```
PROXY="<EGRESS_PROXY_URL>"
TEST_DEST="<REACHABLE_TEST_DESTINATION>"
curl -sS -o /dev/null --max-time 8 --proxy "$PROXY" "$TEST_DEST" >/dev/null 2>&1;
via_proxy_rc=$?
curl -sS -o /dev/null --max-time 8 --proxy "" "$TEST_DEST" >/dev/null 2>&1;
direct_rc=$?
if [ "$direct_rc" -eq 0 ]; then
  echo "FAIL: direct outbound (proxy bypassed) reached the destination, egress is not
locked down"
elif [ "$via_proxy_rc" -eq 0 ]; then
  echo "PASS: direct outbound blocked and the approved proxy path reached the
destination"
else
  echo "REVIEW: neither the proxy path nor the direct path reached the destination,
egress control unverifiable (check the proxy and that the destination is reachable
through it)"
fi
```

A `REVIEW` here means the test could not attribute the result, most often because the destination is unreachable by any path. Resolve it by choosing a destination that is reachable through the approved proxy, so the verdict is trustworthy. Then confirm DNS resolution is restricted to approved internal resolvers, setting `APPROVED_RESOLVERS` to the space-separated list of approved resolver addresses. On a systemd-resolved host, `/etc/resolv.conf` carries only the local stub `127.0.0.53`. In that case the command reads the real upstream resolvers from `resolvectl`.

```
RESOLV="${RESOLV:-/etc/resolv.conf}"
RESOLVECTL="${RESOLVECTL:-resolvectl}"
APPROVED="<APPROVED_RESOLVERS>"
approved_check() {
 set=$1
 if printf '%s' "$APPROVED" | grep -q '<APPROVED_RESOLVERS>'; then
 echo "configured resolvers: $set"
 echo "REVIEW: set APPROVED_RESOLVERS, then confirm each resolver above is approved"
 return
 fi
 bad=""
 for n in $set; do
 case " $APPROVED " in *" $n "*) ;; *) bad="$bad $n" ;; esac
 done
 if [ -n "$bad" ]; then
 echo "FAIL: non-approved resolver(s):$bad"
 else
 echo "PASS: all resolvers approved ($set)"
 fi
}
if [ ! -r "$RESOLV" ]; then
 echo "ERROR: $RESOLV not found or not readable"
else
 ns=$(grep -E '^[[:space:]]*nameserver' "$RESOLV" | awk '{print $2}' | tr '\n' ' ' |
sed 's/[[:space:]]*$//')
 nstrim=$(printf '%s' "$ns" | tr -d '[:space:]')
 if [ -z "$nstrim" ]; then
 echo "REVIEW: no nameserver entries in $RESOLV"
 elif [ "$nstrim" = "127.0.0.53" ]; then
 up=$("$RESOLVECTL" status 2>/dev/null | grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}|([0-
9a-fA-F]{1,4}:)+[0-9a-fA-F:]*[0-9a-fA-F]' | grep -vx '127.0.0.53' | sort -u | tr '\n'
' ' | sed 's/[[:space:]]*$//')
 if [ -z "$up" ]; then
 echo "REVIEW: systemd-resolved stub 127.0.0.53 in $RESOLV, and resolvectl reported
no upstream resolvers or is unavailable, cannot assess"
 else
 approved_check "$up"
 fi
 else
 approved_check "$ns"
 fi
fi
```

This Recommendation is compliant when the egress test reports `PASS`, every configured or upstream resolver is an approved internal resolver, and the FQDN allowlist is enforced at the proxy.

Remediation:

Configure the MCP server's network environment so all outbound access routes through the egress proxy and direct egress is blocked at the network or host-firewall layer, restrict DNS resolution to approved internal resolvers, and define an FQDN allowlist of permitted outbound destinations enforced at the proxy. These are network and host configuration changes, and the exact mechanism (proxy environment, firewall egress rules, resolver configuration) is environment-specific. The audit attributes a blocked direct attempt only when the proxy path succeeds, so the deployment must have both a working approved proxy path to a known test destination and a blocked direct path for the audit to be conclusive. On a systemd-resolved host, set the approved upstream resolvers in the resolver configuration rather than in `/etc/resolv.conf`, which carries only the local stub. Log all outbound attempts, including blocked ones, as security events.

Default Value:

By default, MCP server processes can make direct outbound connections to arbitrary destinations, with DNS resolved by the host's default resolver and no egress proxy or FQDN allowlist in place.

References:

1. MCP specification (security best practices, SSRF, egress proxies, and DNS rebinding): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices
2. MCP specification (Server Features / Resources): https://modelcontextprotocol.io/specification/2026-07-28/server/resources

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.4 Implement and Manage a Firewall on Servers Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.	●	●	●
v8	9.2 Use DNS Filtering Services Use DNS filtering services on all enterprise assets to block access to known malicious domains.	●	●	●

7 Observability & Audit

Every Recommendation in this Benchmark depends on the ability to observe and verify how MCP operated. Without a faithful, time-synchronized record of what was requested, by whom, under which protocol version, and with what result, an operator cannot detect or investigate abuse.

The Recommendations in this section make MCP activity observable and its records reliable. They require lifecycle and invocation metadata to be recorded, alerts to fire on audience and issuer validation failures, non-null JSON-RPC request IDs to be enforced and logged, time synchronization to be in place, and cancellation and progress notifications to be recorded.

7.1 Audit Record Content & Integrity

7.1.1 Ensure lifecycle and invocation metadata is recorded (Automated)

Profile Applicability:

- Level 1

Description:

Structured audit-log entries are generated for MCP lifecycle events and invocation operations. Under the `2026-07-28` stateless model, `server/discover` calls and the per-request protocol metadata in each request's `_meta` (`io.modelcontextprotocol/protocolVersion`, `io.modelcontextprotocol/clientCapabilities`, and `io.modelcontextprotocol/clientInfo`) are recorded, together with the server identity each result reports in `_meta` as `io.modelcontextprotocol/serverInfo`. The invocation operations `tools/call`, `resources/read`, and `prompts/get` are recorded (input metadata, but not full argument payloads for privacy-sensitive tools), along with authentication events, elicitation exchanges, human approval and denial decisions, and Task lifecycle events. Every entry carries a synchronized timestamp, an event type, the producing server's identity, and a correlation identifier tying one request's events together (the JSON-RPC request id or a deployment correlation value). Where distributed tracing is enabled, the `traceparent` trace and span identifiers are recorded alongside that correlation identifier, not in place of it. Logs are shipped to a centralized store with integrity controls, captured at the server (`stderr` or an OpenTelemetry exporter) rather than through the deprecated MCP `logging` utility.

Rationale:

Without structured, correlated logs, an investigator cannot reconstruct what a deployment did: which server was invoked, under whose authority, with what approval, and to what effect. Under the stateless protocol no connection-level session groups a request's events, so a per-request correlation identifier is what makes reconstruction possible, and recording the server identity on each entry is what attributes an event to a server in a multi-server deployment. Distributed trace context, where present, supplements the correlation identifier rather than replacing it. Shipping to a centralized store with integrity controls protects the record from tampering by a compromised host.

Impact:

Structured logging adds storage overhead, which in regulated environments is a compliance requirement rather than a discretionary cost. Propagating a per-request correlation identifier is a one-time instrumentation cost where trace-context propagation is not already in place.

Audit:

Inspect a sample of recent structured audit-log entries and confirm each carries a non-empty timestamp, correlation identifier, event type, and server identity. Review the event-type inventory the command prints against the classes the deployment is expected to produce, and separately confirm logs are shipped to a centralized store with integrity protection. Run the following command, adjusting `LOG` and the field names to the local schema:

```
LOG="${LOG:-<MCP_AUDIT_LOG>}"
N="${N:-200}"
if [ ! -r "$LOG" ]; then echo "ERROR: audit log not readable at $LOG"; exit 0; fi
sample="$(tail -n "$N" "$LOG" 2>/dev/null)"
total="$(printf '%s\n' "$sample" | grep -c .)"
if [ "$total" -eq 0 ]; then echo "ERROR: no log entries found in $LOG"; exit 0; fi
good="$(printf '%s\n' "$sample" | jq -R '
  (fromjson? // null) as $o
  | select($o != null)
  | select(($o.timestamp|type) == "string" and $o.timestamp != "")
  | select(($o.correlation_id|type) == "string" and $o.correlation_id != "")
  | select(($o.event|type) == "string" and $o.event != "")
  | select((($o.server_id // $o.server // "") | tostring) != "")
  | 1' 2>/dev/null | grep -c .)"
bad=$(( total - good ))
distinct_cid="$(printf '%s\n' "$sample" | jq -rR 'fromjson? | .correlation_id //
empty' 2>/dev/null | grep -v '^$' | sort -u | grep -c .)"
echo "--- event types present in last $total entries ---"
printf '%s\n' "$sample" | jq -rR 'fromjson? | .event // empty' 2>/dev/null | grep -v
'^$' | sort | uniq -c | sort -rn
echo "--- field conformance ---"
echo "entries sampled: $total malformed entries: $bad distinct correlation ids:
$distinct_cid"
if [ "$bad" -gt 0 ]; then
  echo "FAIL: $bad of $total sampled entries lack a non-empty string timestamp,
correlation_id, event type, or server identity"
elif [ "$total" -gt 1 ] && [ "$distinct_cid" -le 1 ]; then
  echo "REVIEW: all $total sampled entries share a single correlation id, which
cannot be per-request, verify correlation id assignment"
else
  echo "PASS: every sampled entry carries a non-empty timestamp, correlation id,
event type, and server identity"
fi
```

This Recommendation is compliant when every sampled entry carries the four required fields, the command prints `PASS`, the event-type inventory shows the expected classes, and a reviewer confirms centralized shipping with integrity controls.

Remediation:

Standardize the audit-log schema across all MCP components so every entry carries a timestamp, a correlation identifier, an event type, the server identity, and the event-specific metadata, and extend it to cover elicitation, approval and denial, and Task lifecycle events. Use the JSON-RPC request id or a deployment correlation identifier for the correlation field, and where distributed tracing is available propagate the W3C `traceparent` in `_meta` and record the trace and span identifiers alongside it. Capture logs at the server through `stderr` or an OpenTelemetry exporter rather than the deprecated MCP `logging` utility, and ship them to a centralized store with integrity controls such as write-once storage or signed log batches. The exact mechanism is platform-specific, so this Recommendation states the required schema and pipeline rather than a portable command.

Default Value:

By default, MCP server implementations do not emit structured, correlated audit logs, and elicitation, approval, and Task events are not necessarily recorded unless logging is explicitly configured to capture them. The MCP logging utility is deprecated as of the `2026-07-28` revision, so even where a server still supports it, server-side capture through `stderr` or OpenTelemetry is the intended path instead.

References:

1. MCP specification (Base Protocol / Overview, per-request _meta and request correlation): https://modelcontextprotocol.io/specification/2026-07-28/basic
2. MCP specification (Deprecated Features): https://modelcontextprotocol.io/specification/2026-07-28/deprecated
3. MCP specification (Key Changes): https://modelcontextprotocol.io/specification/2026-07-28/changelog
4. MCP blog (The 2026-07-28 MCP Specification Release Candidate, W3C Trace Context in _meta): https://blog.modelcontextprotocol.io/posts/2026-07-28-release-candidate/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.2 Collect Audit Logs Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.	●	●	●
v8	8.5 Collect Detailed Audit Logs Configure detailed audit logging for enterprise assets containing sensitive data. Include event source, date, username, timestamp, source addresses, destination addresses, and other useful elements that could assist in a forensic investigation.		●	●

7.1.2 Ensure non-null JSON-RPC request IDs are enforced and included in audit logs (Automated)

Profile Applicability:

- Level 1

Description:

The protocol's JSON-RPC id rules are enforced, and the request id is recorded in the audit log. A request id must be a string or an integer, must not be `null`, and must not match the id of another request the sender has issued but not yet received a response for. The uniqueness requirement covers only requests still in flight: once a request completes, its id may be reused, so enforcement applies to ids awaiting a response rather than to every id ever seen. A server that receives a null id, or an id that collides with one of its own outstanding requests, rejects the message as a JSON-RPC Invalid Request with error code `-32600` and HTTP `400`. The JSON-RPC request id is recorded on each audit-log entry so a logged event can be tied back to the request that produced it.

Rationale:

Two kinds of bad request id break the correlation between a request and its response. An id that is `null` cannot be matched to a response at all. An id already in use by another request still awaiting a response lets one response be mistaken for another. Requiring ids to be non-null and unique among the requests still in flight keeps each request paired with its response, and recording the id in the audit log lets an investigator reconstruct which logged events belong to which request. Reuse after a response completes is not a violation, so a rule written against every id ever seen would reject legitimate traffic.

Impact:

Enforcing the id rules is standard JSON-RPC behavior and adds no meaningful overhead. Recording the request id is a schema requirement that pairs with structured audit logging.

Audit:

Send a request that is conformant except for a null id and confirm the server rejects it with HTTP `400` and JSON-RPC `-32600`, then confirm recent audit entries carry the JSON-RPC request id. Set `URL` to the server's MCP endpoint, `LOG` to the audit-log path, and `AUTH_HEADER` to a full authorization header (for example `Authorization: Bearer <token>`) when the endpoint requires authentication. Supplying the header lets the probe reach the id-validation path rather than being rejected for missing credentials:

```
URL="${URL:-https://<SERVER_URL>/mcp}"
LOG="${LOG:-<MCP_AUDIT_LOG>}"
AUTHH="${AUTH_HEADER:-}"
resp="$(curl -sS -m 15 -w '\n%{http_code}' -X POST "$URL" \
  -H 'Accept: application/json, text/event-stream' \
  -H 'Content-Type: application/json' \
  -H 'MCP-Protocol-Version: 2026-07-28' \
  -H 'Mcp-Method: tools/list' \
  -H "$AUTHH" \
  -d
'{"jsonrpc":"2.0","id":null,"method":"tools/list","params":{"_meta":{"io.modelcontext
protocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientInfo":{"name":"cis-benchmark-
audit","version":"1.0"},"io.modelcontextprotocol/clientCapabilities":{}}}}'
2>/dev/null)"
rc=$?
code="$(printf '%s' "$resp" | tail -n1)"
body="$(printf '%s' "$resp" | sed '$d')"
json="$body"
case "$body" in *"data:"*) json="$(printf '%s\n' "$body" | sed -n 's/^data:
\{0,1\}//p')" ;; esac
if [ $rc -ne 0 ] || [ -z "$body" ]; then
  nullid="unreachable"
elif printf '%s' "$json" | jq -e '.error != null' >/dev/null 2>&1; then
  errc="$(printf '%s' "$json" | jq -r '.error.code // empty' 2>/dev/null)"
  if [ "$code" = "400" ] && [ "$errc" = "-32600" ]; then nullid="rejected-clean";
else nullid="rejected-nonstd"; fi
else
  nullid="accepted"
fi
if [ ! -r "$LOG" ]; then
  corr="logmissing"
else
  tot="$(tail -n 200 "$LOG" 2>/dev/null | grep -c .)"
  with="$(tail -n 200 "$LOG" 2>/dev/null | jq -R '(fromjson? // null) as $o |
select($o != null) | select((($o.request_id // $o.jsonrpc_id // "") | tostring)) !=
"") | 1' 2>/dev/null | grep -c .)"
  if [ "${tot:-0}" -eq 0 ]; then corr="logempty"; elif [ "$with" -eq "$tot" ]; then
corr="present"; else corr="partial"; fi
fi
echo "--- null-id wire probe ---"; echo "http status: ${code:-none} null-id outcome:
$nullid"
echo "--- audit-log request-id field ---"; echo "state: $corr"
if [ "$nullid" = "unreachable" ]; then
  echo "ERROR: server unreachable or returned no body, null-id rejection could not be
assessed at $URL"
elif [ "$nullid" = "accepted" ]; then
  echo "FAIL: the server accepted a null JSON-RPC request id instead of returning an
error"
elif [ "$nullid" = "rejected-nonstd" ]; then
```

```
  echo "REVIEW: the null id was rejected but not with HTTP 400 and JSON-RPC -32600
(got HTTP ${code:-none}, code ${errc:-none}), verify the rejection, and if this was a
401 supply AUTH_HEADER so the probe reaches the request-id validation path"
elif [ "$corr" = "logmissing" ] || [ "$corr" = "logempty" ]; then
  echo "REVIEW: the null id was rejected with HTTP 400 and -32600, but the audit log
could not be read to confirm the JSON-RPC request id is recorded"
elif [ "$corr" = "partial" ]; then
  echo "FAIL: the null id was rejected correctly, but some audit entries lack the
JSON-RPC request id field"
else
  echo "PASS: the server rejected a null JSON-RPC request id with HTTP 400 and -32600
and audit entries carry the JSON-RPC request id"
fi
```

This Recommendation is compliant when the server rejects the null id with HTTP `400` and `-32600`, recent audit entries carry the JSON-RPC request `id`, and the command prints `PASS`.

Remediation:

Configure the server to reject a request whose id is `null`, or whose id matches one still awaiting a response, returning a JSON-RPC Invalid Request with error code `-32600` and HTTP `400`. For that check, track only the ids of requests still awaiting a response, and release an id for reuse once its response has been sent, rather than retaining every id seen. Record the JSON-RPC request id on every audit-log entry. Id-rule enforcement is typically provided by a conformant SDK, so the remediation is usually a matter of not overriding it and adding the request id to the log schema.

Default Value:

By default, some MCP server implementations accept null `ids` and may not record the request id as a correlation field.

References:

1. MCP specification (Base Protocol / Overview): https://modelcontextprotocol.io/specification/2026-07-28/basic
2. MCP specification (Schema Reference): https://modelcontextprotocol.io/specification/2026-07-28/schema
3. JSON-RPC 2.0 specification (id member and -32600 Invalid Request): https://www.jsonrpc.org/specification

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.2 Collect Audit Logs Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.	●	●	●
v8	8.5 Collect Detailed Audit Logs Configure detailed audit logging for enterprise assets containing sensitive data. Include event source, date, username, timestamp, source addresses, destination addresses, and other useful elements that could assist in a forensic investigation.		●	●

7.1.3 Ensure MCP audit records carry accurate, monotonic timestamps (Automated)

Profile Applicability:

- Level 1

Description:

Every record in the MCP audit stream carries a timestamp that is present, encoded in a consistent ISO 8601 format, non-decreasing within the audit stream, and close to real time. The property applies to the recorded timestamp values, not to any particular host time service. The timestamp field is read by name from the pinned audit-log schema.

Rationale:

If the recorded timestamps are missing, malformed, or move backward, every downstream use fails silently: two events cannot be ordered, a replay window cannot be computed, and an incident timeline cannot be reconstructed. Checking the recorded timestamps directly is a detective control over the property that matters, independent of which time daemon the host runs.

Impact:

Recording a well-formed timestamp on every audit record is low effort where the audit pipeline is already structured, and the check is read-only. A deployment whose recording host clock is not synchronized surfaces as a `REVIEW` or a non-monotonic `FAIL`, which points at the host time configuration as the thing to fix.

Audit:

Verify that every record in the MCP audit stream carries a parseable timestamp, that timestamps do not move backward within the stream, and that the newest record is within tolerance of the auditing host clock. Run the following command, supplying the audit-log path and the timestamp field name from the pinned schema:

```
LOG="${LOG:-<MCP_AUDIT_LOG>}"
TS_FIELD="${TS_FIELD:-<AUDIT_TIMESTAMP_FIELD>}"
TOL_SEC="${TOL_SEC:-5}"
if [ ! -f "$LOG" ]; then echo "ERROR: audit log $LOG not found"; exit 0; fi
if [ ! -s "$LOG" ]; then echo "ERROR: audit log $LOG is empty, no records to
evaluate"; exit 0; fi
now=$(date -u +%s)
result=$(jq -sRr --arg tsf "$TS_FIELD" --argjson now "$now" 'split("\n") |
map(select(length>0)) | reduce .[] as $l
({total:0,missing:0,unparse:0,nonmono:0,badline:0,prev:null,maxts:null}; .total+=1 |
(try ($l|fromjson) catch null) as $o | if $o==null then .badline+=1 elif
($o|has($tsf)|not) then .missing+=1 else (($o[$tsf]|tostring)|sub("\\.[0-
9]+";"")|sub("\\+00:00$";"Z")) as $n | (try ($n|fromdateiso8601) catch null) as $t |
if $t==null then .unparse+=1 else (if .prev!=null and $t<.prev then .nonmono+=1
else . end)|.prev=$t|(if .maxts==null or $t>.maxts then .maxts=$t else . end) end
end) | "\(.total) \(.missing) \(.unparse) \(.nonmono) \(.badline) \(if .maxts==null
then "" else ((.maxts-$now)|if .<0 then -. else . end|floor) end)"' "$LOG")
set -- $result
total="$1"; missing="$2"; unparse="$3"; nonmono="$4"; badline="$5"; skew="$6"
if ! printf '%s' "$total" | grep -Eq '^[0-9]+$'; then echo "ERROR: audit log could
not be parsed"; exit 0; fi
if [ "$badline" -gt 0 ]; then echo "ERROR: audit log has $badline non-JSON records,
cannot evaluate timestamp integrity"; exit 0; fi
if [ "$total" -eq 0 ]; then echo "ERROR: audit log has no records"; exit 0; fi
if [ "$missing" -gt 0 ] || [ "$unparse" -gt 0 ]; then echo "FAIL: $missing records
missing and $unparse with an unparseable timestamp in field $TS_FIELD"; exit 0; fi
if [ "$nonmono" -gt 0 ]; then echo "FAIL: $nonmono records have a timestamp earlier
than a preceding record, the recording clock is not monotonic"; exit 0; fi
if [ -n "$skew" ] && [ "$skew" -gt "$TOL_SEC" ]; then echo "REVIEW: newest record is
$skew seconds from the auditing host clock, confirm the recording host is time-
synchronized or that this reflects no recent activity"; exit 0; fi
echo "PASS: all $total records carry a parseable, monotonic timestamp within $TOL_SEC
seconds of the auditing host clock"
```

This Recommendation is compliant when the command prints `PASS`. A `REVIEW`, meaning the newest record is far from now, needs a human to decide between an unsynchronized recording clock and simple inactivity before the result is trusted.

Remediation:

Configure the MCP audit pipeline to stamp every record with a timestamp in a consistent parseable format drawn from a synchronized system clock, using the field name declared in the audit-log schema, and ensure the recording host maintains a synchronized clock so recorded timestamps track real time. Enabling time synchronization on a systemd host is:

```
sudo timedatectl set-ntp true
```

After enabling, set the trusted source and fallback in the time client's configuration and wire the drift alert into the monitoring platform. Those steps are environment-specific and have no single portable command.

Default Value:

By default, whether audit records carry a timestamp and in what format is determined by the server or gateway logging configuration, and is not guaranteed to be present, consistent, or drawn from a synchronized clock without configuration.

References:

1. MCP specification (Base Protocol / Authorization, token expiry and time-based validation): https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.4 Standardize Time Synchronization Standardize time synchronization. Configure at least two synchronized time sources across enterprise assets, where supported.		●	●
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

7.2 Detection & Alerting

7.2.1 Ensure alerts are generated on audience and issuer validation failures (Automated)

Profile Applicability:

- Level 1

Description:

A structured audit event is emitted whenever a token audience or an authorization-server issuer fails validation, and each event carries a recognized event identifier and a standardized error code. A SIEM rule alerts the security operations team on those events. Two kinds of failure are covered. In its resource-server role, an MCP server validates that an access token was issued for it as the intended audience and returns HTTP `401` when it is not. As an OAuth client, an MCP client validates the issuer (`iss`) on an authorization response to catch a response returned by the wrong authorization server.

Rationale:

Audience and issuer validation failures are signs of a stolen or misdirected token, or of a response coming back from the wrong authorization server, which is a bigger risk in the single-client, many-server pattern MCP encourages. An alert is only reliable if each failure is a structured event with an error code, because a free-text log line cannot be matched precisely, and loose substring matching also catches harmless records. Alerting on these failures turns a rejection that would otherwise pass unnoticed into a signal someone can act on.

Impact:

Structured failure events and a SIEM alert rule add little overhead. The benefit is early detection of token theft, audience confusion, and authorization-server mix-up that a silent `401` would not surface.

Audit:

Confirm that audience and issuer validation failures are emitted as structured events carrying an error code, then separately confirm a SIEM rule alerts on them and that the deployment rejects a wrong-audience token end to end. The command reads a structured JSON-lines authentication log and detects a failure by matching the structured identifier fields rather than free text, so a benign record that merely mentions a token claim is not counted. Run the following command, adjusting `LOG` and the field names to the local schema:

```
LOG="${LOG:-<MCP_AUTH_LOG>}"
N="${N:-500}"
if [ ! -r "$LOG" ]; then echo "ERROR: auth log not readable at $LOG"; exit 0; fi
sample="$(tail -n "$N" "$LOG" 2>/dev/null)"
total="$(printf '%s\n' "$sample" | grep -c .)"
if [ "$total" -eq 0 ]; then echo "ERROR: no entries found in $LOG"; exit 0; fi
fails="$(printf '%s\n' "$sample" | jq -cR '
 (fromjson? // null) as $o
 | select($o != null)
 | ( ( (($o.event // "") | tostring) + " " + (($o.reason // "") | tostring) + " "
 + (($o.type // "") | tostring) + " " + (($o.error // "") | tostring) + " "
 + (($o.code // "") | tostring) + " " + (($o.error_code // "") | tostring) + " " +
(($o.http_status // "") | tostring) )
 | ascii_downcase ) as $id
 | select(($id | test("audience|issuer|\\baud\\b|\\biss\\b")) and ($id |
test("invalid|mismatch|fail|reject|unauthor|denied|401|403")))
 | $o' 2>/dev/null)"
nfail="$(printf '%s\n' "$fails" | grep -c .)"
coded="$(printf '%s\n' "$fails" | jq -c 'select((((.error_code // .code // .error
// .http_status // "") | tostring)) != "")' 2>/dev/null | grep -c .)"
echo "--- audience/issuer validation failures in last $total entries ---"
echo "failure events (matched on structured event, reason, type, error, code fields):
$nfail with structured error code: $coded"
if [ "$nfail" -eq 0 ]; then
 echo "REVIEW: no audience or issuer validation failures in the sample, which cannot
distinguish no failures from failures not logged as structured events, force one with
a wrong-audience token and confirm the SIEM alert rule exists"
elif [ "$coded" -lt "$nfail" ]; then
 echo "FAIL: $(( nfail - coded )) of $nfail audience or issuer validation failure
events lack a structured error code"
else
 echo "PASS: all $nfail audience or issuer validation failures carry a structured
error code, confirm the SIEM alert rule fires on them"
fi
```

This Recommendation is compliant when the audience and issuer validation-failure events in the sample all carry a structured error code, the command prints `PASS`, and a reviewer confirms a SIEM rule fires on them and a deliberately mis-scoped token is rejected. Where the sample contains no such failures, force one with a wrong-audience token and confirm it is logged as a structured event and alerted on.

Remediation:

Configure the resource-server audience validation and the client issuer validation to emit a structured audit event on every failure, each carrying a recognized event identifier and an error code rather than a free-text log line. Add a SIEM rule that alerts on those event identifiers and routes to the security operations team. As optional defense in depth, a deployment may wire authorization-server introspection and revocation checks into the validation path so revoked tokens also surface, but that is not required to satisfy this Recommendation. The exact event schema and SIEM rule syntax are platform-specific.

Default Value:

By default, audience and issuer validation failures may be logged as generic errors without structured codes, and no alerting rule exists.

References:

1. MCP specification (Base Protocol / Authorization): https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization
2. MCP specification (key changes): https://modelcontextprotocol.io/specification/2026-07-28/changelog
3. RFC 8707 Resource Indicators for OAuth 2.0 (token audience): https://datatracker.ietf.org/doc/html/rfc8707
4. RFC 9207 OAuth 2.0 Authorization Server Issuer Identification (iss parameter): https://datatracker.ietf.org/doc/html/rfc9207

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.2 Collect Audit Logs Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.	●	●	●
v8	8.11 Conduct Audit Log Reviews Conduct reviews of audit logs to detect anomalies or abnormal events that could indicate a potential threat. Conduct reviews on a weekly, or more frequent, basis.		●	●
v8	13.1 Centralize Security Event Alerting Centralize security event alerting across enterprise assets for log correlation and analysis. Best practice implementation requires the use of a SIEM, which includes vendor-defined event correlation alerts. A log analytics platform configured with security-relevant correlation alerts also satisfies this Safeguard.		●	●

7.2.2 Ensure notifications/cancelled and notifications/progress messages are monitored for behavioral anomalies (Automated)

Profile Applicability:

- Level 2

Description:

Cancellation and progress signals are monitored for anomalous patterns, with cancellation, progress, and task lifecycle kept distinct and each correlated by the identifier the protocol defines for it. Cancellation depends on the transport: on stdio it is a `notifications/cancelled` message keyed by `params.requestId`, while Streamable HTTP has no such message and cancellation appears as a response-stream close or abort seen at the server or gateway. Progress is correlated by the `progressToken` carried in `_meta`, and task lifecycle, where the Tasks extension is in use, is a separate mechanism using a server-generated `taskId`. Per-server baselines for cancellation rate, stream-close rate, and progress frequency and payload volume are established over a learning period. An alert is raised when a server deviates from its baseline in a rolling window, or when a cancellation `requestId` or progress `progressToken` matches no request the deployment issued. Because stream resumability was removed at this revision, a request re-issued after a broken stream carries a new id, and the baseline excludes it as a legitimate near-duplicate.

Rationale:

Cancellation and progress signals are low-visibility side channels, and abuse shows up as a shift in rate or pattern rather than a discrete rule violation. Correlating by the correct identifier is what makes an anomaly legible: a `notifications/cancelled` whose `requestId` names no request the deployment issued, or a `notifications/progress` whose `progressToken` matches no originating request, is a spoofed or malformed signal a per-server baseline surfaces only if the monitor keys on the identifier the protocol actually uses. Matching on the wrong field, as an earlier version did, produces confident results about a relationship the protocol does not define. Monitoring progress-payload volume additionally turns the progress channel into a detection surface for data exfiltration through long-running operations.

Impact:

This control requires a learning period to establish baselines and ongoing SIEM rule maintenance. The benefit is detection of notification-abuse patterns that no single-event rule would catch.

Audit:

Confirm that cancellation and progress notifications carry a server identity and map to an originating request by the correct identifier. Separately, confirm the SIEM baseline and rate-anomaly rules exist. This block requires jq and a structured JSON-lines log in which each issued request is recorded with its JSON-RPC id, and its `progressToken` where progress was requested. Adjust `LOG` and the field names to the local schema. Run the following command:

```
LOG="${LOG:-<MCP_AUDIT_LOG>}"
N="${N:-2000}"
if [ ! -r "$LOG" ]; then echo "ERROR: audit log not readable at $LOG"; exit 0; fi
sample="$(tail -n "$N" "$LOG" 2>/dev/null)"
total="$(printf '%s\n' "$sample" | grep -c .)"
if [ "$total" -eq 0 ]; then echo "ERROR: no entries found in $LOG"; exit 0; fi
issued_ids="$(printf '%s\n' "$sample" | jq -rR '(fromjson? // null) as $o |
select($o != null) | (($o.request_id // $o.id) // empty) | tostring' 2>/dev/null |
grep -v '^$' | sort -u)"
issued_tokens="$(printf '%s\n' "$sample" | jq -rR '(fromjson? // null) as $o |
select($o != null) | (($o.progress_token // $o.params."_meta".progressToken //
$o."_meta".progressToken) // empty) | tostring' 2>/dev/null | grep -v '^$' | sort -
u)"
canc="$(printf '%s\n' "$sample" | jq -cR '(fromjson? // null) as $o | select($o !=
null) | select(($o.method // "") == "notifications/cancelled") | $o' 2>/dev/null)"
prog="$(printf '%s\n' "$sample" | jq -cR '(fromjson? // null) as $o | select($o !=
null) | select(($o.method // "") == "notifications/progress") | $o' 2>/dev/null)"
ncanc="$(printf '%s\n' "$canc" | grep -c .)"
nprog="$(printf '%s\n' "$prog" | grep -c .)"
nnotif=$(( ncanc + nprog ))
noid="$( { printf '%s\n' "$canc"; printf '%s\n' "$prog"; } | grep . | jq -c
'select((((.server_id // .server // "") | tostring)) == "")' 2>/dev/null | grep -
c .)"
canc_ids="$(printf '%s\n' "$canc" | jq -r '(.params.requestId // empty) | tostring'
2>/dev/null | grep -v '^$' | sort -u)"
prog_tokens="$(printf '%s\n' "$prog" | jq -r '(.params.progressToken // empty) |
tostring' 2>/dev/null | grep -v '^$' | sort -u)"
canc_orphan="$(comm -23 <(printf '%s\n' "$canc_ids" | grep -v '^$') <(printf '%s\n'
"$issued_ids" | grep -v '^$') 2>/dev/null | grep -c .)"
prog_orphan="$(comm -23 <(printf '%s\n' "$prog_tokens" | grep -v '^$') <(printf
'%s\n' "$issued_tokens" | grep -v '^$') 2>/dev/null | grep -c .)"
orphan=$(( canc_orphan + prog_orphan ))
echo "--- notification correlation in last $total entries ---"
echo "cancelled(by requestId): $ncanc   progress(by progressToken): $nprog   without
server identity: $noid   orphaned: $orphan (cancel $canc_orphan, progress
$prog_orphan)"
echo "(note: on Streamable HTTP cancellation is a closed response stream, not a
notifications/cancelled message; monitor stream-close events out of band. Tasks use
taskId via tasks/cancel, monitored separately.)"
if [ "$nnotif" -eq 0 ]; then
  echo "REVIEW: no cancellation or progress notifications in the sample, so no per-
server baseline can be derived, extend the window or confirm the transport, and
confirm the SIEM baseline rules exist"
elif [ "$orphan" -gt 0 ]; then
  echo "FAIL: $orphan notification(s) reference a requestId or progressToken with no
originating request in the log"
elif [ "$noid" -gt 0 ]; then
  echo "FAIL: $noid notification event(s) lack a server identity, which prevents per-
server baselining"
else
```

```
  echo "PASS: cancellation and progress notifications carry server identity and map
to an originating request, confirm the SIEM baseline and rate-anomaly rules exist"
fi
```

This Recommendation is compliant when the notifications carry a server identity, every `requestId` and `progressToken` maps to an originating request, the command prints `PASS`, and a reviewer confirms that the per-server baselines and rate-anomaly rules exist in the SIEM. The command does not compute a rate baseline itself, and on Streamable HTTP it does not consume stream-close events, which are observed separately at the server or gateway. Those two gaps are the manual part of this Recommendation.

Remediation:

Configure structured logging to record issued requests with their JSON-RPC id and, where progress is requested, their `progressToken`, and to capture cancellation and progress notifications with server identity. On stdio, capture `notifications/cancelled` and correlate it by `params.requestId`. On Streamable HTTP, capture response-stream close and abort events, which are the cancellation signal there, and correlate each to its originating request. Capture `notifications/progress` and correlate it by `params.progressToken`. Where the Tasks extension is in use, capture task lifecycle separately by `taskId`. Establish per-server baselines over a learning period. Then add SIEM rules that alert on three conditions: a per-server cancellation or stream-close rate above a baseline-derived threshold, progress frequency or payload volume outside baseline, and any cancellation `requestId` or progress `progressToken` that references no originating request. Exclude requests re-issued after a broken stream from the anomaly logic. Route alerts to the security operations team. The exact rule syntax is platform-specific.

Default Value:

By default, cancellation and progress notifications are not baselined or alerted on, and a cancellation `requestId` or progress `progressToken` that references no originating request goes undetected.

References:

1. MCP specification (Base Protocol / Message Patterns / Progress): https://modelcontextprotocol.io/specification/2026-07-28/basic/patterns/progress
2. MCP specification (Base Protocol / Transports / Overview, cancellation binding): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports
3. MCP specification (Base Protocol / Transports / stdio, cancellation via notifications/cancelled): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/stdio
4. MCP specification (Schema Reference): https://modelcontextprotocol.io/specification/2026-07-28/schema
5. SEP-2663 (Tasks Extension): https://modelcontextprotocol.io/seps/2663-tasks-extension

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.11 Conduct Audit Log Reviews Conduct reviews of audit logs to detect anomalies or abnormal events that could indicate a potential threat. Conduct reviews on a weekly, or more frequent, basis.		●	●
v8	13.1 Centralize Security Event Alerting Centralize security event alerting across enterprise assets for log correlation and analysis. Best practice implementation requires the use of a SIEM, which includes vendor-defined event correlation alerts. A log analytics platform configured with security-relevant correlation alerts also satisfies this Safeguard.		●	●

8 Supply Chain Security

An MCP server is third-party code that runs with access to the organization's data, tools, and downstream services. A server introduced without review, pulled from an untrusted source, or updated silently to a different artifact than the one that was vetted can subvert every control that assumes the server is what it claims to be. The supply chain is therefore part of the deployment's attack surface, not a separate concern that ends at installation.

The Recommendations in this section govern how third-party servers and connectors enter and stay in the environment. They require servers and connectors to be vetted and approved before use, pinned to a verified container digest or artifact hash so that the running server is provably the reviewed one, and served exclusively from trusted distribution channels. The aim is that no server reaches production without a review, and no reviewed server is silently replaced by another.

8.1 Ensure third-party MCP servers and connectors are vetted and allowlists are enforced (Manual)

Profile Applicability:

- Level 1

Description:

Code review and security assessment are required for every third-party MCP server and connector before it is approved for use. MCP hosts and clients enforce a runtime allowlist so that only approved servers can be connected to. Each allowlist entry is keyed on the server's `server.json` name together with its approved distribution locator, version, and content digest. A remote server is additionally keyed on its exact approved origin. The `server.json` name only identifies who may publish under a namespace, not whether the installed instance is the approved build, so it is one component of the key rather than the whole basis of trust. The allowlist covers every distribution channel the deployment installs from, including remote server URLs, not just a single ecosystem. The current MCP Registry package types are `npm`, `pypi`, `nuget`, `oci`, and `mcpb`, given as an example rather than a fixed set. The approved tool, resource, prompt, and permission surface of each allowlisted server is documented. Any expansion or material change to that surface, or to the executable package, is blocked until it has been reviewed and promoted through the approval workflow.

Rationale:

Third-party servers are untrusted code with access to whatever the host grants them. A local server is a downloaded binary that runs with the client's privileges, so installing one is equivalent to running arbitrary code. Vetting before approval and enforcing a runtime allowlist ensure that only reviewed servers can connect. Tying the allowlist to a documented tool, resource, prompt, and permission surface stops a server from quietly expanding what it does after approval. Presence in the public MCP Registry is not a substitute for vetting, because the registry authenticates namespaces and hosts metadata but does not scan server code. The specification's install-consent requirement for one-click local installs is a per-install prompt, not a persistent approved-server allowlist, so it complements this Recommendation rather than replacing it.

Impact:

Vetting and allowlist maintenance are ongoing costs. A default-deny allowlist will block any currently connected server that lacks a vetting record, so those servers need to be identified and quarantined before enforcement is turned on. Covering every channel, not only `npm` and `pypi`, means keying on remote URLs, `oci` digests, and `mcpb` artifacts. Some hosts cannot do that natively, so enforcement may need to sit at a gateway.

Audit:

Confirm that a vetting record exists for each third-party server in use, and that the host or client enforces a connection allowlist that refuses any server not on the list. Where the allowlist is file-based, run the following command to list the approved servers for review:

```
ALLOWLIST="<MCP_ALLOWLIST_FILE>"
if [ ! -e "$ALLOWLIST" ]; then
  echo "ERROR: no allowlist file at the configured path, confirm where the host or client enforces the approved-server allowlist (it may live in a policy engine or gateway rather than a file)"
  exit 0
fi
if [ ! -r "$ALLOWLIST" ]; then
  echo "ERROR: allowlist file present but not readable, re-run with sufficient privilege"
  exit 0
fi
approved=$(grep -vE '^[[:space:]]*(#|$)' "$ALLOWLIST")
count=$(printf '%s\n' "$approved" | grep -c .)
if [ "$count" -eq 0 ]; then
  echo "Approved servers: (none)"
  echo "REVIEW: allowlist present but lists no approved servers, confirm whether the host is default-deny with nothing yet approved or enforcement is unpopulated"
  exit 0
fi
echo "Approved servers ($count):"
printf '%s\n' "$approved"
echo "REVIEW: confirm each entry has a current vetting record and that the host or client refuses any server not listed, keyed on the server.json name or the pinned package identifier and version"
```

This Recommendation is compliant when every server the host or client can connect to is on the enforced allowlist, each allowlisted server has a current vetting record, and any server without one is refused. Because the enforcement mechanism and the vetting records require human confirmation, the command's final verdict is `REVIEW` on the normal path and `ERROR` when the configured path is absent or unreadable.

Remediation:

Establish a vetting workflow of code review and security assessment that completes before a third-party server is added to the allowlist. Configure the host or client to refuse connections to any server not on the allowlist. Key each allowlist entry on the server's `server.json` name together with the approved locator, version, and content digest, and key a remote server on its exact approved origin as well. Make sure the enforcement point covers every distribution channel the deployment installs from, not only the common package managers. Document each approved server's tool, resource, prompt, and permission surface, and block any expansion or material change pending review. Quarantine or remove any server in use that lacks a completed vetting record. This is host, client, and process configuration, and the exact allowlist mechanism is implementation-specific, so no single portable enforcement command is given.

Default Value:

By default, MCP hosts and clients may connect to any configured server without a vetting record or an enforced approved-server allowlist. Under the `2026-07-28` revision, a client that supports one-click local server configuration shows an install-consent dialog before running the install command. That is a per-install prompt, not a persistent allowlist, so the default posture still permits connecting an unvetted server once consent is given.

References:

1. MCP specification (security best practices): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices
2. MCP Registry (server identity): https://modelcontextprotocol.io/registry/about
3. MCP Registry (supported package types): https://modelcontextprotocol.io/registry/package-types
4. SEP-1024 (MCP Client Security Requirements for Local Server Installation): https://modelcontextprotocol.io/seps/1024-mcp-client-security-requirements-for-local-server-

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	2.1 Establish and Maintain a Software Inventory Establish and maintain a detailed inventory of all licensed software installed on enterprise assets. The software inventory must document the title, publisher, initial install/use date, and business purpose for each entry; where appropriate, include the Uniform Resource Locator (URL), app store(s), version(s), deployment mechanism, and decommission date. Review and update the software inventory bi-annually, or more frequently.	●	●	●
v8	2.5 Allowlist Authorized Software Use technical controls, such as application allowlisting, to ensure that only authorized software can execute or be accessed. Reassess bi-annually, or more frequently.		●	●

8.2 Ensure third-party MCP servers are pinned to a verified content hash and re-vetted before updates (Manual)

Profile Applicability:

- Level 1

Description:

Each approved third-party MCP server has a recorded SHA-256 content hash of its approved artifact, computed the same way the audit recomputes it. The approved artifact is unambiguous: a canonical package archive, an `oci` manifest digest, an `mcpb` artifact, or a deterministic bundle-manifest artifact generated by a documented process. This keeps a single-file entry point from leaving dependencies outside the integrity boundary. Where a native SHA-256 pin exists it is recorded directly: `mcpb` `fileSha256` is SHA-256, and an `oci` digest is SHA-256 when the reference uses sha256:. Otherwise the algorithm is preserved, or an enterprise SHA-256 is computed by the documented procedure. `npm` integrity is SHA-512 and pip hash-checking is operator-provided (SHA-256 recommended). Both are verified once, at install time, and are separate from the recorded hash. The recorded hash is keyed to the server's identity: the `package.json` `mcpName` for `npm` installs, the `server.json` name for registry-style layouts, and the bundle manifest identity for `mcpb`. Before starting a server, the host or gateway recomputes the hash of its artifact and refuses to start it if the value differs from the one recorded for its identity. An installed server with no recorded approved hash is also refused. Any update to a third-party server triggers a re-vetting workflow equivalent to initial onboarding before the new version is permitted. The approved hash and the re-vetting approval are recorded in the enterprise capability baseline.

Rationale:

A content hash detects any change to an approved artifact, whether a legitimate update or a malicious substitution. The MCP Registry requires an `mcpb` package entry to carry a `fileSha256`, which clients validate before installation, but that is a one-time, per-channel check. Recording an approved hash keyed to server identity, and blocking on a mismatch, extends integrity checking to artifacts altered after install and to channels that carry no install-time validation. Each installed instance's own bytes are compared to an approved hash for its identity, rather than merely confirming that the identity appears in the approved set. That distinction stops an artifact from passing on some other approved server's hash, and it catches a second install that shares an approved identity but carries different bytes (a shadow install). Requiring re-vetting on update ensures changes are reviewed rather than trusted automatically. This applies the same discipline to the artifact that the capability baseline applies to a changed tool list: it is quarantined until reviewed.

Impact:

Hash tracking and re-vetting add overhead per update, and legitimate updates are delayed by the re-vetting cycle, which is intended. Because the audit refuses any server with no recorded hash, every server must be pinned before the check passes, so expect to pin or remove unpinned installs. Hashing dependencies needs a deterministic manifest (stable file ordering, no embedded timestamps), which may require a canonicalization step the deployment defines.

Audit:

Verify that every recorded artifact matches its approved SHA-256, and that every installed server instance under the servers root maps to an approved hash for its identity. The approved-hash file has one line per pinned artifact in the form `<sha256> <identity> <artifact_path>`. One identity may carry more than one approved hash if it legitimately runs more than one build. The instance scan takes identities from each `server.json` name and each `package.json mcpName`, and hashes each instance's own entry artifact. This block requires `bash`. Run the following command with `MCP_SERVERS_ROOT` set to the directory of installed servers:

```
APPROVED="<APPROVED_HASHES_FILE>"
SERVERS_ROOT="<MCP_SERVERS_ROOT>"
if [ ! -r "$APPROVED" ]; then
 echo "ERROR: approved-hash list not readable at the configured path, cannot verify
any server"
 exit 0
fi
command -v sha256sum >/dev/null 2>&1 || { echo "ERROR: sha256sum unavailable, cannot
verify artifact hashes"; exit 0; }
norm() { printf '%s' "$1" | tr -d '\r' | sed 's/\xc2\xa0//g'; }
declare -A approved_for
rec_h=(); rec_id=(); rec_art=(); checked=0
while IFS= read -r raw || [ -n "$raw" ]; do
 line=$(printf '%s' "$raw" | tr -d '\r' | sed 's/\xc2\xa0/ /g')
 case "$line" in ""|\#*) continue ;; esac
 set -- $line
 exp=$(printf '%s' "$1" | tr -cd '0-9a-fA-F'); id="$2"; art="$3"
 checked=$((checked+1))
 if [ -z "$exp" ] || [ -z "$id" ] || [ -z "$art" ]; then
 echo "server (malformed line): needs '<sha256> <identity> <artifact_path>'";
continue
 fi
 approved_for["$id"]="${approved_for[$id]} $exp"
 rec_h+=("$exp"); rec_id+=("$id"); rec_art+=("$art")
done < "$APPROVED"
if [ "$checked" -eq 0 ]; then
 echo "ERROR: approved-hash list is empty, no server is pinned"
 exit 0
fi
fail=0; err=0; unpinned=0; shadow=0
i=0
while [ "$i" -lt "${#rec_h[@]}" ]; do
 exp="${rec_h[$i]}"; id="${rec_id[$i]}"; art="${rec_art[$i]}"; i=$((i+1))
 if [ ! -r "$art" ]; then
 echo "approved $id: recorded artifact not readable at $art"; err=$((err+1));
continue
 fi
 got=$(sha256sum "$art" | awk '{print $1}')
 if [ "$got" = "$exp" ]; then
 echo "approved $id: recorded artifact at $art matches"
 else
 echo "approved $id: recorded artifact at $art MISMATCH, computed $got";
fail=$((fail+1))
 fi
done
resolve_entry() {
 d="$1"; e=""
 [ -f "$d/package.json" ] && e=$(norm "$(grep -oE
'"main"[[:space:]]*:[[:space:]]*"[^"]+"' "$d/package.json" | sed -E
's/.*"([^"]+)"$/\1/' | head -1)")
```

```
 { [ -n "$e" ] && [ -f "$d/$e" ] && printf '%s' "$d/$e"; } && return
 [ -f "$d/dist/index.js" ] && { printf '%s' "$d/dist/index.js"; return; }
 [ -f "$d/server.js" ] && { printf '%s' "$d/server.js"; return; }
 ls "$d"/*.js 2>/dev/null | head -1
}
if [ -n "$SERVERS_ROOT" ] && [ -d "$SERVERS_ROOT" ]; then
 declare -A seen_dir
 while IFS= read -r idf; do
 d=$(dirname "$idf")
 [ -n "${seen_dir[$d]:-}" ] && continue
 case "$idf" in
 */server.json) iid=$(tr -d '\r' < "$idf" | grep -oE
'"name"[[:space:]]*:[[:space:]]*"[^"]+"' | sed -E 's/.*:[[:space:]]*"([^"]+)".*/\1/'
| head -1) ;;
 */package.json) grep -q '"mcpName"' "$idf" || continue
 iid=$(tr -d '\r' < "$idf" | grep -oE '"mcpName"[[:space:]]*:[[:space:]]*"[^"]+"' |
sed -E 's/.*:[[:space:]]*"([^"]+)".*/\1/' | head -1) ;;
 *) continue ;;
 esac
 [ -z "$iid" ] && continue
 seen_dir["$d"]=1
 entry=$(resolve_entry "$d")
 if [ -z "$entry" ] || [ ! -r "$entry" ]; then
 echo "installed $iid at $d: no readable entry artifact to hash, cannot integrity-
check this instance"; err=$((err+1)); continue
 fi
 ihash=$(sha256sum "$entry" | awk '{print $1}')
 if [ -z "${approved_for[$iid]:-}" ]; then
 echo "installed $iid at $d: unpinned, no approved-hash entry for this identity";
unpinned=$((unpinned+1)); continue
 fi
 case " ${approved_for[$iid]} " in
 *" $ihash "*) echo "installed $iid at $d: instance artifact matches an approved
hash" ;;
 *) echo "installed $iid at $d: instance artifact hash $ihash is NOT an approved
value for this identity (shadow or tampered install)"; shadow=$((shadow+1)) ;;
 esac
 done < <(find "$SERVERS_ROOT" \( -name server.json -o -name package.json \) -print
2>/dev/null)
fi
if [ "$fail" -gt 0 ] || [ "$unpinned" -gt 0 ] || [ "$shadow" -gt 0 ]; then
 echo "FAIL: $fail recorded mismatch(es), $shadow unapproved installed instance(s),
$unpinned unpinned install(s) across $checked pinned record(s)"
elif [ "$err" -gt 0 ]; then
 echo "ERROR: $err artifact(s) could not be verified across $checked pinned
record(s), not a PASS"
else
 echo "PASS: every pinned record matches and every installed instance maps to an
approved hash for its identity"
fi
```

This Recommendation is compliant when every pinned server's artifact matches the hash recorded for its own identity and no installed server is unpinned. The per-server lines are diagnostics and the final line is the single verdict. Confirm separately that the host or gateway refuses to start a server on a hash mismatch and that an update triggers re-vetting rather than automatic approval, which the audit calls for but does not itself enforce.

Remediation:

Record the SHA-256 hash of each approved third-party server artifact, including its dependency bundle, in the capability baseline, keyed to the server's `server.json` name. Take the value from a native SHA-256 pin where one exists (`mcpb` `fileSha256`, or an `oci` reference that uses a `sha256:` digest). Otherwise compute a SHA-256 over the approved artifact by a documented procedure. Package-registry integrity, such as an `npm`, `pypi`, or `nuget` feeds, is a separate install-time check and is not the recorded value, and only the channels the deployment installs from need to be covered. Configure the host or gateway to refuse to start any server whose recomputed artifact hash does not match the value recorded for its identity, and to refuse any installed server with no recorded approved hash. Implement a re-vetting workflow, equivalent to initial onboarding, triggered by a hash mismatch or an update notification. This is host, gateway, and process configuration, and the blocking enforcement is implementation-specific.

Default Value:

By default, third-party server artifacts are loaded by version or path without a recorded approved-hash check, so an altered artifact at the same version loads silently. The MCP Registry requires an `mcpb` package entry to carry a `fileSha256`, which clients validate before installation, but that check is install-time only and specific to the `mcpb` channel. Without recording and enforcing an approved hash, there is no drift check after install and no equivalent check on the `npm`, `pypi`, `nuget`, or remote channels.

References:

1. MCP specification (security best practices): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices
2. MCP Registry (server identity): https://modelcontextprotocol.io/registry/about
3. MCP Registry (supported package types): https://modelcontextprotocol.io/registry/package-types

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	2.1 Establish and Maintain a Software Inventory Establish and maintain a detailed inventory of all licensed software installed on enterprise assets. The software inventory must document the title, publisher, initial install/use date, and business purpose for each entry; where appropriate, include the Uniform Resource Locator (URL), app store(s), version(s), deployment mechanism, and decommission date. Review and update the software inventory bi-annually, or more frequently.	●	●	●
v8	2.5 Allowlist Authorized Software Use technical controls, such as application allowlisting, to ensure that only authorized software can execute or be accessed. Reassess bi-annually, or more frequently.		●	●

8.3 Ensure third-party MCP server artifacts are served exclusively from an internal private mirror (Manual)

Profile Applicability:

- Level 2

Description:

Enterprises permitting third-party MCP servers maintain an internal private mirror of all approved server artifacts, where each server's provenance has been vetted and its cryptographic hashes verified against official publisher records. Production MCP hosts install packaged servers only from this internal mirror across every installable channel the deployment uses, which may be a self-hosted package mirror and an internal container registry. The current MCP Registry package types are `npm`, `pypi`, `nuget`, `oci`, and `mcpb`, named as examples rather than a set every deployment must have. A remote MCP server is a live endpoint URL rather than a mirrorable artifact, so its equivalent control is to approve its exact origin and route access through an approved enterprise egress gateway or private connector, not to mirror it. An internal MCP registry implementing the MCP Registry OpenAPI interface may hold approved discovery metadata for both, but that metadata catalog is distinct from the package artifact repository that serves the code, and the two roles are not conflated. Direct access from production hosts to external community sources, including `registry.npmjs.org`, `pypi.org`, public container registries, release-hosted `mcpb` artifacts, and the public MCP Registry, is blocked by network egress controls, and remote-server access is confined to the approved origins through the gateway.

Rationale:

Installing directly from public sources exposes production to dependency-confusion, typosquatting, and account-takeover attacks at the source. The public MCP Registry authenticates namespaces and hosts metadata, but it delegates code security scanning to the underlying package registries and downstream aggregators, so it is not itself a vetting gate. That is why provenance vetting and hash verification are enforced at the enterprise mirror instead. An internal mirror of vetted, hash-verified artifacts, combined with egress blocking of public sources, ensures that production installs only reviewed artifacts and cannot be redirected to a malicious upstream.

Impact:

Operating a mirror is infrastructure overhead. Covering every channel the deployment uses can mean mirroring or proxying container images and bundle artifacts as well as the common language-ecosystem packages, not only one package manager. Blocking public egress from production hosts can break a host that legitimately needs some external access, so the egress policy is scoped to the registry endpoints rather than applied as a blanket deny.

Audit:

Confirm that the package managers on production hosts point to the internal mirror. This block requires bash. Run the following command with the internal mirror host set to check each configured source:

```
MIRROR="<INTERNAL_MIRROR_HOST>"
if [ -z "$MIRROR" ]; then
 echo "ERROR: no internal mirror host configured to compare against, set the mirror
host first"
 exit 0
fi
host_of() {
 u=$(printf '%s' "$1" | tr -d '\r' | sed 's/\xc2\xa0//g')
 u="${u#*://}"
 u="${u%%/*}"
 u="${u#*@}"
 u="${u%%\?*}"
 u="${u%%:*}"
 printf '%s' "$u" | tr 'A-Z' 'a-z'
}
present=0; bad=0
check() {
 name="$1"; src="$2"
 case "$src" in "") return ;; esac
 present=$((present+1))
 h=$(host_of "$src")
 if [ "$h" = "$MIRROR" ]; then
  echo "$name source $src resolves to host $h, the internal mirror"
 else
  echo "$name source $src resolves to host $h, not the internal mirror $MIRROR";
bad=$((bad+1))
 fi
}
check_multi() {
 name="$1"; vals="$2"
 oldifs=$IFS
 IFS='
'
 for one in $vals; do
  case "$one" in *://*) IFS=$oldifs; check "$name" "$one"; IFS='
' ;; esac
 done
 IFS=$oldifs
}
if command -v npm >/dev/null 2>&1; then
 check "npm registry" "$(npm config get registry 2>/dev/null)"
 scoped=$(npm config list 2>/dev/null | sed -nE
's/^[[:space:]]*(@[^=:]+:registry)[[:space:]]*=[[:space:]]*"?([^"[:space:]]+)"?.*/\1
\2/p')
 oldifs=$IFS
 IFS='
'
 for line in $scoped; do
  IFS=$oldifs
  key=${line%% *}; val=${line#* }
```

```
  check "npm $key" "$val"
  IFS='
'
 done
 IFS=$oldifs
fi
if command -v pip >/dev/null 2>&1; then
 pl=$(pip config list 2>/dev/null)
 iu=$(printf '%s\n' "$pl" | sed -nE "s/.*\\.index-
url[[:space:]]*=[[:space:]]*'?([^'[:space:]]+)'?.*/\\1/p" | head -1)
 [ -n "$PIP_INDEX_URL" ] && iu="$PIP_INDEX_URL"
 [ -z "$iu" ] && iu="https://pypi.org/ (default, index-url unset)"
 check "pip index-url" "$iu"
 eiu=$(printf '%s\n' "$pl" | sed -nE "s/.*\\.extra-index-
url[[:space:]]*=[[:space:]]*'?([^']*)'?.*/\\1/p")
 check_multi "pip extra-index-url" "$eiu $PIP_EXTRA_INDEX_URL"
 fl=$(printf '%s\n' "$pl" | sed -nE "s/.*\\.find-
links[[:space:]]*=[[:space:]]*'?([^']*)'?.*/\\1/p")
 check_multi "pip find-links" "$fl $PIP_FIND_LINKS"
fi
if [ "$present" -eq 0 ]; then
 echo "ERROR: neither npm nor pip present, confirm the package managers in use and
their configured sources"
elif [ "$bad" -gt 0 ]; then
 echo "FAIL: $bad of $present package manager source(s) do not resolve to the
internal mirror host"
else
 echo "PASS: all $present package manager source(s) resolve to the internal mirror
host"
fi
```

Confirm that direct access from the host to the public source is blocked while the internal mirror is reachable. This block requires bash and curl, and it runs from the production host's network context. Run the following command with the internal mirror URL and the public source URL set (for example `https://registry.npmjs.org/` as the public source):

```
MIRROR_URL="<INTERNAL_MIRROR_URL>"
PUBLIC_URL="<PUBLIC_REGISTRY_URL>"
probe() {
  code=$(curl -sS -o /dev/null -w '%{http_code}' --max-time 8 "$1" 2>/dev/null)
  case "$code" in ""|000) code="000" ;; esac
  printf '%s' "$code"
}
mirror_code=$(probe "$MIRROR_URL")
public_code=$(probe "$PUBLIC_URL")
if [ "$mirror_code" = "000" ]; then
  echo "ERROR: internal mirror unreachable (code $mirror_code), cannot attribute the
egress result, run from the production host network context and confirm the mirror
URL"
  exit 0
fi
case "$public_code" in
  000) echo "REVIEW: public registry not reachable while the mirror is (public
$public_code, mirror $mirror_code), confirm with egress-policy evidence that this is
enforced denial and not a transport failure" ;;
  2*|3*) echo "FAIL: public registry reachable (code $public_code), egress to the
public registry is not restricted" ;;
  *) echo "REVIEW: public registry returned $public_code, distinguish a blocking
proxy from an auth wall" ;;
esac
```

This Recommendation is compliant when every package manager's configured source is the internal mirror, and the public source is not reachable from the production host while the mirror is. A reviewer also confirms that the mirror holds only vetted, hash-verified artifacts, and that the channels the commands do not check (`oci`, `mcpb`, and remote URLs) are likewise served only from internal sources.

Remediation:

Establish an internal artifact mirror (for example a self-hosted registry such as Artifactory or Nexus, an internal container registry, and an internal MCP registry) holding only vetted, hash-verified artifacts, and configure production host package managers and container runtimes to use only those internal sources. Run the following command to point `npm` and `pip` at the internal mirror:

```
npm config set registry https://<INTERNAL_REGISTRY>
pip config set global.index-url https://<INTERNAL_PYPI>
```

Remove or repoint any `npm` scoped registry (`@scope:registry=`) and any pip `extra-index-url` or `find-links` entry that resolves to a source other than the internal mirror, including their environment overrides (`PIP_EXTRA_INDEX_URL`, `PIP_FIND_LINKS`), so that no configured source bypasses the mirror. Configure the container runtime and any `mcpb` install path to pull only from the internal registry. Configure network egress controls on production hosts to block direct access to the public sources for every channel in use, scoped to those registry endpoints rather than a blanket deny.

Default Value:

By default, each channel points at its public source: `registry.npmjs.org`, `pypi.org`, public container registries such as `docker.io` and `ghcr.io`, release-hosted `mcpb` artifacts, and the public MCP Registry. There is no egress restriction, so a production host can pull directly from any of them.

References:

1. MCP specification (security best practices): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices
2. MCP Registry (server identity): https://modelcontextprotocol.io/registry/about
3. MCP Registry (supported package types): https://modelcontextprotocol.io/registry/package-types

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	2.1 Establish and Maintain a Software Inventory Establish and maintain a detailed inventory of all licensed software installed on enterprise assets. The software inventory must document the title, publisher, initial install/use date, and business purpose for each entry; where appropriate, include the Uniform Resource Locator (URL), app store(s), version(s), deployment mechanism, and decommission date. Review and update the software inventory bi-annually, or more frequently.	●	●	●
v8	2.5 Allowlist Authorized Software Use technical controls, such as application allowlisting, to ensure that only authorized software can execute or be accessed. Reassess bi-annually, or more frequently.		●	●

9 Isolation & Execution Safety

A hardened deployment contains what a compromised or misbehaving server can reach. A server that runs with broad host privileges or is unconstrained by mandatory access controls turns a single compromise into access across the environment.

The Recommendations in this section limit the blast radius of execution. They require remote servers to be containerized with least privilege, filesystem ACLs and mandatory access controls to be applied in addition to container boundaries, and local stdio servers to run under least-privilege identities.

9.1 Ensure remote MCP servers run in containers or VMs with least privilege and gateway ingress controls applied (Manual)

Profile Applicability:

- Level 2

Description:

Streamable HTTP MCP servers run in containers or VMs with non-root user identities and read-only root filesystems. All Linux capabilities are dropped, with only an approved set added back, and the process cannot regain privileges through `setuid` or file capabilities (`no-new-privileges`). Kernel-enforced confinement (seccomp and AppArmor, or an equivalent mandatory access control profile) is left enabled rather than disabled. Servers sit behind a gateway that enforces ingress policy. Authorization enforcement, rate limiting, anomaly detection, and a kill switch are centralized at the gateway rather than implemented separately in each server. An ingress gateway does not mediate the server's outbound connections, so egress confinement is enforced at the host or network layer.

Rationale:

A containerized server running as root with a writable filesystem and full capabilities offers an attacker who compromises it a wide path to escalate and persist. Non-root execution, a read-only root filesystem, dropped capabilities, and kernel confinement shrink that path to the minimum the runtime needs. Dropping capabilities alone is incomplete, because a process can regain privileges through a `setuid` binary, so preventing new privileges is what makes the capability drop hold. A gateway that centralizes ingress policy, authorization, rate limiting, and a kill switch provides a single enforcement and shut-off point, rather than relying on each server to police itself. Outbound confinement is enforced separately at the host or network layer, not by the ingress gateway.

Impact:

Hardening the security context and routing through a gateway adds deployment and operational complexity. Each server needs its container configured for a non-root user, a read-only root filesystem, all capabilities dropped, `no-new-privileges`, and, where the server genuinely needs one, an approved capability add-back set. A server with an undeclared runtime need will fail to start until that set is corrected.

Audit:

Confirm the container or containers running the server are hardened. Pass one or more container names or ids in `CONTAINER`, for example a single name or `$(docker compose ps -q)` for a multi-container deployment. A container that exists but is not running returns `ERROR`, because inspecting a stopped object does not prove the running workload is confined.

```
CONTAINER="<CONTAINER_NAME_OR_ID>"
EXPECTED_CAPS="${EXPECTED_CAPS:-}"
insp=$(docker inspect $CONTAINER 2>/dev/null)
if [ -z "$insp" ] || [ "$(printf '%s' "$insp" | jq -r 'length' 2>/dev/null)" = "0" ];
then
 echo "ERROR: no container found for $CONTAINER; confirm the server container is
running, or if the server runs in a VM or directly on a host, audit it with the
systemd command below"
else
 printf '%s' "$insp" | jq -r --arg expected_caps "$EXPECTED_CAPS" '
 def isnum: test("^[0-9]+(:[0-9]+)?$");
 ($expected_caps | ascii_downcase | [splits("[ ,]+")] | map(ltrimstr("cap_")) |
map(select(. != "")) | sort) as $expcaps
 | ([ .[] | select((.State.Running // false) != true) | (.Name // "container") ]) as
$stopped
 | if ($stopped | length) > 0
 then "ERROR: container(s) exist but are not running (" + ($stopped | join(", ")) +
"); runtime confinement cannot be verified"
 else
 ([ .[] | select(((.Config.User // "") != "") and (((.Config.User) | isnum) | not)
and (((.Config.User) | test("^(root|0)(:|$)")) | not)) | "detail: User " +
(.Config.User) + " is a name, confirm it resolves to a non-zero uid in the image" ])
as $un
 | ([ .[] | (.Name // "container") as $n | (.Config // {}) as $cfg | (.HostConfig //
{}) as $h | ($h.SecurityOpt // []) as $so
 | { n:$n, fails: [
 (if (($cfg.User // "")) != "" and (($cfg.User // "") | test("^(root|0)(:|$)") |
not)) then empty else "non-root-user" end),
 (if ($h.Privileged == true) then "privileged" else empty end),
 (if ($h.ReadonlyRootfs == true) then empty else "readonly-rootfs" end),
 (if (($h.CapDrop // []) | any(ascii_upcase == "ALL")) then empty else "cap-drop=ALL"
end),
 (if (($h.CapAdd // []) | length == 0) then empty
 elif ($expcaps | length) > 0 and ((($h.CapAdd // []) | map(ascii_downcase |
ltrimstr("cap_")) | sort) == $expcaps) then empty
 else "cap-add=" + (($h.CapAdd // [])|join(",")) end),
 (if ($so | any(test("seccomp[:=]unconfined";"i"))) then "seccomp-disabled" else
empty end),
 (if ($so | any(test("apparmor[:=]unconfined";"i"))) then "apparmor-disabled" else
empty end),
 (if ($so | any(test("no-new-privileges";"i"))) then empty else "no-new-privileges"
end)
 ] } ]) as $r
 | ([ $r[] | select((.fails|length)>0) ]) as $bad
 | ($un + [ if ($bad|length)==0
 then "PASS: every running container is non-root and unprivileged, with a read-only
root filesystem, all capabilities dropped, no-new-privileges set, and seccomp and
AppArmor confinement not disabled"
 else "FAIL: " + ([ $bad[] | .n + "[" + (.fails|join(" ")) + "]" ] | join("; "))
end ]) | join("\n")
```

```
 end'
fi
```

For a server deployed in a VM or directly on a host under systemd rather than in a container, run the following command to confirm the service unit is hardened to the equivalent properties. Pass the unit name in `UNIT`, and optionally the approved capability set in `EXPECTED_CAPS`.

```
UNIT="<SYSTEMD_UNIT>"
EXPECTED_CAPS="${EXPECTED_CAPS:-}"
props=$(systemctl show "$UNIT" -p LoadState -p User -p NoNewPrivileges -p
CapabilityBoundingSet -p ProtectSystem -p SystemCallFilter -p AppArmorProfile -p
SELinuxContext 2>/dev/null)
load=$(printf '%s\n' "$props" | sed -n 's/^LoadState=//p')
if [ "$load" != "loaded" ]; then
 echo "ERROR: unit $UNIT is not loaded; confirm the server runs under this systemd
unit, or if it runs in a container use the container command above"
else
 user=$(printf '%s\n' "$props" | sed -n 's/^User=//p')
 nnp=$(printf '%s\n' "$props" | sed -n 's/^NoNewPrivileges=//p')
 caps=$(printf '%s\n' "$props" | sed -n 's/^CapabilityBoundingSet=//p')
 psys=$(printf '%s\n' "$props" | sed -n 's/^ProtectSystem=//p')
 scf=$(printf '%s\n' "$props" | sed -n 's/^SystemCallFilter=//p')
 aa=$(printf '%s\n' "$props" | sed -n 's/^AppArmorProfile=//p')
 se=$(printf '%s\n' "$props" | sed -n 's/^SELinuxContext=//p')
 fails=""
 case "$user" in ""|root|0|0:*) fails="$fails non-root-user" ;; esac
 [ "$nnp" = "yes" ] || [ -n "$scf" ] || fails="$fails no-new-privileges"
 case "$psys" in strict|full) : ;; *) fails="$fails read-only-rootfs" ;; esac
 [ -n "$scf" ] || fails="$fails seccomp"
 { [ -n "$aa" ] && [ "$aa" != "unconfined" ]; } || [ -n "$se" ] || fails="$fails mac-
profile"
 if [ -n "$fails" ]; then
  echo "FAIL: unit $UNIT missing required hardening:$fails"
 elif [ -n "$EXPECTED_CAPS" ]; then
  want=$(printf '%s' "$EXPECTED_CAPS" | tr 'A-Z ,' 'a-z\n\n' | sed 's/^cap_//' | grep
-v '^$' | sort | tr '\n' ' ')
  got=$(printf '%s' "$caps" | tr 'A-Z ' 'a-z\n' | sed 's/^cap_//' | grep -v '^$' |
sort | tr '\n' ' ')
  if [ "$want" = "$got" ]; then
   echo "PASS: unit $UNIT runs non-root with no-new-privileges, ProtectSystem=$psys,
a seccomp filter, a MAC profile, and CapabilityBoundingSet matching the approved set"
  else
   echo "REVIEW: unit $UNIT has the required hardening but CapabilityBoundingSet
($caps) does not match EXPECTED_CAPS, confirm the bounding set is minimized"
  fi
 else
  echo "REVIEW: unit $UNIT runs non-root with no-new-privileges, ProtectSystem=$psys,
a seccomp filter, and a MAC profile, confirm CapabilityBoundingSet ($caps) is
minimized to the server's needs"
 fi
fi
```

Separately, confirm from a network position outside the gateway that the server answers only through the gateway. The system is compliant when the applicable hardening command reports `PASS`, or reports `REVIEW` that the reviewer resolves by confirming the capability set is minimized, and the server is reachable only via the gateway.

Remediation:

Configure the container for a non-root user, a read-only root filesystem, all capabilities dropped with only strictly required capabilities added back, and `no-new-privileges`. Leave Docker's default seccomp and AppArmor profiles in place rather than disabling them. Under Docker, run the server with the following hardened configuration, omitting `--cap-add` when no capability is approved:

```
docker run --user <UID>:<GID> --read-only --cap-drop=ALL --cap-add=<APPROVED_CAP> --
security-opt no-new-privileges <IMAGE>
```

Do not use `--privileged`, `--security-opt seccomp=unconfined`, or `--security-opt apparmor=unconfined`. Managed container platforms and Kubernetes set the same properties through the container security context. Apply the following, omitting `add` when no capability is approved, and attach an AppArmor or SELinux profile through the platform's mechanism:

```
securityContext:
  runAsNonRoot: true
  runAsUser: <UID>
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  capabilities:
    drop: ["ALL"]
    add: ["<APPROVED_CAP>"]
  seccompProfile:
    type: RuntimeDefault
```

For a server deployed in a VM or directly on a host rather than a container, configure the equivalent properties on the service manager. Run it under a dedicated non-root account and apply the following directives to the systemd unit, leaving `CapabilityBoundingSet` empty when no capability is approved and replacing `AppArmorProfile` with `SELinuxContext` on an SELinux host. Leave the mandatory access control profile enabled:

```
[Service]
User=<SERVICE_ACCOUNT>
NoNewPrivileges=yes
CapabilityBoundingSet=<APPROVED_CAPS>
ProtectSystem=strict
ReadWritePaths=<REQUIRED_PATHS>
SystemCallFilter=@system-service
AppArmorProfile=<PROFILE>
```

Separately, place the server behind a gateway that enforces ingress policy and centralizes authorization, rate limiting, anomaly detection, and a kill switch. Enforce outbound confinement separately at the host or network layer, for example with an egress proxy or a network policy, because the ingress gateway does not mediate the server's outbound connections. The gateway is distinct from the hardening and does not substitute for it.

Default Value:

By default, a container runs as root with a writable root filesystem and Docker's default capability set, and `no-new-privileges` is off. Docker does apply its default seccomp and AppArmor profiles unless they are explicitly disabled. Servers may be directly reachable without a gateway. These defaults vary by container runtime and platform.

References:

1. MCP specification (Base Protocol / Overview / Statelessness): https://modelcontextprotocol.io/specification/2026-07-28/basic/index#statelessness
2. MCP specification (security best practices): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	13.4 Perform Traffic Filtering Between Network Segments Perform traffic filtering between network segments, where appropriate.		●	●

9.2 Ensure the MCP server process is confined to its working set by OS-level kernel-enforced controls (Automated)

Profile Applicability:

- Level 1

Description:

Each MCP server process is confined to the directories it legitimately needs by OS-level enforcement that the kernel applies regardless of what the server code attempts. Suitable mechanisms include POSIX ACLs, a mandatory access control profile such as AppArmor or SELinux, and container bind mounts for containerized deployments. The deployment selects the mechanism, and whichever is used, the service account's effective filesystem access is confined to its working set. MCP Roots is a deprecated advisory feature that a server can ignore, so it is never the boundary and is not relied on for containment. The boundary is verified as the MCP service account, by reading a known file inside the working set and being denied a known file outside it, and denied attempts are logged for anomaly detection.

Rationale:

MCP Roots does not constrain a server that ignores it, and under the 2026-07-28 revision it is deprecated, so only kernel-enforced OS-level mechanisms actually contain file access. POSIX ACLs, MAC profiles, and bind mounts restrict the service account to its working directory regardless of what the server code attempts, which makes them the real boundary. Logging denials surfaces probing or misbehavior.

Impact:

Confining the process requires identifying the legitimate working set and may need iteration. Where a mandatory access control profile is used, it is authored and confirmed separately from any ACLs.

Audit:

Confirm, as the MCP service account, that a known file inside the working set is readable and a known file outside it is denied. Then review the confinement configuration to confirm it restricts the account to the intended directories. Reading a system file such as `/etc/shadow` does not demonstrate the boundary, because every non-root account is denied it by default. The check therefore reads a sentinel the account should reach and a sentinel it should not. This block requires sudo and the service account. Run the following command, supplying the two sentinel file names in `WORK_SENTINEL` and `OOS_SENTINEL`:

```
SVC_ACCT="<MCP_SERVICE_ACCOUNT>"
WORK_DIR="<MCP_WORK_DIR>"
OUT_OF_SCOPE="<SENSITIVE_DIR_OUTSIDE_SCOPE>"
WORK_SENTINEL="${WORK_SENTINEL:-<WORK_SENTINEL_FILE>}"
OOS_SENTINEL="${OOS_SENTINEL:-<OUT_OF_SCOPE_SENTINEL_FILE>}"
if [ ! -f "$WORK_DIR/$WORK_SENTINEL" ] || [ ! -f "$OUT_OF_SCOPE/$OOS_SENTINEL" ];
then
 echo "ERROR: sentinel files missing, need a readable file at
$WORK_DIR/$WORK_SENTINEL and $OUT_OF_SCOPE/$OOS_SENTINEL so the denial is
attributable to the boundary and not to an empty path"
else
 in_ok=1; out_denied=1
 sudo -u "$SVC_ACCT" cat "$WORK_DIR/$WORK_SENTINEL" >/dev/null 2>&1 || in_ok=0
 if sudo -u "$SVC_ACCT" cat "$OUT_OF_SCOPE/$OOS_SENTINEL" >/dev/null 2>&1; then
out_denied=0; fi
 if [ "$in_ok" != 1 ]; then
 echo "ERROR: service account cannot read its own working file
$WORK_DIR/$WORK_SENTINEL, the profile is wrong or the account is misconfigured"
 elif [ "$out_denied" = 1 ]; then
 echo "PASS: service account reads its working file and is denied the out-of-scope
file $OUT_OF_SCOPE/$OOS_SENTINEL"
 else
 echo "FAIL: service account can read out-of-scope file $OUT_OF_SCOPE/$OOS_SENTINEL
(OS controls not confining it to $WORK_DIR)"
 fi
fi
```

This Recommendation is compliant when the account reads its working sentinel, is denied the out-of-scope sentinel, and the command prints `PASS`, and a reviewer confirms that the mandatory access control profile is present and that denied attempts are logged.

Remediation:

Confine the MCP service account to its working directory with OS-level enforcement and deny it access to sensitive paths. Then confirm the effective access with the sentinel check in the Audit. Where POSIX ACLs are the chosen mechanism, the following commands apply a working-directory-only restriction. They need setfacl from the acl package and a filesystem mounted with ACL support:

```
SVC_ACCT="<MCP_SERVICE_ACCOUNT>"
WORK_DIR="<MCP_WORK_DIR>"
SENSITIVE_DIR="<SENSITIVE_DIR>"
sudo setfacl -R -m u:"$SVC_ACCT":rX "$WORK_DIR"
sudo setfacl -R -m u:"$SVC_ACCT":--- "$SENSITIVE_DIR"
```

Where a mandatory access control profile is the chosen mechanism, author an AppArmor or SELinux profile that confines the process to the same scope. The profile is platform-specific and is not applied by the commands above. For containerized deployments, expose only the working directory through a bind mount, read-only where possible. Configure denied-access logging for anomaly detection.

Default Value:

By default, an MCP service account can read any file its OS user context permits. Roots is deprecated and never restricted file access.

References:

1. MCP specification (Client Features / Roots): https://modelcontextprotocol.io/specification/2026-07-28/client/roots
2. MCP specification (Deprecated Features): https://modelcontextprotocol.io/specification/2026-07-28/deprecated
3. MCP specification (security best practices): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●

9.3 Ensure local stdio MCP servers run under least-privilege identities with sandbox or container isolation (Manual)

Profile Applicability:

- Level 2

Description:

Each local stdio MCP server runs under a least-privilege identity and inside an isolation boundary that limits its filesystem reach, network egress, and ability to spawn subprocesses to what its declared function requires. A stdio server is a subprocess of the host and, without such a boundary, inherits the full privileges of the host user: every file, credential, environment variable, and network destination that user can reach. The isolation boundary may be a container, an operating-system sandbox, systemd sandboxing directives, or a mandatory access control profile combined with capability restrictions. The mechanism varies by platform, but the required properties do not. The execution identity is non-privileged, and where feasible it is distinct from or restricted relative to the interactive user. Filesystem access is limited to the paths the server's declared function requires. Outbound network access is limited to the destinations the server's function requires, which is none for a server with no network function. Subprocess execution is disabled or restricted to declared helper binaries. In higher-risk environments, sandboxed execution is the default for all local servers, with exceptions documented.

Rationale:

A remote, network-exposed server is contained so that a compromise stays inside a boundary, and a compromised local server needs the same containment. The local population is in practice the riskier one. It is dominated by community tool wrappers with a documented record of command injection, the attack class in which a crafted argument to a wrapper yields arbitrary command execution. Filesystem confinement limits what a compromised server can read on disk. It does not stop that server from opening an outbound connection to exfiltrate what it can read, or from spawning a shell, which is exactly the capability this attack class gives an attacker. Constraining egress and subprocess execution turns arbitrary code execution inside the sandbox from a full compromise of the user context into a bounded event: the attacker holds a process that can read its permitted paths and reach its permitted destinations, and nothing else. A least-privilege identity is what makes the rest enforceable, because a sandbox configured by the same unconstrained user it confines is only advisory.

Impact:

There is a meaningful setup cost per server. Each local server needs its isolation profile authored (permitted paths, permitted destinations, permitted helpers). A server with an undeclared runtime dependency will break inside the boundary until the profile is corrected. Developer workflows that install and run ad hoc local servers acquire friction by design. Platforms differ in mechanism, so the properties are portable while the profiles are not.

Audit:

Verify for each approved local stdio server that the running process is confined as its profile declares. Confirm that the execution identity is the designated least-privilege identity, that the isolation mechanism limits filesystem access to the paths documented for the server, that a default-deny egress policy with an allowlist of the declared destinations is applied, and that subprocess execution is disabled or restricted. Under systemd, the command below evaluates the unit's sandbox properties. Supply the approved account in `EXPECTED_USER`, and any approved capability set in `EXPECTED_CAPS` where the deployment defines one. A unit that is loaded but not active returns `ERROR`, because a running process is required to verify runtime confinement. An IPv4 catch-all denial is accepted as default-deny when the unit restricts address families to exclude `AF_INET6`. The command needs systemctl. Run the following command:

```
UNIT="<SYSTEMD_UNIT>"
out=$(systemctl show "$UNIT" -p
LoadState,ActiveState,SubState,User,NoNewPrivileges,ProtectSystem,ProtectHome,Capabil
ityBoundingSet,SystemCallFilter,IPAddressDeny,IPAddressAllow,RestrictAddressFamilies
2>/dev/null)
get() { printf '%s\n' "$out" | sed -n "s/^$1=//p" | head -n1; }
norm_caps() { printf '%s' "$1" | tr 'A-Z' 'a-z' | tr ' ,' '\n\n' | sed
's/^cap_//;/^$/d' | sort | tr '\n' ' '; }
load=$(get LoadState)
if [ -z "$out" ] || [ "$load" = "not-found" ] || [ "$load" = "bad-setting" ] || { [ -
n "$out" ] && [ -z "$load" ]; }; then
  echo "ERROR: unit $UNIT not loaded (LoadState='${load:-unknown}') or systemctl
unavailable; confirm the unit name, or use the container or MAC form of this audit"
else
  active=$(get ActiveState)
  if [ "$active" != "active" ]; then
    echo "ERROR: unit $UNIT is loaded but not active (ActiveState='${active:-
unknown}', SubState='$(get SubState)'); a running process is required to verify
runtime confinement"
  else
    fails=""
    u=$(get User)
    if [ -n "${EXPECTED_USER:-}" ]; then
      [ "$u" = "$EXPECTED_USER" ] || fails="$fails User"
    else
      { [ -n "$u" ] && [ "$u" != "root" ]; } || fails="$fails User"
    fi
    [ "$(get NoNewPrivileges)" = "yes" ] || fails="$fails NoNewPrivileges"
    ps=$(get ProtectSystem); { [ "$ps" = "strict" ] || [ "$ps" = "full" ]; } ||
fails="$fails ProtectSystem"
    ph=$(get ProtectHome); { [ "$ph" = "yes" ] || [ "$ph" = "read-only" ] || [ "$ph"
= "tmpfs" ]; } || fails="$fails ProtectHome"
    cbs=$(get CapabilityBoundingSet)
    if [ -n "${EXPECTED_CAPS:-}" ]; then
      [ "$(norm_caps "$cbs")" = "$(norm_caps "$EXPECTED_CAPS")" ] || fails="$fails
CapabilityBoundingSet"
    else
      [ -z "$cbs" ] || fails="$fails CapabilityBoundingSet"
    fi
    scf=$(get SystemCallFilter)
    [ -n "${scf#\~}" ] || fails="$fails SystemCallFilter"
    deny=$(get IPAddressDeny)
    allow=$(get IPAddressAllow)
    raf=$(get RestrictAddressFamilies)
    ipv6_blocked=0
    if [ -n "$raf" ]; then
      case "$raf" in
        "~"*) case "$raf" in *AF_INET6*) ipv6_blocked=1 ;; esac ;;
        *) case "$raf" in *AF_INET6*) : ;; *) ipv6_blocked=1 ;; esac ;;
      esac
```

```
    fi
    deny_ok=0
    if printf '%s' " $deny " | grep -q ' any '; then deny_ok=1
    elif printf '%s' "$deny" | grep -q '0\.0\.0\.0/0' && printf '%s' "$deny" | grep -
q '::/0'; then deny_ok=1
    elif printf '%s' "$deny" | grep -q '0\.0\.0\.0/0' && [ "$ipv6_blocked" = 1 ];
then deny_ok=1
    fi
    allow_bad=0
    printf '%s' "$allow" | grep -Eq '0\.0\.0\.0/0|::/0|(^| )any( |$)' && allow_bad=1
    { [ "$deny_ok" = 1 ] && [ "$allow_bad" = 0 ]; } || fails="$fails
EgressDefaultDeny"
    for f in $fails; do echo "detail: property not at required setting: $f"; done
    [ -z "${EXPECTED_USER:-}" ] && echo "detail: User=$u not verified against an
approved profile; confirm it is the designated least-privilege identity"
    [ -n "$allow" ] && [ "$allow_bad" = 0 ] && echo "detail: egress allowlist present
(IPAddressAllow=$allow); confirm it matches the server documented destinations"
    [ -z "$fails" ] && echo "PASS: running unit under a restricted identity with
filesystem, capability, syscall, and default-deny egress restrictions applied" ||
echo "FAIL: unit is missing required sandbox properties:$fails"
  fi
fi
```

Then test denials as the confined identity: attempt to read a path outside the profile, open a connection to an undeclared destination, and spawn an undeclared binary, and confirm that each is denied and logged. This Recommendation is compliant when every approved local server runs under its designated identity inside an isolation boundary whose filesystem, egress, and subprocess restrictions match its documented profile, and whose denials can be demonstrated.

Remediation:

Author an isolation profile for each approved local server that documents its required paths, required network destinations, and required helper binaries. Run the server inside a boundary that enforces that profile. The boundary may be a container with a non-root user, a read-only root filesystem, dropped capabilities, and an egress policy. It may be systemd sandboxing directives: a dedicated user, filesystem protection, a default-deny egress policy set with `IPAddressDeny=any` and an `IPAddressAllow` allowlist of the declared destinations, an approved capability set, and restricted system calls. Or it may be a MAC profile with equivalent constraints. Set the execution identity to a dedicated least-privilege account where the host supports launching servers under one. Log denials from the boundary for anomaly detection. The mechanism differs by platform, but the required properties (identity, filesystem, egress, subprocess) do not.

Default Value:

By default, a stdio server spawned by a host runs as the host user, with that user's full filesystem, network, and process-execution reach, and outbound network access is unrestricted. No mainstream host applies an isolation boundary to spawned servers on its own. This varies by host and platform.

References:

1. MCP specification (Base Protocol / Transports / stdio): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/stdio
2. MCP specification (security best practices): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.4 Implement and Manage a Firewall on Servers Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.	●	●	●

10 Resource Limits & Caching

MCP requests and responses can be large, resources can be fetched repeatedly, and a caller or server can consume far more than a task requires. Unbounded consumption threatens availability, and unmanaged caching can serve stale data beyond the point where it is safe to trust.

The Recommendations in this section bound both consumption and freshness. They require resources to be cached with freshness limits so that stale content is not served indefinitely, and they require size limits on request and response bodies, token budgets, and rate limits so that no single caller or server can exhaust the deployment.

10.1 Ensure safe, static resources are cached with freshness limits and per-user data is never shared-cached (Manual)

Profile Applicability:

- Level 1

Description:

Genuinely immutable or infrequently changing resources are cached using `ETag` or `Last-Modified` headers, which enable conditional requests, and static content carries appropriate `Cache-Control` directives for client-side caching. Cached entries are invalidated when a resource updates, and a relevant change notification invalidates a cached result immediately, even before its `ttlMs` elapses. Authorization decisions, security-sensitive resource content, and resources that vary by user identity are never held in a shared cache. This Recommendation covers two caching layers. At the HTTP layer, static resource content served over HTTP carries validators and a `Cache-Control` policy, and dynamic or identity-sensitive content is marked `no-store` or `private`. At the protocol layer, the `2026-07-28` revision requires the cacheable-result fields `resultType`, `ttlMs`, and `cacheScope` on the results of `resources/list`, `resources/read`, `resources/templates/list`, `tools/list`, `prompts/list`, and `server/discover`. Every ordinary result carries `resultType: "complete"`. Any per-user or identity-sensitive result is marked `cacheScope: private`, so a shared intermediary reuses it only within the same authorization context. Only genuinely non-identity-specific data is marked `cacheScope: public`. Because a public result may be shared across callers even from an authenticated endpoint, `cacheScope` is set alongside per-primitive access control and is not relied on alone.

Rationale:

Caching immutable resources reduces load and latency without risk. The danger is caching the wrong thing. Caching an authorization decision can let a revoked permission persist, and serving per-user data from a shared cache can disclose one user's data to another. Under the `2026-07-28` revision, the `cacheScope` hint makes the sharing boundary explicit. Marking identity-sensitive data `public` is a direct confidentiality failure, because a public result may be reused across authorization contexts even when it came from an authenticated endpoint. The default for anything that varies by user is therefore `private`. The hint does not replace access control, so per-primitive authorization is enforced regardless of the cache scope set.

Impact:

Configuring `ETag` and `Cache-Control` for static content is low effort. Scoping per-user data as `private` adds negligible cost.

Audit:

Inspect the caching response headers for a sample of static and dynamic resources. Run the following command against a static resource. It reports a single verdict and rejects a resource that carries a non-shareable directive alongside caching:

```
URL="https://<SERVER_URL>/static/<PATH>"
resp=$(curl -sS -m 10 -D - -o /dev/null "$URL" 2>/dev/null) || resp=""
code=$(printf '%s\n' "$resp" | awk 'NR==1{print $2}')
val=$(printf '%s\n' "$resp" | grep -Ei '^(etag|last-modified):' | tr -d '\r')
cc=$(printf '%s\n' "$resp" | grep -Ei '^cache-control:' | head -1 | sed 's/^[Cc]ache-
[Cc]ontrol:[[:space:]]*//' | tr -d '\r')
tokens=$(printf '%s' "$cc" | tr ',' '\n' | sed 's/^[[:space:]]*//; s/[[:space:]]*$//'
| tr 'A-Z' 'a-z')
has_maxage=$(printf '%s\n' "$tokens" | grep -Eqx 'max-age=[0-9]+' && echo 1)
has_nostore=$(printf '%s\n' "$tokens" | grep -Eqx 'no-store' && echo 1)
has_private=$(printf '%s\n' "$tokens" | grep -Eqx 'private' && echo 1)
if ! printf '%s' "$code" | grep -Eq '^[0-9]+$'; then
  echo "ERROR: static resource unreachable, no attribution possible"
elif [ "$code" -ge 400 ]; then
  echo "ERROR: static path returned $code, cannot attribute cache policy to the
resource"
elif [ -n "$has_nostore" ] || [ -n "$has_private" ]; then
  echo "FAIL: static resource carries a contradictory non-shareable directive (no-
store/private) alongside caching ($code)"
elif [ -n "$val" ] && [ -n "$has_maxage" ]; then
  echo "PASS: static resource carries a validator and a Cache-Control max-age
($code)"
else
  echo "FAIL: static resource lacks a validator or a Cache-Control max-age ($code)"
fi
```

Run the following command against a dynamic or identity-sensitive resource. It matches `no-store` or `private` as directives, so `no-cache` alone does not pass:

```
URL="https://<SERVER_URL>/<DYNAMIC_OR_USER_SPECIFIC_PATH>"
resp=$(curl -sS -m 10 -D - -o /dev/null "$URL" 2>/dev/null) || resp=""
code=$(printf '%s\n' "$resp" | awk 'NR==1{print $2}')
cc=$(printf '%s\n' "$resp" | grep -Ei '^cache-control:' | head -1 | sed 's/^[Cc]ache-
[Cc]ontrol:[[:space:]]*//' | tr -d '\r')
tokens=$(printf '%s' "$cc" | tr ',' '\n' | sed 's/^[[:space:]]*//; s/[[:space:]]*$//'
| tr 'A-Z' 'a-z')
has_nostore=$(printf '%s\n' "$tokens" | grep -Eqx 'no-store' && echo 1)
has_private=$(printf '%s\n' "$tokens" | grep -Eqx 'private' && echo 1)
if ! printf '%s' "$code" | grep -Eq '^[0-9]+$'; then
  echo "ERROR: dynamic/identity-sensitive resource unreachable, no attribution
possible"
elif [ "$code" -ge 400 ]; then
  echo "ERROR: path returned $code, cannot attribute cache policy to the resource"
elif [ -n "$has_nostore" ] || [ -n "$has_private" ]; then
  echo "PASS: dynamic/identity-sensitive response is not shared-cacheable ($code)"
else
  echo "FAIL: dynamic/identity-sensitive response lacks a no-store or private
directive ($code)"
fi
```

At the protocol layer, confirm that a cacheable result for per-user data is a conformant complete result marked `private`. Capture a `resources/read` for an identity-sensitive resource with the `2026-07-28` request envelope and confirm that `resultType` is `complete`, `ttlMs` is an integer at least zero, and `cacheScope` is `private`. The following command reports a single verdict. Substitute the server, the per-user resource URI, and its name:

```
URL="https://<SERVER_URL>/mcp"
RES_URI="<PER_USER_RESOURCE_URI>"
RES_NAME="<RESOURCE_NAME>"
PROTO="2026-07-28"
meta='"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientCapabilities":{},"io.modelcontextprotocol/clientIn
fo":{"name":"cis-mcp-audit","version":"1.0"}}'
body="{\"jsonrpc\":\"2.0\",\"id\":1,\"method\":\"resources/read\",\"params\":{\"uri\"
:\"$RES_URI\",$meta}}"
resp=$(curl -sS -m 10 -X POST -H 'Content-Type: application/json' -H "MCP-Protocol-
Version: $PROTO" -H 'Mcp-Method: resources/read' -H "Mcp-Name: $RES_NAME" --data-
binary "$body" "$URL" 2>/dev/null) || resp=""
fields=$(printf '%s' "$resp" | python3 -c 'import sys,json
try:
    r=json.load(sys.stdin).get("result",{}) or {}
    print("%s|%s|%s" % (r.get("resultType",""), r.get("ttlMs",""),
r.get("cacheScope","")))
except Exception:
    print("||")' 2>/dev/null)
rt=${fields%%|*}; rest=${fields#*|}; ttl=${rest%%|*}; cs=${rest##*|}
if [ -z "$resp" ]; then
  echo "ERROR: server unreachable, no attribution possible"
elif [ -z "$rt" ] && [ -z "$cs" ]; then
  echo "ERROR: no result fields returned, confirm the request envelope and endpoint"
elif [ "$rt" != "complete" ]; then
  echo "FAIL: result is missing resultType complete (got '$rt'), not a conformant
cacheable result"
elif ! printf '%s' "$ttl" | grep -Eq '^[0-9]+$'; then
  echo "FAIL: result lacks a non-negative ttlMs (got '$ttl')"
elif [ "$cs" = "private" ]; then
  echo "PASS: per-user result is complete, carries ttlMs, and is marked cacheScope
private"
elif [ "$cs" = "public" ]; then
  echo "FAIL: per-user result is marked cacheScope public, a cross-user disclosure
path"
else
  echo "REVIEW: cacheScope is '$cs', confirm the intended visibility for this
resource"
fi
```

This protocol-layer command is an optional aid to a per-primitive review rather than a required block, because the correct scope is resource-specific and a generic probe cannot judge every primitive.

This Recommendation is compliant when the static-resource command prints `PASS`, the dynamic-resource command prints `PASS`, and a reviewer confirms that every per-user or identity-sensitive cacheable result is marked `cacheScope: private`, that no such result is marked `public`, and that per-primitive access control is enforced independently of the cache scope.

Remediation:

Enable `ETag` or `Last-Modified` generation for genuinely static content, and set `Cache-Control max-age` values appropriate to how static each resource is, with invalidation on update. Mark all dynamic, identity-sensitive, and authorization-related responses `Cache-Control: no-store`. This is web-server and application configuration. Where a reverse proxy serves static content, the following nginx fragment is illustrative. It sets a validator and `max-age` on the static location and `no-store` on the dynamic location:

```
location /static/ { etag on; add_header Cache-Control "public, max-age=3600"; }
location /api/ { add_header Cache-Control "no-store" always; }
```

At the protocol layer, set `cacheScope: private` on `resources/list`, `resources/read`, and other cacheable results for any per-user or identity-sensitive data. Reserve `cacheScope: public` for genuinely non-identity-specific data. Set `ttlMs` to a freshness bound appropriate to the data. Enforce per-primitive access control independently of the cache scope.

Default Value:

By default, static resources may lack validators, and dynamic responses may not carry `no-store`. Under `2026-07-28`, the `cacheScope` value is the server's choice, and an SDK default is not a substitute for setting it, so it may not reflect the data's sensitivity unless per-user data is explicitly marked `private`.

References:

1. MCP specification (Server Features / Resources): https://modelcontextprotocol.io/specification/2026-07-28/server/resources
2. MCP specification (Server Features / Utilities / Caching): https://modelcontextprotocol.io/specification/2026-07-28/server/utilities/caching

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●
v8	3.12 Segment Data Processing and Storage Based on Sensitivity Segment data processing and storage based on the sensitivity of the data. Do not process sensitive data on enterprise assets intended for lower sensitivity data.		●	●

10.2 Ensure request and response body size limits, token budgets, and per-principal quotas are enforced (Manual)

Profile Applicability:

- Level 1

Description:

MCP servers enforce maximum size limits on all inbound JSON-RPC request bodies. Servers bound the size of a non-streaming result payload, and bound the size of each individual message in a streamed or long-lived response, rather than applying a single total ceiling across an arbitrarily long stream. Servers that proxy model calls enforce token-budget limits per request and per workload. Per-principal and per-tenant rate quotas are enforced independently of global server rate limits. A principal is an authorization context or an application-defined workload scope, not an MCP transport connection, because the protocol is stateless and a connection is not a session. Body-size limits are enforced both at the reverse proxy and in MCP application middleware, so that a proxy misconfiguration does not remove the only line of defense.

Rationale:

Without body-size limits, a large request or response can exhaust memory or bandwidth, which is a straightforward denial-of-service path. Token budgets bound the cost and blast radius of model-proxying calls. Per-principal quotas, enforced independently of global limits, prevent a single principal from consuming shared capacity. Enforcing size limits at both the proxy and the application means a single misconfiguration does not open the denial-of-service path.

Impact:

Size and quota limits can reject legitimate large requests if set too low, so they need tuning to real workloads.

Audit:

Confirm that a maximum request-body size is enforced. Send a normal well-formed request as a baseline and an oversized request, and confirm that the oversized request is rejected while the normal one is accepted. Set `<MAX_REQUEST_BYTES>` to the configured limit so the padded body exceeds it. The request carries the `2026-07-28` envelope (the `MCP-Protocol-Version` and `Mcp-Method` headers and a body `_meta`), so a conforming `2026-07-28` server accepts the baseline. `tools/list` does not name a target, so no `Mcp-Name` header is sent. Run the following command. It reports a single verdict:

```
URL="https://<SERVER_URL>/mcp"
MAX_REQUEST_BYTES="<MAX_REQUEST_BYTES>"
PROTO="2026-07-28"
meta='"_meta":{"io.modelcontextprotocol/protocolVersion":"2026-07-
28","io.modelcontextprotocol/clientCapabilities":{},"io.modelcontextprotocol/clientIn
fo":{"name":"cis-mcp-audit","version":"1.0"}}'
normal="{\"jsonrpc\":\"2.0\",\"id\":1,\"method\":\"tools/list\",\"params\":{$meta}}"
pad=$(head -c "$MAX_REQUEST_BYTES" /dev/zero | tr '\0' 'X')
big="{\"jsonrpc\":\"2.0\",\"id\":1,\"method\":\"tools/list\",\"params\":{$meta,\"pad\
":\"${pad}X\"}}"
post() {
  code=$(printf '%s' "$1" | curl -s -m 20 -o /dev/null -w '%{http_code}' -X POST \
    -H 'Content-Type: application/json' -H "MCP-Protocol-Version: $PROTO" -H 'Mcp-
Method: tools/list' \
    --data-binary @- "$URL" 2>/dev/null)
  [ -n "$code" ] && printf '%s' "$code" || printf '000'
}
cn=$(post "$normal")
cb=$(post "$big")
if [ "$cn" = "000" ]; then
  echo "ERROR: server unreachable on a normal request (cn=$cn), no attribution
possible"
elif ! printf '%s' "$cn" | grep -Eq '^2[0-9][0-9]$'; then
  echo "REVIEW: the normal control request did not succeed (cn=$cn), fix the request
envelope or the server before trusting the size result"
elif [ "$cb" = "413" ]; then
  echo "PASS: oversized body rejected with 413 while a normal request was accepted
(cn=$cn cb=$cb)"
elif [ "$cb" = "000" ]; then
  echo "REVIEW: server closed the connection on the oversized body (cn=$cn cb=$cb),
confirm this is size enforcement and not an unrelated fault"
elif printf '%s' "$cb" | grep -Eq '^2[0-9][0-9]$'; then
  echo "FAIL: oversized body accepted ($cb) at the HTTP layer, no request-body size
limit enforced (cn=$cn)"
else
  echo "REVIEW: oversized body returned $cb (cn=$cn), confirm how the server signals
oversize"
fi
```

The probe proves body-size enforcement at the layer that answers the request. Confirm by configuration review that the limit holds at both the reverse proxy and the application middleware, that a non-streaming response payload and each streamed message are size-bounded, that per-request and per-workload token budgets are configured, and that per-principal quota exhaustion returns a clear error rather than silently queuing.

This Recommendation is compliant when the command prints `PASS`, or prints `REVIEW` for a closed connection that the reviewer confirms is size enforcement, and the configuration review confirms that the remaining limits, budgets, and quotas are in place.

Remediation:

Configure a maximum request-body size at the reverse proxy and, independently, in MCP application middleware. Set a maximum size for non-streaming response payloads and a per-message size bound for streamed responses. Configure per-request and per-workload caps on the token budget for model-proxying calls. Set per-principal and per-tenant rate quotas distinct from global limits. These are server, middleware, and proxy settings. At the proxy, the following nginx directive caps the request body as one layer of the defense in depth. Set the value to the deployment's chosen limit:

```
client_max_body_size 1m;
```

Set the application-layer limits (request and response bytes, token budgets, per-principal quotas) in the server configuration to match, so that a proxy misconfiguration does not remove the only enforcement point.

Default Value:

By default, MCP servers and proxies may not enforce body-size limits, token budgets, or per-principal quotas distinct from global limits.

References:

1. MCP specification (Base Protocol / Transports / Streamable HTTP): https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http
2. MCP specification (security best practices, DoS): https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.10 Apply Secure Design Principles in Application Architectures Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●

Appendix: Summary Table

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
1	**Governance & Versioning**		
1.1	Ensure MCP protocol version is pinned and logged (Automated)	☐	☐
1.2	Ensure server capability baseline is established and documented (Manual)	☐	☐
1.3	Ensure newly advertised server capabilities are re-validated before invocation (Manual)	☐	☐
1.4	Ensure MCP client and server identity metadata is captured and validated against the asset inventory (Automated)	☐	☐
2	**Transport & Connectivity**		
2.1	Ensure stdio is preferred for local, single-user servers (Manual)	☐	☐
2.2	Ensure TLS is required for HTTP and plaintext is disallowed (Automated)	☐	☐
2.3	Ensure authentication propagates through proxies to request-scoped SSE responses (Manual)	☐	☐
2.4	Ensure required MCP request metadata headers are present and consistent with the request body (Automated)	☐	☐
2.5	Ensure Origin header is validated on all Streamable HTTP requests to prevent DNS rebinding attacks (Automated)	☐	☐
3	**Authentication & Authorization**		
3.1	**Authentication and Credential Sourcing**		
3.1.1	Ensure stdio server credentials are sourced from the environment or OS credential store (Automated)	☐	☐
3.1.2	Ensure OIDC/OAuth 2.1 or short-lived API tokens are used for remote servers (Manual)	☐	☐

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
3.2	**Authorization Enforcement**		
3.2.1	Ensure per-tool authorization policies are enforced (Manual)	☐	☐
3.2.2	Ensure token passthrough to downstream APIs is forbidden (Manual)	☐	☐
3.2.3	Ensure server-provided tool annotations are not relied upon for authorization or HITL gating decisions (Manual)	☐	☐
3.3	**OAuth Token and Metadata Hardening**		
3.3.1	Ensure OAuth tokens are audience-bound to the MCP server using Resource Indicators (Automated)	☐	☐
3.3.2	Ensure OAuth discovery metadata is served over TLS and validated against the approved authorization server list (Automated)	☐	☐
3.3.3	Ensure shared downstream service account identities are prohibited across multiple MCP tools or servers (Manual)	☐	☐
3.3.4	Ensure OAuth scopes are minimized and elevated progressively (Manual)	☐	☐
3.3.5	Ensure confused-deputy safeguards are applied for static OAuth client IDs (Manual)	☐	☐
4	**Client (Host) Configuration**		
4.1	**Consent & Human Oversight**		
4.1.1	Ensure per-tool consent or pre-approved allowlists are required (Manual)	☐	☐
4.1.2	Ensure project-scoped MCP server definitions are not enabled or executed without explicit user approval (Manual)	☐	☐
4.1.3	Ensure Elicitation requests are consented, redacted, and identity-bound (Manual)	☐	☐

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
4.1.4	Ensure Sampling consent and redaction controls are implemented where Sampling is in use (Automated)	☐	☐
4.1.5	Ensure a human can deny tool invocations and sensitive or high-impact operations require confirmation (Manual)	☐	☐
4.2	**Resource & Filesystem Restrictions**		
4.2.1	Ensure filesystem scope is restricted to least privilege and enforced by OS-level controls (Manual)	☐	☐
4.2.2	Ensure MCP stdio server subprocess environment is restricted to the minimum required variables (Automated)	☐	☐
4.2.3	Ensure untrusted Resource URIs are not directly dereferenced outside MCP without scheme and destination policy (Manual)	☐	☐
4.3	**MCP Apps**		
4.3.1	Ensure MCP Apps content is rendered in sandboxed isolation with Content Security Policy enforced from declared metadata (Manual)	☐	☐
4.3.2	Ensure MCP Apps permission requests default-deny, all app-to-host messages are logged, and high-risk channels are consented (Manual)	☐	☐
5	**Server Configuration**		
5.1	**Input & Schema Validation**		
5.1.1	Ensure tool schemas and argument types are validated (Automated)	☐	☐
5.1.2	Ensure resource templates with explicit URI patterns and MIME types are used (Automated)	☐	☐
5.1.3	Ensure prompt templates declare and validate their arguments (Automated)	☐	☐
5.2	**Session & Stream Hygiene**		

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
5.2.1	Ensure listChanged notifications are rate-limited to prevent client flooding (Automated)	☐	☐
5.2.2	Ensure sessions are not used as authentication and, where present, are bound to user identity (Manual)	☐	☐
5.2.3	Ensure legacy Streamable HTTP session and stream-resumption mechanisms are disabled (Automated)	☐	☐
5.3	**Protocol Stream Integrity**		
5.3.1	Ensure logs are separated from the protocol stream in stdio mode (Automated)	☐	☐
5.4	**Filesystem Path Safety**		
5.4.1	Ensure path traversal and arbitrary filesystem access are prevented (Automated)	☐	☐
5.5	**MCP Tasks**		
5.5.1	Ensure authorization, scope, and expiry controls are enforced on MCP Tasks (Manual)	☐	☐
5.6	**Idempotency & Side-Effect Control**		
5.6.1	Ensure idempotency keys are required for side-effecting tool calls and duplicate side effects are prevented (Manual)	☐	☐
6	**Data Protection & Privacy**		
6.1	Ensure model context is minimized and MIME handling is honored (Manual)	☐	☐
6.2	Ensure tokens and keys are protected in clients and servers (Automated)	☐	☐
6.3	Ensure outbound proxy and DNS controls are applied for resource fetchers (Manual)	☐	☐
7	**Observability & Audit**		
7.1	**Audit Record Content & Integrity**		

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
7.1.1	Ensure lifecycle and invocation metadata is recorded (Automated)	☐	☐
7.1.2	Ensure non-null JSON-RPC request IDs are enforced and included in audit logs (Automated)	☐	☐
7.1.3	Ensure MCP audit records carry accurate, monotonic timestamps (Automated)	☐	☐
7.2	**Detection & Alerting**		
7.2.1	Ensure alerts are generated on audience and issuer validation failures (Automated)	☐	☐
7.2.2	Ensure notifications/cancelled and notifications/progress messages are monitored for behavioral anomalies (Automated)	☐	☐
8	**Supply Chain Security**		
8.1	Ensure third-party MCP servers and connectors are vetted and allowlists are enforced (Manual)	☐	☐
8.2	Ensure third-party MCP servers are pinned to a verified content hash and re-vetted before updates (Manual)	☐	☐
8.3	Ensure third-party MCP server artifacts are served exclusively from an internal private mirror (Manual)	☐	☐
9	**Isolation & Execution Safety**		
9.1	Ensure remote MCP servers run in containers or VMs with least privilege and gateway ingress controls applied (Manual)	☐	☐
9.2	Ensure the MCP server process is confined to its working set by OS-level kernel-enforced controls (Automated)	☐	☐
9.3	Ensure local stdio MCP servers run under least-privilege identities with sandbox or container isolation (Manual)	☐	☐
10	**Resource Limits & Caching**		

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
10.1	Ensure safe, static resources are cached with freshness limits and per-user data is never shared-cached (Manual)	☐	☐
10.2	Ensure request and response body size limits, token budgets, and per-principal quotas are enforced (Manual)	☐	☐

Appendix: CIS Controls v8 IG 1 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
1.1	Ensure MCP protocol version is pinned and logged	☐	☐
1.2	Ensure server capability baseline is established and documented	☐	☐
1.4	Ensure MCP client and server identity metadata is captured and validated against the asset inventory	☐	☐
3.1.1	Ensure stdio server credentials are sourced from the environment or OS credential store	☐	☐
3.3.4	Ensure OAuth scopes are minimized and elevated progressively	☐	☐
3.3.5	Ensure confused-deputy safeguards are applied for static OAuth client IDs	☐	☐
4.1.1	Ensure per-tool consent or pre-approved allowlists are required	☐	☐
4.1.2	Ensure project-scoped MCP server definitions are not enabled or executed without explicit user approval	☐	☐
4.1.3	Ensure Elicitation requests are consented, redacted, and identity-bound	☐	☐
4.1.4	Ensure Sampling consent and redaction controls are implemented where Sampling is in use	☐	☐
4.2.1	Ensure filesystem scope is restricted to least privilege and enforced by OS-level controls	☐	☐
5.3.1	Ensure logs are separated from the protocol stream in stdio mode	☐	☐
6.1	Ensure model context is minimized and MIME handling is honored	☐	☐
6.3	Ensure outbound proxy and DNS controls are applied for resource fetchers	☐	☐
7.1.1	Ensure lifecycle and invocation metadata is recorded	☐	☐
7.1.2	Ensure non-null JSON-RPC request IDs are enforced and included in audit logs	☐	☐
7.2.1	Ensure alerts are generated on audience and issuer validation failures	☐	☐

Recommendation		Set Correctly	
		Yes	No
8.1	Ensure third-party MCP servers and connectors are vetted and allowlists are enforced	☐	☐
8.2	Ensure third-party MCP servers are pinned to a verified content hash and re-vetted before updates	☐	☐
8.3	Ensure third-party MCP server artifacts are served exclusively from an internal private mirror	☐	☐
9.2	Ensure the MCP server process is confined to its working set by OS-level kernel-enforced controls	☐	☐
9.3	Ensure local stdio MCP servers run under least-privilege identities with sandbox or container isolation	☐	☐
10.1	Ensure safe, static resources are cached with freshness limits and per-user data is never shared-cached	☐	☐

Appendix: CIS Controls v8 IG 2 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
1.1	Ensure MCP protocol version is pinned and logged	☐	☐
1.2	Ensure server capability baseline is established and documented	☐	☐
1.3	Ensure newly advertised server capabilities are re-validated before invocation	☐	☐
1.4	Ensure MCP client and server identity metadata is captured and validated against the asset inventory	☐	☐
2.1	Ensure stdio is preferred for local, single-user servers	☐	☐
2.2	Ensure TLS is required for HTTP and plaintext is disallowed	☐	☐
2.3	Ensure authentication propagates through proxies to request-scoped SSE responses	☐	☐
2.4	Ensure required MCP request metadata headers are present and consistent with the request body	☐	☐
2.5	Ensure Origin header is validated on all Streamable HTTP requests to prevent DNS rebinding attacks	☐	☐
3.1.1	Ensure stdio server credentials are sourced from the environment or OS credential store	☐	☐
3.1.2	Ensure OIDC/OAuth 2.1 or short-lived API tokens are used for remote servers	☐	☐
3.2.2	Ensure token passthrough to downstream APIs is forbidden	☐	☐
3.2.3	Ensure server-provided tool annotations are not relied upon for authorization or HITL gating decisions	☐	☐
3.3.1	Ensure OAuth tokens are audience-bound to the MCP server using Resource Indicators	☐	☐
3.3.2	Ensure OAuth discovery metadata is served over TLS and validated against the approved authorization server list	☐	☐
3.3.3	Ensure shared downstream service account identities are prohibited across multiple MCP tools or servers	☐	☐

Recommendation		Set Correctly	
		Yes	No
3.3.4	Ensure OAuth scopes are minimized and elevated progressively	☐	☐
3.3.5	Ensure confused-deputy safeguards are applied for static OAuth client IDs	☐	☐
4.1.1	Ensure per-tool consent or pre-approved allowlists are required	☐	☐
4.1.2	Ensure project-scoped MCP server definitions are not enabled or executed without explicit user approval	☐	☐
4.1.3	Ensure Elicitation requests are consented, redacted, and identity-bound	☐	☐
4.1.4	Ensure Sampling consent and redaction controls are implemented where Sampling is in use	☐	☐
4.1.5	Ensure a human can deny tool invocations and sensitive or high-impact operations require confirmation	☐	☐
4.2.1	Ensure filesystem scope is restricted to least privilege and enforced by OS-level controls	☐	☐
4.2.2	Ensure MCP stdio server subprocess environment is restricted to the minimum required variables	☐	☐
4.2.3	Ensure untrusted Resource URIs are not directly dereferenced outside MCP without scheme and destination policy	☐	☐
4.3.1	Ensure MCP Apps content is rendered in sandboxed isolation with Content Security Policy enforced from declared metadata	☐	☐
4.3.2	Ensure MCP Apps permission requests default-deny, all app-to-host messages are logged, and high-risk channels are consented	☐	☐
5.1.1	Ensure tool schemas and argument types are validated	☐	☐
5.1.2	Ensure resource templates with explicit URI patterns and MIME types are used	☐	☐
5.1.3	Ensure prompt templates declare and validate their arguments	☐	☐
5.2.1	Ensure listChanged notifications are rate-limited to prevent client flooding	☐	☐
5.2.2	Ensure sessions are not used as authentication and, where present, are bound to user identity	☐	☐
5.2.3	Ensure legacy Streamable HTTP session and stream-resumption mechanisms are disabled	☐	☐

Recommendation		Set Correctly	
		Yes	No
5.3.1	Ensure logs are separated from the protocol stream in stdio mode	☐	☐
5.4.1	Ensure path traversal and arbitrary filesystem access are prevented	☐	☐
5.5.1	Ensure authorization, scope, and expiry controls are enforced on MCP Tasks	☐	☐
5.6.1	Ensure idempotency keys are required for side-effecting tool calls and duplicate side effects are prevented	☐	☐
6.1	Ensure model context is minimized and MIME handling is honored	☐	☐
6.2	Ensure tokens and keys are protected in clients and servers	☐	☐
6.3	Ensure outbound proxy and DNS controls are applied for resource fetchers	☐	☐
7.1.1	Ensure lifecycle and invocation metadata is recorded	☐	☐
7.1.2	Ensure non-null JSON-RPC request IDs are enforced and included in audit logs	☐	☐
7.1.3	Ensure MCP audit records carry accurate, monotonic timestamps	☐	☐
7.2.1	Ensure alerts are generated on audience and issuer validation failures	☐	☐
7.2.2	Ensure notifications/cancelled and notifications/progress messages are monitored for behavioral anomalies	☐	☐
8.1	Ensure third-party MCP servers and connectors are vetted and allowlists are enforced	☐	☐
8.2	Ensure third-party MCP servers are pinned to a verified content hash and re-vetted before updates	☐	☐
8.3	Ensure third-party MCP server artifacts are served exclusively from an internal private mirror	☐	☐
9.1	Ensure remote MCP servers run in containers or VMs with least privilege and gateway ingress controls applied	☐	☐
9.2	Ensure the MCP server process is confined to its working set by OS-level kernel-enforced controls	☐	☐
9.3	Ensure local stdio MCP servers run under least-privilege identities with sandbox or container isolation	☐	☐
10.1	Ensure safe, static resources are cached with freshness limits and per-user data is never shared-cached	☐	☐

Recommendation		Set Correctly	
		Yes	No
10.2	Ensure request and response body size limits, token budgets, and per-principal quotas are enforced	☐	☐

Appendix: CIS Controls v8 IG 3 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
1.1	Ensure MCP protocol version is pinned and logged	☐	☐
1.2	Ensure server capability baseline is established and documented	☐	☐
1.3	Ensure newly advertised server capabilities are re-validated before invocation	☐	☐
1.4	Ensure MCP client and server identity metadata is captured and validated against the asset inventory	☐	☐
2.1	Ensure stdio is preferred for local, single-user servers	☐	☐
2.2	Ensure TLS is required for HTTP and plaintext is disallowed	☐	☐
2.3	Ensure authentication propagates through proxies to request-scoped SSE responses	☐	☐
2.4	Ensure required MCP request metadata headers are present and consistent with the request body	☐	☐
2.5	Ensure Origin header is validated on all Streamable HTTP requests to prevent DNS rebinding attacks	☐	☐
3.1.1	Ensure stdio server credentials are sourced from the environment or OS credential store	☐	☐
3.1.2	Ensure OIDC/OAuth 2.1 or short-lived API tokens are used for remote servers	☐	☐
3.2.1	Ensure per-tool authorization policies are enforced	☐	☐
3.2.2	Ensure token passthrough to downstream APIs is forbidden	☐	☐
3.2.3	Ensure server-provided tool annotations are not relied upon for authorization or HITL gating decisions	☐	☐
3.3.1	Ensure OAuth tokens are audience-bound to the MCP server using Resource Indicators	☐	☐
3.3.2	Ensure OAuth discovery metadata is served over TLS and validated against the approved authorization server list	☐	☐
3.3.3	Ensure shared downstream service account identities are prohibited across multiple MCP tools or servers	☐	☐

Recommendation		Set Correctly	
		Yes	No
3.3.4	Ensure OAuth scopes are minimized and elevated progressively	☐	☐
3.3.5	Ensure confused-deputy safeguards are applied for static OAuth client IDs	☐	☐
4.1.1	Ensure per-tool consent or pre-approved allowlists are required	☐	☐
4.1.2	Ensure project-scoped MCP server definitions are not enabled or executed without explicit user approval	☐	☐
4.1.3	Ensure Elicitation requests are consented, redacted, and identity-bound	☐	☐
4.1.4	Ensure Sampling consent and redaction controls are implemented where Sampling is in use	☐	☐
4.1.5	Ensure a human can deny tool invocations and sensitive or high-impact operations require confirmation	☐	☐
4.2.1	Ensure filesystem scope is restricted to least privilege and enforced by OS-level controls	☐	☐
4.2.2	Ensure MCP stdio server subprocess environment is restricted to the minimum required variables	☐	☐
4.2.3	Ensure untrusted Resource URIs are not directly dereferenced outside MCP without scheme and destination policy	☐	☐
4.3.1	Ensure MCP Apps content is rendered in sandboxed isolation with Content Security Policy enforced from declared metadata	☐	☐
4.3.2	Ensure MCP Apps permission requests default-deny, all app-to-host messages are logged, and high-risk channels are consented	☐	☐
5.1.1	Ensure tool schemas and argument types are validated	☐	☐
5.1.2	Ensure resource templates with explicit URI patterns and MIME types are used	☐	☐
5.1.3	Ensure prompt templates declare and validate their arguments	☐	☐
5.2.1	Ensure listChanged notifications are rate-limited to prevent client flooding	☐	☐
5.2.2	Ensure sessions are not used as authentication and, where present, are bound to user identity	☐	☐
5.2.3	Ensure legacy Streamable HTTP session and stream-resumption mechanisms are disabled	☐	☐

Recommendation		Set Correctly	
		Yes	No
5.3.1	Ensure logs are separated from the protocol stream in stdio mode	☐	☐
5.4.1	Ensure path traversal and arbitrary filesystem access are prevented	☐	☐
5.5.1	Ensure authorization, scope, and expiry controls are enforced on MCP Tasks	☐	☐
5.6.1	Ensure idempotency keys are required for side-effecting tool calls and duplicate side effects are prevented	☐	☐
6.1	Ensure model context is minimized and MIME handling is honored	☐	☐
6.2	Ensure tokens and keys are protected in clients and servers	☐	☐
6.3	Ensure outbound proxy and DNS controls are applied for resource fetchers	☐	☐
7.1.1	Ensure lifecycle and invocation metadata is recorded	☐	☐
7.1.2	Ensure non-null JSON-RPC request IDs are enforced and included in audit logs	☐	☐
7.1.3	Ensure MCP audit records carry accurate, monotonic timestamps	☐	☐
7.2.1	Ensure alerts are generated on audience and issuer validation failures	☐	☐
7.2.2	Ensure notifications/cancelled and notifications/progress messages are monitored for behavioral anomalies	☐	☐
8.1	Ensure third-party MCP servers and connectors are vetted and allowlists are enforced	☐	☐
8.2	Ensure third-party MCP servers are pinned to a verified content hash and re-vetted before updates	☐	☐
8.3	Ensure third-party MCP server artifacts are served exclusively from an internal private mirror	☐	☐
9.1	Ensure remote MCP servers run in containers or VMs with least privilege and gateway ingress controls applied	☐	☐
9.2	Ensure the MCP server process is confined to its working set by OS-level kernel-enforced controls	☐	☐
9.3	Ensure local stdio MCP servers run under least-privilege identities with sandbox or container isolation	☐	☐
10.1	Ensure safe, static resources are cached with freshness limits and per-user data is never shared-cached	☐	☐

Recommendation		Set Correctly	
		Yes	No
10.2	Ensure request and response body size limits, token budgets, and per-principal quotas are enforced	☐	☐

Appendix: Change History

Date	Version	Description